

UART: Hardware, Driver & Integration

UART

Hardware, Driver & Integration

*FPGA meets MCU: a peripheral in VHDL,
its driver in C++, and the bench between them*

Erik Pihl

UART: Hardware, Driver & Integration
Copyright © 2026 QRTECH Academy.

The text and figures of this book, and of the course it is typeset from, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). You may copy and redistribute them, and adapt them, for any non-commercial purpose, provided you give appropriate credit and share what you make under the same license. To view a copy of the license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

The code in this book may also be used under the MIT License, which is the license of all the code in the companion repository. The repository holds the lecture material, the protocol specification, the provided VHDL and its testbenches, the host test suites and the build scripts:

<https://github.com/qrttech-academy/uart>

The VHDL in this book is VHDL-93, simulated with GHDL and synthesized with Quartus Prime Lite. The C++ is C++17 on the host and, wherever it runs on the ATmega328P, a subset that also compiles as C++14 with avr-gcc. Arduino Nano, ATmega328P, DE0-CV, Cyclone V, Quartus, USB-Blaster, CP2102 and FT232 are trademarks of their respective owners.

Set in TeX Gyre Pagella and DejaVu Sans Mono with Lua[®]TeX.

Typeset from the ten lectures of the course, their exercises, the UART register protocol, and the two self-assessment papers. This edition: 11 September 2026.

Contents

Preface	ix
I The FPGA Peripheral	1
1 The Register Package and the Peripheral Top	3
1.1 uart_def.vhd (provided)	4
1.2 Designing uart_top.vhd	8
1.3 Review	13
1.4 Exercises	13
2 UART Framing and the Transmitter	21
2.1 Designing baud_gen.vhd	22
2.2 Designing uart_tx.vhd	24
2.3 Review	28
2.4 Exercises	28
3 The Synchronizer and the Receiver	35
3.1 Designing sync.vhd	36
3.2 Designing uart_rx.vhd	39
3.3 Review	43
3.4 Exercises	44
4 The FIFO and the Receive Path	47
4.1 Designing fifo.vhd	48
4.2 Review	51
4.3 Exercises	52
5 The Register Bank	57
5.1 Designing uart_regs.vhd	58
5.2 Review	67
5.3 Exercises	68

II	The Driver and Integration	73
6	The Driver's Contracts	75
6.1	The Driver Stack (the Contracts)	76
6.2	Review	78
6.3	Exercises	78
7	The Driver, Built	85
7.1	Building the Driver	86
7.2	Review	88
7.3	Exercises	89
8	The Driver, Tested	93
8.1	Testing the Driver on the Host	94
8.2	Review	96
8.3	Exercises	97
9	The Real Transport, and Both Toolchains	103
9.1	The SPI Peripheral and AvrSpi	104
9.2	The Freestanding Target and the Toolchain.	107
9.3	Review	109
9.4	Exercises	110
10	Integration and Bring-Up	115
10.1	The Two-Link Bench	116
10.2	What You Are Building: <code>app::EchoNode</code>	118
10.3	Review	121
10.4	Bringing Up the Bench	122
A	The UART Register Protocol	125
A.1	Part 1: The UART Line Protocol	125
A.2	Part 2: The Register Map.	126
A.3	Part 3: The SPI Transport	128
A.4	A Worked Example	129
B	Answers to the Exercises	131
B.1	Chapter 1: The Register Package and the Peripheral Top.	131
B.2	Chapter 2: UART Framing and the Transmitter	134
B.3	Chapter 3: The Synchronizer and the Receiver.	138
B.4	Chapter 4: The FIFO and the Receive Path	140
B.5	Chapter 5: The Register Bank	142

B.6	Chapter 9: The Real Transport, and Both Toolchains	145
B.7	Chapter 10: Integration and Bring-Up	149
C	The Self-Assessment Papers	153
D	Self-Assessment Paper A	157
E	Self-Assessment Paper B	169
F	Model Answers to Paper A	177
	Question 1: The map, the top, and what a clean analysis does not prove . . .	177
	Question 2: The wire, the divider, and the transmitter	181
	Question 3: The asynchronous line	184
	Question 4: Ports become registers	188
	Question 5: The driver's contracts	191
	Question 6: The five-byte transaction	194
	Question 7: The real transport, and the real toolchain	197
	Question 8: The bench.	201
G	Model Answers to Paper B	207
	Question 1: Building the top before anything it holds	207
	Question 2: The generator, the divider, and the frame	211
	Question 3: The asynchronous line, written out	215
	Question 4: The bank, written out.	220
	Question 5: Writing the contracts	224
	Question 6: Writing the protocol.	229
	Question 7: Writing the transport	233
	Question 8: The application, and the plan	236

Preface

This book is the entry course to the *FPGA-meets-MCU* pattern: build a hardware peripheral in VHDL, write its driver in modern C++, then integrate the two across a real chip boundary. UART is the simplest serial peripheral there is, chosen deliberately so that you meet the *pattern*, the whole loop of hardware, driver and integration, before meeting a hard protocol. Harder protocols apply the same loop later, in far greater depth; this is where its shape is learned.

The full stack, top (the application) to bottom (the wire), spans two chips joined by two serial links. You write every layer of it except the SPI transport, which is provided:

Layer	Runs on	Written by
app::EchoNode	ATmega328P (C++)	you
The driver interface	ATmega328P (C++)	you
driver::uart::Uart	ATmega328P (C++)	you
The AVR SPI master, a driver:: transport::Interface	ATmega328P (C++)	you
<i>The SPI link: four wires and a 5 V to 3.3 V level shifter (the control plane)</i>		
spi_slave and spi_reg_bridge	DE0-CV (VHDL)	provided, a black box
The register bank	DE0-CV (VHDL)	you
uart_top: baud generator, TX, RX, FIFOs	DE0-CV (VHDL)	you
<i>The UART link: tx and rx at 3.3 V (the data plane)</i>		
A PC terminal on a USB-serial adapter	external	—

Note the two serial links, and that they are *not* the same protocol. The **control plane** is SPI: the ATmega328P is the CPU, and it configures and drives the FPGA's UART peripheral by reading and writing the peripheral's registers over SPI. The **data plane** is the UART itself: the peripheral's own tx and rx lines carry bytes to and from the outside world. There is no soft CPU anywhere; the processor is the real ATmega328P on an Arduino Nano.

Why drive a UART over SPI when the ATmega already has one?

Because the ATmega's own UART is not the subject; the *pattern* is. Here the ATmega plays host CPU to a memory-mapped peripheral it does not contain, reaching it across a chip boundary. That is the real situation with FPGA and ASIC peripherals, and it is what makes the skill transfer to the next protocol. Building a UART this way is not the efficient way to get a UART; it is the way to learn the boundary, with a protocol simple enough that nothing about the boundary is hidden by the protocol's own complexity.

What the book covers

The book covers the hardware/software boundary of an embedded system through the simplest serial peripheral there is:

- UART framing from the wire up: the idle-high line, the start bit, the data bits least significant first, optional parity, the stop bit or bits, and how a baud rate maps to a bit period.
- A baud-rate generator, a transmitter and an oversampling receiver in VHDL, each verified by a testbench.
- Register semantics over port semantics: poll-able STATUS bits and sticky ERROR_FLAGS bits built from single-cycle pulses; a write-triggered TX push; a read-then-pop RX path, and why a read that pops a FIFO would need the same commit-on-completion, abort-safe discipline as a write.
- The shared UART register protocol both halves implement against, the book's single source of truth, printed in full as Appendix A.
- A `driver::transport::Interface` seam under the C++ `driver::uart::Uart` implements `driver::uart::Interface` over it, host-tested against a scripted `driver::transport::Stub` before any hardware exists.
- Freestanding C++ on the ATmega328P, and bring-up on the bench: two serial links, a level shifter, a bring-up ladder, and an echo application driven end to end.

What you are assumed to know

The book is self-contained apart from the three foundations below, which it does **not** teach again:

- **Digital design with VHDL:** entity and architecture, processes, synchronizers, state machines, and running a provided self-checking testbench.
- **Modern embedded C++:** classes, interfaces, and the layered-driver idea.

- **Embedded C:** the ATmega toolchain (avr-gcc and avrdude) and register-level programming. It is needed only from Chapter 9, when the driver reaches the real chip.

The provided transport

The link between the MCU and the FPGA, a byte-level `spi_slave` and a transaction-level `spi_reg_bridge`, is handed out as testbench-verified VHDL and used as a black box: you write the peripheral behind it (the register bank and `uart_top`) and the driver above it, not the bridge itself. Part 3 of the protocol (§A.3) documents the transport fully, so nothing about it is magic, only out of scope. The box stays closed for one more course, since the I2C course reuses the same control plane, and is opened in the one after: building `spi_slave` and `spi_reg_bridge` from the wire up, clock-domain crossing and all, is the whole subject of *SPI: The MCU-FPGA Transport, from the Wire Up*.

How the book is organised

The book is typeset from a ten-lecture course, and each chapter is one lecture. Its sections are the lecture's appendices, in the same order and with the same subsections, so a chapter here and a lecture directory in the repository can be read side by side. The lectures come in two parts, and so does the book:

Chapter	Lecture	Side	Topic
<i>Part I: The FPGA Peripheral</i>			
1	L01	VHDL	The register package and the peripheral top
2	L02	VHDL	UART framing and the transmitter
3	L03	VHDL	The synchronizer and the receiver
4	L04	VHDL	The FIFO and the receive path
5	L05	VHDL	The register bank
<i>Part II: The Driver and Integration</i>			
6	L06	C++	The driver's contracts
7	L07	C++	The driver, built
8	L08	C++	The driver, tested
9	L09	C++	The real transport, and both toolchains
10	L10	Both	Integration and bring-up

The rhythm is five chapters of VHDL, four of C++, and one with both halves on the bench at once. Part I ends with a peripheral that passes its own system testbench in simulation; Part II crosses the wire to the ATmega and ends with both chips talking on the bench.

Every chapter opens with what is in it, works through the material, closes with a review of what you should now be able to explain, and ends with its exercises. The appendices hold what every chapter leans on. Appendix A is the UART register protocol, the contract both halves are written against: read its Parts 2 and 3 (§A.2 and §A.3) before Chapter 1, and

Part 1 (§A.1) before Chapter 2. Appendix B answers the exercises that are not code, and the appendices after it hold the two written papers described below.

What the exercises look like

Roughly half the code the course produces lives in its exercises, and a provided self-checking testbench or host suite is what says whether it works. The exercises are numbered within each chapter, and each is labelled with its kind:

Kind	What it asks of you
VHDL	Write a module from its section's specification, and check it with the provided testbench in GHDL.
C++	Write a header or a class from its specification, and check it with the provided host suite, <code>make test</code> .
REASONING	Answer in words or numbers: explain why, trace what happens, or say what a testbench reports when you break a working design on purpose.
BENCH	Work with the hardware in front of you: the Quartus flow, the wiring, and the bring-up ladder.

Most exercises mix the two halves of that table. A VHDL exercise typically starts with the module and goes on to parts that ask what it does, what a deliberate mistake in it would do, and why the specification is the way it is.

The code is not answered in this book. Each exercise names the testbench or the host suite that checks it, and that check is the authority on your module: it is the executable form of the specification, and a listing printed here would only be one answer of many. **Every other part is answered**, in Appendix B: the reasoning, the numbers, the traces, and what a testbench reports when you break a working design on purpose, verified by doing exactly that. Try each part before you read its answer.

Two written papers, and what they are not

Nothing in the course this book comes from is marked. Assessment is the exercises after every chapter, the provided testbench or host suite that grades each one, and the questions in every chapter's review.

Appendices C to G hold two four-hour papers with worked solutions, and they check something else: **what you can reconstruct on paper, with nothing in front of you.** Eight questions each, covering all ten chapters, four of VHDL, three of C++ and one with both halves on the bench, mixing theory with code you either trace and repair or write from scratch, and with no GHDL and no `make test` in the room to tell you which it is.

They exist purely so that you can test your own skills and knowledge after working through the book. They gate nothing, they are not a qualification, and no part of the book requires them.

Take one once you have finished the book, after Chapter 10 and the bring-up ladder. Both papers draw on all ten chapters and on both sides of the wire, so sitting one partway through examines material you have not read yet.

Setting up

Everything short of the bench runs in a Linux terminal: on Linux itself, or on Windows through WSL. The versions below are the floor the build requires; the course is developed against newer ones.

- **GHDL** 3.0 or newer, to analyze and simulate the VHDL. Everything is analyzed with `--std=93`.
- **g++** 7 or newer, for the host build and its tests, which are C++17.
- **avr-gcc** 5 or newer, with **avr-libc** and **avrdude**, to build and flash the ATmega328P. The AVR build is deliberately C++14, so that it also builds on the older compiler Microchip Studio ships.
- **clang-format**, for the style check.
- **Quartus Prime Lite** 20.1 or newer, which is the first with the Cyclone V support the DE0-CV needs, to synthesize the peripheral and program the board. It is not needed until Chapter 9, and it is a multi-gigabyte download, so install it before you get there.
- **A serial terminal**: `picocom`, `minicom` or `screen` on Linux, `PuTTY` on Windows, at 115200 8N1.
- **An editor**; Visual Studio Code is the one the course uses.

Then clone the companion repository, together with the test framework it keeps as a submodule; in a clone made without `--recursive`, run `git submodule update --init` once:

```
git clone --recursive https://github.com/qrtech-academy/uart.git
```

A few commands at its root cover everything the book asks of it:

```
make build           # Everything: the host C++ and its tests, then the VHDL.
make build-vhdl      # Analyze every module and run every testbench it can.
make build-cpp       # Build fw/ and run its host suites.
make format-check    # Check the formatting of the C++ and the VHDL.
```

Code arrives chapter by chapter, and both builds are written for that. A testbench whose modules you have not written yet is reported as skipped rather than failed, and so is a host suite whose headers do not exist yet, so the build stays green while the design is still half-built.

All the VHDL lives in one directory, `hw/`: the modules you write, beside the provided testbenches that check them, the provided transport and the provided register-map package.

Nothing is copied between chapters. A testbench is run from there in three GHDL steps, dependencies first; for the transmitter of Chapter 2:

```
cd hw
ghdl -a --std=93 uart_tx.vhd uart_tx_tb.vhd
ghdl -e --std=93 uart_tx_tb
ghdl -r --std=93 uart_tx_tb --assert-level=error
```

`--assert-level=error` makes a failed check stop the simulation with a non-zero exit code, rather than printing and running on to a misleading pass. Every testbench binds the module it drives **positionally**, and so does every instantiation inside `uart_top`: the contract is the order and the types of the ports, exactly as each section's interface table lists them, and the names are yours.

The C++ lives in `fw/`: headers in `include/`, sources in `source/`, the host suites in `test/`, and the ATmega328P port in `avr/`, which is cross-compiled and flashed with `make -C fw/avr flash` rather than built on the host.

For the bench in Chapter 9 and Chapter 10 you need, per bench, a **Terasic DE0-CV**; an **Arduino Nano** (any 5 V, 16 MHz ATmega328P Nano or clone); a **four-channel bidirectional level shifter**, such as a BSS138-based board, for the four SPI lines; a **3.3 V USB-serial adapter**, such as a CP2102 or an FT232 set to 3.3 V logic, as the far end of the data plane; a breadboard and jumper wires; and the cables and the 12 V supply for the DE0-CV's on-board USB-Blaster. A logic analyzer is recommended but not required: any eight-channel 24 MHz unit is plenty at 1 MHz SCK and 115200 baud, one exercise in Chapter 9 uses one and says so, and nothing else depends on it.

PART I

The FPGA Peripheral

The Register Package and the Peripheral Top

Top-down: the shell the whole peripheral fills in.

IN THIS CHAPTER

- **The provided register map, `uart_def`:** the seven register indices and the STATUS, CTRL and ERROR_FLAGS bit positions from Part 2 of the protocol (§A.2), mirrored on the C++ side in `register_map.hpp`.
- **`uart_top` itself:** the entity as a positional port contract, the provided `reset_sync` that asserts asynchronously and releases synchronously, and the provided SPI transport behind it.
- **Declaring signals as you need them:** each internal signal comes from reading the entity of the block it connects to, so the top grows one block's worth of wiring per chapter rather than starting with a list of names for modules that do not exist yet.
- **What the shape costs and buys:** what a positional contract cannot catch, and how the top grows across Chapters 2 to 5 until the system testbench can finally run.

This chapter starts at the top. Before a single datapath block exists we build the peripheral's shell, because the shell is what fixes every block's interface: once `uart_top` says how it instantiates `baud_gen`, the ports `baud_gen` must expose are settled, and the rest of Part I is filling that shell in. After the chapter you should be able to read the provided register-map package and explain why its bit-**position** constants must match the protocol; build the `uart_top` skeleton so that it analyzes cleanly with no datapath blocks present; and explain why fixing the top first fixes each block's port contract, and why the system testbench is skipped rather than failed until Chapter 5.

Before you start, you need digital design with VHDL: entity and architecture, structural instantiation, clocked processes, and analyzing a design with GHDL. Read the register map (§A.2) and the SPI transport (§A.3) in the protocol appendix; you instantiate the transport, but you do not write it.

Read §1.1 first, for the register map you will wire against, then §1.2, which is the reasoning behind `uart_top` and the design you build from. The exercises in §1.4 carry the build

itself, so skim them as well and know where their tables are. Do open `reset_sync.vhd`, `spi_slave.vhd` and `spi_reg_bridge.vhd` in `hw/`: their entities are where the signals you have to declare come from, and reading a port list to work out what wiring it implies is the habit this chapter is teaching. The exercises tabulate the same ports, but as a check on your reading rather than a substitute for it. Finally, read the system testbench `uart_top_tb.vhd` to see the contract the top must eventually satisfy; you will not run it until Chapter 5.

By the end of the chapter `uart_top` analyzes, though there is no testbench to run yet:

```
cd hw
ghdl -a --std=93 uart_def.vhd reset_sync.vhd spi_slave.vhd spi_reg_bridge.vhd ↵
↵ uart_top.vhd
```

Then, from the repository root, run `make build-vhdl` and note that `uart_top_tb` reports **skipped**: its datapath blocks do not exist yet.

In the exercises, Exercise 1.2 is the build, decomposed into five steps: the entity, the signals this chapter actually wires, the provided `reset_sync`, the two transport blocks, and the check. Exercise 1.1 is the reading: the register map, positions rather than masks, and when a swapped index would surface. Exercise 1.3 is the reasoning, about positional binding, the reset synchronizer, and a register bus that waits four chapters for its bank.

1.1 `uart_def.vhd` (provided)

`uart_def` is the register map in VHDL form: one package of named constants for the seven register indices and the meaning of each `STATUS`, `CTRL` and `ERROR_FLAGS` bit, from Part 2 of the protocol (§A.2). Like the SPI transport, **it is given to you**; you write every module in this course, but not the map. It exists so that the register bank (`uart_regs`, Chapter 5) and the testbenches can name registers and bits instead of writing bare numbers, and so that the two sides of the wire cannot drift apart.

The C++ driver carries the same map in `register_map.hpp`, transcribed once on each side of the wire. Both store **bit positions**, not masks, so `STATUS` bit 1 is the constant 1 in both languages. The C++ side declares only the subset the driver actually uses, under the `reg::`, `status::`, `ctrl::` and `error::` namespaces; where a name exists on both sides, the value is identical. You do not edit `uart_def`, but read it closely: a module that indexes the wrong bit is a bug you would only find on the bench.

1.1.1 The constants

Everything is a natural in package `uart_def`. The register indices are the offset divided by 4; the bit constants are positions into a 32-bit register word.

Group	Constant	Value	Meaning
Register index	REG_STATUS	0	Status register.
	REG_CTRL	1	Control register.
	REG_BAUD_DIV	2	Baud divider.
	REG_TX_DATA	3	Push a byte into the TX FIFO.
	REG_RX_DATA	4	Front byte of the RX FIFO (a pure read).
	REG_RX_POP	5	Advance the RX FIFO.
	REG_ERR_FLAGS	6	Error flag register.
STATUS bit	ST_TX_READY	0	The TX FIFO is not full.
	ST_RX_VALID	1	The RX FIFO is not empty.
	ST_ERROR	2	One or more error flags are set.
	ST_TX_IDLE	3	The TX FIFO is empty and the line is idle.
CTRL bit	CT_ENABLE	0	Enable the peripheral.
	CT_PARITY_LO	1	Parity select, low bit.
	CT_PARITY_HI	2	Parity select, high bit (00 none, 01 even, 10 odd).
	CT_STOP	3	0 for 1 stop bit, 1 for 2 stop bits.
	CT_RX_IRQ	4	RX-valid IRQ mask.
	CT_TX_IRQ	5	TX-ready IRQ mask.
ERROR_FLAGS bit	ER_FRAMING	0	Framing error.
	ER_PARITY	1	Parity error.
	ER_OVERRUN	2	Overrun.

Because they are positions, you index a `std_logic_vector` with them directly, for example `status(ST_RX_VALID)` for the RX-valid bit, exactly as `1U << status::RX_VALID` forms the same mask on the C++ side. Laid out as words, the three flag registers are shown in Figure 1.1.

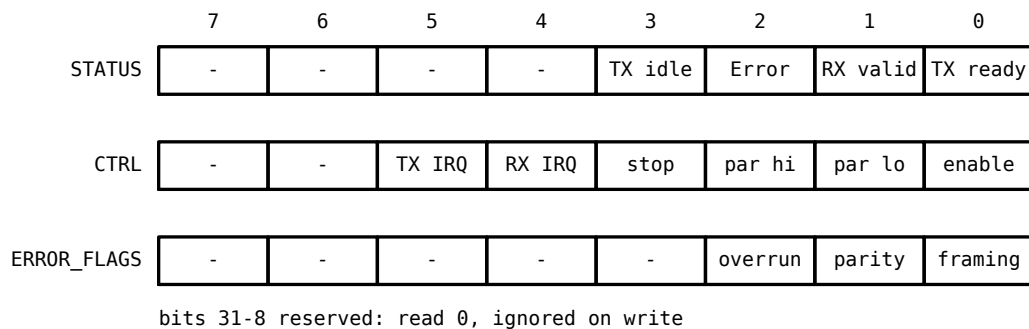


Figure 1.1 The bit fields of STATUS, CTRL and ERROR_FLAGS

CTRL is the only one with bits this course does not act on, and ERROR_FLAGS the only one where two of the three have no producer yet; both are noted where they are built, in §5.1.

1.1.2 The to_hex helper

The package also carries one small convenience for the testbenches: `to_hex`, which renders a `std_logic_vector` as an uppercase, zero-padded hex string. It pads to whole nibbles taken from the argument's own length, so `x"A5"` becomes `"A5"` and a 12-bit bus becomes three digits. It lets a testbench name the offending value in a failure message,

```
report "rx_data byte 1 wrong: expected 0xA5, got 0x" & to_hex(captured_rx) & "!"
severity error;
```

rather than a bare “mismatch”. It is the one piece of behaviour in an otherwise pure constants package: declared in the package, defined in the package body, and called only by testbenches. No synthesizable module uses it.

The whole package, constants and helper, is short enough to print as it stands in the repository:

```
hw/uart_def.vhd

-----
-- uart_def: the register map in VHDL form (L01, provided).
--
-- Named constants for the seven register indices and the meaning of each STATUS,
-- CTRL, and ERROR_FLAGS bit, from Part 2 of the protocol spec. The C++ side's
-- register_map.hpp carries the same map, transcribed once on each side of the
-- wire; it declares only the subset the driver uses, and where a name exists in
-- both, the value is identical. Indices are the byte offset divided by 4; the
-- bit constants are positions into a 32-bit register word.
--
-- This package is given to you. You write every module that uses it (uart_regs,
-- uart_top, and the datapath), but the map itself is fixed so the two sides of
-- the wire cannot drift.
--
-- It also carries one testbench helper, to_hex, so the benches can print a bus
-- value in a failure message.
-----

library ieee;
use ieee.std_logic_1164.all;

package uart_def is
    -- Register indices (byte offset / 4).
    constant REG_STATUS   : natural := 0;    -- Status register.
    constant REG_CTRL     : natural := 1;    -- Control register.
    constant REG_BAUD_DIV : natural := 2;    -- Baud divider.
    constant REG_TX_DATA  : natural := 3;    -- Push a byte into the TX FIFO.
```

```

constant REG_RX_DATA  : natural := 4;  -- Front byte of the RX FIFO (pure ↵
↵ read).
constant REG_RX_POP    : natural := 5;  -- Advance the RX FIFO.
constant REG_ERR_FLAGS: natural := 6;  -- Error flag register.

-- STATUS bits.
constant ST_TX_READY: natural := 0;  -- TX FIFO not full.
constant ST_RX_VALID: natural := 1;  -- RX FIFO not empty.
constant ST_ERROR    : natural := 2;  -- One or more error flags set.
constant ST_TX_IDLE  : natural := 3;  -- TX FIFO empty and line idle.

-- CTRL bits.
constant CT_ENABLE   : natural := 0;  -- Enable the peripheral.
constant CT_PARITY_LO: natural := 1;  -- Parity select, low bit.
constant CT_PARITY_HI: natural := 2;  -- Parity select, high bit (00 none, 01↵
↵ even, 10 odd).
constant CT_STOP      : natural := 3;  -- 0 = 1 stop bit, 1 = 2 stop bits.
constant CT_RX_IRQ    : natural := 4;  -- RX-valid IRQ mask.
constant CT_TX_IRQ    : natural := 5;  -- TX-ready IRQ mask.

-- ERROR_FLAGS bits.
constant ER_FRAMING: natural := 0;  -- Framing error.
constant ER_PARITY : natural := 1;  -- Parity error.
constant ER_OVERRUN: natural := 2;  -- Overrun.

-- Testbench helper: render a std_logic_vector as an uppercase hex string,
-- zero-padded to whole nibbles (e.g. x"A5" -> "A5", a 12-bit bus -> three ↵
↵ digits).
function to_hex(slv: std_logic_vector) return string;
end package;

library ieee;
use ieee.std_logic_1164.all;

package body uart_def is
    function to_hex(slv: std_logic_vector) return string is
        constant DIGITS : string(1 to 16) := "0123456789ABCDEF";
        constant NIBBLES: natural := (slv'length + 3) / 4;
        variable v      : std_logic_vector(NIBBLES * 4 - 1 downto 0) := (others => ↵
↵ '0');
        variable d      : natural;
        variable res: string(1 to NIBBLES);
    begin
        v(slv'length - 1 downto 0) := slv;

```

```

    for i in 0 to NIBBLES - 1 loop
        -- Fold the four bits of nibble 'i' into its value, most significant ←
        -- first.
        d := 0;
        for b in 3 downto 0 loop
            d := d * 2;
            if (v(i * 4 + b) = '1') then
                d := d + 1;
            end if;
        end loop;
        res(NIBBLES - i) := DIGITS(d + 1);
    end loop;
    return res;
end function;
end package body;

```

1.1.3 Where it fits

uart_def sits underneath everything that speaks in registers. uart_regs (Chapter 5) is the module that actually reads and writes these bits; the package only names them. uart_top never uses it at all, because the top wires the register bus without interpreting anything carried on it. Six testbenches analyze against the package, most of them only for to_hex.

On the far side of the SPI wire, the driver you build from Chapter 6 on reaches the same registers through the same positions. That is the point of transcribing the map twice rather than inventing it twice: the protocol is the single source, and each language is checked against it rather than against the other.

1.1.4 What's ahead

§1.2 is the peripheral top, uart_top, the structural skeleton built against this package. The exercises in §1.4 read uart_def, then build uart_top from the ground up. The datapath and register modules the top composes arrive in Chapters 2 to 5, and are instantiated into uart_top as each one lands.

1.2 Designing uart_top.vhd

uart_top is the peripheral: the datapath (baud_gen, uart_tx, uart_rx), the register bank (uart_regs), and the provided SPI transport (spi_slave, spi_reg_bridge) that lets an off-chip processor reach the registers. It is built **first**, top-down, before any of those blocks exist. That is the point of starting here: the top defines every block's interface, so when baud_gen arrives in Chapter 2 or uart_regs in Chapter 5, the ports each one must expose are already fixed by how uart_top instantiates it.

What this chapter produces is the top's *skeleton*: the entity, the provided `reset_sync`, the provided transport, and the handful of signals those three need. It is wiring, almost entirely: the pieces of real logic this file eventually holds, the TX feeder and one type conversion, are written later, in the chapters that bring the signals they act on. The datapath and register blocks do not exist yet; each is instantiated here as it is created across Chapters 2 to 5, and the system testbench turns green once the last of them lands.

The signals follow the same rule. Rather than declaring names for modules that will not exist for weeks, each chapter reads the entity of the block it is about to instantiate and declares exactly the signals that block's ports imply. Two conventions keep it readable: a **`spi_`** prefix for signals carrying SPI traffic between the two transport blocks, and an **`_s2`** suffix for anything that has been through a two-flop synchronizer, which is why every submodule expects `reset_s2_n`.

This section is the reasoning behind the file. The exercises in §1.4 are where it is actually typed, port by port and signal by signal.

1.2.1 Interface

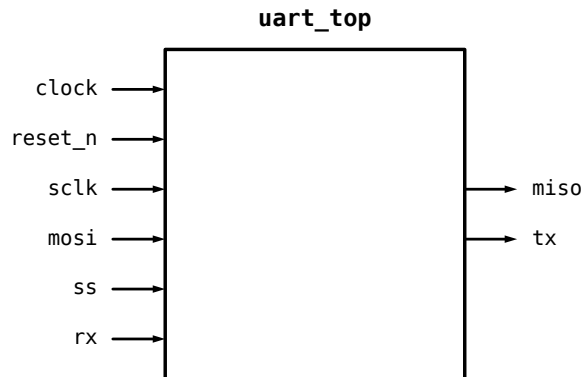


Figure 1.2 Module `uart_top`

Port	Dir	Type	Meaning
clock	in	std_logic	50 MHz system clock.
reset_n	in	std_logic	Asynchronous reset, active low (a button or a supervisor).
sclk, mosi, ss	in	std_logic	SPI from the processor.
rx	in	std_logic	The UART serial input.
miso	out	std_logic	SPI to the processor.
tx	out	std_logic	The UART serial output.

The ports are declared all inputs first, then all outputs; `uart_top_tb` binds them positionally, so the order above is the contract. Nothing in this course associates a port by name, which means a transposed pair of same-type pins is invisible to the analyzer, and stays invisible until the system testbench runs at the end of Chapter 5.

`uart_top` is the only block that takes `reset_n`, the raw asynchronous reset, from outside. The one block inside it that also takes `reset_n` is `reset_sync`, because manufacturing a clean reset is its whole job; every other block takes `reset_s2_n`, the synchronized version it produces.

1.2.2 Skeleton behaviour (wiring, and one placeholder)

The transport comes first, because it is what makes a register bus exist at all. `spi_slave` and `spi_reg_bridge` are given to you, testbench-verified; their wire protocol is Part 3 of the protocol (§A.3), and their port lists are shown in Exercise 1.2. You instantiate them; you do not write them. `spi_slave` is the byte engine: it shifts bytes in and out on `sclk` and reports each completed one. `spi_reg_bridge` assembles those bytes into the five-byte register transactions the protocol defines, drives `reg_addr`, `reg_wdata` and `reg_write`, and reads back whatever `reg_rdata` presents to it. The bank that will answer on that bus arrives in Chapter 5; until then the bus exists with nothing behind it.

The **reset synchronizer** is provided, as `reset_sync`, and you instantiate it. `reset_n` comes from off-chip, so it can be released at any instant relative to `clock`. A release landing near a clock edge could let different flip-flops leave reset on different cycles, and a design that starts half-reset can sit in a state its own logic never expects. `reset_sync` asserts asynchronously, so the moment `reset_n` goes low everything resets with no clock needed, and releases synchronously, two flip-flops later, so the release is lined up with the clock. Its two internal flops stay inside it, so the only signal it adds to your top is `reset_s2_n`. The whole module is short enough to print:

```
hw/reset_sync.vhd
-----
-- Double-flop synchronizes an asynchronous reset.
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_n: Active-low asynchronous reset.
--
-- Outputs:
--   - reset_s2_n: Active-low synchronized reset.
-----
library ieee;
use ieee.std_logic_1164.all;
```

```

entity reset_sync is
    port(clock, reset_n: in std_logic;
          reset_s2_n    : out std_logic);
end entity;

architecture behaviour of reset_sync is
    signal reset_s1_n: std_logic;
begin
    process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            reset_s1_n <= '0';
            reset_s2_n <= '0';
        elsif (rising_edge(clock)) then
            reset_s1_n <= '1';
            reset_s2_n <= reset_s1_n;
        end if;
    end process;
end architecture;

```

That output then feeds every submodule, and the blocks you write in Chapters 2 to 5 all take it the same way the provided ones do: `reset_s2_n` goes in the process's sensitivity list, so the assertion still needs no clock edge, while the *release* is already lined up because `reset_sync` lined it up. That is the house style throughout hw/, and it is what makes an asynchronous assertion safe in the first place: the dangerous edge, the release, was dealt with once, here.

Note that this job is not `sync`'s (Chapter 3), even though that module is also two flops in series. `sync` takes `reset_s2_n` as an *input*, so it consumes the very signal `reset_sync` produces, and a block that needs a clean reset cannot be the block that manufactures one.

The **TX feeder** is written in Chapter 5, once both signals it needs exist. `uart_tx` (Chapter 2) sends a byte when its start is pulsed, and the TX FIFO inside `uart_regs` (Chapter 5) holds the bytes waiting to go. The feeder is what connects them: `tx_load` is FIFO-not-empty AND transmitter-not-busy, and `tx_pop` is that same signal, so the transmitter latches the front byte and the FIFO advances in the same cycle. The instant the transmitter goes busy, `tx_load` drops, so exactly one byte moves per idle transmitter, never two. The RX side needs no feeder at all: the receiver's `valid` and `data_out` become `uart_regs`' `rx_push` and `rx_byte`, and the bank pushes its own FIFO.

The other piece is a **type conversion**, written in Chapter 2 alongside `baud_gen`. `baud_gen` takes its divider as a natural range 1 to 65535, but `BAUD_DIV` leaves the register bank as a `std_logic_vector(15 downto 0)`, so `uart_top` converts it. That conversion is guarded, because until `uart_regs` exists in Chapter 5 nothing drives the vector, and `to_integer` on an all-'U' value warns and substitutes a zero of its own choosing, which is a value that port's

range does not accept. Picking a legal value yourself costs one conditional expression, and this is the only place in `uart_top` where a `std_logic_vector` becomes an integer.

The one thing this chapter does write is a **placeholder**: `reg_rdata` tied to zeros, so the bridge reads a defined value rather than 'U' while the register bank is missing. It is deleted in Chapter 5, when `uart_regs` starts driving that vector for real.

1.2.3 What the testbench pins down

`uart_top_tb` is the system testbench, and it does not run until the last block lands in Chapter 5. It drives the peripheral entirely over SPI, as the processor would, and loops tx back into rx. A single byte written over SPI is therefore transmitted, received through the loopback, pushed into the RX FIFO, and read back over SPI: an end-to-end check that the reset synchronizer, the transport, the register bank, the FIFOs, the feeder and both datapath halves all agree. It also writes `BAUD_DIV` and reads it back, and checks `STATUS`, so the register plumbing is exercised alongside the datapath.

Its one line of instantiation is also why the entity looks the way it does. The testbench binds port map(`clock`, `reset_n`, `sclk`, `mosi`, `ss`, `rx`, `miso`, `tx`), and that order, not the drawing in Figure 1.2, is what the port table records.

1.2.4 Where it fits

`uart_top` faces two directions. Outward, its eight pins are the FPGA boundary: `sclk`, `mosi`, `ss` and `miso` reach the ATmega328P on the other side of the wire, and the driver built in Chapters 6 to 9 talks to exactly these ports; `rx` and `tx` are the serial line itself. In Chapter 9 a board wrapper sits above it and maps those pins onto DE0-CV package pins, which is the only thing that wrapper does.

Inward, it is the one file revisited in every VHDL chapter, and each visit adds a block, the signals that block's entity implies, and any glue those signals now make possible. This chapter leaves it with the entity, `reset_sync`, the transport and a register bus nothing answers. Chapter 2 adds `baud_gen` and `uart_tx`, and with them the `BAUD_DIV` conversion, so the top can transmit. Chapter 4 adds `sync` on the `rx` pin, producing `rx_s2`, and `uart_rx` behind it, so the receiver never sees the raw asynchronous input. Chapter 5 adds `uart_regs`, which owns both FIFOs, and the TX feeder that connects it to the transmitter; with every block present, the system testbench elaborates for the first time. The rule that keeps it compiling throughout is that you only ever instantiate an entity after you have created it.

1.2.5 What's ahead

The exercises in §1.4 are, for `uart_top`, also the build sheet: everything needed to type the file, from the entity down to the last internal signal, is there. In Chapter 2 you build the first two datapath blocks, `baud_gen` and `uart_tx`, and instantiate them here.

1.3 Review

What you should be able to do now

After this chapter, you should be able to:

- **Read the provided register-map package** (`uart_def`) and explain why its natural bit-**position** constants must match the protocol, and why the C++ `register_map.hpp` agrees with it value for value wherever a name exists on both sides.
- **Build the `uart_top` skeleton**, meaning the entity, the provided `reset_sync`, the provided transport, and the signals those three need, so that it analyzes cleanly with no datapath blocks present.
- **Derive the internal signals from the entities they connect**, rather than from a list, and apply the `spi_` prefix and `_s2` suffix conventions the provided modules already use.
- **Explain top-down structural design**: why fixing the top first fixes each block's port contract, and why the system testbench is gated, skipped rather than failed, until Chapter 5.

Questions to test yourself

You should be able to explain:

1. Why build `uart_top` before any block it instantiates, and what does fixing the top first fix about every datapath module you write later?
2. The reset is asserted asynchronously but released synchronously: what can go wrong if the release is not synchronized, and why can `sync` (Chapter 3) not do this job?
3. `uart_top_tb` binds `uart_top`'s ports positionally: what class of wiring mistake does that leave the analyzer unable to catch, and at which point in the book does it finally surface?

Looking ahead

The datapath begins in Chapter 2: `baud_gen`, the shared time base, and `uart_tx`, the transmitter, each verified against its own testbench and instantiated into the `uart_top` you built here.

1.4 Exercises

How to work these. Exercise 1.1 reinforces §1.1; Exercises 1.2 and 1.3 go with §1.2. `uart_def` is **provided** and only needs reading. `uart_top` is yours, and Exercise 1.2 is self-contained: everything needed to type `hw/uart_top.vhd`, from the entity down to the last internal signal, is below. §1.2 is where the reasoning behind it lives, and Exercise 1.3 asks for that reasoning back. **How to check your work.** Neither module has a testbench of its own in this chapter. `uart_def` is a package of constants, and `uart_top`'s system testbench, `uart_top_tb`, stays skipped until

its datapath exists in Chapter 5. So the check here is that `uart_top` **analyzes cleanly** against the transport it instantiates. Run GHDL from `hw/`; the preface's *Setting up* has the full flow and what `--assert-level=error` does.

The answers to every part that is not code are in §B.1. Try each part before you read its answer.

Exercise 1.1 `uart_def`

REASONING

a) `uart_def.vhd` is provided. Read it against §1.1, then, without looking, name the seven register indices and the STATUS, CTRL and ERROR_FLAGS bit positions. Analyze it, so that the names are in your work library for everything that follows:

```
cd hw
ghdl -a --std=93 uart_def.vhd
```

There is nothing to elaborate or run; a clean analysis means the package compiles and its names are available to every module that later writes `use work. uart_def.all`.

b) These are bit **positions**, not masks: `ST_RX_VALID` is 1, not 2. Show how you would test the RX-valid bit of a status vector `status` using the constant, and contrast it with how the C++ driver forms the same test (`1U << status::RX_VALID`). Why does storing positions keep both sides identical to the protocol's "bit N" wording?

c) A teammate swaps two index values, setting `REG_RX_POP` to 6 and `REG_ERR_FLAGS` to 5. Nothing in this chapter notices. At what point in the course would the swap first surface as a failure, and in which testbench, if any? Say why it does not surface before then.

Exercise 1.2 `uart_top`

VHDL

Build the skeleton in the order the file is written: the entity, the declarations, then the three instantiations and the one placeholder assignment. Each part below carries what that step needs. Do not instantiate `baud_gen`, `uart_tx`, `sync`, `uart_rx` or `uart_regs` yet, and do not declare their signals: those blocks arrive in Chapters 2 to 5, and each brings its own wiring with it.

a) The entity. Declare the eight ports in exactly this order. `uart_top_tb` binds them by position, so the numbers are the contract; the names are yours.

#	Port	Dir	Type
1	clock	in	std_logic
2	reset_n	in	std_logic
3	sclk	in	std_logic
4	mosi	in	std_logic
5	ss	in	std_logic
6	rx	in	std_logic
7	miso	out	std_logic
8	tx	out	std_logic

The file needs `ieee.std_logic_1164` for `std_logic`. It does not need `ieee.numeric_std` yet, since nothing here converts a vector to an integer, and it does **not** need use `work.uart_def.all`: the top wires the register bus but never names a bit on it, and `uart_regs` (Chapter 5) is the only module that does. Follow the provided files' house style: a banner comment listing the inputs and outputs, then architecture behaviour of `uart_top` is.

b) The internal signals. Declare only the signals **this** chapter wires. The method matters more than the list: open the entity of every module you are about to instantiate, and for each port that does not connect straight to a `uart_top` port, declare a signal of the same type to carry it. Here that means `reset_sync`, `spi_slave` and `spi_reg_bridge`, and nothing else. Each later chapter adds its own signals the same way, when it adds the block that needs them.

Two naming conventions run through the whole design, and both are worth adopting now. Signals that carry SPI traffic between the two transport blocks take a **spi_ prefix**. Signals that have been through a two-flop synchronizer take an **_s2 suffix**, which is why the provided modules all expect `reset_s2_n` rather than `reset_n`.

Signal	Type	Driven by	Read by
reset_s2_n	std_logic	reset_sync, in c)	every submodule
spi_rx_data	std_logic_vector(7 downto 0)	spi_slave	spi_reg_bridge
spi_rx_valid	std_logic	spi_slave	spi_reg_bridge
spi_ss_active	std_logic	spi_slave	spi_reg_bridge
spi_tx_data	std_logic_vector(7 downto 0)	spi_reg_bridge	spi_slave
reg_addr	std_logic_vector(3 downto 0)	spi_reg_bridge	uart_regs (Chapter 5)
reg_wdata	std_logic_vector(31 downto 0)	spi_reg_bridge	uart_regs (Chapter 5)
reg_write	std_logic	spi_reg_bridge	uart_regs (Chapter 5)
reg_rdata	std_logic_vector(31 downto 0)	zeros, in d); then uart_regs (Chapter 5)	spi_reg_bridge

The four register-bus signals are here even though the block that answers on them is four chapters away, because `spi_reg_bridge` has ports for them and an instantiation must connect every port. That is the register bus existing with nothing behind it, which is the shape this chapter is really about.

`reg_addr` is four bits, not three, even though there are only seven registers: the protocol's command byte reserves bits 3 to 0 for the index.

c) **The reset synchronizer.** `reset_n` arrives from off-chip, so it must assert asynchronously and release synchronously, two flip-flops later. That job is done by the provided `reset_sync`, which you instantiate rather than write; its two internal flops are its own business, so the only signal it adds to your top is `reset_s2_n`. Everything inside `uart_top` takes `reset_s2_n`, and nothing but this instance sees `reset_n` directly.

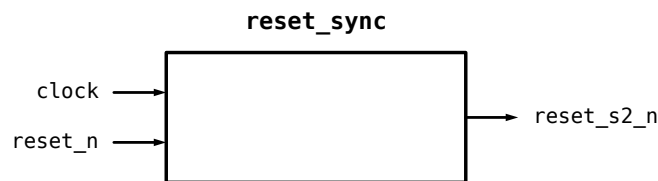


Figure 1.3 Module `reset_sync`

#	reset_sync port	Dir	Connect to
1	clock	in	clock
2	reset_n	in	reset_n
3	reset_s2_n	out	reset_s2_n

```

reset_sync: entity work.reset_sync
  port map(clock, reset_n, reset_s2_n);

```

d) **The transport.** Instantiate `spi_slave` and `spi_reg_bridge`, both positionally. The last column names a signal from part b) or a port from part a).

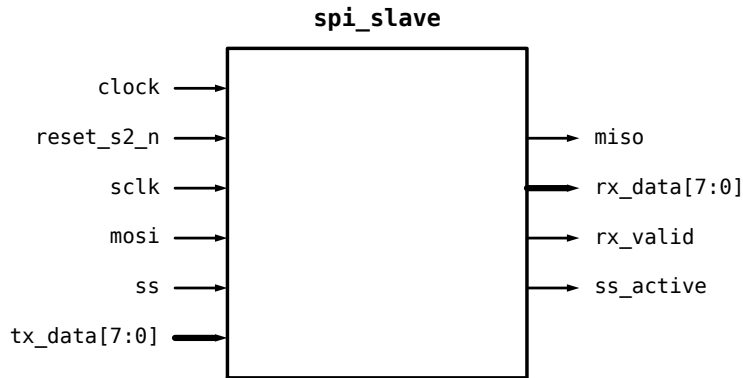


Figure 1.4 Module `spi_slave`

#	spi_slave port	Dir	Type	Connect to
1	clock	in	std_logic	clock
2	reset_s2_n	in	std_logic	reset_s2_n
3	sclk	in	std_logic	sclk
4	mosi	in	std_logic	mosi
5	ss	in	std_logic	ss
6	tx_data	in	std_logic_vector(7 downto 0)	spi_tx_data
7	miso	out	std_logic	miso
8	rx_data	out	std_logic_vector(7 downto 0)	spi_rx_data
9	rx_valid	out	std_logic	spi_rx_valid
10	ss_active	out	std_logic	spi_ss_active

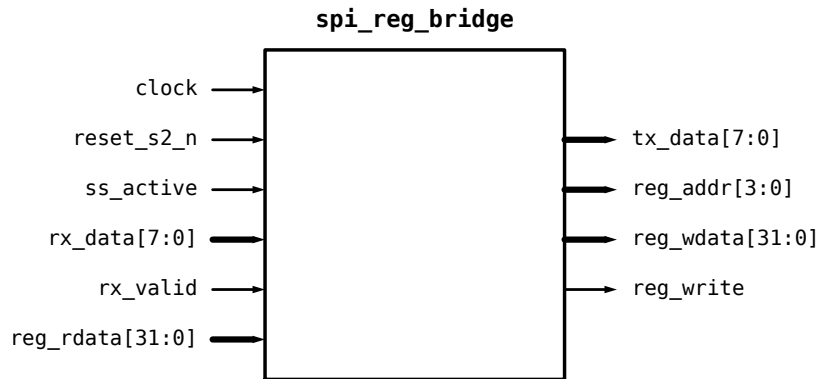


Figure 1.5 Module spi_reg_bridge

#	spi_reg_bridge port	Dir	Type	Connect to
1	clock	in	std_logic	clock
2	reset_s2_n	in	std_logic	reset_s2_n
3	ss_active	in	std_logic	spi_ss_active
4	rx_data	in	std_logic_vector(7 downto 0)	spi_rx_data
5	rx_valid	in	std_logic	spi_rx_valid
6	reg_rdata	in	std_logic_vector(31 downto 0)	reg_rdata
7	tx_data	out	std_logic_vector(7 downto 0)	spi_tx_data
8	reg_addr	out	std_logic_vector(3 downto 0)	reg_addr
9	reg_wdata	out	std_logic_vector(31 downto 0)	reg_wdata
10	reg_write	out	std_logic	reg_write

The names that look shared are not one bus. `spi_slave`'s `rx_data` is an output and the bridge's `rx_data` is an input, so `spi_rx_data` runs from the slave to the bridge; `spi_tx_data` runs the other way, from the bridge's `tx_data` output into the slave's `tx_data` input. Read them as two one-directional lanes and the wiring falls out.

Nothing drives `reg_rdata` yet, so give it a value:

```
reg_rdata <= (others => '0');
```

The file would analyze without this; an undriven signal is legal. It is here so that the bridge reads a defined value instead of 'U' while the bank is missing. **Delete this line in Chapter 5**, when `uart_regs` starts driving `reg_rdata`. Leave it in and the vector has two drivers: every '1' from the bank resolves against the '0' here to 'X', and the system testbench fails on a read-back caused by a line you wrote four chapters earlier.

e) **The check.** Analyze the skeleton against the package and the transport it instantiates:

```
cd hw
ghdl -a --std=93 uart_def.vhd reset_sync.vhd spi_slave.vhd spi_reg_bridge.vhd ↵
↳ uart_top.vhd
```

Then run `make build-vhdl` from the repository root. Confirm that `uart_def` and `uart_top` both analyze cleanly, that the two transport testbenches pass (they are the only ones that can run this early, since they check provided modules), and that `uart_top_tb` reports **skipped**. Which modules does the skip message name as missing, and which chapter (2, 3, 4 or 5) builds each? One of the six is never instantiated by `uart_top` at all; find it, and say why the build needs it anyway.

Exercise 1.3 What a clean analysis does not prove

REASONING

`uart_top` analyzes, and almost nothing about it is proven. That gap is the lesson of this chapter.

- a) Swap two same-type ports in your entity, say `sclk` and `mosi`, and re-analyze. Does `ghdl -a` catch it? If not, at which point in the course does the mistake finally show up, and what does that tell you about the cost of a positional contract?
- b) `reg_rdata` is tied to zeros and no register bank answers on the bus. What does an SPI read transaction return with that placeholder in place, and why is that harmless until Chapter 5? What would it return instead if you left the placeholder out altogether?
- c) The provided `reset_sync` asserts asynchronously but releases synchronously. Suppose it released asynchronously too, wiring `reset_n` straight to every block. Describe the failure that a near-edge release could cause. Then explain why `sync` (Chapter 3) cannot do this job, given that it is also two flip-flops in series: look at its port list and say what it would need that only `reset_sync` can give it.
- d) Why does building `uart_top` before any block it instantiates fix each block's port list in advance? What has to be true about `baud_gen`'s ports in Chapter 2 for its instantiation here to bind without you touching the top again?

UART Framing and the Transmitter

The wire, and the easy direction first.

IN THIS CHAPTER

- **UART framing itself:** the idle-high line, the start bit, eight data bits sent least significant first, optional parity, and one or two stop bits, and how a baud rate maps to a bit period. The line protocol (§A.1) fixes the default 8N1 frame and the BAUD_DIV divider taken from the 50 MHz system clock.
- **The baud-rate generator,** which emits one enable tick at $16 \times \text{baud}$, so that the transmitter written here and the receiver written in Chapter 3 share a single time base rather than each keeping its own.
- **baud_gen.vhd and then uart_tx.vhd,** each verified against its provided testbench.

This chapter works from the wire up. After it you should be able to draw a tx waveform for a given byte, baud rate and frame format, and mark the bit boundaries on it; explain why the transmitter needs a baud tick at $16 \times \text{baud}$ rather than at the baud rate itself, given that the receiver of Chapter 3 oversamples; and implement a transmitter state machine that loads, shifts and frames, with correct busy and done timing, verified against a provided testbench.

Before you start, you need Chapter 1, where `uart_def` and the `uart_top` skeleton were built; `baud_gen` and `uart_tx` are the first two blocks you instantiate into that top. From digital design with VHDL the chapter assumes entity and architecture, clocked processes, state machines, and running a provided self-checking testbench with GHDL. Read the UART line protocol (§A.1) first.

Read §2.1 for `baud_gen` and §2.2 for `uart_tx`; these are the specifications you build the two modules from. Then read the provided testbenches, `hw/baud_gen_tb.vhd` and `hw/uart_tx_tb.vhd`, which are the executable form of this chapter's contract and whose case comments each state what a failure means. Confirm your GHDL install beforehand by running an existing testbench from your digital design course.

Each module is run against its testbench as it lands, `baud_gen` first:

```
cd hw
ghdl -a --std=93 baud_gen.vhd baud_gen_tb.vhd
ghdl -e --std=93 baud_gen_tb
ghdl -r --std=93 baud_gen_tb --assert-level=error
```

Then both are instantiated into the `uart_top` skeleton from Chapter 1, so the top now drives tx. The system testbench still waits for the receiver and the register bank.

In the exercises in §2.4 you build `baud_gen` and `uart_tx` from §2.1 and §2.2 (Exercises 2.1 and 2.2), instantiate them into `uart_top`, and then extend the transmitter with parity and a second stop bit (Exercise 2.3).

2.1 Designing `baud_gen.vhd`

A UART has no clock line. The transmitter and the receiver each keep their own time and agree only on a number: the baud rate, the number of bits sent per second. Everything in this peripheral that has to happen “once per bit” is paced from a single enable pulse, and `baud_gen` is what produces it.

It does not produce one pulse per bit, though; it produces sixteen. The receiver you will build in Chapter 3 cannot see where a bit begins, so it samples the line sixteen times per bit and picks the middle sample (§3.2 works through that sampling). For the transmitter and the receiver to share one time base, that time base has to tick at the finer, sixteen-times rate; the transmitter simply counts sixteen ticks per bit. So `baud_gen` divides the 50 MHz system clock down to $16 * \text{baud}$, and every other datapath block treats one tick as “one sixteenth of a bit has passed”.

2.1.1 Interface

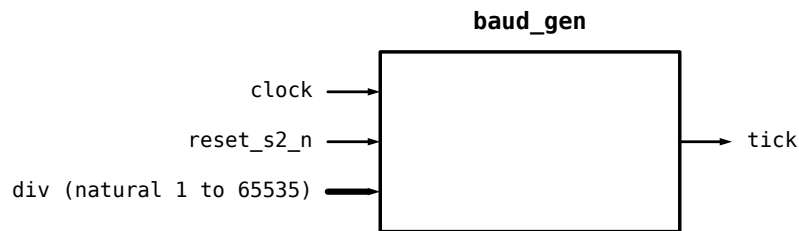


Figure 2.1 Module `baud_gen`

Port	Dir	Type	Meaning
clock	in	std_logic	50 MHz system clock.
reset_s2_n	in	std_logic	Active-low synchronized reset.
div	in	natural range 1 to 65535	Clocks per tick: $\text{div} = \text{clock} / (16 * \text{baud})$.
tick	out	std_logic	One-cycle pulse every div clocks.

`baud_gen_tb` binds these ports positionally, as everything in this course does, so the order and types above must match exactly. `div` is a plain natural, not a vector: the register bank (Chapter 5) holds the value as `BAUD_DIV` and hands it over already converted, so this block never touches a `std_logic_vector` or `numeric_std`.

`div` is expected to be at least 1. The formula gives, for a 50 MHz clock, $\text{div} = 27$ for 115200 baud and 326 for 9600. The testbench uses $\text{div} = 4$, so that a tick is only four clocks apart and the simulation is short.

2.1.2 Behaviour

One counter, one comparison:

- On system reset (`reset_s2_n` is low):
 - tick goes low.
 - The counter is reset to zero.
- On each rising edge of `clock`, while not in reset:
 - If the counter has reached $\text{div} - 1$: tick goes high for this one cycle, and the counter is reset to zero.
 - If the counter has not reached $\text{div} - 1$: tick stays low, and the counter is incremented.

So tick is high for exactly one clock in every `div`, and the mean tick rate is $\text{clock} / \text{div}$. That is $16 * \text{baud}$ only to within the rounding of `div`, which is an integer: at 115200 the exact divider is 27.126, so $\text{div} = 27$ runs about 0.47% fast. That margin is what the receiver's mid-bit sampling in Chapter 3 is there to absorb.

Two details are worth getting right. The first is that **the comparison is `counter >= div - 1`, not `counter = div - 1`**. With a stable `div` the two behave identically, since the counter reloads the instant it reaches the threshold and never passes it. The `>=` form matters only at the edges: if `div` were ever 0, the equality test would never fire and the counter would run free until it overflowed its range, but `>=` catches it and simply ticks every cycle. It also means a `div` changed mid-count is corrected on the next edge rather than after a wrap. Robustness for one character of typing.

The second is that **the reset is asserted asynchronously**. `reset_s2_n` sits in the process's sensitivity list, so the moment it goes low the counter clears and tick drops, with no clock edge needed. That is the house style for every block here, provided and student-written alike. It is safe precisely because `reset_s2_n` has *already* been synchronized: `uart_top` runs

the raw `reset_n` through the provided `reset_sync` once (§1.2), so the release is lined up with the clock before any block sees it.

`tick` is a registered output, so it is glitch-free and one clock wide, which is exactly what a downstream “count sixteen of these” expects.

2.1.3 What the testbench pins down

`baud_gen_tb` drives `div = 4`, releases `reset`, then finds the first `tick` and measures the gap to each of the next three. Every gap must be exactly four clocks, and a tick that arrives early or late fails with the measured count in the message.

That is the whole contract: a tick every `div` clocks, steadily. The testbench does not check the $16 * \text{baud}$ arithmetic, because that lives in the *value* of `div`, which the register bank supplies; `baud_gen` only has to divide by whatever number it is given.

2.1.4 Where it fits

`baud_gen` sits at the root of the datapath. Its single `tick` feeds the transmitter you build next in this chapter and the receiver in Chapter 3; both count sixteen ticks to a bit, so both stay locked to the same time base without any handshake between them. In `uart_top` (Chapter 1) its `div` comes from the `BAUD_DIV` register, so software sets the baud rate by writing one number.

2.1.5 What’s ahead

§2.2 builds `uart_tx`, the transmitter. It takes the tick you just produced and turns a byte into a framed serial waveform on the `tx` line.

2.2 Designing `uart_tx.vhd`

The transmitter turns one byte into the waveform a UART line carries: a low **start bit**, then the eight data bits **least significant first**, then a high **stop bit**, with the line sitting **idle high** between frames. That default framing is called 8N1 (8 data bits, No parity, 1 stop bit), and it is the only framing this transmitter produces; parity and a second stop bit are left as an exercise.

The transmitter owns no time of its own. It counts the ticks from `baud_gen`, sixteen to a bit, and changes the line only on bit boundaries. So the whole of its timing is “hold this bit for sixteen ticks, then move to the next”.

2.2.1 Interface

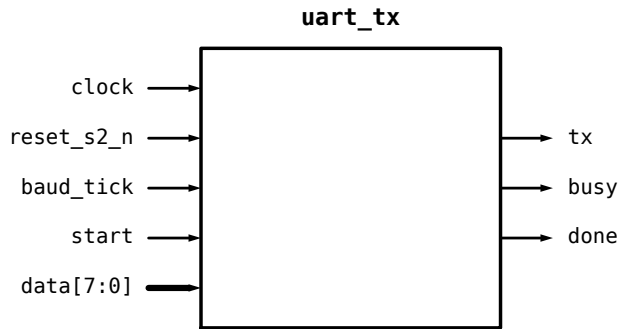


Figure 2.2 Module uart_tx

Port	Dir	Type	Meaning
clock	in	std_logic	50 MHz system clock.
reset_s2_n	in	std_logic	Active-low synchronized reset.
baud_tick	in	std_logic	The 16x oversample tick from baud_gen.
start	in	std_logic	A one-cycle pulse: latch data and begin sending.
data	in	std_logic_vector(7 downto 0)	The byte to send.
tx	out	std_logic	The serial line, idle high.
busy	out	std_logic	High for the whole frame.
done	out	std_logic	A one-cycle pulse as the frame completes.

uart_tx_tb binds these ports positionally, as everything in this course does, so the order and types above must match exactly. start is a load command, not a level: it is looked at only while the transmitter is idle, so holding it high does not re-send. In uart_top (Chapter 1) the TX feeder pulses start and pops the FIFO in the same cycle, so data is the FIFO's front byte.

2.2.2 The frame

Build the whole frame once, as a 10-bit vector, then just index it. This is the trick that keeps the rest of the module short:

```
frame <= STOP_BIT & data & START_BIT;  -- '1' & data(7 downto 0) & '0'
```

Concatenation puts the leftmost item in the highest bits, so that vector reads:

frame	9	8	7	6	5	4	3	2	1	0
carries	stop	d7	d6	d5	d4	d3	d2	d1	d0	start
value	'1'									'0'

where d7 to d0 stand for data(7) to data(0). Read it from index **0 upward** and you get exactly the wire order: the start bit, then data(0) through data(7) least significant first, then the stop bit. So the transmitter never has to think about bit order again. It drives `tx <= frame(bit_idx)` and walks `bit_idx` from 0 to 9.

2.2.3 Behaviour

Two states, `STATE_IDLE` and `STATE_SEND`, and two counters between them: `ticks` counts 0 to 15 within one bit, and `bit_idx` counts 0 to 9 across the frame.

- On reset (`reset_s2_n` is low):
 - `tx` goes high, the idle level.
 - `done` goes low and the state returns to `STATE_IDLE`.
 - `frame`, `bit_idx` and `ticks` are cleared.
- On each rising edge of `clock`, while not in reset:
 - `done` goes low, so any pulse from the previous cycle lasts exactly one clock.
 - In **STATE_IDLE**: `tx` is held high. If `start = '1'`, build frame from data, as above; drive `tx` low, beginning the start bit; and move to `STATE_SEND`.
 - In **STATE_SEND**: drive `tx <= frame(bit_idx)`, the bit currently being sent. If `baud_tick = '1'`:
 - If `ticks` has reached 15, this bit is finished: clear `ticks`. Then, if `bit_idx` has reached 9, the stop bit is finished: raise `done` for this one cycle, clear `bit_idx` and return to `STATE_IDLE`. If `bit_idx` has not reached 9, increment it, moving to the next bit.
 - If `ticks` has not reached 15: increment `ticks`, holding the current bit.

`busy` is not part of that process at all. It is one concurrent line, high whenever the state machine is sending:

```
busy <= '1' when state = STATE_SEND else '0';
```

Three details are worth getting right.

The first is that **ticks and bit_idx advance only on a baud_tick**. Every other clock edge in `STATE_SEND` does nothing but re-drive `tx` with the same bit. At 115200 baud `BAUD_DIV` is 27, so that is 26 idle edges for every one that counts, which is the point: the transmitter is paced by `baud_gen`, not by the system clock.

The second is that **done is cleared at the top of every clock edge and set only in the single cycle the frame ends**. Writing the default first and letting the specific case override

it is what guarantees a one-cycle pulse with no extra counter. The two status outputs then carry different information. `busy` is a *level* covering the whole frame, and is what the feeder in `uart_top` watches so it does not load another byte mid-frame. `done` is an *event*, useful for counting frames or raising an interrupt, and it is not what moves the next byte along.

The third is that **the frame is captured on start**, so the caller may change data on the very next cycle. The byte in flight lives in `frame`, not in the caller's register.

2.2.4 What the testbench pins down

`uart_tx_tb` generates `baud_tick` locally at one pulse per four clocks, so a bit is 64 clocks and the frame simulates quickly. It checks that:

- `tx idles high` before anything is sent.
- `tx goes low within two bit periods` of start. This catches a transmitter that never leaves the idle state; without it the testbench would wait for a falling edge that never comes, and hang.
- The **start bit is low** at its centre.
- The **eight data bits match 0x53, least significant first**, sampled a full bit apart. This is the check that catches a transmitter sending most significant first.
- The **stop bit is high**.

Sampling at bit centres rather than edges is the same discipline the receiver uses in Chapter 3, and it is why a one-tick timing slip does not fail the test while a wrong bit order does.

The **choice of 0x53 is not arbitrary**, and it is worth knowing why before you write a testbench of your own. 0x53 is 0101_0011, and reversed it is 1100_1010, or 0xCA: a different byte, so a frame packed most significant first fails on the very first data bit. A byte that reads the same backwards would not test the ordering at all: 0xA5 is 1010_0101 either way round, so both packings put exactly the same ten levels on the line and the testbench passes on both. Sixteen of the 256 bytes are such **bit-reversal palindromes** — 0xA5, 0x3C and 0x5A among them — and choosing one of them turns a real check into a decoration. Exercise 2.2 makes you demonstrate it.

Note what the testbench does **not** check. `busy` and `done` are bound but never asserted on, so a `busy` that rises one cycle late still passes here. That bug has no effect until Chapter 5, where the TX feeder, still seeing `busy` low on the cycle after a load, pops a second byte that the transmitter, already sending, ignores. The byte is lost, and because `uart_top_tb` sends a single byte, not even the system testbench shows it.

2.2.5 Where it fits

`uart_tx` is the output half of the datapath. In `uart_top` (Chapter 1) its data comes from the TX FIFO and its start and busy drive the feeder that empties that FIFO one byte at a time;

its tx pin leaves the FPGA as the peripheral's serial output. It shares baud_gen's single tick with the receiver built in Chapter 3, which is what keeps the two halves on one time base.

2.2.6 What's ahead

The exercises in §2.4 build baud_gen and uart_tx from these two sections, run their testbenches, and probe the design decisions the tests depend on.

2.3 Review

What you should be able to do now

After this chapter, you should be able to:

- **Draw a tx waveform** for a given byte, baud rate and frame format, and mark the bit boundaries on it.
- **Explain why the transmitter needs a baud tick at $16 \times \text{baud}$** rather than at the baud rate itself, given that the receiver of Chapter 3 oversamples.
- **Implement a transmitter state machine** that loads, shifts and frames, with correct busy and done timing, verified against a provided testbench.

Questions to test yourself

You should be able to explain:

1. Why is the line idle-high, and what does a receiver detect to know a frame has started?
2. A byte is sent at 115200 baud with the 50 MHz clock: how many system cycles long is one bit, and what BAUD_DIV produces it?
3. If busy rises one cycle late, what does the TX feeder of Chapter 5 do on that cycle, and what happens to the byte it acts on?

Looking ahead

Chapter 3 takes the hard direction: recovering bytes from an asynchronous line, with a two-flop synchronizer on rx, 16x oversampling, mid-bit sampling and framing-error detection.

2.4 Exercises

How to work these. Exercise 2.1 reinforces §2.1; Exercises 2.2 and 2.3 reinforce §2.2. Design each module from the specification in its own section.

How to check your work. The provided testbenches are the check that the modules behave correctly. Run them from `hw/` with the three GHDL steps; the preface's *Setting up* has the full flow and what `--assert-level=error` does.

The answers to every part that is not code are in §B.2. Try each part before you read its answer.

Exercise 2.1 baud_gen

VHDL

- a) Write your own `baud_gen.vhd` from the specification in §2.1 and run its testbench:

```
cd hw
ghdl -a --std=93 baud_gen.vhd baud_gen_tb.vhd
ghdl -e --std=93 baud_gen_tb
ghdl -r --std=93 baud_gen_tb --assert-level=error
```

- b) The testbench uses `div = 4`. Work out on paper what `div` produces 115200 baud from the 50 MHz clock, and confirm it matches §2.1. Why is the divide by `16 * baud` and not by `baud`? Which later module spends those sixteen ticks, and on what?
- c) Change the comparison in your design from `counter >= div - 1` to `counter = div - 1` and re-run. The testbench still passes; explain why the two are indistinguishable for `div = 4`. Then say what each version does if `div` is ever 0, and which of the two keeps the counter inside its declared range.
- d) `tick` is a registered output, one clock wide. Suppose you drove it combinatorially instead, asserting it whenever `counter = div - 1`. Nothing in this testbench would notice; name a downstream consumer (in Chapter 2 or Chapter 3) that counts ticks, and argue whether a glitchy or wider `tick` would break it.

Exercise 2.2 uart_tx

VHDL

- a) Write your own `uart_tx.vhd` from the specification in §2.2 and run its testbench:

```
cd hw
ghdl -a --std=93 uart_tx.vhd uart_tx_tb.vhd
ghdl -e --std=93 uart_tx_tb
ghdl -r --std=93 uart_tx_tb --assert-level=error
```

- b) The testbench sends `0x53` and checks the data bits **least significant first**. Change your frame packing so it sends most significant first and re-run. Which bit does the testbench report first as wrong, and does that match `0x53 = 0101_0011` shifted the other way? Put it back.

c) Now the reason that constant is what it is. Leave your frame packed most significant first, change TXBYTE in `uart_tx_tb` to `x"A5"`, and re-run: the testbench **passes**, with the transmitter still wrong. Write out the eight data-bit levels under both packings and explain why. What property must a byte have for this check to be able to fail at all, and how many of the 256 bytes lack it?

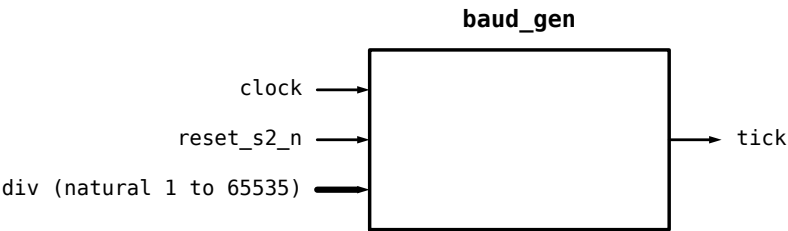
Then look at `uart_rx_tb` and `uart_top_tb` with the same question in mind, and say why a loopback testbench could never catch `uart_tx` and `uart_rx` being reversed **together**, however good its constant was, while each unit testbench catches its own half. Put both files back.

d) `busy` and `done` carry different information. Give one job each does that the other cannot: what would go wrong in `uart_top`'s feeder if it watched `done` instead of `busy` to decide when to load the next byte?

e) The frame is captured on start into a 10-bit vector. Show, by writing out the bits, that reading '1' & `data` & '0' from index 0 upward gives start, then `data(0)` to `data(7)`, then stop. Then explain why the caller is free to change `data` on the very next cycle.

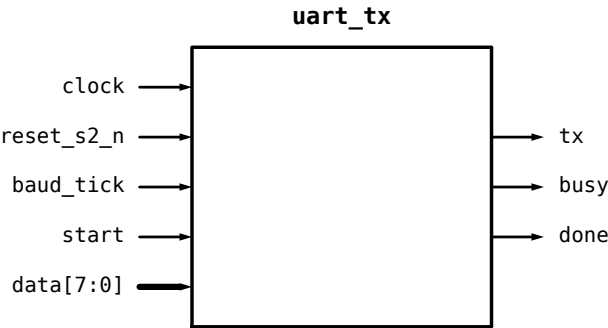
f) Instantiate `baud_gen` and `uart_tx` into the `uart_top` skeleton from Chapter 1, positionally, and declare the signals they need as you go. Read each entity and let it tell you what to add: `baud_gen` takes a natural divider and drives `baud_tick`; `uart_tx` takes `baud_tick`, a byte and a start pulse, and drives `tx`, `busy` and `done`. So `uart_top` gains these signals, and no more than that:

Signal	Type	Driven by	Read by
<code>baud_div</code>	<code>std_logic_vector(15 downto 0)</code>	<code>uart_regs</code> (Chapter 5)	the conversion below
<code>baud_div_int</code>	natural range 1 to 65535	the conversion below	<code>baud_gen</code>
<code>baud_tick</code>	<code>std_logic</code>	<code>baud_gen</code>	<code>uart_tx</code> , and <code>uart_rx</code> (Chapter 3)
<code>tx_byte</code>	<code>std_logic_vector(7 downto 0)</code>	<code>uart_regs</code> (Chapter 5)	<code>uart_tx</code>
<code>tx_load</code>	<code>std_logic</code>	the feeder (Chapter 5)	<code>uart_tx</code> , and <code>tx_pop</code> (Chapter 5)
<code>tx_busy</code>	<code>std_logic</code>	<code>uart_tx</code>	the feeder and <code>uart_regs</code> (Chapter 5)
<code>tx_done</code>	<code>std_logic</code>	<code>uart_tx</code>	nothing, in this build



#	baud_gen port	Dir	Type	Connect to
1	clock	in	std_logic	clock
2	reset_s2_n	in	std_logic	reset_s2_n
3	div	in	natural range 1 to 65535	baud_div_int
4	tick	out	std_logic	baud_tick

```
-- Generate the 16x oversample tick from the divider the register bank will hold.
baud_generator: entity work.baud_gen
    port map(clock, reset_s2_n, baud_div_int, baud_tick);
```



#	uart_tx port	Dir	Type	Connect to
1	clock	in	std_logic	clock
2	reset_s2_n	in	std_logic	reset_s2_n
3	baud_tick	in	std_logic	baud_tick
4	start	in	std_logic	tx_load
5	data	in	std_logic_vector(7 downto 0)	tx_byte
6	tx	out	std_logic	tx, the top-level port
7	busy	out	std_logic	tx_busy
8	done	out	std_logic	tx_done

```
-- Drive the serial line. Port 6 is the transmitter's own tx output, so it binds
-- to the top-level tx pin, not to a signal.
uart_transmitter: entity work.uart_tx
  port map(clock, reset_s2_n, baud_tick, tx_load, tx_byte, tx, tx_busy,
           tx_done);
```

Position 6 is the one to check twice. Every port here is a `std_logic` except data, so binding a signal there instead of the tx pin analyzes cleanly, elaborates cleanly, and leaves the pin undriven: the loopback in `uart_top_tb` then feeds 'U' back into rx, and the system test fails three chapters later with nothing pointing at this line.

Three signals need a word. `tx_byte` has no driver yet, because the FIFO that will fill it arrives with `uart_regs` in Chapter 5, and neither does `tx_load`, whose feeder you write in the same chapter: an undriven signal analyzes and elaborates without complaint, and the transmitter simply sits idle. `baud_div` is the third, and it needs the **guarded conversion** that turns the register bank's vector into the natural that `baud_gen` expects:

```
-- baud_gen's div starts at 1, so substitute a legal value rather than hand it a
-- zero or a metavalue: baud_div has no driver until uart_regs arrives in L05.
baud_div_int <= 1 when Is_X(baud_div) or unsigned(baud_div) = 0
               else to_integer(unsigned(baud_div));
```

`Is_X` comes from `std_logic_1164`, and `to_integer` and `unsigned` from `numeric_std`, so add that second use clause now. The guard covers two cases, and both of them would otherwise land on 0, which `baud_gen` does not accept: its div port is a natural range 1 to 65535, so a 0 reaching it is a bound check failure at run time, not a warning you can ignore.

The first case is a **metavalue**. `baud_div` has no driver until Chapter 5 either, so it sits at all-'U', and `to_integer` on an all-'U' vector prints `NUMERIC_STD.TO_INTEGER: metavalue detected`, returning 0 and carries on. The library quietly picks a value you did not choose, and the value it picks is out of range.

The second is a **real zero**, which arrives with `uart_regs` in Chapter 5: `BAUD_DIV` holds whatever the bank resets it to until software writes a divider, and that reset value is a legitimate 0. The substitute is 1 because it is the smallest value the port accepts; nothing useful is transmitted at either value, so the only thing that matters is that the design elaborates and runs.

VHDL's or on booleans is short-circuit, so the comparison is never evaluated on the all-'U' vector, and the metavalue case stays warning-free.

```
cd hw
ghdl -a --std=93 uart_def.vhd reset_sync.vhd baud_gen.vhd uart_tx.vhd \
      spi_slave.vhd spi_reg_bridge.vhd uart_top.vhd
```

The top now drives tx, but the system testbench stays skipped: the receiver and the register bank are still missing. Do the extension in Exercise 2.3 on a copy, or after this step, since adding ports to `uart_tx` changes the positional contract `uart_top` binds against.

Exercise 2.3 Parity and a second stop bit

VHDL

8N1 is the only framing `uart_tx` produces. Real UARTs let software pick even or odd parity and one or two stop bits; in this peripheral those live in the CTRL register (Chapter 5), but the transmitter is where the extra bits would be sent.

- a) Extend your `uart_tx` with two inputs, `parity_en` and `parity_odd`, and insert a parity bit between the last data bit and the stop bit. Even parity makes the total number of 1s (data plus parity) even; odd parity makes it odd. Widen your frame vector and index range to match. Give each new input a default of '0' and declare it after the eight existing ports: `uart_tx_tb` and `uart_top` bind positionally, and a trailing input with a default may be left out of a positional map, so this is the one place the inputs-first convention gives way.
- b) Add a `two_stop` input, declared the same way, that appends a second high stop bit. Which field of the frame changes length with `two_stop`, and what does that do to the total frame time at a fixed baud rate?
- c) `uart_tx_tb` checks 8N1 only, so it will still pass with your additions as long as the new inputs default to off and come after the existing ports. Confirm that. Then describe, in words, the extra testbench case you would add to pin down even parity: what byte would you send, and what would you expect on the parity bit?

The Synchronizer and the Receiver

The hard direction: recovering bytes from an asynchronous line.

IN THIS CHAPTER

- **The synchronizer:** before any receive logic can run, the asynchronous rx input has to pass through a two-flop synchronizer, `sync`, which `uart_top` will instantiate on the pin so that the crossing happens in the module that owns it. This is the metastability lesson from digital design with VHDL, reused here on a genuinely asynchronous input.
- **Oversampling:** the receiver watches rx at $16 \times \text{baud}$ rather than clocking on it, detecting the start-bit edge and then sampling each bit at its midpoint, tick 8 of 16.
- **Leaving idle on an edge, not a level:** one flip-flop of history for `rx_s2`, and why it is the difference between a receiver that works on clean data and one that survives a cable being unplugged.
- **The error family:** what framing, parity and overrun each mean, and when each can be detected. The receiver you build here surfaces a framing error as a one-cycle pulse for the register bank to latch in Chapter 5; parity is left as an extension, and overrun is not something the receiver can judge at all, so it belongs to the closing discussion of Chapter 4 and to Chapter 5.
- **`sync.vhd` and `uart_rx.vhd`,** each verified against its provided testbench.

Receiving is the hard direction, and this chapter is about why. The buffer the register bank needs, and the two instantiations that put this chapter's modules into `uart_top`, belong to Chapter 4; this chapter is the receiver and nothing else. After it you should be able to explain why an asynchronous input sampled directly can violate setup and hold, and place a two-flop synchronizer correctly; explain why sampling at mid-bit, tick 8 of 16, maximizes the margin against a baud mismatch, and implement start-bit detection along with mid-bit sampling; explain why the receiver leaves idle on a falling edge rather than a low level, and what a break does to a receiver that tests the level instead; and detect a framing error, emitting it as a clean single-cycle pulse alongside `valid`, and explain why parity is a natural

extension of the same sampling loop while overrun is not the receiver's question to answer at all.

Before you start, you need Chapter 1, which produced the `uart_top` skeleton the receive path is instantiated into in Chapter 4, and Chapter 2, which produced the baud generator and the frame format; the receiver reuses the same `16 x baud tick` rather than generating its own. From digital design with VHDL the chapter assumes metastability and the two-flop synchronizer. Read the notes on the three error flags in the register semantics (§A.2.3), since this chapter produces the pulses that Chapter 5 latches.

Read §3.1 for `sync` and §3.2 for `uart_rx`; these are the specifications you build the two modules from. Then read the provided testbenches, `hw/sync_tb.vhd` and `hw/uart_rx_tb.vhd`. The receiver's testbench has three cases, and each case comment names what it expects: a clean frame that must produce exactly one `valid` byte and no error; a frame with a low stop bit that must raise `frame_err` and produce no byte at all; and a break, the line held low for longer than a frame, that must produce framing errors and no byte at all, on the way in or on recovery.

Each module is run against its testbench as it lands:

```
cd hw
ghdl -a --std=93 uart_def.vhd sync.vhd uart_rx.vhd sync_tb.vhd uart_rx_tb.vhd
ghdl -e --std=93 uart_rx_tb && ghdl -r --std=93 uart_rx_tb --assert-level=error
```

Neither module is instantiated yet. `uart_top` gains both in Chapter 4, together with the FIFO the register bank will own, which is why this chapter can stay on the one hard idea: detecting the start edge and sampling at tick 8 of 16, and the break case, where a level test instead of an edge test delivers a byte assembled out of nothing.

In the exercises in §3.4 you build `sync` and `uart_rx` (Exercises 3.1 and 3.2), run both testbenches, and reason about sampling margin, glitch rejection, and the bit order the shift produces.

3.1 Designing `sync.vhd`

The `rx` line arrives from another chip. Nothing in this FPGA has ever governed it, so it can change at any instant relative to `clock`, including inside a flip-flop's setup-and-hold window. Sample it directly and that flop can go metastable: sit between 0 and 1 for a while before resolving, and resolve either way. Digital design with VHDL solved this with two flip-flops in series, and that is all `sync` is, written once, properly, for this design to keep.

It is also the only module in the course with nothing UART-specific in it, which is exactly why it is reusable. `uart_top` instantiates it on the `rx` pin, producing `rx_s2` for the receiver of §3.2, so the asynchronous crossing is handled by the module that owns the pins. The same two-flop idea also guards the three SPI inputs inside the provided `spi_slave`, though that

block, being provided and self-contained, spells it out inline rather than depending on your module.

3.1.1 Interface

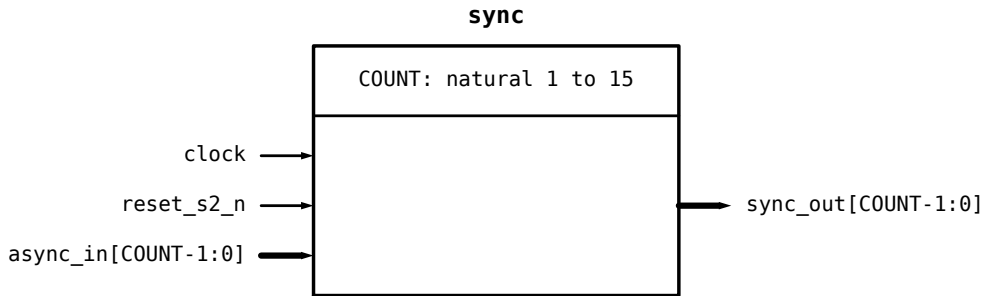


Figure 3.1 Module sync

Port / generic	Dir	Type	Meaning
COUNT	generic	natural range 1 to 15 := 1	Number of independent bits to synchronize.
clock	in	std_logic	50 MHz system clock, the domain being crossed <i>into</i> .
reset_s2_n	in	std_logic	Active-low synchronized reset.
async_in	in	std_logic_vector(COUNT-1 downto 0)	The asynchronous input or inputs.
sync_out	out	std_logic_vector(COUNT-1 downto 0)	The same bits, safe to use in this domain.

sync_tb binds the ports positionally, and the **generic map is positional too**, so COUNT must be the module's only (or first) generic. The testbench instantiates the module twice, at the default count and at count 4, so both the scalar case and the generic are exercised. sync reads no package, so its analyze line is just the module and its testbench, nothing else.

uart_top uses it at COUNT = 1 for rx. The generic earns its place by being exercised and by staying reusable; a wider instance is a one-line change should a later design want to synchronize a bus of switches or pins.

3.1.2 Behaviour

Two flip-flops per bit, in series, on the rising edge of clock. The first may go metastable; the second gives it a full clock period to settle before anything downstream sees it. So sync_out

follows `async_in` after **exactly two** rising edges: not one (one flop is not a synchronizer) and not three.

Two choices are deliberate. The first is the **reset, and which reset it is**. `sync` clears both flops on `reset_s2_n`, asserted asynchronously so the chain starts from a known state with no clock needed, exactly as the rest of the design does. Note carefully *which* reset that is: `reset_s2_n` is the **already-synchronized** one, produced by the provided `reset_sync` in `uart_top`.

That asymmetry is the point, and it is why `sync` cannot be the reset synchronizer, however alike the two look. `reset_sync` takes the raw asynchronous `reset_n` and manufactures a clean one; `sync` **consumes** the clean one it produces. A module that needs `reset_s2_n` as an input cannot be the module that creates it, so the two stay separate even though both are two flops in series.

The second is **the count generic**, under which each bit crosses independently. That is fine for independent signals, and it is the reason a plain synchronizer is *not* a general answer for a multi-bit value whose bits must stay coherent: two bits that change together may land one clock apart. `rx` is a single bit, so the question does not arise here, but it is worth knowing before reaching for a wide instance on a counter.

3.1.3 What the testbench pins down

`sync_tb` checks that `sync_out` follows `async_in` after exactly two rising edges, low-to-high and high-to-low alike; that at `COUNT = 4` all four bits arrive together, two edges after the input changed; and that a steady input leaves `sync_out` steady, so the output tracks the input rather than the intermediate flop.

It also checks the reset from both sides. Held low, the outputs stay clear whatever the input does. Asserted *between* clock edges with a known non-zero output standing, the outputs must clear **immediately** rather than at the next edge, a check that fails against a synchronous reset and is how the testbench pins down that this one is asynchronous. Released, the chain must re-synchronize and take the full two edges again.

Deleting one flip-flop is the exact bug this testbench exists to catch, and it is a bug no functional simulation would otherwise reveal, because nothing in a simulator is ever metastable.

3.1.4 Where it fits

In `uart_top`, the `rx` pin goes through `sync` before any logic looks at it, and the receiver of §3.2 reads only the synchronized copy, `rx_s2`. Because `sync`'s ports are vectors even at `COUNT = 1`, a scalar `rx` needs a one-element signal on each side; that wiring is part of the exercise in Chapter 4 that adds the receive path to the top.

3.1.5 What's ahead

§3.2 builds `uart_rx`, the receiver, which takes the synchronized `rx` and recovers a byte from it by oversampling.

3.2 Designing `uart_rx.vhd`

Receiving is harder than transmitting for one reason: the transmitter chooses when each bit begins, and the receiver has to *find out*. There is no clock on the wire, only the line itself, idle high, falling low to start a frame. The receiver has to notice that falling edge, then read eight data bits and a stop bit whose timing it can only infer from the baud rate it was told.

The technique is **oversampling**. The receiver runs at the same $16 * \text{baud tick}$ as the transmitter, from `baud_gen`, so there are sixteen ticks per bit. It does not try to sample on the bit edge, where the line is changing; it aims for the **middle** of each bit, tick 8 of 16, where the level is settled and a small timing error either way is harmless. Sampling at the centre is the single idea that makes an asynchronous line readable.

3.2.1 Interface

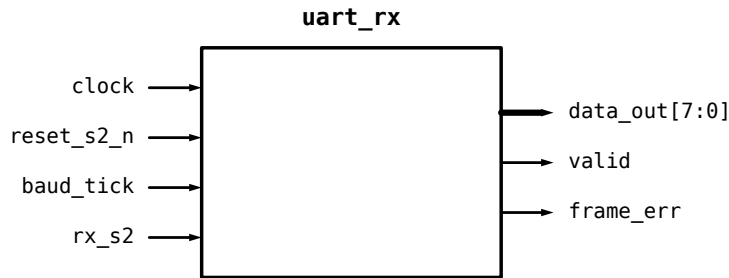


Figure 3.2 Module `uart_rx`

Port	Dir	Type	Meaning
<code>clock</code>	in	<code>std_logic</code>	50 MHz system clock.
<code>reset_s2_n</code>	in	<code>std_logic</code>	Active-low synchronized reset.
<code>baud_tick</code>	in	<code>std_logic</code>	The 16x oversample tick from <code>baud_gen</code> .
<code>rx_s2</code>	in	<code>std_logic</code>	The serial input, already synchronized.
<code>data_out</code>	out	<code>std_logic_vector(7 downto 0)</code>	The received byte.
<code>valid</code>	out	<code>std_logic</code>	A one-cycle pulse: <code>data_out</code> is fresh.
<code>frame_err</code>	out	<code>std_logic</code>	A one-cycle pulse: the stop bit was not high.

`uart_rx_tb` binds these positionally. Note what this port is **not**: it is not the raw pin. The two-flop synchronizer that cleans up `rx` lives one level up, in `uart_top`, which instantiates `sync` (§3.1) on the pin and passes the result down as `rx_s2`. By the time the receiver sees the line it has already been resynchronized to the system clock, so every process in here treats it as an ordinary synchronous input and samples it directly.

Putting the synchronizer in the top rather than inside the receiver keeps the responsibility where the asynchronous boundary actually is. `uart_top` owns the pins, so it owns the crossing; `uart_rx` is then a purely synchronous design, which is easier both to reason about and to test.

3.2.2 Behaviour

Four states, `STATE_IDLE`, `STATE_START`, `STATE_DATA` and `STATE_STOP`, and two counters between them: `ticks` counts 0 to 15 within one bit, and `bit_idx` counts 0 to 7 across the data bits. Apart from the output defaults, the whole process sits inside `if baud_tick = '1'`, so the receiver advances in oversample periods rather than clock cycles.

- On reset (`reset_s2_n` is low):
 - `data_out` is cleared, and `valid` and `frame_err` go low.
 - The state returns to `STATE_IDLE`.
 - `frame`, `bit_idx`, `ticks` and `sample_data` are cleared.
 - `rx_prev` is set to `'1'`, the idle level, so the first tick after reset cannot look like a falling edge.
- On each rising edge of clock, while not in reset:
 - `data_out` is cleared and `valid` and `frame_err` go low, so any pulse from the previous cycle lasts exactly one clock.
 - If `baud_tick = '1'`, the oversample counter runs first, ahead of the state machine. If `ticks` has reached 15, a whole bit period has passed: raise `sample_data` and clear `ticks`. If `ticks` has not reached 15: lower `sample_data` and increment `ticks`.
 - Then, on that same tick, in **STATE_IDLE**: if `rx_s2` has just **fallen**, low on this tick and high on the one before it, a start bit may be beginning, so clear `ticks`, restarting the count from this edge, and move to `STATE_START`.
 - In **STATE_START**: if `sample_start_bit = '1'`, that is, `ticks` has reached 8, half a bit on from the edge, then if `rx_s2` is still low the start is genuine, so clear `ticks` and move to `STATE_DATA`. If `rx_s2` has gone high, the low was a glitch rather than a start, so return to `STATE_IDLE`. That mid-bit re-check is what rejects noise on an idle line.
 - In **STATE_DATA**: if `sample_data = '1'`, a full bit has passed since the last sample, so store the line into the frame, `frame(bit_idx) <= rx_s2`. If `bit_idx` has reached 7, the eighth and last data bit is in: clear `ticks` and `bit_idx`, and move to `STATE_STOP`. Otherwise increment `bit_idx`, moving to the next bit.

- In **STATE_STOP**: if `sample_data = '1'`, a full bit has passed since the last data bit. If `rx_s2` is high, the stop bit is where it should be and the frame is good: present frame on `data_out` and raise `valid` for this one cycle. If `rx_s2` is low, the framing is broken: raise `frame_err` for this one cycle instead, clear frame, and drop the byte. Either way, return to **STATE_IDLE**.
- Last on the tick, after the state machine has read it, `rx_prev` takes the current `rx_s2`. Updating it on the tick is what makes the comparison above “this tick against the previous one”: a signal assignment takes effect only once the process has run, so wherever it is written inside the tick the state machine sees the value from the previous tick, whereas updating it on every clock would compare samples one clock apart and miss the edge.

`sample_start_bit` is not part of that process at all. It is one concurrent line, high whenever the tick counter sits half a bit past the edge that restarted it:

```
sample_start_bit <= '1' when ticks = HALF_TICKS else '0';
```

Two details in the process are easy to miss. The counter block and the state machine both assign `ticks`, and the counter block comes first, so wherever a state clears `ticks` that clear is what takes effect: VHDL gives the last assignment in a process the final say. And `sample_data` is a registered flag rather than a comparison, so it is raised on the tick where `ticks` wraps and acted on at the next one. Because the state machine runs only on a `baud_tick`, it still fires once per bit rather than once for every clock in the tick.

Between them those two details also decide *where* in each bit the sample lands, and it is not exactly tick 8. Three things each cost a tick: the falling edge is only noticed at the next `baud_tick`; `ticks` is compared before it is incremented; and every `ticks <= 0` on a state change spends one more. So the start-bit re-check arrives nine ticks after the edge rather than eight, the first data sample seventeen ticks after that re-check rather than sixteen, and the stop bit seventeen after the last data bit, while data bit to data bit is exactly sixteen. Against the frame `uart_rx_tb` sends, that puts the start-bit check two ticks past the centre of its bit, each data bit three, and the stop bit four.

The spacing between data bits is right, so that offset is fixed rather than accumulating across the byte, and a sample three ticks late still has five ticks of settled line ahead of it rather than eight. The receiver works, and the testbench passes, but it begins with roughly half the trailing margin the ideal tick-8 sample would have. That is worth holding on to for the margin question in Exercise 3.2: the budget a real receiver spends on baud mismatch is whatever the sampling phase has not already spent.

Walking `bit_idx` from 0 upward and storing with `frame(bit_idx) <= rx_s2` puts the first-received bit in bit 0, which is **least significant first**, the order the transmitter sent them in, so `data_out` reads out as the byte that was sent.

`valid` and `frame_err` are one-cycle pulses, so `uart_top` can turn a `valid` straight into a FIFO push and latch a `frame_err` into the `ERROR_FLAGS` register (Chapter 5). Both default low every cycle and rise for exactly one clock at the stop-bit sample.

Parity and overrun are deliberately absent here. Parity would be another mid-bit sample between the last data bit and the stop bit, gated by `CTRL`; overrun, a new byte arriving before the last was read, is not something the receiver can judge, because it does not know whether anyone has read the byte. That belongs to the register bank, where the RX FIFO's full flag answers it (Chapter 5).

3.2.3 Why idle waits for an edge, not a level

Keeping one tick of history for `rx_s2` costs a single flip-flop, and leaving idle only on the high-to-low transition is what that flip-flop buys. Testing the level instead, “if `rx_s2` is low, start a frame”, passes every frame `uart_rx_tb` sends and fails on the one case that matters.

Consider a **break**: the line held low far longer than a frame, which is what a transmitter losing power or a cable coming loose looks like. With a level test the receiver re-arms the moment the previous frame's stop bit is judged, so it spends the break marching through back-to-back all-zero frames. Each of those ends on a low stop bit and is correctly rejected as a framing error, and that part is fine. The problem is the frame still in flight when the break *ends*: its stop bit is sampled after the line has returned high, so it is a well-formed frame as far as the receiver can tell, and a byte assembled out of nothing is pushed into the FIFO as real data.

With an edge test there is no second falling edge until the line has gone high again, so no frame is ever in flight across that boundary. One flip-flop, and it is the difference between a receiver that works on clean data and one that survives a cable being unplugged.

3.2.4 What the testbench pins down

`uart_rx_tb` drives framed bytes onto `rx_s2` at sixteen ticks per bit and checks three cases. A clean byte must arrive on `data_out` with `valid` and no `frame_err`, and with the right value, which is what pins down the least-significant-first shift. A byte whose stop bit is driven low must raise `frame_err` and produce no `valid` byte. And a **break**, the line held low for longer than a frame and then released, must produce framing errors and no byte at all, which is the case discussed above and the only one that requires leaving idle on an edge rather than a level.

The clean byte is `0x53`, and the choice matters for the same reason it does in Chapter 2 (§2.2.4): `0x53` reversed is `0xCA`, so a receiver that walked `bit_idx` the wrong way would deliver a visibly different byte and fail. A **bit-reversal palindrome** such as `0xA5` would be delivered unchanged by a receiver storing its bits backwards, and the check would pass on a broken design.

It counts the `valid` and `frame_err` pulses with a small monitor process rather than racing the one-cycle strobes, which is worth copying whenever a testbench has to catch a pulse.

3.2.5 Where it fits

uart_rx is the input half of the datapath. In uart_top (Chapter 1) its valid and data_out become uart_regs' rx_push and rx_byte, so the bank pushes the received byte into the RX FIFO it owns, and its frame_err feeds the same bank's error latch. Its rx pin is the peripheral's serial input, looped back from tx in the system testbench so that a sent byte comes straight back.

3.2.6 What's ahead

The exercises in §3.4 build sync and uart_rx, run their testbenches, and reason about over-sampling margin and glitch rejection. Chapter 4 then adds the FIFO, puts both modules into uart_top, and works out where overrun really lives.

3.3 Review

What you should be able to do now

After this chapter, you should be able to:

- **Explain why an asynchronous input sampled directly can violate setup and hold**, and place a two-flop synchronizer correctly.
- **Explain why sampling at mid-bit, tick 8 of 16, maximizes margin against a baud mismatch**, and implement start-bit detection along with mid-bit sampling.
- **Explain why the receiver leaves idle on a falling edge rather than a low level**, and what a break does to a receiver that tests the level instead.
- **Detect a framing error**, emitting it as a clean single-cycle pulse alongside valid, and explain why parity is a natural extension of the same sampling loop while overrun is not the receiver's question to answer at all.

Questions to test yourself

You should be able to explain:

1. Why must rx pass through sync before any logic looks at it, and what can go wrong without it?
2. The receiver samples at tick 8 of 16: as the transmitter's baud rate drifts from the receiver's, what breaks first, the early bits or the late ones, and why?
3. Idle is left on a falling edge rather than on a low level. Which of the three testbench cases fails if you test the level instead, and what does the receiver deliver that it should not?

Looking ahead

Chapter 4 builds the buffer the register bank will need two of: a small synchronous FIFO with a look-then-advance read. Then the receive path goes into `uart_top`, `sync` on the pin and `uart_rx` behind it, and we work out why overrun is not a question the receiver can answer.

3.4 Exercises

How to work these. Exercise 3.1 reinforces §3.1 and Exercise 3.2 §3.2. Design each module from the specification in its own section. The FIFO, the two instantiations that put the receive path into `uart_top`, and the overrun question belong to Chapter 4 (§4.3), not to this chapter.

How to check your work. Check each module with its provided testbench, run from `hw/`; the preface's *Setting up* has the full flow.

The answers to every part that is not code are in §B.3. Try each part before you read its answer.

Exercise 3.1 `sync`

VHDL

a) Write your own `sync.vhd` from the specification in §3.1 and run its testbench:

```
cd hw
ghdl -a --std=93 sync.vhd sync_tb.vhd
ghdl -e --std=93 sync_tb
ghdl -r --std=93 sync_tb --assert-level=error
```

Note that no package appears in that first line: `sync` reads none, which is part of what makes it reusable.

b) Delete one of the two flip-flops, leaving a single register, and re-run. Which check fails, and what does its message tell you? Put it back. Why would this bug pass every *other* simulation in the course if this one testbench did not exist?

c) `sync` clears on `reset_s2_n`, and the testbench asserts it *between* two clock edges and requires the output to clear before the next one. Change your reset to a synchronous one, tested inside `if rising_edge(clock)`, and re-run. Which check fails, and what is the message? Put it back, then say why the assertion has to be asynchronous while the *release* does not need to be, given where `reset_s2_n` comes from.

d) `sync` takes `reset_s2_n`, the already-synchronized reset, as an input. Explain why that one fact makes it impossible for `sync` to be the module that produces `reset_s2_n`, and why `reset_sync` therefore exists as a separate provided block even though both are two flops in series.

e) `uart_top` uses `sync` at `COUNT = 1`. Two bits of a `COUNT = 8` instance change on the same edge; explain why `sync_out` may show one bit's new value and the other's old one for a clock, why that is not a bug in `sync`, and why it would matter if those eight bits were a counter read across clock domains rather than eight unrelated pins.

Exercise 3.2 `uart_rx`

VHDL

a) Write your own `uart_rx.vhd` from the specification in §3.2 and run its testbench:

```
cd hw
ghdl -a --std=93 uart_def.vhd uart_rx.vhd uart_rx_tb.vhd
ghdl -e --std=93 uart_rx_tb
ghdl -r --std=93 uart_rx_tb --assert-level=error
```

`uart_def.vhd` comes first because `uart_rx_tb` reads the package, for the `to_hex` in its failure messages. `sync.vhd` is not on that line at all: `uart_rx` reads `rx_s2`, the already-synchronized line, and instantiates nothing. The crossing belongs one level up, in `uart_top`, which is where Exercise 4.2 puts it.

b) The receiver samples each bit at tick 8 of 16. Suppose the transmitter's baud rate is a few percent faster than the receiver's, so the receiver's ticks are slightly long. Which end of the byte drifts furthest from the bit centre, the first data bit or the stop bit, and why? Roughly how much faster could the transmitter run before the stop-bit sample landed outside its bit?

c) The `STATE_START` state re-checks `rx_s2` at tick 8 before committing to a frame. Remove that check (go straight from the falling edge to `STATE_DATA`) and describe what a single-tick glitch on an idle line would now produce. Which real-world condition makes this more than a theoretical worry?

d) Trace the store `frame(bit_idx) <= rx_s2` for the byte `0x53` (`0101_0011`) sent least significant first. Write the eight received bits in the order they arrive, name the index each one lands in, and show that after eight bits `frame` holds `0x53`. Then: what would `data_out` read if `bit_idx` counted **down** from 7 instead of up from 0, and what is the relationship between that byte and the one that was sent?

The FIFO and the Receive Path

The buffer the register bank needs two of, and the wiring that puts Chapter 3's work into the top.

IN THIS CHAPTER

- **A small synchronous FIFO** (`fifo.vhd`): a ring buffer with a head, a tail and a count, written once as a generic module because the register bank in Chapter 5 needs two of them, one per direction.
- **The look-then-advance read**: `rdata` shows the front entry continuously and `rd` is what *discards* it. That split is not a stylistic choice — it is exactly what lets Chapter 5 keep `RX_DATA` a pure read with a separate `RX_POP`, and it is the first place in the course where a register's semantics reach down into a datapath block's ports.
- **The corner cases that a count gets wrong**: a write to a full FIFO and a read from an empty one, both dropped rather than allowed to cross the pointers, and a `wr` and `rd` on the same edge, where a count adjusted in two independent branches quietly loses an entry.
- **The receive path in `uart_top`**: `sync` on the `rx` pin producing `rx_s2`, then the `uart_rx` of Chapter 3 behind it, so the pin crosses into the clock domain once, in the module that owns the pin.
- **Where overrun lives**: the receiver reports a framing error but can never report an overrun, because it does not know whether anyone has read the previous byte. The RX FIFO's `full` flag is what makes it detectable, and the register bank is where it would be reported: Chapter 5 reserves `ER_OVERRUN` for it, but gives it no producer in this build.

One module and one wiring job, both of which Chapter 5 depends on. After this chapter you should be able to build a synchronous FIFO with a look-then-advance read, and explain why that split is what lets Chapter 5 keep `RX_DATA` a pure read; say what `full` and `empty` guard, and what a write to a full FIFO or a read from an empty one must do instead of wrapping the pointers past each other; instantiate the receive path into `uart_top` in the right order, with the synchronizer on the pin and the receiver behind it, and explain why binding `rx` straight to `uart_rx` would analyze cleanly and still be wrong; and explain what an overrun is, why the receiver cannot detect one on its own, and which block has the one piece of information that makes it detectable.

Before you start, you need the `sync` and `uart_rx` of Chapter 3, which Exercise 4.2 instantiates, and the `uart_top` skeleton from Chapter 1 that they go into, with `baud_gen` and `uart_tx` already in it from Chapter 2. Nothing new is assumed from the foundations: the FIFO is an ordinary clocked design with an array, two pointers and a count.

Read §4.1 for `fifo`; that is the specification you build the module from. Then read the provided testbench, `hw/fifo_tb.vhd`, which uses a depth-4 instance: it checks empty after reset and full after four pushes, drops a fifth push rather than overwriting, drains the four bytes in the order they went in, and finally raises `wr` and `rd` on the same edge with two entries queued — one in and one out, so the depth must not move. The FIFO is run against it as it lands:

```
cd hw
ghdl -a --std=93 uart_def.vhd fifo.vhd fifo_tb.vhd
ghdl -e --std=93 fifo_tb && ghdl -r --std=93 fifo_tb --assert-level=error
```

The two places the FIFO goes wrong are the guards that keep the pointers from crossing, and the count on a simultaneous push and pop. Then the chapter returns to `uart_top` and adds the receive path, which is two instantiations rather than one: `sync` on the `rx` pin, then `uart_rx` reading `rx_s2` rather than `rx`. They are short, and they are the point of the chapter as much as the module is: the pin crosses once, in the module that owns it. The top then analyzes with only `uart_regs` still missing, and the system testbench stays skipped until Chapter 5 supplies it. The FIFO is not instantiated in the top at all; it belongs to the register bank that owns it.

In the exercises in §4.3 you build `fifo` and run its testbench (Exercise 4.1), add `sync` and `uart_rx` to `uart_top` (Exercise 4.2), then reason about why overrun detection belongs to the register bank rather than the receiver (Exercise 4.3).

4.1 Designing `fifo.vhd`

The datapath produces and consumes bytes at different rates:

- Software writes a byte to send whenever it likes.
- The transmitter sends one every ten bit-times.
- The receiver delivers a byte every frame.
- Software reads it whenever it gets round to it.

A **FIFO** (a first-in, first-out queue) absorbs that mismatch: bytes go in one end and come out the other in the same order, and each side can run without waiting for the other, up to the depth of the queue.

The register bank (§5.1) needs two of these, one for each direction, so `fifo` is written once as a generic module and instantiated twice.

4.1.1 Interface

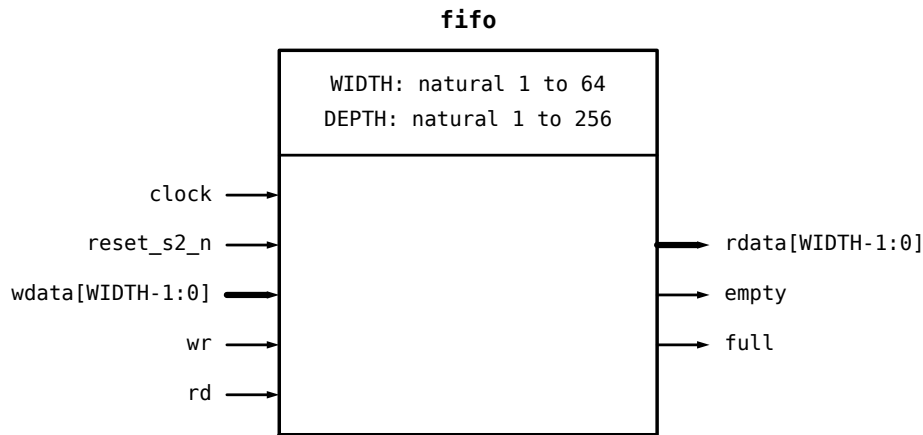


Figure 4.1 Module fifo

Port / generic	Dir	Type	Meaning
WIDTH	generic	natural range 1 to 64 := 8	Bits per entry.
DEPTH	generic	natural range 1 to 256 := 8	Number of entries.
clock	in	std_logic	50 MHz system clock.
reset_s2_n	in	std_logic	Active-low synchronized reset.
wdata	in	std_logic_vector(WIDTH-1 downto 0)	The entry to push.
wr	in	std_logic	'1' to push wdata this cycle.
rd	in	std_logic	'1' to pop the front entry this cycle.
rdata	out	std_logic_vector(WIDTH-1 downto 0)	The entry at the front (combinational).
empty	out	std_logic	'1' if the FIFO is empty.
full	out	std_logic	'1' if the FIFO is full.

fifo_tb binds the ports positionally and drives the generic map too, instantiating a depth-4 FIFO. rdata shows the front entry continuously, without a rd; rd is what *discards* it and moves to the next. That split, look then advance, is exactly what the RX path in Chapter 5 needs.

4.1.2 Behaviour

A ring buffer: an array of DEPTH entries, a write pointer (head), a read pointer (tail), and a count. Each pointer wraps around from DEPTH-1 back to 0.

On reset (`reset_s2_n` low), `head`, `tail` and `count` all go to 0, so the FIFO comes out of reset empty and not full; as everywhere else in this peripheral the reset is asserted asynchronously and tested ahead of `rising_edge(clock)`. The entries themselves need no clearing, since with `count = 0` nothing in the array is reachable and the first write lands on entry 0 anyway. What does matter is that the two pointers start *equal*, not merely at some known value: `empty` and `full` are read off `count`, so a `head` and `tail` that disagreed out of reset would keep the `count` honest while handing back a stale entry instead of the byte just pushed. `rdata` still shows an entry while the FIFO is empty (entry 0 out of reset, a stale one after a drain), whatever that entry happens to hold, which is one more reason the caller watches `empty` rather than trusting `rdata`.

On each rising edge, while not in reset, a **write** happens only if `wr = '1'` and the FIFO is not full, storing `wdata` at `head` and advancing `head`; a **read** happens only if `rd = '1'` and the FIFO is not empty, advancing `tail`. The `count` follows from the pair: it rises by one on a write without a read, falls by one on a read without a write, and is unchanged when both or neither happen.

`full` is `count = DEPTH`, `empty` is `count = 0`, and `rdata` is combinationally the entry at `tail`. Guarding writes on not `full` and reads on not `empty` means the pointers can never cross; a write to a full FIFO and a read from an empty one are simply dropped, which is why the caller must watch the flags.

4.1.3 What the testbench pins down

`fifo_tb` uses a depth-4 instance. It checks `empty` after reset and `full` after four pushes, then pushes a fifth entry while `full` and confirms it is dropped rather than stored over an existing one, and finally drains the FIFO and checks the four bytes come back in the order they went in, leaving it empty again. A last case raises `wr` and `rd` on the same edge with two entries queued: one entry in and one out, so the `depth` must not move, and the byte pushed alongside the `pop` must still be there after the next one. That is the case where a `count` adjusted in two independent branches quietly loses an entry.

That is the whole contract: order preserved, and the flags honest at both ends.

4.1.4 Where it fits

The register bank (§5.1) instantiates two `fifos`, both `WIDTH = 8`, `DEPTH = 8`: a TX FIFO that a `TX_DATA` register write pushes and the transmitter drains, and an RX FIFO that the receiver pushes and an `RX_DATA` read plus an `RX_POP` write drains. The FIFO's `full` and `empty` flags become the `STATUS` register's TX-ready and RX-valid bits.

4.1.5 What's ahead

The exercises in §4.3 build `fifo` and run its testbench, put the receive path of Chapter 3 into `uart_top`, and work out why overrun is not the receiver's question to answer. Then §5.1

builds `uart_regs`, the register bank, which wraps two of these FIFOs and the datapath in the register map that software drives.

4.2 Review

What you should be able to do now

After this chapter, you should be able to:

- **Build a synchronous FIFO** with a look-then-advance read, and explain why that split is what lets Chapter 5 keep `RX_DATA` a pure read.
- **Say what full and empty guard**, and what a write to a full FIFO or a read from an empty one must do instead of wrapping the pointers past each other.
- **Instantiate the receive path into `uart_top`** in the right order, with the synchronizer on the pin and the receiver behind it, and explain why binding `rx` straight to `uart_rx` would analyze cleanly and still be wrong.
- **Explain what an overrun is**, why the receiver cannot detect one on its own, and which block has the one piece of information that makes it detectable.

Questions to test yourself

You should be able to explain:

1. `rdata` shows the front entry with no `rd` at all. What would break in the `RX_DATA` and `RX_POP` split of Chapter 5 if the FIFO instead popped on every read?
2. A write arrives while the FIFO is full. What must happen to `wdata`, to head and to count, and what tells the caller it happened?
3. What is an overrun, why can the receiver not detect one on its own, and which block would have to do it instead?

Looking ahead

In Chapter 5 ports become registers: poll-able STATUS bits, sticky error flags, a write-triggered TX push, the read-then-pop RX path over two of the FIFOs you just built, and completing `uart_top` behind the provided SPI transport, so that the system testbench finally runs.

4.3 Exercises

How to work these. Exercise 4.1 reinforces §4.1. Exercise 4.2 puts the receiver built in Chapter 3 into `uart_top`, behind the synchronizer that owns the crossing. Exercise 4.3 is the reasoning that links the receiver to the register bank you build next. Design the module from the specification in its section.

How to check your work. Check the module with the provided testbench, run from `hw/`; the preface's *Setting up* has the full flow.

The answers to every part that is not code are in §B.4. Try each part before you read its answer.

Exercise 4.1 `fifo`

VHDL

a) Write your own `fifo.vhd` from the specification in §4.1 and run its testbench:

```
cd hw
ghdl -a --std=93 uart_def.vhd fifo.vhd fifo_tb.vhd
ghdl -e --std=93 fifo_tb
ghdl -r --std=93 fifo_tb --assert-level=error
```

b) The testbench pushes a fifth byte into a full depth-4 FIFO and checks it is dropped. Remove the `not full` guard on writes and re-run. What does the run report, and what has your FIFO done to count and to the head and tail pointers to cause it? Note that the answer depends on how you declared count: constrained as `natural range 0 to DEPTH`, the fifth push drives it out of range and GHDL stops on a bound check before any assertion runs; left as an unconstrained `natural`, the count simply passes `DEPTH` and it is the testbench's own check that fires. Both are failures, but only one of them is the testbench talking. Put the guard back.

c) `rdata` shows the front entry with no `rd`; `rd` only advances. Explain why the RX path you will build in Chapter 5 needs exactly this “look, then advance” split rather than a single “read-and-pop” operation.

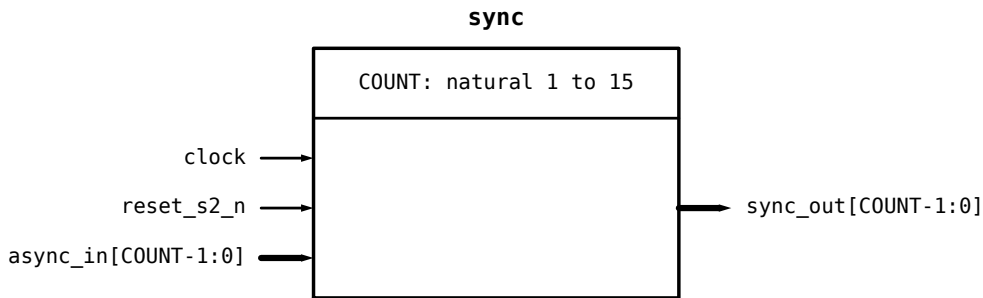
d) Both FIFOs in the finished peripheral are depth 8. Suppose software is slow and the receiver delivers bytes faster than software reads them. At depth 8, how many back-to-back frames can arrive before a byte is lost, and what flag would tell software it happened?

Exercise 4.2 The receive path in uart_top

VHDL

a) Add the receive path to the `uart_top` skeleton, which this time is two instantiations rather than one: `sync` on the `rx` pin, then `uart_rx` behind it, reading `rx_s2` rather than `rx`. The signals this step adds:

Signal	Type	Driven by	Read by
<code>rx_vec</code>	<code>std_logic_vector(0 downto 0)</code>	the <code>rx</code> pin, in the adapter below	<code>sync</code>
<code>rx_vec_s2</code>	<code>std_logic_vector(0 downto 0)</code>	<code>sync</code>	the adapter below
<code>rx_s2</code>	<code>std_logic</code>	the adapter below	<code>uart_rx</code>
<code>rx_byte</code>	<code>std_logic_vector(7 downto 0)</code>	<code>uart_rx</code>	<code>uart_regs</code> (Chapter 5)
<code>rx_push</code>	<code>std_logic</code>	<code>uart_rx</code> 's valid	<code>uart_regs</code> (Chapter 5)
<code>frame_err</code>	<code>std_logic</code>	<code>uart_rx</code>	<code>uart_regs</code> (Chapter 5)

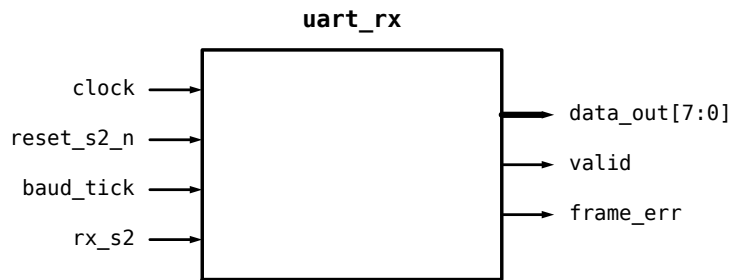


`COUNT` defaults to 1, so this instance needs no generic map — but it does work on vectors, which is why the two one-element vectors above exist.

#	sync port	Dir	Type	Connect to
1	<code>clock</code>	in	<code>std_logic</code>	<code>clock</code>
2	<code>reset_s2_n</code>	in	<code>std_logic</code>	<code>reset_s2_n</code>
3	<code>async_in</code>	in	<code>std_logic_vector(COUNT-1 downto 0)</code>	<code>rx_vec</code>
4	<code>sync_out</code>	out	<code>std_logic_vector(COUNT-1 downto 0)</code>	<code>rx_vec_s2</code>

```
-- Cross the rx pin into the clock domain, once, in the module that owns the pin.
sync: entity work.sync
    port map(clock, reset_s2_n, rx_vec, rx_vec_s2);

-- sync works on vectors, so the single rx bit goes in and comes back out
-- through one.
rx_vec(0) <= rx;
rx_s2    <= rx_vec_s2(0);
```



#	uart_rx port	Dir	Type	Connect to
1	clock	in	std_logic	clock
2	reset_s2_n	in	std_logic	reset_s2_n
3	baud_tick	in	std_logic	baud_tick, the same tick uart_tx uses
4	rx_s2	in	std_logic	rx_s2, not the rx pin
5	data_out	out	std_logic_vector(7 downto 0)	rx_byte
6	valid	out	std_logic	rx_push
7	frame_err	out	std_logic	frame_err

```
-- Recover bytes from the synchronized line. Its valid output is the RX FIFO's
-- push strobe, so it binds straight to rx_push: that side needs no feeder,
-- unlike the transmit side in L05.
uart_receiver: entity work. uart_rx
    port map(clock, reset_s2_n, baud_tick, rx_s2, rx_byte, rx_push, frame_err);
```

The order matters and is the point of the exercise: the pin crosses into the clock domain once, in the module that owns the pin, and everything downstream of that is ordinary synchronous logic. Binding rx straight to port 4 would analyze — both are std_logic — and would sample an asynchronous pin directly, which is the metastability bug the synchronizer of Chapter 3 exists to prevent.

Both rx_byte and rx_push are read by nothing until Chapter 5, so nothing yet consumes what the receiver produces. That is expected: an output with no reader analyzes and elaborates fine. The FIFO from Exercise 4.1 is not instantiated here either: it belongs to the register bank that owns it, which arrives in Chapter 5.

```
cd hw
ghdl -a --std=93 uart_def.vhd reset_sync.vhd baud_gen.vhd sync.vhd uart_tx.vhd \
    uart_rx.vhd spi_slave.vhd spi_reg_bridge.vhd uart_top.vhd
```

The top still analyzes and the system testbench is still skipped; only uart_regs is missing now.

Exercise 4.3 **Overflow belongs to the register bank**

REASONING

uart_rx reports a *framing* error but never an *overflow*: a second byte arriving before the first was read.

- a) Explain why uart_rx, on its own, cannot detect an overflow. What single piece of information does it lack?
- b) In Chapter 5 the RX FIFO's full flag is what would make overflow detectable: a valid byte arriving while the FIFO is full is dropped. Sketch, in words, the extra flag you would add so software could tell that a byte was lost, and say which register it would live in (see ERROR_FLAGS in Part 2 of the protocol, §A.2).
- c) A framing error and an overflow have different causes. Give a physical cause for each: what on the wire, or in the software's timing, produces one but not the other?

The Register Bank

Ports become registers; the FPGA half is complete.

IN THIS CHAPTER

- **The two FIFOs from Chapter 4**, one per direction, instantiated by the bank rather than by the top.
- **The register bank** (`uart_regs.vhd`) around them: `STATUS` computed straight from the FIFO flags, the `ERROR_FLAGS` latches, `CTRL` and `BAUD_DIV` as plain storage, the write-triggered `TX_DATA` push, and the read-then-pop `RX` path in which `RX_DATA` is a pure read and `RX_POP` is the separate, abort-safe write that advances the FIFO.
- **Completing `uart_top`**: instantiating `uart_regs`, which owns both FIFOs, into the skeleton from Chapter 1, the last block it was waiting for, so the full system testbench runs for the first time.

This chapter is about the distance between port semantics and register semantics. The TX and RX cores speak in pulses and levels, while the register map (§A.2) promises poll-able bits, sticky error flags and FIFO-backed data; bridging the two is the whole job. After the chapter you should be able to explain why a single-cycle error pulse cannot be polled, and implement pulse-to-level latching with a defined set and clear pair for each `ERROR_FLAGS` bit, while deriving every `STATUS` bit combinationally from the FIFO flags instead; explain why popping a FIFO on read would need write-like commit-and-abort discipline, and why splitting the pure read (`RX_DATA`) from the explicit pop (`RX_POP`) avoids it; and instantiate the register bank into `uart_top` positionally, completing the peripheral so that it passes the system testbench.

Before you start, you need what Chapter 1 started: the `uart_top` skeleton and the `uart_def` package. You need the datapath blocks `baud_gen`, `uart_tx` and `uart_rx` from Chapters 2 and 3, along with the pulses and levels they expose, wired into the top across Chapters 2 and 4, and the `fifo` from Chapter 4, which this bank instantiates twice. Read the whole protocol (Appendix A) first, especially the register semantics (§A.2.3) and the SPI transport (§A.3), which you instantiate but do not write.

Read §5.1 for `uart_regs`; that is the specification you build it from. The two FIFOs it owns were built in Chapter 4 and are already in `hw/`. Revisit §1.2 for the top you now complete.

Then read the provided testbench `hw/uart_regs_tb.vhd`, along with the system testbench `hw/uart_top_tb.vhd`, which plays SPI transactions through the provided transport and watches a byte cross the whole peripheral.

The two ideas of the chapter are `STATUS` computed straight from the FIFO flags rather than stored, and the `RX_DATA` and `RX_POP` split with the reasoning behind it. The rest of the register decode is seven near-identical cases, and the `ERROR_FLAGS` latch is one more. With `uart_regs` instantiated into `uart_top`, every block is present, and the system testbench elaborates and runs for the first time, which is the moment the FPGA half becomes a thing that works:

```
make build-vhdl # From the repository root: every testbench whose modules exist.
```

In the exercises in §5.3 you build `uart_regs` (Exercise 5.1), then instantiate the bank into the `uart_top` skeleton, delete the `reg_rdata` placeholder you left there in part d) of Exercise 1.2, and run the full system testbench (Exercise 5.2).

5.1 Designing `uart_regs.vhd`

The datapath speaks in pulses and levels: a `valid` that is high for one clock, a `busy` that rises and falls, a tick. Software speaks in **registers**: it writes a byte to a fixed address, polls a status word, reads a byte back. The register bank is the translator between the two, and it is the layer the whole protocol (Appendix A) describes.

It owns two of the `fifo` you built in Chapter 4 (§4.1), the `CTRL`, `BAUD_DIV` and `ERROR_FLAGS` registers, and it computes the read-only `STATUS` word. It is reached over a small register bus that the provided SPI bridge drives; the bridge, not this module, is what turns SPI transactions into `reg_write` strobes, so `uart_regs` can trust that a `reg_write` it sees is a real, completed write.

5.1.1 Interface

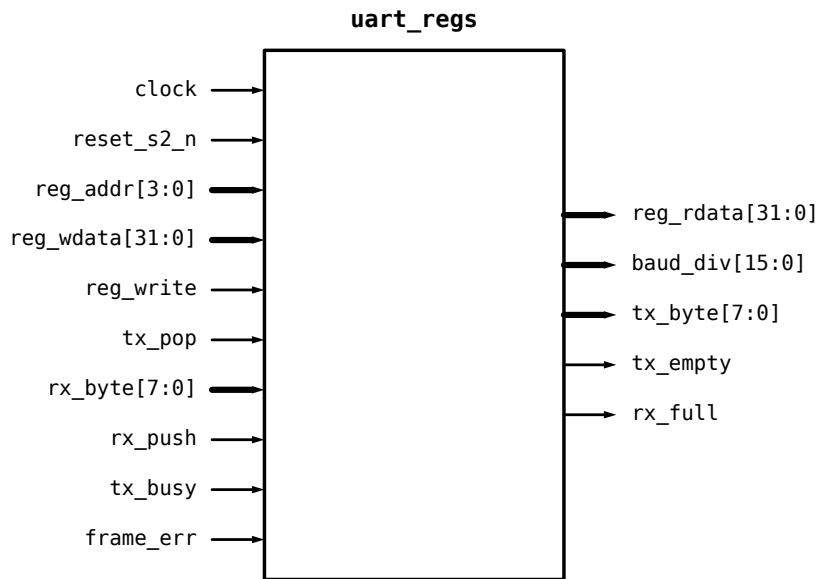


Figure 5.1 Module `uart_regs`

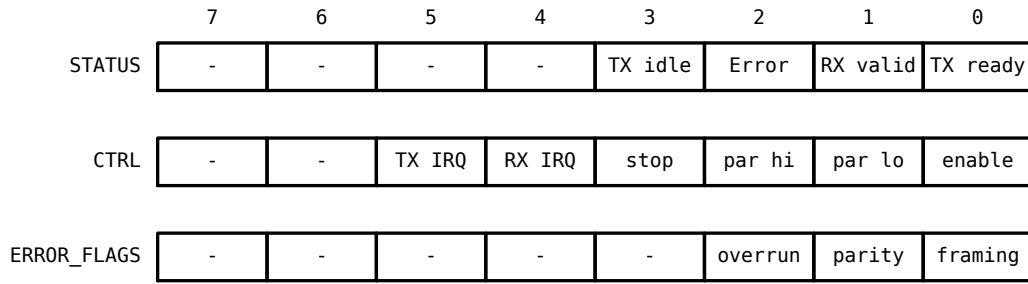
Port	Dir	Type	Meaning
clock, reset_s2_n	in	std_logic	Clock, and an active-low synchronized reset.
reg_addr	in	std_logic_vector(3 downto 0)	Register index (offset / 4).
reg_wdata	in	std_logic_vector(31 downto 0)	Write data.
reg_write	in	std_logic	One-cycle commit strobe from the bridge.
tx_pop	in	std_logic	The transmitter has taken the front byte.
rx_byte	in	std_logic_vector(7 downto 0)	A received byte, from the receiver.
rx_push	in	std_logic	Push rx_byte (the receiver's valid).
tx_busy	in	std_logic	The transmitter is mid-frame.
frame_err	in	std_logic	The receiver saw a bad stop bit.
reg_rdata	out	std_logic_vector(31 downto 0)	Combinational read value for reg_addr.
baud_div	out	std_logic_vector(15 downto 0)	BAUD_DIV, for baud_gen.
tx_byte	out	std_logic_vector(7 downto 0)	TX FIFO front, to the transmitter.
tx_empty	out	std_logic	TX FIFO empty.
rx_full	out	std_logic	RX FIFO full.

The ports are declared all inputs first, then all outputs; `uart_regs_tb` binds them positionally, so the order above is the contract. The testbench drives the register bus directly, standing in for the bridge, and models the datapath side with plain signals, so the register semantics are tested with no SPI in sight.

The register map itself (indices, offsets, bit positions) lives in the provided `uart_def` package, so the bank and the testbench name registers and bits (`REG_STATUS`, `ST_TX_READY`, ...) instead of magic numbers.

5.1.2 Register semantics (the whole point of this module)

The map promises sticky, poll-able bits and FIFO-backed data. The datapath offers pulses and levels. Bridging the two, bit by bit, is what the bank does. The three flag registers, from §1.1, are shown again in Figure 5.2.



bits 31-8 reserved: read 0, ignored on write

Figure 5.2 The bit fields of STATUS, CTRL and ERROR_FLAGS

STATUS is computed, never stored. Bit 0 (TX ready) is not `tx_full`, bit 1 (RX valid) is not `rx_empty`, bit 2 (Error) is the OR of the error flags, and bit 3 (TX idle) is `tx_empty` and not `tx_busy`. Software polls these; nothing writes them.

TX_DATA is a write-triggered action, not storage. A `reg_write` to its index pushes `reg_wdata(7 downto 0)` into the TX FIFO. It is an edge event, one write for one byte, and a separate feeder in `uart_top` (§1.2) drains the FIFO into the transmitter.

RX_DATA is a pure read; RX_POP is the advance. Reading `RX_DATA` returns the FIFO's front byte and changes nothing, so it obeys the same latch-once rule as STATUS, and discarding that byte is a *separate* write to `RX_POP`. The split is deliberate. A read that also popped the FIFO would be a read with a side effect, and it would need the same commit-on-completion, abort-safe handling as a write: what should happen if the SPI transaction is aborted mid-read? Keeping the read pure and the pop an explicit write sidesteps the question entirely, and it is the single most important idea in this module.

ERROR_FLAGS latches. A `frame_err` pulse from the receiver sets the framing bit, and it stays set until software writes 0 to clear it. STATUS bit 2 is just the OR of these flags.

CTRL and BAUD_DIV are plain read/write registers. `baud_div` is driven straight out to `baud_gen`, so a `BAUD_DIV` write takes effect on the next baud tick the generator produces.

CTRL is different, and the difference matters: in this course it is **storage only, and gates nothing**. Look at the port list above and you will not find a `ctrl` output, so there is no path by which `CT_ENABLE`, the parity select, the stop-bit select or the two IRQ masks could reach the datapath. They are stored, they read back, and nothing acts on them. Do **not** gate your TX push or your receiver on `CT_ENABLE`: `uart_regs_tb` writes CTRL only in its last case, long after its transmit case, and the system testbench `uart_top_tb` never writes it at all, so a bank that waits to be enabled simply never transmits, and fails with messages that say nothing about the cause: TX FIFO front should be 0x5A, got 0x00! in the one, a timeout waiting for RX valid in the other. The bits exist in `uart_def` because the register map is the shared contract with the C++ side and with the protocol; wiring them to something real is a natural extension, not part of the build here.

The bank never worries about aborted or partial transactions. The bridge only asserts `reg_write` on a completed, non-aborted write (Part 3 of the protocol, §A.3), so a strobe here is always a real commit. That contract is exactly what lets `TX_DATA` and `RX_POP` be simple single-cycle actions.

5.1.3 Behaviour

Very little of this module is clocked. It owns two `fifo` instances, three stored registers, and a pile of decode; the register bus itself is combinational, and the only state that advances on an edge is `ctrl_reg`, `baud_div_reg`, `err_flags`, and whatever the two FIFOs do internally.

Everything below is in the order you would type it: what to declare, what to instantiate, the concurrent lines, then the one process. The blocks and what connects them come first, in Figure 5.3.

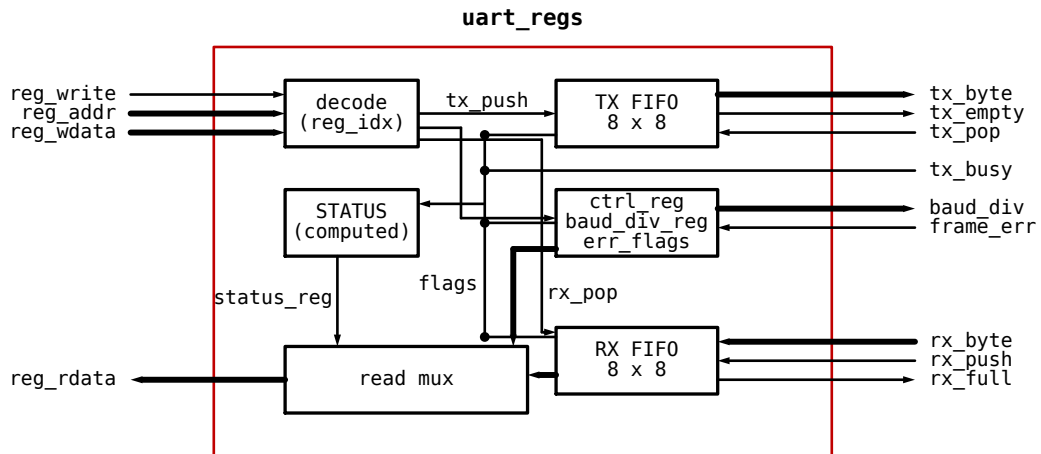


Figure 5.3 Inside `uart_regs`

Read the two queues against each other and the asymmetry is the whole module. The TX FIFO takes its data from the bus and its pop from the datapath; the RX FIFO takes its data from the datapath and its pop from the bus. Everything else is the decode that turns one `reg_write` into one action, the `flags` the `STATUS` word is assembled from, and the mux that answers reads.

Then the control flow. The left half of the module is one clocked decode, the right half is pure combinational read-back, and they never touch:

```
on each rising edge, if reg_write = '1', decode reg_idx:
    REG_CTRL      : ctrl_reg(CT_TX_IRQ downto 0) <= reg_wdata(CT_TX_IRQ downto 0)
    REG_BAUD_DIV  : baud_div_reg <= reg_wdata(15 downto 0)
```

```

REG_ERR_FLAGS : err_flags    <= reg_wdata(2 downto 0)
REG_TX_DATA   : nothing here - tx_push already pushed the FIFO
REG_RX_POP    : nothing here - rx_pop already advanced the FIFO
anything else : ignored
then, last in the process, so that it beats a clear in the same cycle:
  if frame_err = '1' then err_flags(ER_FRAMING) <= '1'

combinationally, every cycle, reg_rdata follows reg_idx:
  reg_rdata <= (others => '0')          -- default, then override
  when REG_STATUS => reg_rdata          <= status_reg
  when REG_CTRL   => reg_rdata          <= ctrl_reg
  when REG_BAUD_DIV => reg_rdata(15 downto 0) <= baud_div_reg
  when REG_RX_DATA => reg_rdata(7 downto 0) <= rx_front -- never a pop
  when REG_ERR_FLAGS => reg_rdata(2 downto 0) <= err_flags
  when others     => null                -- TX_DATA, RX_POP, 7-15

```

A TX_DATA write is the one part of this that is easier to see in time than in structure. The bridge holds the address and the word, raises `reg_write` for exactly one cycle, and the byte lands on the edge inside it, as Figure 5.4 shows.

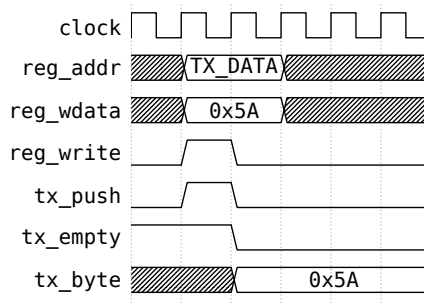


Figure 5.4 A TX_DATA write, cycle by cycle

`tx_push` is combinational, so it is high for the same cycle as `reg_write`; the FIFO registers the push on the edge at the end of that cycle, which is where `tx_empty` falls and `tx_byte` starts showing the byte. That one-cycle strobe is the shape `uart_regs_tb's bus_write` procedure drives, and it is why TX_DATA needs no storage of its own.

What to declare. The port list already gives you every input and output, so the architecture adds only what the ports cannot hold:

- `ctrl_reg[31:0]`: the control register (CTRL). Only bits 5–0 are ever written; bits 31–6 are reserved, so they stay at their reset value and read back as zero, exactly as BAUD_DIV and ERROR_FLAGS do with theirs.

- `baud_div_reg[15:0]`: the baud divider register (`BAUD_DIV`).
- `err_flags[2:0]`: the three error flags.
- `tx_push`, `rx_pop`: the two decoded write strobes.
- `tx_empty_s`, `tx_full_s`, `rx_empty_s`, `rx_full_s`, all `std_logic`: the four FIFO flags.
- `rx_front[7:0]`: the RX FIFO's front byte, which the read mux needs.
- `status_reg[31:0]`: the assembled `STATUS` value.

The two fifo instances. Both are generic `map(8, 8)`, and both bind positionally:

- The TX FIFO, in port order: `clock`, `reset_s2_n`, `reg_wdata(7 downto 0)`, `tx_push`, `tx_pop`, `tx_byte`, `tx_empty_s`, `tx_full_s`.
- The RX FIFO, in port order: `clock`, `reset_s2_n`, `rx_byte`, `rx_push`, `rx_pop`, `rx_front`, `rx_empty_s`, `rx_full_s`.

Read those two lines against each other and the asymmetry is the whole module: the TX queue takes its data from the bus and its pop from the datapath, and the RX queue takes its data from the datapath and its pop from the bus. `tx_byte` binds straight to the output port because nothing in here reads it; `rx_front` is internal because the read mux does.

The concurrent assignments. One line each, outside any process. Everything the read mux and the clocked process below need is built here:

- `reg_idx` is `to_integer(unsigned(reg_addr))`, which needs `ieee.numeric_std`. Four address bits means 0 to 15, so the decode's others branch covers the reserved indices 7 to 15 for free.
- `tx_push` is '1' when `reg_write = '1'` and `reg_idx = REG_TX_DATA`, else '0'.
- `rx_pop` is '1' when `reg_write = '1'` and `reg_idx = REG_RX_POP`, else '0'.
- `tx_empty` follows `tx_empty_s`, and `rx_full` follows `rx_full_s`: the two flags that leave the module.
- `baud_div` follows `baud_div_reg`, straight out to the conversion in `uart_top`.
- `status_reg` is built one bit at a time, and every bit not named below is '0':
 - `status_reg(ST_TX_READY)` is not `tx_full_s`: there is room to push.
 - `status_reg(ST_RX_VALID)` is not `rx_empty_s`: there is a byte to read.
 - `status_reg(ST_ERROR)` is '1' when `err_flags` is not all zeros: the OR of the three.
 - `status_reg(ST_TX_IDLE)` is `tx_empty_s` and not `tx_busy`: nothing queued *and* no frame in flight. TX ready is about room, TX idle is about quiet; software waits on the second before reconfiguring.
 - `status_reg(31 downto 4)` is (`others => '0'`), so the unused bits read back as the protocol promises. Every bit needs exactly one driver, which is why this line exists at all.

The read mux. A combinational process: no clock, no register, one case on `reg_idx`. Assign the zeros first and let each index override the bits it owns:

```

process(reg_idx, status_reg, ctrl_reg, baud_div_reg, rx_front, err_flags) is
begin
    reg_rdata <= (others => '0');           -- TX_DATA, RX_POP and 7-15 read zero
    case reg_idx is
        when REG_STATUS    => reg_rdata      <= status_reg;
        when REG_CTRL      => reg_rdata      <= ctrl_reg;
        when REG_BAUD_DIV  => reg_rdata(15 downto 0) <= baud_div_reg;
        when REG_RX_DATA   => reg_rdata(7  downto 0) <= rx_front;  -- never pops
        when REG_ERR_FLAGS => reg_rdata(2  downto 0) <= err_flags;
        when others        => null;
    end case;
end process;

```

Four things that shape holds down:

- **The default assignment comes first**, so every path through the process assigns `reg_rdata` and no latch is inferred. It also spares you five zero-extension expressions: assign the narrow register to the low bits and the leading zeros are already there.
- **when others => null;** is honest here, not lazy, precisely because of that default. Written without it, the same process would need `(others => '0')` spelled out in three more branches.
- **The reserved indices come free.** `reg_idx` is natural range 0 to 15, so when `others` covers 7 to 15 as well as the two write-only registers, which is what Part 2 of the protocol asks for.
- **The sensitivity list is spelled out.** VHDL-93 has no `process(all)` and this course analyzes with `--std=93`, so every signal the process reads has to be listed, or simulation and synthesis disagree about a mux that looked fine on the page. The provided `spi_reg_bridge` does exactly this in `hw/spi_reg_bridge.vhd`, and is worth reading as the pattern.

Because that path is combinational, `reg_rdata` follows `reg_addr` within the cycle. Nothing registers it on the way out: the bridge latches the word once, just after the command byte, and `uart_regs_tb` allows it a nanosecond (`SETTLE_NS`) to settle before it samples.

The clocked process. It is `process(clock, reset_s2_n)`, with its reset asserted asynchronously like every other module here, and it is the only *clocked* process in the file — the read mux above is the other one, and it has no clock at all:

- On reset (`reset_s2_n` is low):
 - `ctrl_reg` is cleared, so every CTRL bit reads back low.
 - `baud_div_reg` is cleared, so BAUD_DIV reads 0. A zero divider is not a legal input to `baud_gen`, which is why `uart_top` guards the conversion (§1.2); the driver is expected to write a real divider before using the line.
 - `err_flags` is cleared, so STATUS bit 2 reads low.

- Nothing else. The FIFOs take `reset_s2_n` on their own ports and come out empty, which is what makes the first STATUS read after reset show TX ready and TX idle set with RX valid clear.
- On each rising edge of clock, while not in reset:
 - First, if `reg_write` is high, decode `reg_idx`. **REG_CTRL**: `ctrl_reg(CT_TX_IRQ downto 0)` takes `reg_wdata(CT_TX_IRQ downto 0)`, so the six defined bits are stored and the reserved ones above them stay zero. Nothing else happens: no `ctrl` output exists, so the bits are stored, read back, and act on nothing. **REG_BAUD_DIV**: `baud_div_reg` takes `reg_wdata(15 downto 0)`. The new divider reaches `baud_gen` combinationally, so it applies from the next tick the generator produces. **REG_ERR_FLAGS**: `err_flags` takes `reg_wdata(2 downto 0)`, so the documented write of `0x0` clears all three latches. **REG_TX_DATA** and **REG_RX_POP**: nothing happens *here*. Their whole effect is the concurrent strobe into a FIFO, which that FIFO registers on this same edge. Writing FIFO logic in this process as well is the most common way to get two bytes out of one write. **REG_STATUS**, **REG_RX_DATA**, and indices 7 to 15: ignored. A write to a read-only or reserved register does nothing at all.
 - Then, *after* that decode, if `frame_err` is high, set `err_flags(ER_FRAMING)`.
 - Nothing else is clocked. No byte passes through a register on its way to `reg_rdata`, and no output is registered.

Three things that catch people out.

- **The order of the last two steps.** An `ERROR_FLAGS` clear and a fresh `frame_err` can land in the same cycle, and the last assignment in a process wins — the same rule `uart_rx` leans on for its tick counter. Putting the latch after the write decode means the new error survives the clear; the other order loses a real error to a write issued before it happened, and a dropped error flag is far worse than a stale one.
- **The FIFOs already guard themselves.** A `TX_DATA` write while the TX FIFO is full is dropped by the FIFO, and an `RX_POP` on an empty RX FIFO does nothing, both because Chapter 4 guarded them on `not full` and `not empty`. The bank adds no guards of its own; STATUS is what the driver is supposed to poll first.
- **Nothing is gated on CT_ENABLE.** As above: no testbench writes CTRL before it transmits, so a bank that waits to be enabled never transmits.

Two things are deliberately absent. `ER_PARITY` and `ER_OVERRUN` have no producer in this build, so they hold whatever a write last left in them, which is zero; `rx_full` is exported precisely so that overrun detection can be added later without touching the map. And there is no TX feeder in here: `tx_pop` is an *input*, driven by the feeder in `uart_top`, so this module never decides when the transmitter takes a byte.

5.1.4 What the testbench pins down

uart_regs_tb works entirely over the register bus. It checks that BAUD_DIV writes and reads back and drives baud_div, and that STATUS after reset shows TX ready and TX idle set with RX valid and Error clear. On the transmit side, a TX_DATA write must appear at the FIFO front and clear TX idle, and a tx_pop must empty it again. On the receive side, an rx_push must set RX valid, RX_DATA must return the byte, a bare RX_DATA read must **not** pop, and an RX_POP write must clear RX valid. Finally, a frame_err must latch into ERROR_FLAGS and STATUS, and a write of 0 must clear it. Two checks close the map off: a CTRL write of all ones must read back as 0x0000003F, proving that the six defined bits are stored and the reserved ones above them are not, and a write to reserved index 7 must be ignored, with the read returning zero.

The read-does-not-pop check is the one that pins down the read/pop split; get it wrong and RX valid would clear on a plain read.

5.1.5 Where it fits

uart_regs is the middle of uart_top (§1.2): the bridge on one side driving the register bus, the datapath (baud_gen, uart_tx, uart_rx) on the other. Its two FIFOs are the buffers that let software and the serial line run at their own speeds.

5.1.6 What's ahead

The exercises in §5.3 build uart_regs, then instantiate the bank into the uart_top skeleton from Chapter 1, the last block it was waiting for, and run the full system testbench for the first time.

5.2 Review

What you should be able to do now

After this chapter, you should be able to:

- **Explain why a single-cycle error pulse cannot be polled**, and implement pulse-to-level latching with a defined set and clear pair for each ERROR_FLAGS bit, while deriving every STATUS bit combinationally from the FIFO flags instead.
- **Explain why popping a FIFO on read would need write-like commit-and-abort discipline**, and why splitting the pure read (RX_DATA) from the explicit pop (RX_POP) avoids it.
- **Instantiate the register bank into uart_top positionally**, completing the peripheral so that it passes the system testbench.

Questions to test yourself

You should be able to explain:

1. Why can STATUS bit 1 (RX valid) not simply be wired to the receiver's "byte ready" pulse?
2. A TX_DATA write arrives while STATUS bit 0 (TX ready) is clear: what happens, and why is dropping the byte the right choice here?
3. SS rises after the third byte of an RX_POP write: what must the peripheral do, and which part of the transport contract guarantees it?
4. If reading RX_DATA returns a byte but never advances the FIFO, what did the driver forget, and what would go wrong if RX_DATA popped on its own?

Looking ahead

The FPGA half is done. The driver stack begins in Chapter 6, on the host, with no hardware: the `driver::uart::Interface` and its `driver::transport::Interface` seam, then a first `driver::uart::Stub` written against that interface.

5.3 Exercises

How to work these. Exercise 5.1 reinforces §5.1, and Exercise 5.2 completes `uart_top` from §1.2. The `fifo` the register bank owns was built in Chapter 4, so it is already in `hw/` and ready to instantiate. Build each module from its section.

How to check your work. Check each module with its provided testbench, run from `hw/`; the preface's *Setting up* has the full flow.

The answers to every part that is not code are in §B.5. Try each part before you read its answer.

Exercise 5.1 `uart_regs`

VHDL

- a) Write your own `uart_regs.vhd` from the specification in §5.1 and run its testbench:

```
cd hw
ghdl -a --std=93 uart_def.vhd fifo.vhd uart_regs.vhd uart_regs_tb.vhd
ghdl -e --std=93 uart_regs_tb
ghdl -r --std=93 uart_regs_tb --assert-level=error
```

`uart_def.vhd` (the register map) and `fifo.vhd` come first because `uart_regs` reads the package and instantiates two FIFOs.

- b) Make `RX_DATA` pop the FIFO on read (advance `tail` whenever that index is read). Re-run. Which check fails first, and why is it not the "a bare `RX_DATA` read must not pop" check? Does `uart_top_tb` notice? Then explain, in terms of the SPI transport's abort rule, why a

read with a side effect is a genuinely harder thing to get right than the pure read plus a separate `RX_POP`.

c) `STATUS` is computed, not stored. Name the four things each of its bits is derived from, and say why none of them can simply be a stored register that the datapath writes.

d) The bank acts on every `reg_write` it sees as a real commit. What does it rely on the provided SPI bridge to guarantee for that to be safe, and what would break if a half-finished (aborted) transaction could reach it as a `reg_write`?

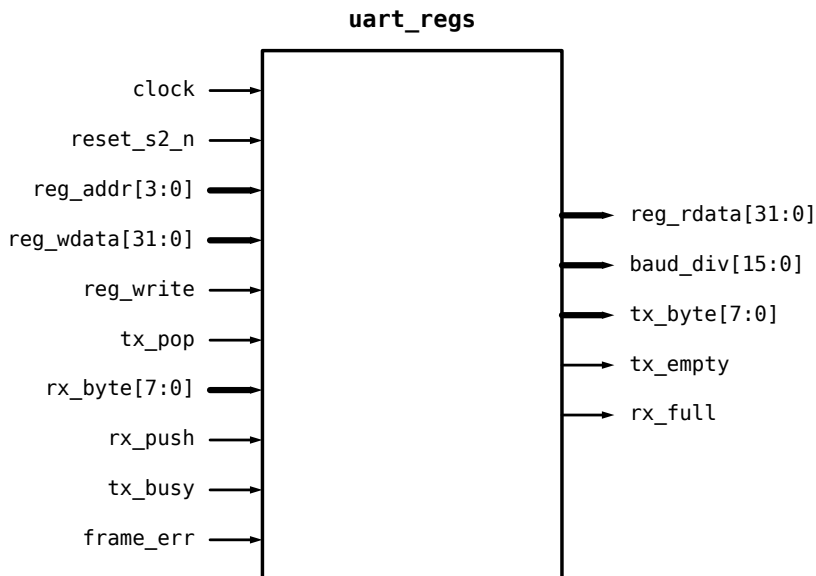
Exercise 5.2 `uart_top`

VHDL

a) You built the `uart_top` skeleton in Chapter 1 and added `baud_gen`, `uart_tx`, `sync` and `uart_rx` across Chapters 2 and 4, declaring each block's signals as you went. Now do it once more for `uart_regs`, the last block the top was waiting for. Three signals are new here:

Signal	Type	Driven by	Read by
<code>tx_empty</code>	<code>std_logic</code>	<code>uart_regs</code>	the feeder below
<code>rx_full</code>	<code>std_logic</code>	<code>uart_regs</code>	nothing, in this build
<code>tx_pop</code>	<code>std_logic</code>	the feeder below	<code>uart_regs</code>

Every other port below is a signal you already declared in Chapters 1, 2 or 4 — which is the point of having declared them where they were needed.



#	uart_regs port	Dir	Type	Connect to
1	clock	in	std_logic	clock
2	reset_s2_n	in	std_logic	reset_s2_n
3	reg_addr	in	std_logic_vector(3 downto 0)	reg_addr (Chapter 1)
4	reg_wdata	in	std_logic_vector(31 downto 0)	reg_wdata (Chapter 1)
5	reg_write	in	std_logic	reg_write (Chapter 1)
6	tx_pop	in	std_logic	tx_pop
7	rx_byte	in	std_logic_vector(7 downto 0)	rx_byte (Chapter 4)
8	rx_push	in	std_logic	rx_push (Chapter 4)
9	tx_busy	in	std_logic	tx_busy (Chapter 2)
10	frame_err	in	std_logic	frame_err (Chapter 4)
11	reg_rdata	out	std_logic_vector(31 downto 0)	reg_rdata (Chapter 1)
12	baud_div	out	std_logic_vector(15 downto 0)	baud_div (Chapter 2)
13	tx_byte	out	std_logic_vector(7 downto 0)	tx_byte (Chapter 2)
14	tx_empty	out	std_logic	tx_empty
15	rx_full	out	std_logic	rx_full

```
-- The register bank: the bridge's bus on one side, the datapath on the other.
-- This closes every loop in the design - the bus now has something answering on
-- it, the transmitter has a byte source, and the receiver has somewhere to put
-- what it recovers.
uart_regs: entity work.uart_regs
  port map(clock, reset_s2_n, reg_addr, reg_wdata, reg_write, tx_pop, rx_byte,
           rx_push, tx_busy, frame_err, reg_rdata, baud_div, tx_byte, tx_empty,
           rx_full);
```

Delete the `reg_rdata <= (others => '0')`; placeholder from Chapter 1 in the same edit: the bank drives that vector now, and leaving the placeholder in gives it two drivers.

With `tx_empty` finally driven, the **TX feeder** can be written, the one piece of real logic the top has been waiting to hold. It is two concurrent statements:

```
-- The TX feeder: load the transmitter whenever a byte is queued and the line is
-- free, and pop the FIFO on that same edge, so exactly one byte moves per idle
-- transmitter. The instant the transmitter goes busy, tx_load drops - which is
-- what stops a second byte following it.
tx_load <= (not tx_empty) and (not tx_busy);
tx_pop <= tx_load;
```

Note why this could not have been written earlier: `tx_empty` comes from the register bank and `tx_busy` from the transmitter, so the feeder needs a block from Chapter 2 and a block from Chapter 5 to exist at the same time. That is the general shape of this design, and the reason signals are declared where they are needed rather than all at once in Chapter 1.

The receive side needs no equivalent: `uart_rx`'s `valid` went straight to `rx_push` in Chapter 4, so the receiver pushes its own byte into the RX FIFO.

Then run the system testbench:

```
cd hw
ghdl -a --std=93 uart_def.vhd reset_sync.vhd baud_gen.vhd sync.vhd fifo.vhd \
      uart_tx.vhd uart_rx.vhd uart_regs.vhd spi_slave.vhd spi_reg_bridge.vhd \
      uart_top.vhd uart_top_tb.vhd
ghdl -e --std=93 uart_top_tb
ghdl -r --std=93 uart_top_tb --assert-level=error
```

Everything the design contains appears on that line, in dependency order, including `fifo`, which `uart_top` never instantiates itself: `uart_regs` owns both of them. With the last block in place, the system testbench elaborates the whole peripheral for the first time.

If `uart_top_tb` fails on the `BAUD_DIV` read-back with every 1 bit missing from the value you wrote (got `0x0000` for the `0x0004` it wrote), the placeholder from Chapter 1 is still there: each '1' the bank drives is resolving against its '0' to 'X', and `to_hex` prints an 'X' as a zero.

- b) The provided `reset_sync` asserts asynchronously but releases synchronously. Explain what could go wrong if it released `reset_s2_n` asynchronously too (the moment `reset_n` rises), and why asserting asynchronously is nonetheless the right choice.
- c) Why can `reset_sync` not just be a `sync` instance, given that both are two flip-flops in series? Name the port that settles it, and describe the circular dependency you would create by trying.
- d) Study the TX feeder you wrote in a): `tx_load <= (not tx_empty) and (not tx_busy)`. Argue that it loads **exactly one** byte each time the transmitter goes idle with the FIFO non-empty, never zero and never two. What holds `tx_load` low for the rest of the frame?
- e) The system testbench loops tx back to rx. Trace a single byte: an SPI write of `TX_DATA`, through the FIFO and the transmitter, out on tx, back in on rx, through the receiver into the RX FIFO, and out again on an SPI read of `RX_DATA`. Name the block that acts at each step.

PART II

The Driver and Integration

The Driver's Contracts

Crossing to C++: the three abstractions the driver rests on, before any algorithm.

IN THIS CHAPTER

- **The register map in C++:** the seven register indices and the STATUS, CTRL and ERROR_FLAGS bits from Part 2 of the protocol (§A.2), transcribed as named constants the whole driver shares with `uart_def.vhd`.
- **The transport interface,** `driver::transport::Interface`: a byte-level SPI seam of begin, transfer and end, and why putting that seam at the byte level is exactly what makes the driver host-testable.
- **The driver interface,** `driver::uart::Interface`: the abstract, non-blocking API the application codes against, being configure, write, read, poll status and clear errors.
- **A first implementation of it,** `driver::uart::Stub`: a test double that plays back scripted bytes and records what was written. It needs nothing but the interface, so it can be written the moment the interface exists, and it keeps this chapter from being declarations only.

This chapter crosses from VHDL to C++, and it stays entirely with *shapes*: the driver stack top to bottom, and the three abstractions it rests on, before any algorithm is written. After it you should be able to transcribe the register map into `register_map.hpp` as `constexpr` constants, and explain why a value there that disagrees with the hardware is a bug you would only find on the bench; design the transport interface and say what it buys, the same driver over a stub in Chapter 8 and over the real AVR SPI in Chapter 9, with nothing above the seam changing; turn the register-level flow into a clean, abstract `Interface`, and explain why non-blocking write and read map directly onto the FIFO-backed STATUS bits of Chapter 5; and implement that interface once, as a `Stub` backed by two linear buffers, and explain what a test double has to be faithful about and what it may ignore.

Before you start, you need abstract classes, pure virtual methods and dependency injection from modern embedded C++. Embedded software testing is recommended but not required, for why a seam exists and what a stub sits behind. Read Parts 2 and 3 of the protocol (§A.2 and §A.3).

Read §6.1, which lays out the full driver stack and specifies the three contracts you build here: the register map, the byte-transport seam, and the interface. They are designed top-

down, in the order the design demands rather than the order the files are written: the interface the application wants, the seam the driver will run over, and the register map both sides of the wire share. These are declarations only; the driver that implements them arrives in Chapter 7, and the tests that check it in Chapter 8. The first `make test` of the C++ half therefore comes in Chapter 8, once both the `Uart` and the stub that scripts it exist.

In the exercises (§6.3) you build `driver/uart/register_map.hpp`, `driver/transport/interface.hpp` and `driver/uart/interface.hpp`, keeping the names the tests of Chapter 8 will bind to (Exercises 6.1 to 6.3), then write `driver/uart/stub.hpp` against the interface you just declared (Exercise 6.4). The stub is the first concrete class of the C++ half and about as long as the three headers together, but it is buffer bookkeeping rather than design, and by then you have the interface it must satisfy.

6.1 The Driver Stack (the Contracts)

This chapter crosses from VHDL to C++, and it starts with the *shapes*: the three abstractions the driver will rest on. None of them contains an algorithm yet; they are the register map, the transport interface, and the driver interface. Designing them well is what makes the driver (Chapter 7) short and host-testable.

Two of the three are abstract interfaces, so each lives in its own namespace: the driver's public API is `driver::uart::Interface`, and the transport seam it runs over is `driver::transport::Interface`. Naming both `Interface`, in separate namespaces, is deliberate: each subsystem exposes one seam by that name.

Everything here is written by you. The three contracts are declarations; the exercises then implement the driver interface once, as `driver::uart::Stub`, so the chapter ends with a concrete class rather than three headers. The provided host tests arrive in Chapter 8, once the `Uart` that implements the same interface exists.

The full stack, top to bottom:

Layer	Chapter	Role
The application, <code>app::EchoNode</code>	10	Uses the driver through its abstract interface.
<code>driver::uart::Interface</code>	6	The abstract driver API the application depends on.
<code>driver::uart::Stub</code>	6	A test double implementing that interface; the whole harness for <code>app::EchoNode</code> in Chapter 10.
<code>driver::uart::Uart</code>	7	Implements the driver interface using the register protocol.
<code>driver::transport::Interface</code>	6	The abstract SPI byte seam the driver runs over.
A scripted stub, or the real SPI	8, 9	<code>driver::transport::Stub</code> for the host tests; the AVR SPI on hardware.

Every layer is C++. The register map both halves share lives in `driver/uart/register_map.hpp`.

6.1.1 The register map

The C++ side needs the same register map the hardware has: the seven register indices and the meaning of each STATUS, CTRL and ERROR_FLAGS bit, from Part 2 of the protocol (§A.2). It comes from the same source as `uart_def.vhd` on the other side of the wire, and where a name exists on both sides the value must be identical; you declare only the bits this driver actually uses. You transcribe it once, as named constants, so that nothing above ever writes a bare 3 where it means “the TX_DATA register”. A value here that disagrees with the hardware is a bug you would only find on the bench, so it is worth copying carefully.

6.1.2 The transport seam

The driver will never talk to SPI directly. It talks to an abstract seam, `driver::transport::Interface`, that does one thing: exchange raw bytes over one SPI transaction. A transaction is *select* (pull SS low), some byte exchanges, then *deselect* (release SS), matching the framing in Part 3 of the protocol (§A.3). Each byte exchange is duplex: the same call sends one byte and returns the byte that arrived on MISO at the same time, because that is exactly how SPI works, a bit out and a bit in on every clock.

Putting the seam at the *byte* level, rather than exposing register reads and writes directly, is the deliberate choice. It keeps the SPI-specific knowledge (five-byte transactions, the command byte, the byte order) in one place, the driver, and lets the layer below be a dumb byte pipe that a stub can imitate perfectly. The seam is the whole reason the driver is host-testable: the tests of Chapter 8 inject a stub here, Chapter 9 injects the real AVR SPI, and nothing above the seam changes.

6.1.3 The driver interface

`driver::uart::Interface` is the abstract API the application codes against, and it promises five things: configure the peripheral, write a byte, read a byte, report the status and error state, and clear errors. Making it abstract is what lets the application, and its tests, be written against a promise rather than a concrete driver, and it is why the same application later runs unchanged over the real hardware driver.

`write()` and `read()` are both **non-blocking**. `write()` pushes a byte only if the transmitter has room, and reports whether it did; `read()` hands back a byte only if one has arrived. That maps directly onto the FIFO-backed STATUS bits of Chapter 5 and makes every operation deterministic to test. Blocking versions, which spin until the peripheral is ready, are a thin convenience built on top, an exercise in Chapter 8 rather than part of the core.

6.1.4 What’s ahead

The exercises in §6.3 build the register map, the transport interface and the driver interface, then implement that interface once as `driver::uart::Stub`. In Chapter 7 you implement the Uart driver over these contracts, using the register protocol, and in Chapter 8 you test it with a scripted `driver::transport::Stub`.

6.2 Review

What you should be able to do now

After this chapter, you should be able to:

- **Transcribe the register map** into `register_map.hpp` as `constexpr` constants, and explain why a value there that disagrees with the hardware is a bench-only bug.
- **Design the transport interface** (`driver::transport::Interface`) and say what it buys: the same driver over a stub in Chapter 8 and over the real AVR SPI in Chapter 9, with nothing above the seam changing.
- **Turn the register-level flow into a clean, abstract Interface**, and explain why non-blocking write and read map directly onto the FIFO-backed STATUS bits of Chapter 5.
- **Implement that interface once**, as a Stub backed by two linear buffers, and explain what a test double has to be faithful about and what it may ignore.

Questions to test yourself

You should be able to explain:

1. Why is the transport seam placed at the raw-byte level rather than exposing `readReg` and `writeReg` directly, and what does the byte-level choice let a stub do?
2. `write` and `read` are non-blocking and report whether they did anything: which STATUS bits of Chapter 5 make that possible, and why is a non-blocking core easier to test than a blocking one?
3. The register map is written once in VHDL (`uart_def.vhd`) and once in C++ (`register_map.hpp`): what goes wrong if the two disagree, and at which stage would you first notice?

Looking ahead

In Chapter 7 the contracts get an implementation: the `Uart` driver over the transport interface, and the register read and write protocol underneath it, the five-byte transaction, most significant byte first. The stub that verifies it on the host follows in Chapter 8.

6.3 Exercises

How to work these. These exercises build the driver's three contracts from §6.1: the register map, the transport interface, and the driver interface. All three are header-only declarations, so there is nothing to run yet; Chapter 7 implements the driver over them and Chapter 8 brings the first tests. Exercise 6.4 then implements the driver interface once, as a UART stub.

How to check your work. For now the check is simply that each header compiles. Nothing in fw/Makefile compiles a header on its own yet (make build waits for the source/main.cpp of Chapter 8, and make test for the headers of Chapter 8), so compile each one by hand, from fw/:

```
cd fw
for h in driver/uart/register_map.hpp driver/transport/interface.hpp driver/uart/stub.hpp; do
    echo "#include \"$h\" | g++ -std=c++17 -Wall -Wextra -Werror -Iinclude -fsyntax-only -x c++ -"
done
```

Every exercise in this chapter is code, so Appendix B has nothing to add here. The names below are the contract the host suite of Chapter 8, fw/test/uart/uart_test.cpp, binds to, and that suite is where these headers are first put to work.

Work in fw/. Create the following files:

```
include/
  driver/
    uart/
      register_map.hpp
      interface.hpp
      stub.hpp
    transport/
      interface.hpp
```

The register map and the driver interface live in the driver::uart namespace; the transport interface lives in driver::transport. The names below are the contract the tests of Chapter 8 bind to, so keep them as written.

Write it AVR-portable. These headers compile on the host (g++) now and, from Chapter 9, on the ATmega328P. **We deliberately skip several modern C++ features here because the AVR toolchain does not support them**, and the same driver code has to build for both targets, so stay on the subset that toolchain accepts. The reference point is the AVR/GNU C++ compiler bundled with Microchip Studio, which is several GCC releases behind a desktop g++; something compiling on your host, or even on a Linux avr-g++, does not mean it will compile there.

Include <stdint.h>, and <stddef.h> if you need size_t, and use the bare types uint8_t, uint16_t, uint32_t and size_t: the AVR toolchain ships no C++ standard library, so there is no <cstdint> and no std:: namespace at all, and anything reaching for either will fail.

Write namespaces as nested **blocks**, namespace driver { namespace uart { ... } }, because the compact C++17 form namespace driver::uart { ... } does not compile there. Leave out [[nodiscard]], which it rejects for the same reason. What you can rely on is noexcept, override, final, = default, and pure = 0.

EXERCISE SET 1

The Register Map

Exercise 6.1 register_map.hpp

C++

In `include/driver/uart/register_map.hpp`, transcribe the register map from Part 2 of the protocol (§A.2) as `constexpr` constants. Plain `constexpr`, not `inline constexpr`: *inline variables* are a C++17 feature the AVR toolchain does not have. At namespace scope `constexpr` already implies internal linkage, so each translation unit gets its own copy of a compile-time constant, which is exactly what you want and costs nothing. These are the same values `uart_def.vhd` holds on the other side of the wire; a mismatch is a bug you would only find on the bench.

a) Register indices

In a nested namespace `reg`, add one `uint8_t` constant per register index, each one the register's offset divided by 4: `STATUS = 0`, `CTRL = 1`, `BAUD_DIV = 2`, `TX_DATA = 3`, `RX_DATA = 4`, `RX_POP = 5` and `ERROR_FLAGS = 6`.

b) STATUS bit positions

In a nested namespace `status`, add one `uint8_t` **bit-position** constant per STATUS bit, meaning the bit's index rather than a mask: `TX_READY = 0`, `RX_VALID = 1`, `ERROR = 2` and `TX_IDLE = 3`. These are the same positions `uart_def.vhd` defines, so the two sides agree.

You form a mask by shifting at the use site: `1U << status::TX_READY` tests that bit, and `status & (1U << status::RX_VALID)` is non-zero when RX has data.

c) CTRL and error bit positions

In a nested namespace `ctrl`, add `ENABLE = 0` as a `uint8_t` bit position. In a nested namespace `error`, add `FRAMING = 0`, `PARITY = 1` and `OVERRUN = 2`, all `uint8_t` bit positions. The VHDL package also defines parity, stop-bit and interrupt-mask positions in CTRL; this driver does not use them yet, so it does not declare them.

Positions rather than masks keep the C++ constants identical to the VHDL `uart_def.vhd`, which also stores positions, and the one-line `1U << pos` at each use site is explicit about which bit you mean.

EXERCISE SET 2

The Transport Interface

Exercise 6.2 driver::transport::Interface

C++

In `include/driver/transport/interface.hpp`, design an abstract class `driver::transport::Interface` representing a byte-level SPI transport. It exchanges raw bytes over one transaction and knows nothing about registers. All methods except the destructor shall be pure virtual (`= 0`).

a) Destructor

Add a destructor that is virtual, marked `noexcept`, and implemented using `= default`.

b) Begin a transaction

Add a pure virtual method `begin()` that starts a transaction by pulling SS low. It takes no parameters, returns nothing, and cannot throw exceptions.

c) Exchange one byte

Add a pure virtual method `transfer()` that exchanges a single byte in full duplex. It takes one parameter of type `uint8_t`, the byte to send, returns the `uint8_t` received at the same time, and cannot throw exceptions.

d) End a transaction

Add a pure virtual method `end()` that ends the transaction by releasing SS. It takes no parameters, returns nothing, and cannot throw exceptions.

EXERCISE SET 3

The Driver Interface

Exercise 6.3 Interface

C++

In `include/driver/uart/interface.hpp`, design an abstract class `driver::uart::Interface`, the driver's public API. All methods except the destructor shall be pure virtual.

a) Destructor

Add a destructor that is virtual, `noexcept`, and `= default`.

b) Configure

Add a pure virtual method `configure()` that sets the baud divider and enables the peripheral. It takes one parameter of type `uint16_t`, the baud divider, returns nothing, and cannot throw exceptions.

c) Write a byte

Add a pure virtual method `write()` that attempts to send one byte. It takes one parameter of type `uint8_t`, returns `true` if the byte was accepted or `false` if the transmit FIFO was full, and cannot throw exceptions.

d) Read a byte

Add a pure virtual method `read()` that attempts to receive one byte. It takes a reference to a `uint8_t` where the received byte will be stored, returns `true` if a byte was received and `false` otherwise, and cannot throw exceptions.

e) Status and errors

Add three more pure virtual methods, none of which throws. `status()` returns the raw STATUS register as a `uint32_t`, takes no parameters, and does not modify the object (`const`). `errorFlags()` returns the raw ERROR_FLAGS register as a `uint32_t`, also taking no parameters and also `const`. `clearErrors()` clears the error flags, takes no parameters, and returns nothing.

EXERCISE SET 4

The UART Stub

Exercise 6.4 `driver::uart::Stub`

C++

The three headers above are all declarations. This one is the first concrete class of the C++ half, and it is worth writing now rather than later, because it needs nothing except the interface you just declared: a **UART stub**, a `driver::uart::Interface` a test can drive.

It plays back scripted bytes on `read()` and records the bytes an application `write()`s, so a test can queue input, run something over it, and check what came out. It is the same idea as the `driver::transport::Stub` you write in Chapter 8, one layer up: that one fakes the wire under the driver, this one fakes the driver under an application. Write it in `include/driver/uart/stub.hpp`, header-only, in the `driver::uart` namespace, in the same AVR-portable style as everything else here; it holds no dynamic memory, so it compiles for the target as well as the host.

Because a UART is a byte stream, both directions are **FIFO**: bytes are received and recorded in the order they arrive. But a test queues a bounded number of bytes and reads them once, so the storage is two plain **linear buffers** with no wraparound to reason about: two fixed `uint8_t` arrays, each with a length. The **RX** buffer is read front to back through an advancing index, so `read()` hands back the byte at the index and advances it, and the length is how many bytes were queued. The **TX** buffer is a write-only append log, so `write()` appends one byte and bumps the length, and the test reads the whole log back afterwards.

Member variables, using a fixed capacity `OurBufLen` and `uint8_t` throughout, with no `size_t`. `myRxBuf[OurBufLen]`, with `myRxLen` and `myRxDx`, holds the scripted bytes to receive

and tracks how far `read()` has advanced through them. `myTxBuf[0:myTxLen]`, with `myTxLen`, holds the bytes the application has sent, appended in order.

The third is `volatile bool& myStop`, a reference to a caller-owned flag, set true when `read()` reaches the end of the RX buffer. That one needs a word of explanation this early: in Chapter 10 you write an application whose `run(const volatile bool& stop)` loops until the flag is set, and a single-threaded test has no other way to end it. The stub feeds every scripted byte and then, once the input is exhausted, asks the loop to stop. The flag is `volatile` because it is written outside the loop that reads it; Chapter 10 says why that matters. Because it is a reference member, copy and move are deleted, as is the default constructor, since you need a stop flag to build one.

Methods, the six from `driver::uart::Interface` plus scripting helpers. The constructor, `Stub(volatile bool& stop)`, is explicit, stores the stop reference, and leaves both buffers empty. `configure(uint16_t)` does nothing, since the stub has no baud rate to set. `write(uint8_t byte)` appends `byte` to `myTxBuf` if there is room and returns true, as a UART with room in its TX FIFO would, and returns false if the buffer is full. `read(uint8_t& byte)` hands back `myRxBuf[myRxIdx++]` and returns true when `myRxIdx < myRxLen`, and otherwise sets `myStop` to true and returns false; that empty-case flag is the loop's off switch. `status()`, `errorFlags()` and `clearErrors()` are trivial, returning 0 or doing nothing, because the applications tested this way drive the UART through `read()` and `write()` rather than the status register.

Three helpers exist for the tests. `injectRxByte(uint8_t byte)` appends one scripted byte to `myRxBuf`, returning false if it is full, and is called once per byte the application should receive. `txBuf()` and `txLen()` return a pointer to the recorded TX bytes and how many there are, so a test reads `txBuf()[i]` for `i < txLen()` in the order they were sent. And `reset()` zeroes both lengths, the read index and the stop flag, so the stub can be reused across tests.

Nothing exercises it yet, which is the honest state of a test double written before the thing it doubles for. It compiles, and in Chapter 10 it becomes the entire test harness for `app::EchoNode`.

What's ahead

In Chapter 7 you implement `driver::uart::Uart`, which inherits this interface and drives it through the register protocol over a `driver::transport::Interface`; Chapter 8 then writes the scripted `driver::transport::Stub` and gets the host tests that check it green. The UART stub written here waits until Chapter 10, where it becomes the test harness for `app::EchoNode`.

The Driver, Built

The contracts get an implementation. Nothing runs it yet, which is what makes the reasoning matter.

IN THIS CHAPTER

- **The register protocol in code:** `readReg()` and `writeReg()` as the five-byte SPI transactions of Part 3 of the protocol (§A.3), each moving its bytes through the transport interface most significant byte first.
- **const at a hardware boundary:** why `readReg()` is a `const` member driving a non-`const` transport, what `const_cast` is doing there, and what it is *not* doing, since `const` never propagated through the reference member in the first place.
- **The Uart driver** on that core, implementing the Interface of Chapter 6: `configure`, the non-blocking `write`, and the `poll`, `read` and separate `RX_POP` that make up the read path.
- **Why the read path is three accesses and not one:** `RX_DATA` is a pure read, `RX_POP` is what advances the FIFO, and the register bank of Chapter 5 is on the other side of the wire enforcing exactly that.

Chapter 6 designed the contracts; this chapter implements them. Nothing is tested yet, which is Chapter 8's job, so this chapter is entirely about the code that has to be right before a test can say so. After it you should be able to implement `readReg()` and `writeReg()` as byte sequences that match the protocol exactly, and explain why the four data bytes go most significant first; explain what `const` does and does not reach through a reference member, and justify the `const_cast` at the one line where a `const` method starts driving hardware; implement the Uart methods over that register core, including the `poll`, `read` and `RX_POP` split, injecting the transport through the constructor; and say what each of the three RX accesses would break if it were omitted, reordered, or folded into the one before it.

Before you start, you need all of Chapter 6: the register map, the transport interface (`driver::transport::Interface`), and the driver interface (`driver::uart::Interface`) that the driver implements. The chapter also assumes the register semantics of Chapter 5, since the driver is the other side of that contract, and inheritance, override and dependency injection from modern embedded C++.

Read §7.1, which explains the register read and write core and the driver built over it. Re-read the register semantics of the protocol (§A.2.3): every method in this chapter is one half of a promise the VHDL already keeps.

Check that the driver compiles by running the compiler by hand, from fw/:

```
cd fw
g++ -std=c++17 -Wall -Wextra -Werror -Iinclude -fsyntax-only source/driver/uart/↵
    ↵ uart.cpp
```

That the compiler is run by hand is expected, and worth pausing on. `make build-cpp` has nothing to do for fw/ yet: `make build` waits for the provided `source/main.cpp` that arrives in Chapter 8, and the provided suite is guarded on four headers, two of which, `driver/transport/stub.hpp` and `driver/uart/blocking.hpp`, belong to Chapter 8, so it stays switched off for one more chapter. Until then the compiler is the only thing checking this code, which is exactly why the byte order and the three-access read path are reasoned through rather than guessed at.

`readReg` and `writeReg` are the whole protocol in about sixty lines, and the one place byte order can go wrong; `read()` matters as much, because the poll, read and separate pop is the contract of Chapter 5 showing up on the other side of the wire. The exercises in §7.3 declare class `Uart final : public Interface` (Exercise 7.1), implement the two private register helpers (Exercise 7.2), and implement every public method of the interface over them (Exercise 7.3); the remaining public methods follow the same shape as `read()`.

7.1 Building the Driver

Chapter 6 designed the three contracts: the register map, the `driver::transport::Interface` seam, and the `Interface` the application codes against. This chapter puts them to work by implementing the `Uart` driver over the seam: the private register core that turns each access into one five-byte transaction, and the public methods built on it. Proving it, with a scripted stub and the provided host suite, is Chapter 8.

Everything here is written by you. The exercises give the precise build instructions, and this section explains the ideas they rest on. Nothing is run yet: the suite that grades this code needs the stub of Chapter 8 before it will do anything, which is why the byte order and the ordering of the three RX accesses are worth getting right by reasoning rather than by trial.

7.1.1 The register protocol: reading and writing a register

The driver's private core is two operations that turn each register access into one **five-byte transaction**, as defined in Part 3 of the protocol (§A.3). Both move their bytes through the injected `driver::transport::Interface`.

Every transaction is exactly five bytes: one command byte, then four data bytes. The command byte carries the operation type in bit 7, 0 for a register read and 1 for a register write, the register index in the low nibble, and any unused bits set as the protocol requires.

A **register read** sends a command byte with the write bit clear and the register index in the low nibble, then exchanges four dummy bytes after it. Whatever came back while the command byte was going out is ignored; the four bytes returned during the dummy exchanges are the answer, and they assemble into a 32-bit value with the first returned byte as the most significant and the last as the least significant. This relies on the peripheral's latch-once-then-shift-out behaviour: the register value is captured at the end of the command byte, once the address is known, so all four returned bytes belong to the same captured value.

A **register write** sends a command byte with the write bit set and the register index in the low nibble, then splits the 32-bit value into four bytes and sends them most significant first, least significant last. The whole command and value goes out as one five-byte transaction, and the peripheral commits the new register value once the fifth byte has been received.

Both operations go through `driver::transport::Interface`, so the driver never depends on a particular SPI or hardware implementation. Each transmitted byte passes through the injected transport, and the byte each exchange returns is what the read operation collects. That separation is what makes the register protocol easy to test against a fake transport.

Byte order is critical. Both reads and writes are most significant byte first. A read must reconstruct the result in the same order the peripheral shifts it out, and a write must split the value in the order the peripheral expects; reverse the bytes and you have a different 32-bit value, even though the same four bytes crossed the wire. It is the detail most likely to be implemented incorrectly, and one of the tests exists specifically to catch it, using a value whose reversed byte sequence is visibly different to the hardware.

7.1.2 The driver

Uart implements the Interface of Chapter 6 using the private register read and write core and the register map you transcribed in Chapter 6. Its public methods stay short, because the register bank built in Chapter 5 already provides most of the UART semantics.

Writing one byte reads `STATUS`, checks whether the transmitter can accept another byte, and if it can, writes the byte to `TX_DATA` and reports that it was accepted. If it cannot, the driver writes nothing and reports that the operation could not be completed yet. The driver never manages the transmit FIFO directly; it watches the ready flag and writes the next byte when permitted.

Reading one byte reads `STATUS` and checks whether a valid byte is waiting in the receive FIFO. If one is, it reads the byte from `RX_DATA`, keeps it as the result, performs a separate write to `RX_POP`, and returns the byte to the caller. If no byte is available it neither reads `RX_DATA` nor writes `RX_POP`, and simply reports that nothing was available.

That receive sequence is deliberately three steps: poll `STATUS`, read `RX_DATA`, write `RX_POP`. Reading `RX_DATA` is a pure read, returning the byte at the front of the FIFO without removing

it, so repeated reads return the same byte until an explicit pop happens. Writing `RX_POP` is what advances the FIFO, removing the current byte only after it has been read successfully and making the next queued byte visible through `RX_DATA`.

The order matters in every direction. Polling first prevents the driver from reading an empty FIFO; reading before popping ensures the current byte is not discarded; popping afterwards ensures the next call observes the next byte. Omit the pop and the same byte comes back indefinitely, pop before the read and the current byte is skipped, and hiding the pop inside the `RX_DATA` read would violate the register semantics defined in §5.1. The read path is therefore the delicate one, and it mirrors the register bank's required **read-then-pop** sequence exactly: establish that data is available, read it without side effects, then issue a separate command that advances the FIFO.

Transport injection keeps the driver hardware-independent. The `Uart` receives a `driver::transport::Interface` through its constructor and stores it by reference rather than constructing or owning a concrete SPI implementation, and every register read and write passes through it. The UART logic therefore depends only on the transport interface, never on AVR registers or SPI-specific code, and the same `Uart` runs over the stub transport Chapter 8 scripts under it, over any fake or mock a test cares to write, and over the real AVR SPI transport introduced in Chapter 9. Nothing above the seam changes: the register protocol, the UART behaviour and the public `Interface` implementation stay identical, and only the injected transport differs.

7.1.3 What's ahead

The exercises in §7.3 implement the `Uart` class, the private register core, and every public method of the interface of Chapter 6. Chapter 8 then scripts a transport under it and turns the provided host suite green, and Chapter 9 replaces that stub with `AvrSpi` over the ATmega328P's real SPI, with nothing above the seam changing.

7.2 Review

What you should be able to do now

After this chapter, you should be able to:

- **Implement `readReg()` and `writeReg()`** as byte sequences that match the protocol exactly, and explain why the four data bytes go most significant first.
- **Explain what `const` does and does not reach** through a reference member, and justify the `const_cast` at the one line where a `const` method starts driving hardware.
- **Implement the `Uart` methods** over that register core, including the poll, read and `RX_POP` split, injecting the transport through the constructor.
- **Say what each of the three `RX` accesses would break** if it were omitted, reordered, or folded into the one before it.

Questions to test yourself

You should be able to explain:

1. A `writeReg()` sends the four data bytes least significant first: what does the peripheral do with the value, and at which point in the book would you first find out?
2. `readReg()` is `const` and calls `begin()`, `transfer()` and `end()`, which are not. Explain why that compiles with no cast at all, and why the book writes the cast anyway.
3. The read path is poll STATUS, read RX_DATA, write RX_POP. Take each of the three away in turn and say what the caller observes.

Looking ahead

In Chapter 8 the driver gets a wire it can be tested against: a scripted `driver::transport::Stub`, the provided host suite that drives the real Uart over it with no hardware in sight, and the blocking helpers built on top of the non-blocking core.

7.3 Exercises

How to work these. These exercises implement the Uart driver over the contracts of Chapter 6: the private register core that turns each access into one five-byte transaction, and every public method of the interface built on it.

How to check your work. Nothing runs yet. The provided host suite, `fw/test/uart/uart_test.cpp`, is guarded on four headers, two of which belong to Chapter 8, so `make test` still reports there is nothing to do; the check on this work arrives in the next chapter, and until then the specification below is the contract. `make build` has nothing to build yet either, since it waits for the `source/main.cpp` that Chapter 8 provides, so check that the driver compiles by hand, from `fw/`: `g++ -std=c++17 -Wall -Wextra -Werror -Iinclude -fsyntax-only source/driver/uart/uart.cpp`.

Every exercise in this chapter is code, so Appendix B has nothing to add here.

Work in `fw/`. You will add:

```
include/
  driver/
    uart/
      uart.hpp
source/
  driver/
    uart/
      uart.cpp
```

The driver lives in the `driver::uart` namespace. These conventions apply to `include/` and `source/`, which have to cross-compile for the ATmega; `test/` and `include/arch/test/` are host-only and never reach `avr-gcc`, which is why the provided suites next to them use `<stdint>`, `std::`, `namespace driver::uart::test` and `[[nodiscard]]` freely. Follow the same AVR-portable conventions as in Chapter 6 (§6.3): `<stdint.h>` and bare `uint8_t` and `size_t` (no `std::`), nested namespace blocks (not `namespace driver::uart { ... }`), and no `[[nodiscard]]`.

EXERCISE SET 1

The Uart Driver

Exercise 7.1 The class

C++

In `include/driver/uart/uart.hpp`, declare `class Uart final : public Interface`. It holds the transport as a reference member, `transport::Interface& myTransport`, injected through the constructor, so that the same driver runs over the stub of Chapter 8 and over the real SPI of Chapter 9; the constructor therefore takes a `driver::transport::Interface&` and stores it. Delete copy and move, since a reference member cannot be reassigned and the intent is worth making explicit. Declare each `Interface` method with `override`, keeping the `noexcept` and `const` qualifiers from the interface exactly.

Put the two private register helpers (Exercise 7.2) in the private section. Implement the bodies in `source/driver/uart/uart.cpp`.

Exercise 7.2 The register access core (private)

C++

Add two private helpers. They are the **only** methods that touch `myTransport`, and each performs one five-byte SPI transaction from Part 3 of the protocol (§A.3).

`readReg(index)` returns a register's 32-bit value. It begins the transaction with `myTransport.begin()`, then sends the command byte, which is the register index with the write bit clear (bit 7 is 0), using `myTransport.transfer(command)` and ignoring the byte that comes back, since the reply to the command byte is meaningless. It then clocks out the four data bytes by calling `myTransport.transfer(0x00)` four times, sending dummy zeros, and assembles the four returned bytes into the result **most significant byte first**, so the first byte back is bits 31 to 24. Finally it ends the transaction with `myTransport.end()` and returns the assembled value.

Every byte here moves through the injected `myTransport`, using `myTransport.transfer()`, the raw duplex SPI exchange. Do **not** call the driver's own `write()`: that is the public "send a UART byte" operation, a different thing entirely, and calling it here would recurse.

`writeReg(index, value)` commits a 32-bit value to a register. It begins the transaction, sends the command byte, which is the register index with the write bit set (bit 7 is 1), via `myTransport.transfer(command)` while ignoring the reply, then sends the four bytes of

value **most significant first** (bits 31 to 24 first), each with `myTransport.transfer(...)` and each reply ignored, and ends the transaction.

Make `readReg` a `const` member: it does not modify the `Uart` object; it exchanges bytes through the transport, whose state is its own. `writeReg` need not be `const`. Marking `readReg` `const` is exactly what lets `status()` and `errorFlags()` be `const`.

Most significant first, in both helpers, is the detail the byte-order test exists to catch. A reversed value is a different value to the hardware.

Reaching a non-const transport from a const method (`const_cast`)

`readReg` is `const`, but the transport's `begin()`, `transfer()` and `end()` are **not**: driving SPI is an action with side effects, not an observation. So a `const` method that has to perform a register read must reach a non-const operation, and this book writes that reach out explicitly:

```
uint32_t Uart::readReg(uint8_t addr) const noexcept
{
    auto& transport = const_cast<transport::Interface&>(myTransport);
    transport.begin();
    // ... transport.transfer(...) ...
    transport.end();
    // ...
}
```

Read that `cast` carefully, because it is doing less than it looks like it is doing. **The enclosing `const` never reached the transport in the first place.** `myTransport` is a *reference* member, and `const` does not propagate through a reference: inside a `const` method, `myTransport` still names a plain `transport::Interface&`, so the compiler would accept `myTransport.begin()` with no cast at all. The cast yields the type the expression already had. In general, `const_cast` is what you reach for when a logically `const` method (an observer such as `status()`) has to invoke a non-const operation underneath, and you have decided the method should stay `const` to its callers. It is a deliberate compromise, and worth understanding precisely.

The alternatives are marking the method non-const, which is honest but then means `status()` cannot be `const`, or using a mutable member, which is right for a cache and awkward for a reference. `const_cast` keeps the public `const` promise while admitting that the implementation must drive something underneath. So why write a cast the compiler does not ask for? Because it makes the intent explicit at the one line where a `const` method starts driving hardware, and because it becomes genuinely *required* the moment the transport is held differently — by value, or behind a `const`-propagating handle — at which point the object's `const` does reach the member and the calls stop compiling. Writing it now makes that change a one-word edit rather than a redesign. Be clear which of the two you are relying on: here it is documentation, not necessity.

The one hard rule is to cast away `const` only on an object that is not actually `const`. `myTransport` refers to a real, non-const transport the caller owns, so driving it is defined

behaviour; casting away `const` on a genuinely `const` object and then mutating it is undefined behaviour.

Exercise 7.3 The public methods

C++

Implement each Interface method in terms of the two helpers and the register map. The register semantics of Chapter 5 make them short.

`configure(baudDiv)` sets the baud rate first and enables afterwards: `writeReg(BAUD_DIV, baudDiv)`, then `writeReg(CTRL, 1U << ENABLE)`. Note the shift: `ENABLE` is a bit *position*, as every constant in the register map is, so writing it directly would send 0 and clear the register rather than set bit 0.

`write(byte)` reads `STATUS`; if `TX_READY` is set it calls `writeReg(TX_DATA, byte)` and returns `true`, and otherwise it writes nothing and returns `false`.

`read(byte&)` reads `STATUS`; if `RX_VALID` is set it reads `RX_DATA` into the out-parameter, then writes `writeReg(RX_POP, 1)` to advance the FIFO, and returns `true`. If `RX_VALID` is clear it returns `false` and issues **no** `RX_POP`.

That success path is three distinct register accesses, in this order, which is the contract of §5.1. **Poll** `STATUS` to learn whether a byte is even available. **Read** `RX_DATA`, which is a *pure read*: it returns the front byte but does **not** remove it from the FIFO. Then **pop** by writing `RX_POP`, a separate write that discards the front byte so the next call sees the next one.

Because `RX_DATA` does not pop on its own, the `RX_POP` write is not optional. If you leave it out, whether by forgetting it or by assuming the read already advanced the FIFO, the front byte is never discarded, so every following read polls `RX_VALID`, finds it still set, re-reads `RX_DATA`, and returns the **same byte forever**. Popping *before* the read is the opposite mistake: it discards the current byte unread and skips data.

The last three are one-liners. `status()` returns `readReg(STATUS)`, `errorFlags()` returns `readReg(ERROR_FLAGS)`, and `clearErrors()` calls `writeReg(ERROR_FLAGS, 0)`.

What's ahead

Chapter 8 scripts a transport under this driver: `driver::transport::Stub`, the provided host suite that drives the real Uart over it, and the blocking helpers on top. Every mistake this section warns about — the reversed byte order, the missing `RX_POP`, the bit position used as a mask — is something a case in that suite exists to catch.

The Driver, Tested

The wire, scripted, and the first make test the C++ half of the book has had.

IN THIS CHAPTER

- **The scripted stub:** `driver::transport::Stub`, a concrete `driver::transport::Interface` that records every byte the driver sends and plays back bytes a test queued for it. It is the C++ counterpart of a self-checking testbench.
- **Host testing with QAcademy Test:** the provided suite constructs a `Stub`, constructs a `Uart` over it, scripts the replies, calls the real driver, and asserts on the exact transactions that came out — the five-byte framing, the command byte, the byte order, and the “no `RX_POP` on empty” rule.
- **Testing the test double first:** four `TransportStub` cases run before any driver case, because every `Uart` case reads what the stub was scripted to reply, so a stub bug reports itself as a driver failure.
- **Blocking write and read** as thin free functions over the non-blocking core, taking `Interface&` rather than `Uart&`, plus a provided demo that exercises the whole API end to end.

Chapter 7 built the driver and nothing ran it. This chapter supplies the wire, scripted, and turns on the first `make test` the C++ half of the book has had. After it you should be able to build a scripted stub and use it to assert the exact transactions the driver produces, including the byte order and the “no `RX_POP` on empty” behaviour; explain what a test double has to be faithful about and what it may ignore, and why the stub’s two buffers are linear logs rather than FIFOs; read a red test and place the bug, telling a fault in the stub’s script from a fault in the driver it is scripting; and build blocking helpers over a non-blocking core, and explain why the spinning lives above the driver rather than inside it.

Before you start, you need the `Uart` of Chapter 7 and, through it, the three contracts of Chapter 6; the driver is the thing under test, so it has to exist before the suite can say anything about it. The chapter also assumes unit and component testing with stubs and dependency injection, applied here to a real driver; embedded software testing is recommended but not required.

Read §8.1, which explains the scripted stub, what the provided host suite pins down, and why the blocking helpers sit above the interface rather than inside the driver. Then read

the provided suite itself, `fw/test/uart/uart_test.cpp`: it is the executable specification of everything Chapter 7 wrote, and reading it before writing the stub tells you exactly which member names the stub owes it. The suite comes on the moment the last header exists:

```
make build-cpp          # builds fw/, runs its QAcademy Test suite
```

Get the four `TransportStub` cases green first, then watch the `Uart` cases of Chapter 7 go green behind them without a line of driver code changing — which is the point of having built the driver against an interface rather than against SPI. Of the stub, `injectRxWord` deserves the most care: most significant first there is the mirror of most significant first in `readReg`, and a mistake in one looks exactly like a mistake in the other.

The exercises in §8.3 finish the scripted Stub and get the whole host suite green (Exercise 8.1), add `writeBlocking` and `readBlocking` (Exercise 8.2), then copy in the provided `main.cpp` and run it (Exercise 8.3).

8.1 Testing the Driver on the Host

Chapter 7 built the `Uart`: the private register core and every public method of the interface of Chapter 6. Nothing has run it. This chapter supplies the missing half — a transport that can be scripted, and the provided host suite that drives the real driver over it — so that a byte-order or sequencing bug is caught on your machine in milliseconds, long before a logic analyzer is involved.

The stub is written by you; the host suite is provided and is the check, exactly as the `*_tb.vhd` testbenches were on the VHDL side. The exercises give the precise build instructions, and this section explains the ideas they rest on.

8.1.1 The scripted stub

To test the driver with no hardware, you build a stub that stands in for the `driver::transport::Interface` seam. It records every byte the driver sends and plays back bytes you have queued for it to return, so a test drives the *real* driver and then asserts the exact transactions it produced. This is the C++ counterpart of a self-checking testbench: the stub is the wire, scripted.

The stub is a concrete `driver::transport::Interface`. The class is named `driver::transport::Stub`, it publicly inherits the interface so that a `Uart` accepts it wherever the seam is expected, and it overrides the three seam methods, `begin()`, `transfer()` and `end()`. It is injected into the `Uart` through the constructor, exactly like the real transport of Chapter 9. It lives in `include/driver/transport/stub.hpp`, beside the interface it implements, so that the host demo in `source/main.cpp` can drive the driver with no hardware too.

On the MOSI side it records what the driver sends. Every byte passed to `transfer()` is appended to a log the test can read back, and that log is the exact sequence of bytes the driver put on the wire, so a test asserts against it to check the five-byte transaction, the

command byte and the byte order. Counting `begin()` and `end()` calls is part of the contract the provided suite asserts on: it checks `beginCalls()` in eight of its cases, which confirms each transaction was framed exactly once, and `endCalls()` in two, which confirms the two stay balanced.

On the MIS0 side it plays back what the driver reads. The test loads a queue of bytes before the driver runs, each `transfer()` call returns the next byte from that queue, and once the queue is empty `transfer()` returns `0x00`. That is how a test makes `readReg` observe a chosen register value: queue the four bytes, most significant first, that the peripheral would have shifted out. The suite reaches for that often enough that the stub owes it a word-at-a-time helper, which queues a 32-bit register value most significant byte first, and a reset that clears the queued script between phases.

The result is a self-checking, hardware-free test. It constructs a `Stub`, constructs a `Uart` over it, and scripts the responses; it calls the real driver methods under test; and it then asserts on the stub's recorded bytes and on the driver's return values. No SPI, no AVR and no bench are involved, and the whole driver is exercised on the host.

8.1.2 What the host tests pin down

Once the driver and stub exist, the provided host suite checks the behaviour that matters:

- **The transactions themselves.** A register read and a register write must produce the exact five-byte sequences, with the right command byte and four data bytes **most significant first**. This is the byte-order check.
- **Configuring** must write the baud divider first and then enable the peripheral.
- **`write()`** must read `STATUS` and write `TX_DATA` only when there is room, writing nothing when there is not.
- **`read()`, on valid data**, must read `STATUS`, then `RX_DATA`, then write `RX_POP`, in that order.
- **`read()`, on no data**, must report failure and issue **no** `RX_POP`.
- **The error flags** must read back, and clearing must write zero to them.

The “no pop on empty” and “bytes most significant first” checks are the two that catch the mistakes which would otherwise only surface on the bench.

The suite is all-or-nothing by design: it is guarded on all four headers existing, so it reports nothing to do until `register_map.hpp`, `uart.hpp`, `stub.hpp` and `blocking.hpp` are all present. The first four cases check the *stub*, before any driver case runs, and they are the ones to get green first. Every `Uart` case reads what the stub was scripted to reply, so a stub bug surfaces as a driver failure: queue a register value least significant first and the suite reports `Uart.WriteWhenReadyPushesTxData` red, pointing at the `readReg()` of Chapter 7, which is correct and unhelpful.

8.1.3 Blocking on top, not inside

`write()` and `read()` are non-blocking because that is what makes them deterministic to test: each one reports whether it did anything, and a test can drive both outcomes from the stub's script. The convenience of “send this byte, I'll wait” is real, though, so it is built *over* the interface as two free functions that spin on the non-blocking core until it succeeds.

Building them that way rather than into the driver keeps the core deterministic and keeps the spinning where a caller opts into it. They take `Interface&` rather than `Uart&`, so they work over any implementation — including the `driver::uart::Stub` of Chapter 6, which is what lets the `app::EchoNode` of Chapter 10 be tested with no driver at all underneath it.

8.1.4 What's ahead

The exercises in §8.3 build the scripted `Stub`, get the host suite green, add the blocking helpers, and run the provided demo. Chapter 9 then replaces the stub with `AvrSpi` over the ATmega328P's real SPI, and nothing above the seam changes.

8.2 Review

What you should be able to do now

After this chapter, you should be able to:

- **Build a scripted stub** and use it to assert the exact transactions the driver produces, including the byte order and the “no `RX_POP` on empty” behaviour.
- **Explain what a test double has to be faithful about** and what it may ignore, and why the stub's two buffers are linear logs rather than FIFOs.
- **Read a red test and place the bug**, distinguishing a fault in the stub's script from a fault in the driver it is scripting.
- **Build blocking helpers over a non-blocking core**, and explain why the spinning lives above the driver rather than inside it.

Questions to test yourself

You should be able to explain:

1. Why is the driver tested against a scripted `driver::transport::Stub` rather than the real SPI at this stage, and what class of bug does that let you find first?
2. `injectRxWord` queues its four bytes least significant first by mistake. Which cases go red, and why does the failure point at `readReg()` rather than at the stub?
3. The suite counts `begin()` calls in eight cases, and `end()` calls in two of them. What would be wrong with a driver that passed every byte-sequence assertion but failed those counters?

4. Why do `writeBlocking` and `readBlocking` take `Interface&` rather than `Uart&`, and which class of Chapter 6 does that choice make usable in Chapter 10?

Looking ahead

In Chapter 9 the bottom layer becomes real: the AVR's own SPCR, SPSR and SPDR, `AvrSpi` implementing `driver::transport::Interface`, and what a freestanding target removes. The driver, the interface, the register map and this suite all run unchanged.

8.3 Exercises

How to work these. These exercises supply the half Chapter 7 left out: a transport that can be scripted, so that the provided host suite can drive the real `Uart` with no hardware, plus the blocking helpers on top of the non-blocking core. You write the stub and the helpers. A small demo, `main.cpp`, is provided too, listed in full in Exercise 8.3.

How to check your work. The provided host suite, `fw/test/uart/uart_test.cpp`, run by `make test`, is the check, and it is the first one the C++ half of the book has been able to run. It is guarded on all four driver headers — `register_map.hpp` and `uart.hpp` from Chapters 6 and 7, `stub.hpp` and `blocking.hpp` from here — so it reports there is nothing to do until the last of them exists; adding the two files below is what switches it on.

Every exercise in this chapter is code, so Appendix B has nothing to add here.

Work in `fw/`. You will add:

```
include/
  driver/
    uart/
      blocking.hpp
      transport/
        stub.hpp
source/
  main.cpp          (provided; the host demo, copied from the last exercise)
test/
  uart/
    uart_test.cpp   (provided; the host suite, already in the repository)
```

The stub joins the transport interface in `driver::transport` (the demo may use a small app namespace). These conventions apply to `include/` and `source/`, which have to cross-compile for the ATmega; `test/` and `include/arch/test/` are host-only and never reach `avr-gcc`, which is why the provided suites next to them use `<cstdint>`, `std::`, `namespace driver::uart::` `test` and `[[nodiscard]]` freely. Follow the same AVR-portable conventions as in Chapter 6 (§6.3): `<stdint.h>` and bare `uint8_t` and `size_t` (no `std::`), nested namespace blocks (not

namespace driver::uart { ... })), and no `[[nodiscard]]`. Build the application with `make build` and the tests with `make test`.

EXERCISE SET 1

The Scripted Stub

Exercise 8.1 driver::transport::Stub

C++

In `include/driver/transport/stub.hpp`, build a header-only `driver::transport::Stub` that implements `driver::transport::Interface` so that the driver can be tested with no hardware. It is the wire, scripted, and it has three jobs.

The provided test suite binds to this stub by name, so the names below are part of the exercise rather than suggestions: a stub with the right behaviour but different member names will not compile against `test/uart/uart_test.cpp`. It needs a default constructor, since every test builds one with `transport::Stub stub{};`.

That suite opens with four `TransportStub` cases that check the stub itself, before any driver case runs. They are there because every `Uart` case reads what the stub was scripted to reply, so a stub bug surfaces as a *driver* failure: queue the four bytes least significant first, for instance, and the suite reports `Uart.WriteWhenReadyPushesTxData` red, pointing at `readReg()`, which is correct. Get the `TransportStub` cases green first and the failures below are genuinely yours.

It must **record** every call. Append each byte passed to `transfer()` to a fixed-size `uint8_t` buffer with a running length, where a capacity of 100 bytes is plenty for these tests. Expose that record through `txLen()`, returning the number of bytes captured, and `txByte(index)`, returning the byte at a position and 0 for an index past the end. Both must be **const**, because the tests inspect the stub through a `const transport::Stub&`. Keep two `uint16_t` counters as well, one bumped by `begin()` and one by `end()`, exposed as `beginCalls()` and `endCalls()`, also **const**. These are not optional: the suite asserts on `beginCalls()` in eight separate cases and on `endCalls()` in two, because they are what proves a transaction was framed exactly once and that the two stay balanced.

It must **play back** bytes on `transfer()`, returning the next byte from a preloaded response buffer tracked by an index, and returning 0x00 once that buffer is exhausted. This is the MIS0 the driver reads.

And it must offer helpers to script that response. `injectRxByte(uint8_t)` queues a single byte, and `injectRxWord(uint32_t)` queues a whole register value **most significant first**, so that a `readReg` sees exactly that value. `clearRxData()` resets the reply buffer and its index, so that one test can script several reads.

Give the two a *different name* rather than overloading one name on width. An integer literal converts equally well to `uint8_t` and to `uint32_t`, so an overload pair would make `stub.injectRx(0x41)` ambiguous and force every call site to spell out the type of its argu-

ment. Two names cost nothing and keep the call sites readable, which is why the UART stub of Chapter 6 is built the same way.

Two fixed `uint8_t` buffers with a length each, one for what the driver sent and one for the scripted replies, are all it takes; no dynamic containers are needed. Both are **linear buffers**, **not FIFOs**: the record buffer is a write-only append log the test reads back afterwards, and the reply buffer is read front to back through an advancing index. No circular wraparound, no pop-with-shift; just reset each length, and the read index, to zero at the start of a test.

Keep it dependency-injected: a test constructs a `Stub`, constructs a `Uart` over it, drives the driver, and then asserts against the `Stub`'s record.

EXERCISE SET 2

Using the Driver

Exercise 8.2 Blocking helpers

C++

The interface's `write()` and `read()` are non-blocking by design. Build the thin blocking convenience on top, in a new header `include/driver/uart/blocking.hpp`, as free functions in the `driver::uart` namespace. Keep the same AVR-portable style as the rest of the driver (`<stdint.h>` and bare types, nested namespace blocks). Both take `Interface&`, not `Uart&`, so they work over any implementation; `uart.write` and `uart.read` dispatch to the concrete driver at run time.

a) Write a byte, blocking

Add a free function `writeBlocking()` that sends one byte, waiting until the transmitter accepts it. It takes a `driver::uart::Interface&`, the driver, and a `uint8_t`, the byte to send; it spins on `uart.write(byte)` until that returns `true` and then returns nothing. Mark it `inline`, since it is defined in a header and that avoids a one-definition-rule violation once more than one file includes it, and mark it `noexcept`.

b) Read a byte, blocking

Add a free function `readBlocking()` that receives one byte, waiting until one is available. It takes a `driver::uart::Interface&`, the driver, and a reference to a `uint8_t` where the received byte is stored; it spins on `uart.read(byte)` until that returns `true` and then returns nothing. It is `inline` and `noexcept` for the same reasons.

Each is a one-line spin loop, so header-only `inline` functions are all you need; no `.cpp`. They belong above the driver, not inside it: the core stays deterministic and testable, and the spinning lives where a caller opts into it.

Exercise 8.3 The provided demo

C++

Use this test program to exercise communication end to end. It is **provided**, like the host suite; you do not write it. Create `source/main.cpp` and copy it in as it stands:

```
/**
 * @brief Application entry point.
 */
#include <stdint.h>

#include "driver/transport/stub.hpp"
#include "driver/uart/blocking.hpp"
#include "driver/uart/register_map.hpp"
#include "driver/uart/uart.hpp"

using namespace driver;

/**
 * @brief Application entry point.
 *
 * On the host, the demo uses a scripted transport::Stub because no real
 * UART is available. It reports TX ready and returns one received byte so
 * the blocking calls can complete. On the ATmega328P (L09), the real
 * transport reports the actual hardware status.
 *
 * Each register read transfers one command byte followed by four data
 * bytes, so scripted responses begin with a command-phase placeholder.
 * Because writes also consume scripted bytes, each response is installed
 * immediately before use and clearRxData() resets the script between
 * phases.
 *
 * @return 0 on termination of the program.
 */
int main()
{
    constexpr uint16_t baudDiv{27U};
    constexpr uint8_t txByte{0x7FU};
    constexpr uint8_t dummyCmd{0U};

    constexpr uint32_t rxData{txByte};
    constexpr uint32_t txReady{static_cast<uint32_t>(1U << uart::status::TX_READY) ←
    ↪ };
    constexpr uint32_t rxValid{static_cast<uint32_t>(1U << uart::status::RX_VALID) ←
    ↪ };
```

```

transport::Stub transport{};
uart::Uart uart{transport};
uart.configure(baudDiv);

// Report TX ready, then send a byte.
transport.injectRxByte(dummyCmd);
transport.injectRxWord(txReady);
uart::writeBlocking(uart, txByte);

// Report RX valid and hand back the byte, then receive it.
transport.clearRxData();
transport.injectRxByte(dummyCmd);
transport.injectRxWord(rxValid);
transport.injectRxByte(dummyCmd);
transport.injectRxWord(rxData);

uint8_t rxByte{};
uart::readBlocking(uart, rxByte);
return 0;
}

```

It shows the API end to end: it constructs a `Uart` over a `driver::transport::Stub`, `configure()`s it, then uses your `writeBlocking` and `readBlocking` to send a byte and receive one back. Because the blocking calls spin until the transport reports ready, the demo scripts the stub just before each call so that it completes instead of spinning forever. On the target there is nothing to script: Chapter 9 writes a separate freestanding main in `fw/avr/` that builds the same `Uart` over `AvrSpi`, and the real transport reports the actual hardware status.

Read it, then run `make build` and `./uart_firmware`: it exits cleanly once your `Uart`, `Stub` and blocking helpers are correct. The demo is illustration, not a test; the real checking is `make test`.

What's ahead

Chapter 9 replaces the stub with `AvrSpi`, the real SPI on the ATmega328P, and the driver, the interface and the register map all run unchanged over the wire. The suite you just turned green runs unchanged too, which is the return on putting the seam where Chapter 6 put it.

The Real Transport, and Both Toolchains

The bottom layer becomes real; nothing above the seam changes, and both chips get programmed.

IN THIS CHAPTER

- **The AVR's own SPI peripheral:** SPCR, SPSR and SPDR as genuine memory-mapped volatile registers, configured as an SPI master at $f_{\text{osc}}/16$, which is 1 MHz, mode 0, most significant bit first, matching Part 3 of the protocol (§A.3).
- **AvrSpi on top of it,** implementing the same `driver::transport::Interface` seam that the `driver::transport::Stub` of Chapter 8 did, only now clocking real bytes. Nothing above the seam changes, which is the whole return on the design work of Chapter 6.
- **What a freestanding target removes,** namely the heap, exceptions, RTTI and iostreams, and what replaces each: static storage, return codes, virtual dispatch in place of `dynamic_cast`, and logging over the AVR's own hardware UART.
- **Building with `avr-gcc` and flashing with `avrdude`.**
- **The other toolchain:** synthesizing `uart_board.vhd` in Quartus, assigning pins, and programming the DE0-CV over USB-Blaster. Both halves of the course reach real hardware in this chapter, and doing the FPGA side here rather than on bring-up day means a toolchain problem surfaces while there is still time to solve it.

This is where the stub gives way to real hardware. After this chapter you should be able to configure the AVR SPI master registers correctly for the transport contract, and explain each bit they set; implement `AvrSpi` so that the existing driver and its host tests are reused unchanged; explain which host-side idioms do not survive a freestanding build, and what replaces them; and synthesize and program the DE0-CV, and read a fit and timing report well enough to say whether the design met timing at 50 MHz.

Before you start, you need the `driver::transport::Interface` seam of Chapter 6, which is what the new transport implements, plus the `Uart` driver of Chapter 7 and the host tests of Chapter 8, both reused here unchanged. You also need the finished `uart_top` of Chapter 5,

passing its system testbench, because the second half of the chapter puts that design on the FPGA. From embedded C the chapter assumes the AVR toolchain, `avr-gcc` and `avrdude`, and register-level programming; from modern embedded C++, the `volatile` semantics of memory-mapped registers. Install **Quartus Prime Lite** before you start: it is a multi-gigabyte download, and it is the one tool the book has not needed until now.

Read §9.1 for the SPI peripheral and `AvrSpi`, and §9.2 for the freestanding target and the toolchain. Alongside them, read the ATmega328P datasheet’s SPI section covering `SPCR`, `SPSR` and `SPDR`, and the provided `avr/` runtime and Makefile under `fw/` (`fw/README.md`).

Write `AvrSpi`, confirm that the host tests still pass unchanged with `make build-cpp`, then cross-compile and flash a minimal bring-up main to see the toolchain end to end:

```
make -C fw/avr flash # avr-gcc + avrdude; see fw/README.md.
```

`AvrSpi` is about forty lines; go through `SPCR` bit by bit as you write it, because a wrong bit here is a bug you would otherwise meet on a logic analyzer at the bench.

Then take the same journey on the other chip: synthesize the `uart_top` finished in Chapter 5 through the provided `uart_board.vhd` wrapper, read the fit and timing report, and program the DE0-CV over USB-Blaster. What the board can show on its own is the data-plane pin loopback: with the wrapper’s `rx` pin wired straight to its `tx` pin, bypassing `uart_top`, a terminal on the USB-serial adapter echoes what you type. The peripheral itself cannot be exercised yet: it transmits only what an SPI master writes to `TX_DATA`, and that master is the Nano, in Chapter 10.

The exercises in §9.4 implement `AvrSpi` (Exercise 9.1) and the freestanding bring-up main (Exercise 9.2), add logging over the AVR’s own hardware UART and work out one `readReg()` round trip against the 1 MHz `SCK` budget, measuring it on a logic analyzer if you have one (Exercise 9.3). Synthesize and program your own `uart_top` as well (Exercise 9.4), so that both chips are running your code before you reach the bench.

9.1 The SPI Peripheral and `AvrSpi`

This is the bottom of the stack, and the one place the code finally touches real hardware. Everything above the seam, the register map, the Interface, the `Uart` driver, was built and host-tested in Chapters 6 to 8 against a scripted `Stub`. Here you implement the *other* `driver::transport::Interface`, the one that drives the ATmega328P’s own SPI peripheral, so that exactly the same driver runs over a real 1 MHz SPI bus with nothing above the seam changing.

`AvrSpi` is deliberately thin: it configures the SPI master once, then translates the three seam calls into register accesses. All the protocol knowledge (five-byte transactions, the command byte, the byte order) already lives in the driver above it.

9.1.1 The SPI registers

The ATmega328P's SPI is three volatile memory-mapped registers. This is the volatile lesson from modern embedded C++, now on the master side of the wire: the compiler must not cache or reorder these accesses, because each one is a real bus event.

Register	Bit	Meaning
SPCR	SPE	SPI enable.
SPCR	MSTR	Master (1) or slave (0).
SPCR	DORD	Data order: 0 for most significant bit first, 1 for least.
SPCR	CPOL	Clock polarity: 0 means SCK idles low.
SPCR	CPHA	Clock phase: 0 means sample on the leading edge.
SPCR	SPR1, SPR0	SCK rate select (with SPI2X).
SPSR	SPIF	Transfer complete: set when a byte has finished shifting.
SPSR	SPI2X	Double the SCK rate.
SPDR	—	Data register: write to start a transfer, read to get the byte clocked in.

The transport contract of Part 3 of the protocol (§A.3) pins every one of these. The ATmega is the **master**, running **mode 0** (CPOL = 0, CPHA = 0), **most significant bit first** (DORD = 0), with SCK at $f_{\text{osc}}/16 = 1 \text{ MHz}$, the Nano's 16 MHz clock divided by 16 (SPR1:SPR0 = 01, SPI2X = 0).

1 MHz is not a limit the FPGA imposes: spi_slave synchronizes scl_k into the 50 MHz domain and edge-detects it, so it keeps up with far more than this. It is simply the nearest prescaler value below the rate at which a breadboard and a passive level shifter stop being reliable, and it leaves comfortable margin for both.

9.1.2 Configuring the master

Configuration is a one-time setup of SPCR, plus the port pins. On the ATmega328P the SPI pins are fixed: SCK is PB5, MOSI PB3, MISO PB4, and the hardware SS PB2. Two details matter. The first is direction: SCK, MOSI and SS are outputs while MISO is an input, and you set that in DDRB. The second is that SS (PB2) **must be an output** in master mode. If it is left an input and something pulls it low, the peripheral clears MSTR and demotes itself to a slave mid-flight. PB2 doubles as the chip select to the FPGA, so driving it is the transport's job anyway.

9.1.3 Releasing the registers on destruction

The constructor is the one place that *sets* bits, so the destructor is the one place that clears them: AvrSpi acquires the SPI hardware when it is constructed and **must release it when it is destroyed**, leaving the registers exactly as the reset left them. Clear every bit the constructor set, and nothing else: the DDRB direction bits for SCK, MOSI and SS, putting them back to inputs; the SS bit in PORTB, which is the chip-select drive and its pull-up; and all of SPCR, written to 0, which disables the SPI peripheral.

MISO was never configured, so leave it untouched. The destructor is noexcept and, because the transport Interface declares a virtual destructor, is marked override, so destroying an

AvrSpi through an Interface& still runs it. This is plain RAI: whatever the constructor took, the destructor gives back, so the next owner of those pins — a second AvrSpi, or any other code — starts from the power-on state rather than a half-configured SPI master.

9.1.4 The three seam methods

The seam maps directly onto the peripheral. `begin()` drives the chip select low (SS, PB2), opening a transaction, and `end()` releases it high again, closing it. Between them, `transfer(byte)` exchanges one byte in full duplex: write the byte to SPDR, wait for the hardware to finish, then read SPDR for the byte that arrived on MISO.

That wait is the whole subtlety. A byte takes eight SCK periods to shift, and SPDR does not hold the received byte until the transfer completes, signalled by SPIF going high:

```
SPDR = byte;
while (0U == (SPSR & (1U << SPIF))) {} // Spin until the transfer completes.
return SPDR;                          // Now SPDR holds the received byte.
```

Read SPDR *before* SPIF is set and you get stale data from the previous exchange, so the poll is not optional. (Reading SPSR then SPDR also clears SPIF, arming the next transfer.) This is the same three-method seam the Stub of Chapter 8 implemented; the difference is that these bytes are real.

9.1.5 Testing it on the host

The transport touches hardware, but its *logic* is still worth pinning without a bench, so it is host-tested exactly as the Stub of Chapter 8 was, over a mocked register file. On the target, AvrSpi reaches the registers through a platform header (`arch/avr/hw_platform.hpp`), which forwards to `<avr/io.h>` and resolves SPCR, SPSR, SPDR, DDRB and PORTB to the real memory-mapped hardware. On the host, that same platform header selects a provided mock instead, under `-DHOST_TEST`, which backs DDRB, PORTB and SPCR with plain storage and makes SPDR and SPSR proxy objects, so that reading them has an effect the way it does on the part. It records the bytes written to SPDR as a MOSI log and returns a scripted byte on each SPDR read from a MISO queue, and it models the one property that matters here: a transfer takes time. Writing SPDR starts it, SPIF stays **clear**, and only a read of SPSR — the driver's own poll — completes the transfer and makes the received byte readable. Delete the poll and `transfer()` hands back a stale byte (on the mock, the value left from reset, since without the poll no transfer ever completes), and the suite goes red. It is the direct counterpart of the Stub.

With that mock, a provided host suite pins the register-level behaviour the bench cannot easily show: the master configuration bits, the chip-select framing (`begin` low, `end` high), the SPIF poll, and that `transfer` returns the byte the mock presented, in order. Only the register file needs mocking; the transport uses no interrupts and no `F_CPU`-derived timing, so neither `<avr/interrupt.h>` nor `F_CPU` comes into it. The whole transport is then proven end to end on the bench in Chapter 10.

9.1.6 What's ahead

§9.2 covers the target the transport is built for: a freestanding avr-gcc build, what it removes, the `env.cpp` runtime stubs, and flashing with avrdude. The exercises in §9.4 implement AvrSpi, the freestanding bring-up main, and UART-based logging.

9.2 The Freestanding Target and the Toolchain

The host build (`g++`) links against a full hosted C++ implementation: a heap, exceptions, RTTI, iostreams, the whole standard library. The ATmega328P has none of that. Its build is **freestanding**: avr-gcc, no operating system, and only the thin runtime avr-libc provides. This section is about what that removes, why the driver survives it unchanged, and how you build and flash.

9.2.1 What freestanding removes, and what replaces it

A freestanding avr-gcc build is compiled with `-std=c++14 -mmcu=atmega328p -Os -DF_CPU=16000000UL -fno-exceptions -fno-rtti -fno-threadsafe-statics`, plus `-ffunction-sections -fdata-sections` and a linker `-Wl,--gc-sections` so that the linker can drop what virtual dispatch pulls in on a 32 KB part. Everything the driver uses is C++11, so the standard here is a floor rather than a requirement; `-std=c++14` is chosen because every avr-gcc in circulation accepts it, while `-std=c++17` is not even a recognized option on the older toolchains, including the one Microchip Studio ships. Three things go away with the rest of the flags. **The heap** goes first: avr-libc does provide `malloc` and `free`, but there is no C++ operator `new` or operator `delete`, so dynamic allocation is off the table in practice and you use static and automatic storage. **Exceptions and RTTI** go next, which means no `throw`, no `dynamic_cast` and no `typeid`, and errors become return values, which is exactly what the driver already does when `write` returns `false` on a full FIFO. Finally **iostreams and most of the standard library** go, so there is no `std::cout`, and logging goes out a real UART instead, as described below.

The payoff of the AVR-portable style of Chapter 6 lands here. Because the register map, Interface, Uart, Stub and the blocking helpers were all written with `<stdint.h>` and bare types, no `std::`, nested namespace blocks, and none of the newer C++ attributes the AVR toolchain rejects, they cross-compile to the target **untouched**. Nothing above the seam is rewritten for the AVR; only the transport below it is new. That is the whole point of having built portable in the first place.

9.2.2 `env.cpp`, the runtime the compiler still expects

avr-libc ships no C++ runtime, yet the compiler still emits references to a few runtime symbols for ordinary C++ constructs, and the link fails without them. The provided `avr/env.cpp` hand-defines three groups of them. `operator delete` is needed because a class with a virtual destructor names the deleting operator `delete` in its vtable, even if you never

delete through a base pointer. `__cxa_pure_virtual()` is named by every abstract class's vtable, and is called only if a pure virtual is somehow invoked. And `__cxa_guard_acquire()`, `__cxa_guard_release()` and `__cxa_guard_abort()` form the guard around a function-local static's one-time initialization.

This build passes `-fno-threadsafe-statics`, so `avr-gcc` never emits the `__cxa_guard_*` calls at all; `env.cpp` defines them anyway, so that the link succeeds with or without the flag and on any toolchain that does emit them. `env.cpp` lives in `avr/`, not `source/`, on purpose: the host build already has these from `libstdc++`, so keeping the file out of the host link avoids replacing the host's thread-safe static guards with the single-threaded versions. The details are in `fw/README.md`.

9.2.3 Logging over the AVR's own UART

With no iostreams, debug logging goes out the ATmega328P's **own hardware USART** (its TXD and RXD pins, the "serial" a Nano exposes over USB). Do not confuse it with the UART peripheral this whole book builds: that one lives on the FPGA and the ATmega drives it over SPI; this one is the ATmega's built-in serial port, used here purely as a debug console to a PC terminal.

Using it is a small, poll-based routine: set the baud divider `UBRR0` from `F_CPU` and the target baud, enable the transmitter in `UCSR0B`, then for each byte wait for the data register to be empty (`UCSR0A & (1U << UDRE0)`) and write it to `UDR0`. This is why the logging path, unlike the SPI transport, needs `F_CPU`: the baud divider is computed from the clock.

9.2.4 Building and flashing the MCU

Two steps turn the source into something on the chip. First you **compile and link** with `avr-gcc` for `-mmcu=atmega328p`, then `avr-objcopy` the ELF into an Intel HEX image. Then you **flash** it with `avrdude` over the Nano's USB bootloader. Both are wrapped by the `avr-gcc` Makefile in `avr/`:

```
make -C fw/avr flash
```

`F_CPU` must match the board's actual clock, 16 MHz on a standard Nano, set by its crystal, with the fuses selecting that external oscillator as the clock source, because it drives every computed timing, including the debug UART's baud. The `avr/` build is cross-compiled and flashed only; it is excluded from host CI, while the transport's *logic* is still checked on the host, as §9.1.5 describes.

9.2.5 The other toolchain: getting the peripheral onto the FPGA

The FPGA half has the same journey, and this is where it happens, because the two are the same idea twice: source becomes an image, and an image gets programmed onto a part.

Everything up to now has been GHDL, which only ever simulated your VHDL. Simulation says the design is *correct*; it says nothing about whether it **fits** on the device or **meets timing** at 50 MHz. Only synthesis answers those, and the tool for the DE0-CV is **Quartus Prime Lite**.

`uart_top` is not the top level on the board. It has no notion of which physical pin `sclk` or `tx` is, so it is wrapped by `uart_board.vhd`, the provided Quartus top level that maps its ports onto DE0-CV package pins and feeds it the board's 50 MHz clock. That wrapper is board I/O rather than peripheral logic, which is why it lives with the Quartus project instead of in `hw/`.

The flow is: open the project, add your `hw/*.vhd` alongside the wrapper, **compile**, then read the two numbers that matter. **Fit** tells you the design is small enough, which for this peripheral it comfortably is. **Timing** tells you every path from flip-flop to flip-flop settles inside one 20 ns clock period; a design that simulates perfectly can still fail here, and that failure is invisible to every testbench you have run. Then **program** the board over USB-Blaster.

What the board can show with nothing else attached is less than it may seem. `uart_top` has no path of its own from `rx` to `tx`: it transmits only what an SPI master writes to `TX_DATA`, and it holds what it receives in the RX FIFO until a master reads it, so with no Nano on the SPI pins `BAUD_DIV` reads zero and `tx` simply idles high. The loopback of `uart_top_tb` worked because the testbench was that master. What can be checked alone is the path around the peripheral: wire the wrapper's `rx` pin straight back out to its `tx` pin, bypassing `uart_top`, re-synthesize, and a terminal on the USB-serial adapter echoes what you type. That proves the programming, the pin assignment, the adapter and its logic levels (not the terminal's baud rate: the adapter sends and receives at the same setting, so a loopback agrees with itself at any rate), and it is the first rung of the bring-up ladder of Chapter 10; the peripheral itself is first exercised there, once the Nano writes `BAUD_DIV` and `TX_DATA` over SPI.

9.2.6 What's ahead

The exercises in §9.4 implement `AvrSpi`, write the freestanding bring-up main, add UART logging, and measure a register round trip against the 1 MHz `SCK` budget. Chapter 10 then puts both halves on the bench.

9.3 Review

What you should be able to do now

After this chapter, you should be able to:

- **Configure the AVR SPI master registers** correctly for the transport contract, and explain each bit they set.
- **Implement `AvrSpi`** so that the existing driver and its host tests are reused unchanged.
- **Explain which host-side idioms do not survive a freestanding build**, and what replaces them.
- **Synthesize and program the DE0-CV**, and read a fit and timing report well enough to say whether the design met timing at 50 MHz.

Questions to test yourself

You should be able to explain:

1. Which idiom from the host build fails to compile freestanding, and how does this chapter replace it?
2. The AVR SPI runs at 1 MHz: which prescaler setting produces it from a 16 MHz clock, and what actually limits SCK on this bench, given that the FPGA slave could keep up with considerably more?
3. If SPDR is read a cycle too early, before SPIF is set, what value comes back, and how does the transport avoid it?

Looking ahead

In Chapter 10 both halves are on the bench: two links, a level shifter, and a bring-up ladder that ends with an echo application driven end to end.

9.4 Exercises

How to work these. Exercise 9.1 reinforces §9.1; Exercises 9.2, 9.3 and 9.4 reinforce §9.2. `AvrSpi` joins the other transport code in `driver/transport/`; the bring-up main and the logging go in the AVR-only tree `fw/avr/`, next to the provided `env.cpp` and `avr-gcc` Makefile. The runtime stubs, the Makefile and the host register mock are all provided.

`AvrSpi` reaches the SPI registers through the platform header (`arch/avr/hw_platform.hpp`, which forwards to `<avr/io.h>` on the target and to the mock under `-DHOST_TEST`), so the same source compiles for the target (real registers) and for the host suite (the mocked register file). Because it touches AVR registers, the host make build skips `avr_spi.cpp`; the AVR build compiles it for the target, and the host test suite compiles it against the mock. Keep the AVR-portable style of Chapter 6: `<stdint.h>` and bare types, nested namespace blocks, no `[[nodiscard]]`.

How to check your work. The provided host suite `fw/test/transport/avr_spi_test.cpp`, run by `make test`, checks `AvrSpi` over the mocked register file; the rest is checked on the hardware. The answers to every part that is not code are in §B.6. Try each part before you read its answer.

Exercise 9.1 `AvrSpi`

C++

a) In `include/driver/transport/avr_spi.hpp` (the declaration) and `source/driver/transport/avr_spi.cpp` (the definitions), create a new class `AvrSpi`, a second implementation of `driver::transport::Interface` (the first was the Stub of Chapter 8), beside `interface.hpp` and `stub.hpp`. Reach the registers through `arch/avr/hw_platform.hpp`.

Configure the SPI master once, from the bit table in §9.1.1: set the `DDRB` directions, with `SCK`, `MOSI` and `SS` as outputs and `MISO` as an input; idle the chip select (`SS`) high by writing to `PORTB`; and set `SPCR` for enable, master, mode 0, most significant bit first and $f_{\text{osc}}/16$.

Then give the destructor the mirror job: clear exactly the bits the constructor set — `SCK`, `MOSI` and `SS` in `DDRB`, `SS` in `PORTB`, and all of `SPCR` — so that an `AvrSpi` leaves the registers as it found them, the ATmega’s reset state (§9.1.3). Leave `MISO` alone; the constructor never touched it.

Do not restate the protocol; the driver above already owns it.

b) Implement the three seam methods: `begin()` drives `SS` low, `end()` drives it high, and `transfer(byte)` writes `SPDR`, spins until `SPIF` is set, then returns `SPDR`. Explain, in one sentence, why the spin cannot be skipped.

c) `SS` (`PB2`) is configured as an output even though the transport drives the chip select itself. What does the SPI peripheral do if `SS` is left an input and something pulls it low mid-transaction, and which `SPCR` bit changes?

d) A colleague writes `return SPDR`; immediately after `SPDR = byte;`, with no poll. What byte comes back, and why? Tie your answer to the transfer waveform: how many `SCK` periods must pass first?

e) The check: run the provided host suite (`make test`), which drives `AvrSpi` over the mocked register file and asserts the configuration bits, the chip-select framing, the `SPIF` poll, and the returned byte. Confirm that the driver suite of Chapter 8 still passes unchanged, since nothing above the seam moved. The transport itself is finally proven on the bench in Chapter 10.

Exercise 9.2 The freestanding bring-up main

C++

a) In `fw/avr/`, write a small `main()` that constructs a `Uart` over an `AvrSpi`, `configure()`s it for 115200 8N1 (`BAUD_DIV = 27`), sends a byte with `writeBlocking`, and reads one back. Cross-compile and flash it:

```
make -C fw/avr flash
```

b) The host demo (`source/main.cpp`) compiles unchanged in this freestanding build. Say why, then name the idioms of ordinary host C++ that would **not** compile here, and what replaces each on the AVR. (Consider `<cstdint>`, the heap, exceptions, RTTI, and `iostreams`.)

c) Delete `env.cpp` from the link and rebuild. Which symbol does the linker report as undefined, which C++ construct in your code emitted the reference to it, and why does the host build not need the file?

Exercise 9.3 Logging and timing

C++

a) Add debug logging over the ATmega328P's **own** hardware USART (not the FPGA peripheral): set `UBRR0` from `F_CPU` and the log baud, enable the transmitter in `UCSR0B`, then send each byte by waiting on `UCSR0A & (1 << UDRE0)` and writing `UDR0`. Log one line per register transaction, so that you can watch the driver work from a PC terminal.

b) (*Needs a logic analyzer; do the arithmetic either way and skip the capture if you have none.*) Put a logic analyzer on the four SPI lines and capture one `readReg()`. A register read is a five-byte transaction at 1 MHz SCK; work out how many SCK cycles that is and how long it should take, then compare against what you measure. Where does the extra time between bytes come from?

c) The SPI transport needs no `F_CPU`, but this logging does. Explain the difference: what sets the SPI SCK rate, and what sets the USART baud rate?

Exercise 9.4 Your own `uart_top` on the board

BENCH

The Quartus flow is where you synthesize your own design, so that the FPGA is already carrying your code when you reach the bench rather than on the day.

a) Open the provided Quartus project, add your own `hw/*.vhd` alongside the provided `uart_board.vhd` wrapper, and compile. Record two numbers from the reports: how much of the Cyclone V the design uses, and the **worst-case slack** on the 50 MHz clock. Did it meet timing, and how do you know from the report rather than from the design working?

b) Program the DE0-CV over USB-Blaster. Then build the wrapper once more with its rx pin wired straight to its tx pin, bypassing `uart_top`, program that, and open a terminal at 115200 8N1 on the USB-serial adapter: typing a character should echo it back. This is the

only loopback the board can run on its own. Jumpering the peripheral's own tx to its rx shows nothing yet, because `uart_top` transmits only what an SPI master writes to `TX_DATA`; `uart_top_tb` could loop a byte through it only because the testbench was that master, and on the bench the Nano takes that role in Chapter 10.

c) The pin loopback in b) proves rather less than the system testbench did, and rather more. Name one thing each establishes that the other cannot.

Integration and Bring-Up

Both halves on the bench; the pyramid gets its top.

IN THIS CHAPTER

- **Two serial links**, carrying different traffic over different protocols. SPI, from the Nano to the DE0-CV through the level shifter, is the **control plane**: the ATmega driving the peripheral's registers. The peripheral's own UART tx and rx are the **data plane**, running out to a 3.3 V USB-serial adapter.
- **Level shifting**, since the Nano is 5 V and the DE0-CV 3.3 V, together with the four SPI lines and the wiring table.
- **The bring-up ladder**, climbed one rung at a time, each rung proving something the previous one could not.
- **app::EchoNode**, written and host-tested against the UART stub of Chapter 6 first, and only then run on the ATmega over your own VHDL peripheral, with one byte traced end to end.

Everything the book built meets on one bench, and the first thing to get straight is that two serial links run across it. After this chapter you should be able to write `app::EchoNode` against the driver interface and host-test it on the UART stub of Chapter 6 with no hardware, then bring it up as the final rung of the ladder; wire the two-link bench correctly, level shifter included, and explain what each SPI line and each UART line carries; climb the bring-up ladder methodically and state what each rung establishes that the previous one cannot; and explain what an integration test catches that no host test can, and where the top of the test pyramid sits for this system.

Before you start, you need a `uart_top` from Chapter 5 that passes the system testbench, synthesized and programmed onto the DE0-CV in Chapter 9 through the board wrapper (`uart_board.vhd`) in the Quartus project, and the ATmega firmware of Chapter 9, flashed, with its host tests passing. You also need the hardware listed in the preface's *Setting up*: a DE0-CV, a Nano, a level shifter, a 3.3 V USB-serial adapter, and, recommended but not required, a logic analyzer.

Read §10.1 for the two links, level shifting and the board wrapper. Then read §10.2, and write and host-test `app::EchoNode` (Exercise 10.1) before you go near the bench; that needs no hardware, only the driver interface and the UART stub of Chapter 6. Because the class

arrives at the bench already proven, the only open questions there are wiring and links. Read §10.4 for the wiring tables and the bench procedure, and build the wiring table yourself from the SPI parameters in Part 3 of the protocol (§A.3) and the board pinout. Have it checked before powering anything.

The work at the bench is climbing the **bring-up ladder**, stopping at the first rung that misbehaves. Each rung adds exactly one layer, so the first rung that breaks names the culprit; the rungs are Exercise 10.4. Rung a is a **data-plane pin loopback**: the board wrapper wires its rx pin straight back to its tx pin, bypassing the peripheral, so a PC terminal echoes itself and only the adapter, the levels, the wiring and the pin assignment are on trial (not the terminal's baud rate: the adapter sends and receives at the same setting, so a loopback agrees with itself at any rate). Rung b brings up the **control plane**, writing BAUD_DIV over SPI and reading it back. Rung c is **peripheral loopback**, with tx tied to rx on the board so that the driver can send a byte and read the same byte back without the outside world. Rung d is the **real data plane**, the same send and receive against the terminal. Rung e is **EchoNode**, the full application.

The order is forced: the peripheral cannot transmit until BAUD_DIV is written, and BAUD_DIV is only reachable over SPI, so the control plane must be proven before any rung that uses the peripheral.

The chapter closes with Exercise 10.5: trace a single byte through every layer, from the PC terminal through the FPGA's UART receiver, the FIFO, the register bank, the SPI bridge, AvrSpi, the driver and EchoNode, and back out again, naming each module and the chapter that built it.

10.1 The Two-Link Bench

Everything the book built meets on one bench: your VHDL peripheral on the DE0-CV, your C++ driver on the Nano's ATmega328P, and a PC terminal at the far end. What makes the bench worth understanding is that **two different serial links** run across it, for two different jobs, and confusing them is the most common bring-up mistake.

The control plane is SPI. The ATmega328P is the CPU, and it configures and drives the peripheral by reading and writing its registers over SPI, in the five-byte transactions of Part 3 of the protocol (§A.3). This is AvrSpi on one end and the provided spi_slave and spi_reg_bridge on the other, and it carries *register* traffic, not user data.

The data plane is the UART itself. The peripheral's own tx and rx pins carry bytes to and from the outside world, out to a USB-serial adapter and a PC terminal. This is the line the whole peripheral exists to drive.

So a byte the user types travels **in** on the data plane (UART rx), is read by the CPU **over the control plane** (SPI, RX_DATA), and, in the echo application, is written back **over the control plane** (SPI, TX_DATA) to leave again on the data plane (UART tx). Two protocols, one byte.

10.1.1 Level shifting: 5 V meets 3.3 V

The Nano runs at 5 V, the DE0-CV at 3.3 V, and that mismatch is the one electrical hazard on the bench.

The **four SPI lines cross a level shifter**, a bidirectional BSS138-style board: SCK, MOSI and SS go from 5 V on the Nano down to 3.3 V at the FPGA, while MISO goes from 3.3 V up to 5 V. The shifter's HV side ties to the Nano's 5 V, its LV side to the DE0-CV's 3.3 V, and the grounds are common. The **data-plane UART needs no shifting** at all, because the DE0-CV's tx and rx are already 3.3 V and the USB-serial adapter is set to 3.3 V logic, so those two connect directly.

Drive a 3.3 V FPGA input straight from a 5 V pin and, best case, it misbehaves; worst case, you damage the pin. The ladder's control-plane rung is where a crossed or unshifted SPI line first shows itself, but a rung can only report the damage, not prevent it: by the time any rung runs, the line has been powered. That is why the wiring is checked before power is applied (Exercise 10.2).

10.1.2 The board wrapper and Quartus

uart_top passed its system testbench in simulation (Chapter 5), but a testbench is not a pin assignment. To run on real silicon, uart_top is wrapped by **uart_board.vhd**, the Quartus top level that maps the peripheral's ports (clock, reset_n, the SPI lines, rx, tx) onto specific DE0-CV pins: an on-board 50 MHz clock circuit, a reset button, and GPIO-header pins for SPI and the UART.

That wrapper is board I/O, not peripheral logic, so it lives with the Quartus project rather than in hw/, and it is provided. You already synthesized it and programmed the board in Chapter 9, so nothing about the FPGA toolchain is new here; the only change now is the loopback edit each rung of the ladder asks for.

10.1.3 Why a ladder, and what integration proves

The peripheral works in simulation and the driver passes its host tests, yet the assembled bench can still fail, because simulation and host tests cannot see a level shifter, a crossed wire, a clock that is not really 50 MHz, or SPI and UART timing on real edges. Integration is the top of the test pyramid: it is the only level that exercises the actual chip boundary.

The **bring-up ladder** climbs that gap one rung at a time, each rung proving something the last could not, so that a failure points at one layer instead of the whole bench. **Data-plane pin loopback** bypasses your logic entirely and proves only the adapter, the levels, the wiring and the pin assignment (not the terminal's baud rate: the adapter sends and receives at the same setting, so a loopback agrees with itself at any rate). **The control plane** adds the SPI link, the level shifter, AvrSpi and the register bank, checked by writing BAUD_DIV and reading it back. **Peripheral loopback** adds the whole datapath, with tx tied to rx on the board so that the byte never leaves the FPGA. **The real data plane** adds the outside world,

and with it two independently generated baud rates that have to agree. And **EchoNode** adds the application on top.

The order is forced, not arbitrary. The peripheral cannot transmit until `BAUD_DIV` is written, and `BAUD_DIV` is only reachable over SPI, so the control plane has to be proven before anything that uses the peripheral. Any ladder that puts a peripheral rung first is really testing two layers at once and calling it one.

Climbed in order, the first rung that misbehaves names the layer at fault. That is the point of the method, and it is what an integration test catches that no host test can.

10.1.4 What's ahead

§10.2 describes `app::EchoNode`, the application you write and host-test before the bench. §10.4 is the bench procedure: assemble the hardware, program the FPGA and flash the Nano, climb the ladder, run `app::EchoNode`, and trace one byte through every layer the book built.

10.2 What You Are Building: `app::EchoNode`

10.2.1 The `app::EchoNode` application

`EchoNode` is the application that finishes the book: it receives a byte over the UART and sends it straight back. On the bench, what you type in the terminal comes back to you, having crossed every layer the book built. The application itself is tiny, because all the hard work is already done and tested underneath it: the driver, the transport, the peripheral. `EchoNode` just spends that stack.

The important idea is that `EchoNode` is **software you write and host-test before you touch the bench**. It depends only on `driver::uart::Interface`, not on `Uart`, `AvrSpi`, or any hardware, so you build it, test it against the UART stub, and only *then* flash it and bring it up, as the last rung of the ladder (Exercise 10.4). By the time it reaches the bench, its logic is already proven; the only thing the bench adds is the wiring. This is the same discipline the whole book runs on — host-test behind the interface seam first, hardware last — applied one level up, to the application.

This section **describes** `EchoNode`; you write it yourself. The UART stub it is tested against was written in Chapter 6, as soon as the interface it implements existed.

10.2.2 The application seam: `app::Interface`

Applications sit behind their own interface, `app::Interface`, exactly as drivers sit behind `driver::uart::Interface`. It declares a single operation, `run(const volatile bool& stop)`, which runs the application until `stop` becomes true. `main()` holds an `app::Interface&` and calls `run()`, so it neither knows nor cares which application it runs; swapping `EchoNode` for another is a one-line change in `main()`.

`stop` is a `const volatile bool&`, a flag the application only *reads*, hence the `const`, owned by whoever started it. Setting it `true` asks the application to return from `run()`, which is a clean shutdown and, in a test, the way the loop is ended. It is `volatile` because it is written outside the loop that reads it: by the UART stub in a test, and on a target by whatever else may set it, an interrupt handler for instance. `run()` never writes the flag, and where nothing the loop calls could write it either, as with an interrupt handler, the compiler may read it once, keep it in a register and never see it change; `volatile` makes every pass read it from memory. It is a `volatile bool` rather than a `std::atomic`, because `avr-libc` is freestanding and ships no `<atomic>`, and on the single-core ATmega a byte read is already indivisible: atomicity is not the problem here, visibility is.

10.2.3 `app::EchoNode`

`EchoNode` implements `app::Interface`. Write it as `include/app/echo_node.hpp` (the declarations) and `source/app/echo_node.cpp` (the definitions), in the `app` namespace, in the same AVR-portable style as the driver (`<stdint.h>` and bare types, nested namespace blocks, no `[[nodiscard]]`). It is `final`, non-copyable, and non-movable.

Member variable. There is one: `driver::uart::Interface& myUart`, the UART driver to echo over, injected through the constructor and held by reference. That reference is what lets the *same* class run over the concrete `Uart` on hardware and over the UART stub in a test, with no change. Because it is a reference it is set once at construction and never rebound, which is also why copy and move are deleted: a reference member cannot be reassigned, the same reason `Uart` deletes them.

Methods. The constructor, `EchoNode(driver::uart::Interface& uart)`, is explicit and `noexcept`. It takes the driver to echo over and stores it in `myUart`, and it does no I/O, since construction only wires the dependency and therefore cannot fail.

`run(const volatile bool& stop)` is the application loop, `noexcept`, overriding `app::Interface::run()`. It repeats until `stop` is `true`, and on each pass it checks `stop`, asks `myUart` for a byte with the **non-blocking** `read()`, and, if one arrived, echoes it straight back with `writeBlocking()`.

The receive is a poll rather than `readBlocking()` on purpose. A blocking read would wait forever for a byte and never look at `stop` again, so the loop could not be stopped, and a test of it would hang. Polling `read()` returns to the top of the loop every pass to re-check `stop`, which is exactly what lets a caller, or the test, end it. The echo write may block, using `writeBlocking()`, because the byte is already in hand and simply has to go out.

10.2.4 Host-testing it, before any hardware

Because `EchoNode` depends only on `driver::uart::Interface`, you test it against the **UART stub** you wrote back in Chapter 6 (Exercise 6.4), the moment that interface existed: no `Uart`, no transport, no FPGA. This is where it finally earns its keep. The trick that makes `run()` return in a test is the `stop` flag: the stub sets `stop` to `true` the moment its scripted RX runs out,

so `run()` echoes every queued byte and then ends on the next check. The test queues the input, constructs the `EchoNode` over the stub, calls `run(stop)`, and asserts that the stub's recorded TX equals what it queued, in order. The suite lives in `test/app/echo_node_test.cpp` and, like the other host tests, may use full modern C++; run it with `make test`.

That termination trick is also a check on your implementation: because it depends on `run()` polling `read()` and re-checking `stop`, a `run()` that blocked in `readBlocking()` would hang the test — the test telling you the loop is not actually stoppable.

10.2.5 On the bench

Nothing about the class changes for hardware. The application `main()` on the Nano constructs the real stack under the same interface — an `AvrSpi`, a `Uart` over it, `configure()`d for 115200 8N1, then an `EchoNode` over the `Uart` — and calls `run()` with a stop flag it leaves false, so the node echoes forever. That is the last rung of the bring-up ladder (Exercise 10.4): the exact class you host-tested, now echoing real characters typed in the terminal, over real SPI, through your own VHDL peripheral.

How to check your work. The provided host suite `fw/test/app/echo_node_test.cpp`, run by `make test`, checks `EchoNode` over the UART stub; it switches on once `app/echo_node.hpp` and `driver/uart/stub.hpp` both exist. The answers to the parts that are not code are in §B.7. Try each part before you read its answer.

Exercise 10.1 `app::EchoNode`

C++

a) Write `include/app/interface.hpp` first: the `app::Interface` seam described above, with a virtual noexcept destructor that is = default, and the single pure virtual `run(const volatile bool& stop)` noexcept. Then write `include/app/echo_node.hpp` and `source/app/echo_node.cpp` from the description above — the `myUart` member, the constructor, `run(const volatile bool& stop)`, and the deleted copy and move — so that `EchoNode` implements `app::Interface`. Run `make test` and confirm that the echo test passes with no hardware.

b) The test drives `run()` with the stub, which stops once its scripted RX is exhausted, queuing the bytes `0x00`, `0x41` and `0xFF`. Add a case that queues **nothing** and confirm that `run()` returns immediately, having sent nothing. Which implementation mistake is this the only case that catches?

c) The method `run(const volatile bool& stop)` is the unit under test, ended by the flag. Explain why `run()` must poll the non-blocking `read()` rather than call `readBlocking()`, and connect it to why `stop` is passed by reference (and read every pass) rather than returned or checked once, and why it is volatile.

d) Change EchoNode to echo each byte back **uppercased** (leave non-letters alone), and update the test. Keep the transformation inside `run()`; the point is that the application layer is exactly where such policy belongs, above a driver that only moves bytes.

10.2.6 What's ahead

With EchoNode written and host-tested, §10.4 is the bench: flash it and climb the bring-up ladder to watch it echo on real hardware, then trace one byte through every layer of the stack the book built.

10.3 Review

What you should be able to do now

After this chapter, you should be able to:

- **Write `app::EchoNode`** against the driver interface and host-test it on the UART stub of Chapter 6 with no hardware, then bring it up as the final rung of the ladder.
- **Wire the two-link bench correctly**, level shifter included, and explain what each SPI line and each UART line carries.
- **Climb the bring-up ladder methodically** and state what each rung establishes that the previous one cannot.
- **Explain what an integration test catches that no host test can**, and where the top of the test pyramid sits for this system.

Questions to test yourself

You should be able to explain:

1. The real-data-plane rung (d) works but the EchoNode rung (e) does not: which layer is implicated, and which layers has the ladder already cleared?
2. Rung c loops tx back to rx on the board and passes, yet rung d fails against the terminal: what does that isolate, given that the peripheral's own datapath is common to both?
3. What does the control-plane rung (b) prove that a passing `uart_top_tb` in simulation did not?
4. If one SPI line is left unshifted, wired 5V straight to a 3.3V pin, what is the likely symptom, and which rung exposes it first?

Course review

The pattern you have just completed — a peripheral in VHDL, a driver in C++, one shared register contract, integrated across a real chip boundary — is the whole FPGA-meets-MCU method. The same method scales to far harder protocols, and the SPI transport this book handed you as a black box is itself built from the wire up two courses later, in *SPI: The MCU-FPGA Transport, from the Wire Up*.

10.4 Bringing Up the Bench

How to work these. This is the bench work that ends the book: the DE0-CV, the Nano, a level shifter and a USB-serial adapter, wired into one working node. The hardware is listed in the preface's *Setting up*. Read §10.1 first: it explains the two links, the level shifting, and why the bring-up ladder climbs one rung at a time. `app: :EchoNode` (Exercise 10.1) should already be written and passing its host test.

How to check your work. The bench is the check: every rung of the ladder has an observable outcome, and the first one that misbehaves names the layer at fault. The answers to the parts that are not bench work are in §B.7.

Exercise 10.2 Assemble the bench

BENCH

Wire it with everything **powered off**, and have the wiring checked before applying power. A single 5 V line into a 3.3 V FPGA pin can damage it.

The control plane (SPI), through the level shifter. The Nano's SPI pins are fixed; the DE0-CV signals are the GPIO-header pins assigned in `uart_board.vhd`.

Signal	Nano	Level shifter	DE0-CV
SCK	D13 (PB5)	HV to LV	<code>sclk</code>
MOSI	D11 (PB3)	HV to LV	<code>mosi</code>
MISO	D12 (PB4)	LV to HV	<code>miso</code>
SS, the chip select	D10 (PB2)	HV to LV	<code>ss</code>
5 V reference	5V	HV	—
3.3 V reference	—	LV	3.3 V
GND	GND	GND (both sides)	GND

The data plane (UART), 3.3 V direct, with no shifter, since the adapter is set to 3.3 V logic. The tx/rx crossover is the usual one.

Signal	DE0-CV	USB-serial adapter
The peripheral's tx	tx	RX
The peripheral's rx	rx	TX
GND	GND	GND

The adapter's USB goes to the PC, where a terminal at 115200 8N1 is the far end of the data plane.

Exercise 10.3 Program the FPGA and flash the Nano

BENCH

Two chips, two toolchains, both of which you have already used. For the **DE0-CV**, re-synthesize `uart_board.vhd`, the provided Quartus top level wrapping your `uart_top`, and program the FPGA over USB-Blaster exactly as in Chapter 9; confirm that the design fits and that the pin assignments match the wiring table above. For the **Nano**, flash the ATmega firmware:

```
make -C fw/avr flash          # avr-gcc + avrdude; see fw/README.md
```

With both programmed and the bench wired, you are ready to climb the ladder.

Exercise 10.4 Climb the bring-up ladder

BENCH

Climb one rung at a time and **stop at the first that misbehaves**; that rung names the layer at fault (§10.1.3 explains why).

a) Data-plane pin loopback. This is the loopback you already ran on the board alone in Chapter 9. In `uart_board.vhd`, temporarily wire the board's UART receive pin straight back out to its transmit pin, bypassing `uart_top` altogether, and re-synthesize. Type in the PC terminal; every character should come straight back. None of your logic is in this path, and that is the point: it proves the USB-serial adapter, its 3.3 V logic levels, the two data-plane wires, and the FPGA pin assignment, and it proves nothing else, because nothing else is in the circuit yet. Note what it cannot prove: the terminal's baud and frame settings. The adapter sends and receives at the same settings, so a loopback agrees with itself at any rate; they are first on trial in rung d. Fix any problem here before going further, since every later rung depends on this one.

b) The control plane. Restore the wrapper. Flash a bring-up `main()` that does nothing but write `BAUD_DIV` over SPI and read it straight back, comparing the two values. A matching read-back is the first evidence that the whole control plane works: the level shifter on all four SPI lines, `AvrSpi`'s register setup, the provided `spi_slave` and `spi_reg_bridge`, and your register bank's read path. This rung has to come before any rung that uses the peripheral, because until `BAUD_DIV` is written the divider reads zero and `uart_top` substitutes 1, so a byte written to `TX_DATA` would leave at $50 \text{ MHz} / 16 = 3.125 \text{ Mbaud}$ — transmitted, but at a rate nothing on the bench can decode — and the only route to a usable `BAUD_DIV` is SPI. A crossed, unshifted or floating SPI line surfaces here and nowhere earlier.

c) The peripheral, looped back on itself. Wire the peripheral's own tx to its own rx in the wrapper, exactly as `uart_top_tb` does in simulation, and re-synthesize. Have the bring-up `main()` configure the peripheral, write a byte to `TX_DATA`, then poll `STATUS` and read `RX_DATA`

back. The same byte coming back proves the entire datapath on real silicon: `baud_gen`, `uart_tx`, `uart_rx`, both FIFOs and the register bank, running at an actual 50 MHz rather than a simulated one. The PC terminal is not involved at all, so a failure here is the FPGA and not the link.

d) The real data plane. Undo the loopback and wire tx and rx to the USB-serial adapter. Repeat the send and the receive from `rung c`, but against the terminal this time: a byte the driver writes should appear in the terminal, and a character you type should come back through `RX_DATA`. This adds the one thing `rung c` could not, which is the peripheral talking to a *different* device, with two independently generated baud rates that now have to agree with each other.

e) `app::EchoNode`. Flash the echo application you wrote and host-tested in Exercise 10.1: `main()` constructs a `Uart` over `AvrSpi`, `configure()`s it for 115200 8N1, and calls `node.run(stop)`. What you type in the terminal is received by the driver over SPI and echoed back out the peripheral's tx, one byte crossing every layer. Because the class was already proven on the host, anything that fails at this `rung` is the wiring or the link, not the echo logic.

Exercise 10.5 Trace a byte

REASONING

The closing exercise, and the book's summary. Take a single character typed in the terminal and trace it through **every** layer, out and back, naming each module and the chapter that built it.

It travels in on the data plane, from the PC terminal to the USB-serial adapter to the DE0-CV's rx pin, through `sync` into `uart_rx` (both Chapter 3), then the RX FIFO (`fifo`, Chapter 4) inside `uart_regs` (Chapter 5). It then crosses the control plane, out through `spi_reg_bridge` and `spi_slave` (provided) to `AvrSpi` (Chapter 9), up through `readReg` and `Uart` (Chapter 7), and into `EchoNode` (Chapter 10). Finally it goes back out, from `writeBlocking()` (Chapter 8) and `Uart::write()` to `writeReg()` to `AvrSpi`, back across `spi_slave` and `spi_reg_bridge` into `uart_regs` and the TX FIFO, through the TX feeder in `uart_top` (Chapter 5) and `uart_tx` (Chapter 2) to the DE0-CV's tx pin, and back via the adapter to the terminal.

Name each place the byte changes representation (a UART frame on the wire, a register field over SPI, a `uint8_t` in the driver), and you have described the whole FPGA-meets-MCU stack the book built.

The UART Register Protocol

The contract at the centre of the course.

The FPGA side (Chapters 1 to 5) implements this contract as a peripheral behind the provided SPI transport; the MCU side (Chapters 6 to 9) implements it as a master; and Chapter 10 proves the two agree. Both halves are written against this appendix, not against each other's code. **If an implementation and this appendix disagree, the implementation is wrong.**

There are two layers here, and they are independent:

1. **The UART line protocol:** how the peripheral's tx and rx pins carry bytes to the outside world, the *data plane*.
2. **The register map and its SPI access:** how the ATmega328P reaches the peripheral's registers across the chip boundary, the *control plane*.

A.1 Part 1: The UART Line Protocol

This is the data plane: the peripheral's own asynchronous serial line, on tx and rx.

Parameter	Value
Frame	1 start bit (low), then the data bits, then an optional parity bit, then the stop bit or bits (high).
Data bits	8, least significant first .
Idle	The line idles high .
Parity	None, even or odd, selected by CTRL. Default none.
Stop bits	1 or 2, selected by CTRL. Default 1.
Default frame	8N1 : 8 data bits, no parity, 1 stop bit.
Voltage	3.3 V logic, that of the DE0-CV. The data-plane peer must be 3.3 V too, for example a CP2102 or an FT232 set to 3.3 V.

A.1.1 Baud rate

The system clock is **50 MHz**. The receiver oversamples the line at **16 times** the baud rate, so the internal tick rate is $16 * \text{baud}$. BAUD_DIV holds the divider from the 50 MHz clock to that tick:

```
BAUD_DIV = round( 50_000_000 / (16 * baud) )
```

Baud	BAUD_DIV
9600	326
19200	163
57600	54
115200	27

The transmitter emits one bit per 16 ticks; the receiver samples each bit at tick 8, mid-bit, after detecting a start edge. BAUD_DIV is 16 bits, so with the oversampling factor of 16 the lowest representable rate is $50_000_000 / (16 * 65535)$, about 48 baud.

Because BAUD_DIV is an integer, the achieved rate is only as close as the rounding allows. At 115200 the exact divider is 27.126, so $\text{BAUD_DIV} = 27$ gives 115740.7 baud, about 0.47 % fast; at 9600 the error is -0.15% . Both are comfortably inside what an 8-bit frame tolerates: the receiver resynchronizes on every start bit, and only has to be still inside the stop bit when it samples it, nine and a half bit periods later, which allows a few percent of combined error between the two ends.

A.2 Part 2: The Register Map

This is the control plane: seven 32-bit registers, reached over the provided SPI transport of Part 3. Only the low bits of each register are meaningful; the upper bits read back as zero and are ignored on a write.

In- dex	Register	Offset	Ac- cess	Description
0	STATUS	0x00	R	Bit 0: TX ready (the TX FIFO is not full). Bit 1: RX valid (the RX FIFO is not empty). Bit 2: Error (any ERROR_FLAGS bit set). Bit 3: TX idle (the TX FIFO is empty and the line is idle).
1	CTRL	0x04	R/W	Bit 0: enable. Bits 2–1: parity (00 none, 01 even, 10 odd; 11 reserved). Bit 3: stop bits (0 for 1, 1 for 2). Bit 4: RX-valid IRQ mask. Bit 5: TX-ready IRQ mask. Reserved in this course: see §A.2.1.
2	BAUD_DIV	0x08	R/W	The baud divider (bits 15–0), per Part 1.
3	TX_DATA	0x0C	W	Bits 7–0: push one byte into the TX FIFO.
4	RX_DATA	0x10	R	Bits 7–0: the byte at the front of the RX FIFO. Reading it does not pop it.
5	RX_POP	0x14	W	Any write discards the front RX byte and advances the FIFO; the value is ignored. Write 0x1 by convention.
6	ERROR_FLAGS	0x18	R/W	Bit 0: framing error. Bit 1: parity error. Bit 2: overrun. Write 0x0 to clear all three.

Indices 7 to 15 are reserved: a read returns 0x00000000, and a write is ignored.

A.2.1 What this course implements, and what it reserves

The map above is the full contract. The peripheral built in Chapters 1 to 5 implements the part of it that carries data, and stores but does not act on the rest:

- **CTRL is stored and read back, but gates nothing.** The register bank has no ctrl output, so CT_ENABLE, the parity select, the stop-bit select and the two IRQ masks reach no logic. The peripheral is always enabled and always 8N1. The driver's `configure()` still writes the enable bit, so the two halves agree on the wire; the write simply has no effect on the hardware yet.
- **There is no interrupt line.** `uart_top` has eight ports and none of them is an IRQ output, so the two IRQ mask bits mask nothing. The system is poll-only, end to end.
- **Only the framing error flag has a producer.** The receiver emits `frame_err` and nothing else, so `ER_PARITY` and `ER_OVERRUN` are defined positions that read zero unless software writes a one to them. Parity detection and overrun detection are the natural extensions of Chapter 3 and Chapter 5 respectively.

These are reservations, not omissions: the positions are fixed so that an implementation which adds them later stays compatible with both halves as written.

A.2.2 The driver flow this map implies

- **Transmit:** poll STATUS bit 0 (TX ready); when it is set, write a byte to TX_DATA.

- **Receive:** poll STATUS bit 1 (RX valid); when it is set, read RX_DATA, then write RX_POP to advance to the next byte.

This mirrors the poll-status, read-data, acknowledge shape of a typical hardware FIFO.

A.2.3 Register semantics

This is what the register bank must implement. The TX and RX cores expose single-cycle pulses and levels (Chapters 2 and 3); the register map promises poll-able bits, sticky error flags and FIFO-backed data. Bridging the two is the register bank's whole job (Chapter 5):

- **STATUS bit 0 (TX ready):** a level, meaning the TX FIFO is not full. A TX_DATA write while it is clear is dropped, because there is no room, which is why the driver must poll it first.
- **STATUS bit 1 (RX valid):** a level, meaning the RX FIFO is not empty. It is set by the receiver pushing a byte, and cleared when RX_POP empties the FIFO.
- **STATUS bit 2 (Error) and ERROR_FLAGS:** the bank latches the receiver's framing, parity and overrun pulses into the three ERROR_FLAGS bits, and bit 2 of STATUS is their OR. An ERROR_FLAGS write of 0x0 clears the latches.
- **TX_DATA:** a write pushes exactly one byte into the TX FIFO. It is an **edge event**, one write for one byte, never a held level. On a transaction abort (Part 3), no byte is pushed.
- **RX_DATA and RX_POP:** RX_DATA is a *pure read*. It returns the front byte and has no side effect, so it obeys the same latch-once rule as STATUS. Popping the FIFO is a *separate write*, RX_POP, so that the side effect is a committed, abort-safe action and never a by-product of reading. This split is the point of Chapter 5: **a read that popped a FIFO would need write-like commit-and-abort discipline; keeping the read pure and the pop explicit avoids that entirely.**
- **BAUD_DIV and CTRL:** plain read/write registers. baud_gen reads BAUD_DIV continuously, so a new divider takes effect from the next baud tick, in the middle of a frame if one is in flight; CTRL is stored only. Reconfiguring mid-frame is the driver's problem, not the bank's, which is what STATUS bit 3 (TX idle) is for: the driver waits for it before changing the baud rate.

A.3 Part 3: The SPI Transport

The register map above is reached over SPI by the provided `spi_slave` and `spi_reg_bridge`. This section documents that transport fully: it is a black box in *this* course, which you use and do not write, but it is not magic.

Parameter	Value
Roles	The MCU is the SPI master, the FPGA the slave.
Mode	SPI mode 0 (CPOL = 0, CPHA = 0): SCK idles low, and both sides sample on the rising edge.
Bit order	Most significant bit first, in every byte.
SCK frequency	At most 1 MHz. The Nano uses $f_{\text{osc}}/16$, which is 1 MHz.
SS	Active low; one transaction per low period.
Voltage	The Nano is 5 V and the DE0-CV 3.3 V, so every SPI line crosses a level shifter (Chapter 10).

A.3.1 Transactions

Every transaction is exactly **5 bytes**: one command byte, then four data bytes. SS falls before the command byte and rises after the fifth byte, staying low for the whole transaction. The command byte is laid out as:

```

Bit:      7   6   5   4   3   2   1   0
          +---+---+---+---+---+---+---+---+
          | W | 0 | 0 | 0 |   register index   |
          +---+---+---+---+---+---+---+

```

- **Bit 7 (W)**: 1 for a write, 0 for a read.
- **Bits 6 to 4**: reserved, and must be 0.
- **Bits 3 to 0**: the register index, which is the offset divided by 4, so 0 to 6 in the map above.

The two kinds of transaction, and the one way either can end early:

- **Write (W = 1)**: the master sends the 32-bit value in the four data bytes, most significant byte first. The slave commits it to the addressed register only when the fifth byte completes.
- **Read (W = 0)**: at the end of the command byte the slave **latches the addressed register's value once**; the four data bytes clock it out on MISO, most significant byte first, while the master sends dummy 0x00 bytes. The latch-once rule is part of the contract: the four bytes are one coherent sample.
- **Abort**: if SS rises before the fifth byte, the transaction is abandoned with **no side effects**. Nothing commits, no TX_DATA push happens and no RX_POP. This is what makes the peripheral self-recovering.

A.4 A Worked Example

Configuring 115200 8N1, sending 'H' (0x48), then polling for and reading one received byte. Each line is one transaction, as the master sends it; on a read, the four bytes after the command byte are what comes back on MISO.

```
Write BAUD_DIV : 82 00 00 00 1B (0x1B = 27)
Write CTRL     : 81 00 00 00 01 (enable, 8N1)
Read  STATUS   : 00 xx xx xx xx -> bit 0 set means TX ready
Write TX_DATA  : 83 00 00 00 48 ('H' pushed into the TX FIFO)
Read  STATUS   : 00 xx xx xx xx -> bit 1 set means RX valid
Read  RX_DATA  : 04 00 00 00 XX -> XX is the received byte
Write RX_POP   : 85 00 00 00 01 (advance the RX FIFO)
```

Answers to the Exercises

Every part that is not code, answered in full.

These are the answers to the parts of the exercises that you answer in words or numbers: the reasoning, the calculations, the traces, and the parts that ask you to break a working design on purpose and say what happens. They are in the order the exercises are set, one section per chapter that has any. Chapters 6 to 8 have none: every exercise there is a header or a class, and the host suite of Chapter 8 is its check. Every answer opens with the exercise it answers, linked back to it and to the page it is on, and each part's answer is lettered as the part is. A part that is not listed is code, and has nothing to answer here.

The VHDL and C++ the exercises ask for are not answered. Each exercise names the testbench or the host suite that checks it, and that check is the authority on your module: it is the executable form of the specification the chapter sets, and a module that passes it has met that specification, however it is written. A listing printed here would be one answer of many, and it would teach less than the testbench's failure message does.

Where a part asks what a testbench or a suite reports when you break something, the answer quotes the message a design built to the chapter's specification actually produced when it was broken in exactly that way. If yours reports something else, the difference is worth understanding before you move on: it usually means your design differs from the specification somewhere the testbench does not look.

Try each part before you read its answer. That matters most for the parts that ask you to predict: an outcome read before it is predicted is an outcome you can no longer disagree with, and the disagreement is the lesson.

B.1 Chapter 1: The Register Package and the Peripheral Top

Exercise 1.2 is the build of `uart_top`, checked by a clean analysis now and by `uart_top_tb` from Chapter 5 on; only its closing question, part e), is answered here.

Exercise 1.1 `uart_def`

page 14

a) The register indices are REG_STATUS 0, REG_CTRL 1, REG_BAUD_DIV 2, REG_TX_DATA 3, REG_RX_DATA 4, REG_RX_POP 5 and REG_ERR_FLAGS 6, each the byte offset divided by four. The STATUS bits are ST_TX_READY 0, ST_RX_VALID 1, ST_ERROR 2 and ST_TX_IDLE 3; the CTRL bits CT_ENABLE 0, CT_PARITY_LO 1, CT_PARITY_HI 2, CT_STOP 3, CT_RX_IRQ 4 and CT_TX_IRQ 5; the ERROR_FLAGS bits ER_FRAMING 0, ER_PARITY 1 and ER_OVERRUN 2.

b) In VHDL the position indexes the vector directly:

```
if (status(ST_RX_VALID) = '1') then
```

In C++ the same number is shifted into a mask at the use site: `(status & (1U << status::RX_VALID)) != 0U`. The protocol speaks of “STATUS bit 1”, and a position is that number and nothing else, so both files can be checked against the protocol by reading it off, digit for digit. A mask would be a derived value, 2 here and 4 for bit 2, which the reader has to recompute to compare, and which only means something for one word width. Storing positions also keeps the two languages identical, because the only difference between them is the operator that uses the constant, and that operator is written where it is used.

c) **In no testbench at all.** Every testbench that touches those two registers names them through `uart_def`, and so does the register bank. `uart_regs_tb` writes RX_POP to “REG_RX_POP” and the bank decodes “REG_RX_POP”, so after the swap both sides mean 6 and the bench still passes; the same holds for ERROR_FLAGS at 5. `uart_top_tb` never reads or writes index 5 or 6. Both were run with the swapped package, and both report that they passed. A constant that is wrong in the one place everything reads it from is wrong consistently, and a consistent error is invisible to any check built from the same source.

The C++ side is a separate transcription, and it still says RX_POP is 5 and ERROR_FLAGS is 6, so the host suites in Chapters 8 and 9 pass too. The swap surfaces only when the two transcriptions finally meet, on the bench in Chapter 10. There the driver’s `read()` ends with `writeReg(RX_POP, 1)`, which the peripheral now takes as an ERROR_FLAGS write: it sets the framing flag, so STATUS bit 2 comes up, and it never advances the RX FIFO, so RX_VALID stays set and every later `read()` returns the same byte. `clearErrors()` writes index 6, which the peripheral now takes as a pop, and silently throws a received byte away.

Rung c of the ladder can still pass, because it sends one byte and reads one byte back, and that first read is correct; it shows the fault only if the bring-up program looks at STATUS after the read, or sends a second byte. Rung d shows it for certain: the first character typed comes back correctly and so does every read after it, the same character each time. At rung e, `EchoNode` echoes the first character typed without end.

Exercise 1.2 `uart_top`

page 15

e) With only the provided files and your skeleton in `hw/`, make `build-vhdl` analyzes five modules, runs the two transport testbenches, and skips the rest:

```
==> uart_top_tb (skipped: no baud_gen.vhd sync.vhd fifo.vhd uart_tx.vhd uart_rx.vhd
    ↳ vhd uart_regs.vhd yet)
2 testbench(es) run, 7 skipped.
```

`baud_gen` and `uart_tx` arrive in Chapter 2, `sync` and `uart_rx` in Chapter 3, `fifo` in Chapter 4 and `uart_regs` in Chapter 5. The one that `uart_top` never instantiates is `fifo`: `uart_regs` owns both FIFOs. The build needs it anyway because elaboration needs every entity in the hierarchy, not only the top's direct children, and `uart_regs` cannot be analyzed until the entity it instantiates twice is already in the work library.

Exercise 1.3 What a clean analysis does not prove

page 19

a) `ghdl -a` accepts it. Both ports are in `std_logic`, so the entity is as legal with them swapped as without, and the architecture refers to them by name, so it still analyzes. Nothing binds to the entity positionally until `uart_top_tb` runs in Chapter 5, and there the first check fails: the bench's clock now arrives on the peripheral's `mosi` and its data on `sclk`, so the first write never lands and the read-back reports

```
uart_top_tb: BAUD_DIV read-back mismatch, got 0x0000!
```

four chapters after the mistake was made, in a message about `BAUD_DIV`. That is the cost of a positional contract: the order of the ports is checked by nothing except a simulation of the whole system, and when that simulation fails it points at the first symptom rather than the cause. A named association would have caught it at analysis; the course binds positionally so that the order in each interface table is something you have to get right.

b) All zeros. Every read transaction clocks out `0x00000000`, whatever register it addresses, because the bridge latches `reg_rdata` one cycle after the command byte and shifts it out on `MISO`. It is harmless until Chapter 5 because nothing reads it: the only testbench that drives `uart_top` is skipped until then, and no register exists yet to be misreported. Without the placeholder the vector is undriven, so every bit is 'U', and the master samples 'U' on all 32 data bits of every read. Both were simulated: the skeleton with the placeholder returned `00000000000000000000000000000000` for `BAUD_DIV` and for `STATUS`, and the skeleton without it returned 32 'U's for each. The placeholder makes the answer a defined value; it does not make it a right one.

c) A release that lands inside the recovery and removal window of the flip-flops around a clock edge leaves some of them out of reset on that edge and others on the next, and any

of them may go metastable on the way. The design then starts half-reset: a state machine whose state register has left reset while its outputs have not, a FIFO whose pointers start one cycle apart, the SPI slave's edge detector seeing a false edge on a synchronizer stage that left reset a cycle early. None of those states is one the logic was designed to leave, and nothing brings it back short of another reset.

`sync` cannot do this job because of its port list. It takes `reset_s2_n` as an input: it needs a clean reset in order to reset itself, so it cannot be the thing that produces one. A reset synchronizer needs the raw `reset_n` on its flops' asynchronous clear, so that the assertion reaches them with no clock at all, and a constant '1' clocked through the chain, so that the release arrives a clean two edges later. `sync` has no port for either: its only asynchronous input is the already-synchronized reset, and its data input is what it synchronizes.

d) Because the top's instantiation line is the block's contract. `uart_top` declares the signals it connects and binds them by position, so writing that line fixes how many ports the block has, in which order, of which types and in which directions, before the block exists. For `baud_gen` to bind without the top changing, its entity must declare exactly `clock` and `reset_s2_n` (in `std_logic`), then `div` (in natural range 1 to 65535), then `tick` (out `std_logic`), in that order, which is also the order `baud_gen_tb` binds. The names inside may be anything; the positions and the types may not.

B.2 Chapter 2: UART Framing and the Transmitter

The modules themselves are code: `baud_gen` and `uart_tx` in the a) parts of Exercises 2.1 and 2.2, checked by `baud_gen_tb` and `uart_tx_tb`; their instantiation into `uart_top` in Exercise 2.2 f), checked by a clean analysis now and by `uart_top_tb` in Chapter 5; and the parity and second stop bit of Exercise 2.3 a), checked by your own testbench case.

Exercise 2.1 `baud_gen`

page 29

b) $\text{div} = 50000000 / (16 * 115200) = 27.13$, which rounds to **27**, the value in the protocol's table (§A.1.1). One bit is then $16 \times 27 = 432$ clocks, $8.64 \mu\text{s}$, which is 115740.7 baud, 0.47% fast.

The divide is by $16 * \text{baud}$ because the receiver cannot see where a bit begins, so it samples the line sixteen times per bit, and the transmitter and the receiver share the one time base. `uart_rx` (Chapter 3) spends the sixteen ticks: it detects the start edge on one of them, counts to the middle of the start bit to re-check it, and then counts sixteen per bit to land each later sample in the middle of its bit. `uart_tx` simply holds each bit for sixteen ticks. A generator dividing by baud alone would give the transmitter everything it needs and leave the receiver nothing to find the middle of a bit with.

c) The testbench passes with `counter = div - 1`. The counter reloads to zero on the very edge it reaches `div - 1`, so with a constant `div` it never passes the threshold, and “has reached” and “is at least” are the same test: both fire exactly when the counter is 3.

They part company only when the counter is already past the threshold, and a `div` of zero is the extreme case, since `div - 1` is then -1 . With `>=`, every counter value is at least -1 , so tick is high on every clock and the counter stays at zero. With `=`, the counter is never -1 , so it never reloads: tick never rises, and the counter climbs until its next increment leaves its declared range. Simulated with `div` forced to zero, the `>=` version ticked on 99 of the first 100 clocks, and the `=` version never ticked and stopped after 65536 clocks with

```
ghdl:error: bound check failure at baud_gen.vhd:22
```

so `>=` is the one that keeps the counter inside its range. (In hardware the counter would wrap silently instead, and still never tick.) In this design a zero cannot actually reach the port, since `div` is natural range 1 to 65535: that is why `uart_top` substitutes 1 for a zero `BAUD_DIV`, and the robustness of `>=` is a second line of defence. It also covers the case the range does allow, a `div` lowered while the counter is above the new threshold: `>=` reloads on the next edge, `=` counts on to the top of its range first.

d) Both `uart_tx` and `uart_rx` count ticks, and both are synchronous: each looks at `baud_tick` only inside `rising_edge(clock)`. A tick decoded combinatorially from the registered counter is still exactly one clock wide, because the counter holds `div - 1` for exactly one clock. A glitch on it between edges, while the counter's bits change, is invisible to both consumers as long as it has settled by the next edge, which is ordinary timing closure. So a glitchy tick would not break them.

A wider tick would. Every clock on which `baud_tick` is high counts as one sixteenth of a bit, so a tick two clocks wide advances `uart_tx`'s ticks counter twice per period: bits come out half as long as they should, and `uart_rx` samples at the wrong points. The registered output is the design that makes “one clock wide” true by construction. A glitch would

matter only where tick is used as a clock or crosses into another clock domain, and it is neither here.

Exercise 2.2 `uart_tx`

page 29

b) The testbench stops at the first data bit:

```
uart_tx_tb: data bit 0 mismatch!
```

0x53 is 0101_0011, so least significant first the data bits go out as 1, 1, 0, 0, 1, 0, 1, 0. Packed most significant first they go out as 0, 1, 0, 1, 0, 0, 1, 1: the first bit on the line is data(7), a 0, where the testbench expects data(0), a 1. That is the byte shifted the other way, 0xCA.

c) The testbench passes. 0xA5 is 1010_0101. Least significant first, the eight data levels are 1, 0, 1, 0, 0, 1, 0, 1; most significant first they are 1, 0, 1, 0, 0, 1, 0, 1 as well, because the byte reads the same in both directions, so the two packings put the same ten levels on the line and a check of the levels cannot tell them apart.

For the check to be able to fail, the byte must **not be a bit-reversal palindrome**: its bit i must differ from its bit $7 - i$ for at least one i . A palindrome fixes the top four bits by the bottom four, so there are $2^4 = 16$ of them, and those 16 lack the property: 0x00, 0x18, 0x24, 0x3C, 0x42, 0x5A, 0x66, 0x7E, 0x81, 0x99, 0xA5, 0xBD, 0xC3, 0xDB, 0xE7 and 0xFF.

uart_rx_tb sends 0x53 and 0x2D, neither a palindrome, and uart_top_tb sends 0x53, so each constant is good. A loopback still cannot catch the two halves reversed *together*, however good its constant: a transmitter that sends data(7) first and a receiver that stores the first bit it gets in bit 7 undo each other exactly, and the byte that comes back is the byte that went in. With both halves reversed, uart_top_tb reports that it passed. With either half reversed alone it fails with

```
uart_top_tb: received byte mismatch, expected 0x53, got 0xCA!
```

and the receiver reversed alone fails its own testbench with the same message from uart_rx_tb. Each unit testbench compares its module with the protocol, a fixed wire order, rather than with the other module, so each catches its own half, and together they catch the pair the loopback cannot.

d) busy is a level that is true for the whole frame, so it answers “may another byte be loaded now?”, which is the feeder’s whole question; it is also half of STATUS’s TX-idle bit. done is a one-clock event at the moment the stop bit ends, so it can count frames or raise a transmit-complete interrupt exactly once per frame, which a level can only do with an edge detector of its own.

A feeder that loaded on done instead of on not_busy would never send anything: done only pulses at the end of a frame, and no frame starts until something is loaded, so the first byte waits for an event that only a sent byte can produce. Built that way, `uart_top_tb` reports

```
uart_top_tb: timed out waiting for RX valid!
```

because the byte written to `TX_DATA` never leaves the FIFO. Even with a first load forced some other way, a one-clock pulse is the wrong thing to gate on: miss it, because the FIFO was empty in that one clock and a byte arrived a cycle later, and that byte waits for a frame that will never end.

e) `frame <= '1' & data & '0'` puts the leftmost item in the highest bit, so

frame index	9	8	7	6	5	4	3	2	1	0
Bit	stop	d7	d6	d5	d4	d3	d2	d1	d0	start
Value	1									0

Read from index 0 upward, that is the start bit, then `d0`, `d1` and so on to `d7`, then the stop bit: exactly the wire order of an 8N1 frame. The caller may change data on the very next cycle because `frame` is a register of the transmitter's own, loaded once, in `STATE_IDLE`, on the edge that sees start. From then on every bit on the line comes from `frame`, and data is not read again until the next start in the next idle period. In `uart_top` that is what lets the FIFO advance on the same edge the transmitter loads.

Exercise 2.3 Parity and a second stop bit

page 33

b) The stop field, which is now one or two bits long. (With the parity bit of part a), the parity field is present or absent too, so the frame is 10, 11 or 12 bits; the start bit and the eight data bits never change.) At a fixed baud rate the frame time is the number of bits over the baud rate, so a second stop bit makes every frame one bit period longer, 11 instead of 10: at 115200 baud that is 95.5 μ s instead of 86.8 μ s, and the throughput falls by a tenth, from 11520 to 10473 bytes per second.

c) `uart_tx_tb` binds positionally and names eight ports, so it binds the eight existing ones and leaves the three new inputs, declared after them, at their defaults of '0'. With `parity_en` and `two_stop` low the frame is the old 8N1 frame, bit for bit, and the testbench passes unchanged. (Declared anywhere else, or without a default, a new input shifts or breaks the positional binding, and the testbench no longer analyzes against the module.)

To pin down even parity, drive `parity_en = '1'` and `parity_odd = '0'`, send 0x53, and after the eight data bits sample one more bit centre: expect '0', because 0x53 has four 1s, already an even number. Then sample one bit later and expect the stop bit's '1'. The choice of a byte whose parity bit is '0' is deliberate: a transmitter that forgot to insert the parity

bit would put its stop bit, a '1', in that position, and one that computed odd parity would send a '1' too, so both fail. Add a second byte of odd weight, such as 0x52 with three 1s, and expect a '1', so that a parity bit stuck at '0' fails as well.

B.3 Chapter 3: The Synchronizer and the Receiver

The a) parts of both exercises are code: `sync`, checked by `sync_tb`, and `uart_rx`, checked by `uart_rx_tb`.

Exercise 3.1 `sync`

page 44

b) The first two-edge check fails and the run stops:

```
sync: output changed after only one clock edge!
```

It says exactly what was deleted: the input reached the output one edge after it changed, so there is one register between them where there should be two.

Every other simulation in the course would pass, because **nothing in a simulator is ever metastable**. A one-flop synchronizer is, functionally, a correct synchronizer with one cycle less latency, and no other testbench cares about one cycle: `uart_rx_tb` drives `rx_s2` directly and never instantiates `sync`, and in `uart_top_tb` the looped-back line changes in step with the same clock, so it never violates anything. The bug is purely electrical: on the bench, the single flop samples a genuinely asynchronous line, sometimes inside its setup and hold window, and its output occasionally hangs between levels for long enough to reach the receiver's logic. Only a testbench that checks the latency itself can see the missing flop, which is why `sync_tb` checks it.

c) Cases 0 to 4 still pass: the reset is held low across two clock edges there, so a synchronous reset has time to clear the chain. Case 5 asserts `reset_s2_n` a quarter of a period after an edge with 1011 standing on the wide output and looks one nanosecond later:

```
sync: reset_s2_n did not clear the output asynchronously!
```

The assertion has to be asynchronous so that the chain clears the moment reset is asserted, whether or not a clock edge follows: a design is reset when its clock has not started yet, or has stopped, and every other block in hw/ clears the same instant, so a block that waited for an edge would be the one block out of step. The release, on the other hand, is not `sync`'s problem. `reset_s2_n` comes from `reset_sync`, which already lines its rising edge up with the clock, two flip-flops after `reset_n` rises. By the time the release reaches `sync` it is an ordinary synchronous signal, so testing it in the sensitivity list is safe.

d) A module that needs a signal as an input cannot also be where that signal comes from. `sync` resets its flops from `reset_s2_n`; to produce `reset_s2_n` it would have to be reset by

its own output, which is undefined until the module has already run, and the raw `reset_n` would have to go through its data input instead, where the assertion would take two clock edges to arrive and would not arrive at all without a clock. `reset_sync` is built the other way round: the raw `reset_n` is the asynchronous clear of both its flops, and a constant '1' is clocked through them, so the assertion needs no clock and the release is synchronous. The two modules look alike, two flops in series, and are wired in opposite directions, so the course keeps both.

e) Each bit of a wide sync is its own two-flop chain. If two input bits change on the same edge of their own domain and the change lands inside the first flops' setup and hold window, each first flop may go metastable and each resolves independently, one to the new value and one to the old. The one that resolved to the new value appears at the output two edges later, the other three edges later, and for one clock `sync_out` shows one bit's new value beside the other's old one.

That is not a bug in sync: it promises that every bit it delivers is a clean level, and that each input change arrives within two or three edges. It does not promise that bits which changed together arrive together, and nothing built from independent flops can. For eight unrelated pins it does not matter, since each pin is read on its own. For a counter it is fatal: going from 0111 to 1000, every bit changes, and a mixed sample can be any of sixteen values, including 1111 and 0000, which the counter never held. A multi-bit value crosses domains as a Gray code, where one bit changes per step, or through a handshake or an asynchronous FIFO, never through a wide synchronizer.

Exercise 3.2 `uart_rx`

page 45

b) **The stop bit** drifts furthest. The receiver resynchronizes only on the start edge; every later sample is timed by counting its own ticks from there, so a rate error accumulates with the time since that edge. The first data bit is sampled about 1.6 bit periods after the edge, the stop bit about 9.7, so the stop bit carries six times the drift.

With the transmitter faster by a fraction f , its bit period is $1/(1+f)$ of the receiver's. Its stop bit, the tenth bit of the frame, ends ten of its own bit periods, 160 of its ticks, after the edge. The stop-bit sample has to land before that.

- **An ideal tick-8 sampler** samples the stop bit at the centre of its tenth bit, 9.5 bit periods, 152 of its ticks, after the edge. It needs $152 < 160/(1+f)$, so $f < 160/152 - 1 = 5.3\%$. Counting the up to one tick a 16-times sampler inherently takes to notice the edge, $f < 160/153 - 1 = 4.6\%$.
- **This receiver** samples the stop bit 155 ticks after the tick that noticed the edge (9 to the start re-check, 17 to the first data bit, 16 for each of the other seven, 17 to the stop bit), and the edge is noticed up to one tick after it happens, so up to 156 ticks after the edge. It needs $156 < 160/(1+f)$, so $f < 160/156 - 1 = 2.6\%$.

So the transmitter can run about 5% fast against the ideal sampler, and only about 2.5% fast against this one, which spends half the budget on its own late sampling phase before any mismatch starts. The budget is shared by both ends: the DE0-CV's divider alone already runs 0.47% fast at 115200. The same late phase buys margin the other way: a transmitter up to $155/144 - 1 = 7.6\%$ slower still has its stop bit under the sample.

c) With the re-check removed, a single tick of low on an idle line starts a frame. The receiver then samples a line that has already gone back to idle, reads eight 1s and a high stop bit, and delivers a perfectly well-formed 0xFF with valid and no framing error. Simulated with a one-tick glitch, the receiver without the re-check delivered one byte, 0xFF, and no framing error; with the re-check it delivered nothing. Note that `uart_rx_tb` still passes without the re-check: none of its three cases contains a glitch, so the bench cannot see the difference.

It is more than theoretical because real lines are noisy, and the moments they are noisiest are the ones nobody is watching: a cable plugged in or pulled out, the far end powering up or down, a long unshielded run beside a motor or a switching supply, a receive pin left floating with no pull-up. Each of those can put a short low pulse on an idle line, and without the re-check each pulse is a phantom 0xFF in the RX FIFO.

d) 0x53 is 0101_0011, so least significant first the bits arrive in this order:

Arrival	1st	2nd	3rd	4th	5th	6th	7th	8th
Value	1	1	0	0	1	0	1	0
bit_idx, index stored	0	1	2	3	4	5	6	7

so `frame(7 downto 0)` holds 0, 1, 0, 1, 0, 0, 1, 1 from bit 7 down, which is 0101_0011, 0x53. Counting down from 7 instead stores the first bit in bit 7, so `frame` holds 1100_1010: `data_out` reads 0xCA, the **bit reversal** of the byte sent, and `uart_rx_tb` says so:

```
uart_rx_tb: received byte mismatch, expected 0x53, got 0xCA!
```

B.4 Chapter 4: The FIFO and the Receive Path

Exercise 4.1 a) is code, `fifo`, checked by `fifo_tb`; so is all of Exercise 4.2, the receive path in `uart_top`, which is checked by a clean analysis now and by `uart_top_tb` in Chapter 5.

Exercise 4.1 `fifo`

page 52

b) After four pushes head has wrapped round to 0, equal to tail, and count is 4. The ungarded fifth push stores 0x99 in entry 0, over 0x11, the oldest byte, which has not been read; advances head to 1, one past tail; and raises count to 5. The FIFO now claims five entries in a four-entry ring, and its front byte is gone. What the run reports depends on the declaration of count:

- As natural range 0 to DEPTH, the assignment of 5 is out of range, and GHDL stops on the fifth push's edge, before any assertion runs, with a `ghdl:error: bound check failure at fifo.vhd:n`, where *n* is the line that increments count.
- As an unconstrained natural, count becomes 5, and full, which is `count = DEPTH`, drops. The testbench's own check fires:

```
fifo_tb: still full after a dropped write!
```

Both are failures, and only the second is the testbench talking; the first is the language refusing a value the declaration said could not happen, which is what a constrained type is for.

c) Because the register bus has no read strobe. A read in this design is not an event: the bridge sets `reg_addr`, the bank's read mux presents `reg_rdata` combinationally, and the bridge latches the word once, one cycle after the command byte. The bank never learns that a read has happened, let alone when it finished. A FIFO whose only read operation was "read and pop" would force the bank to invent a pop out of that non-event, and then decide what it means when the transaction is abandoned half-way. Look-then-advance lets `RX_DATA` be wired straight to `rdata`, a pure read with no side effect, like `STATUS`, and puts the advance on `rd`, where the separate `RX_POP` write drives it. Writes are exactly what the bridge already commits once, on completion, and never on an abort, so the pop inherits that for free.

d) **Eight.** Eight back-to-back frames fill the depth-8 RX FIFO with nothing read; the ninth is the first that can be lost. The receiver holds no finished byte of its own: it pushes each byte on the one-clock `valid` at its stop-bit sample and keeps only the frame it is still assembling. So while the ninth frame is on the wire, software has until that frame's stop-bit sample to pop one byte, about one frame time, 86.8 μ s at 115200 baud; if the FIFO is still full then, the ninth byte is dropped by the FIFO's not_full guard, and every frame after it too.

No flag tells software, in this build. `STATUS` bit 1 stays set throughout, which says only that there is something to read, and `ER_OVERRUN` has no producer, so it reads 0 (see §A.2.1). The information exists: `uart_regs` exports `rx_full`, and a `valid` arriving while it is high is precisely a lost byte. Exercise 4.3 is about where to latch it.

Exercise 4.3 Overrun belongs to the register bank

page 55

a) An overrun is a byte arriving when there is nowhere to put it, and whether there is anywhere to put it depends on whether the previous bytes have been read. `uart_rx` does not know that. It emits a one-clock `valid` and forgets the byte; it has no view of the FIFO behind it or of the software behind that. The one piece of information it lacks is **the consumer's state**, which here is the RX FIFO's full flag.

b) Add a third latch beside the framing one in `uart_regs`: in the clocked process, after the write decode, set `err_flags(ER_OVERRUN)` when `rx_push` and `rx_full_s` are both high, since that is a push the FIFO is about to drop. It lives in `ERROR_FLAGS` bit 2, at the position the protocol reserves for it, latched until software writes `0x0`, and `STATUS` bit 2 shows it with no further change, because that bit is already the OR of the three flags. The driver's existing `errorFlags()` and `clearErrors()` then reach it unchanged, which is what reserving the position bought.

c) A **framing error** is a wire problem: the stop bit was low when it was sampled. The causes are on the line or in its configuration: the two ends at different baud rates, a divider written wrongly, noise on the stop bit, a sender using a different frame format such as a parity bit where the receiver expects the stop bit, or a break, the line held low because a cable came loose or the transmitter lost power. An **overrun** is a software timing problem: every frame on the wire was perfect, and the driver simply did not read them as fast as they came, because it was busy elsewhere, blocked in a delay, or polling too slowly. A framing error says do not trust this byte; an overrun says a good byte was thrown away.

B.5 Chapter 5: The Register Bank

The a) parts of both exercises are code: `uart_regs`, checked by `uart_regs_tb`, and the completed `uart_top`, checked by `uart_top_tb`.

Exercise 5.1 `uart_regs`

page 68

b) Advancing the FIFO “whenever that index is read” means decoding a pop from the address alone, since the bus has no read strobe: `rx_pop` is high whenever `reg_idx` is `REG_RX_DATA`. The run then fails one check earlier than you might expect, in Case 5 rather than Case 6:

```
uart_regs_tb: RX_DATA mismatch, got 0x00!
```

`bus_set_addr` puts the address on the bus, waits for a clock edge and then samples. That edge pops the FIFO, so by the time the bench reads `RX_DATA` the byte it pushed is gone and the front shows an entry that was never written. The “a bare `RX_DATA` read must not pop” check in Case 6 would fail too, but the run never gets there.

The system testbench, by contrast, *passes* with the change. The bridge latches `reg_rdata` on the same edge as the first pop, so it captures the byte before it disappears, and `uart_top_tb` reads `RX_DATA` once and never looks again. What it does not see is the rest: the bridge holds `reg_addr` at the last command's index until the next command, so the pop repeats on every clock and an eight-entry FIFO is empty 160 ns after the command byte.

That is the general problem, and the abort rule makes it plain. A read with a side effect has to decide *when* the side effect happens. If the pop happens when the value is latched, after the command byte, then an aborted read, `SS` rising before the fifth byte, has discarded a byte the master never received, and the protocol promises that an abort has no side effects. If the pop waits for the fifth byte, it is a commit on completion, the discipline a write has; but the bridge commits only writes, with `reg_write`, and has no “read completed” strobe to give the bank, so both the bridge and the bank would have to change. The pure read plus a separate `RX_POP` needs none of that: reading has no side effect to abort, and the pop is a write, which the bridge already commits exactly once, on completion, and never on an abort.

c) STATUS bit 0, TX ready, is not `tx_full_s`, from the TX FIFO. Bit 1, RX valid, is not `rx_empty_s`, from the RX FIFO. Bit 2, Error, is the OR of the three `err_flags` latches. Bit 3, TX idle, is `tx_empty_s` and not `tx_busy`, from the TX FIFO and the transmitter. Bits 31 to 4 are zero.

None of them can be a stored register the datapath writes, because each is a copy of state that already lives somewhere else: the FIFOs' counts, the error latches, the transmitter's state machine. A stored copy would have to be updated by every event that changes its source, on the same edge, by logic that duplicates the FIFO's own; any cycle in which it lagged would be a cycle in which the driver could read “TX ready” for a FIFO that had just filled and push a byte into it that is then dropped. Computed combinatorially, each bit is correct by construction, and the bridge's latch-once read takes a coherent snapshot of all four at once.

d) It relies on the bridge raising `reg_write` only for a **completed, non-aborted write**: one clock, once, when the fifth byte arrives with `SS` still low, with `reg_addr` and `reg_wdata` already holding the transaction's index and all four data bytes. The provided `spi_reg_bridge` does exactly that: it counts the bytes, resets the count whenever `SS` rises, and asserts `reg_write` only on byte four of a write command, which `spi_reg_bridge_tb` checks, abort included.

If a half-finished transaction could reach the bank as a `reg_write`, its data word would be partly stale, the bytes left over from an earlier transaction, and the bank would act on it: a `BAUD_DIV` of garbage, and the line at a rate nobody chose; a `TX_DATA` push of a byte the master never finished sending, which then goes out on `tx`; an `RX_POP` that discards an unread byte; an `ERROR_FLAGS` write that clears an error nobody saw. Every one of those is a single-cycle action with nothing to undo it, which is why the guarantee has to hold before the strobe reaches the bank.

Exercise 5.2 `uart_top`

page 69

b) A release that lands near a clock edge would leave some flip-flops out of reset on that edge and others on the next, with any of them possibly metastable, and the peripheral would start half-reset. Here that could mean a FIFO whose head leaves reset a cycle before its count, a receiver whose `rx_prev` leaves reset after its state and sees a falling edge that never happened, or the transmitter's state machine out of step with its `tx` register: states the logic was never designed to leave. Asserting asynchronously is nonetheless right, because entering reset has no such hazard. Every flop goes to its reset value immediately, together, and without needing a clock, so the design is safe the instant `reset_n` falls, even if the clock has stopped, and `tx` goes to its idle high at once. Only the release has to line up with the clock, and `reset_sync` lines it up.

c) The port that settles it is `sync's reset_s2_n` input. To make `reset_s2_n` with a `sync` instance, you would have to connect the instance's own output, `reset_s2_n`, to its own reset input: the synchronizer would be reset by the signal it has not produced yet, which is undefined until it has been clocked, and it would never assert while it was holding itself in reset. The raw `reset_n` would have to go in through `async_in`, so even the assertion would wait two clock edges and would never come at all without a clock. The circle cannot be closed; `reset_sync` breaks it by taking the raw reset on its flops' asynchronous clear.

d) `tx_load` is (not `tx_empty`) and (not `tx_busy`), and `tx_pop` is the same signal.

- **Never zero.** Whenever a byte is queued and the transmitter is idle, `tx_load` is already high, combinationaly, so the transmitter loads on the next edge.
- **Exactly one.** On that edge, `uart_tx` sees start in `STATE_IDLE`, latches the front byte and moves to `STATE_SEND`, and the FIFO pops the same byte. Because `busy` is decoded combinationaly from the state register, it is high from that edge on, so `tx_load` falls in the very next cycle. It cannot reload while the transmitter is busy, and the FIFO cannot pop a second byte for the same frame.
- **Held low** for the rest of the frame by `tx_busy`, which stays high from the load edge until the edge on which the stop bit's sixteenth tick returns the state machine to `STATE_IDLE`. In that first idle cycle, a queued byte raises `tx_load` again.

Simulated with three bytes queued at once, the feeder popped exactly three times, once per frame, each pop on the first idle edge after the previous frame's done. The argument depends on `busy` rising on the load edge itself. With `busy` registered one cycle late, as a second flip-flop, `uart_tx_tb` and `uart_top_tb` both still pass, since each sends one byte and a second pop from an empty FIFO is ignored; but with three bytes queued the feeder popped twice on consecutive edges, the transmitter took the first and ignored the second, and the third byte never arrived.

e) One byte, around the loop:

1. The testbench plays a five-byte write of TX_DATA on sclk, mosi and ss. spi_slave synchronizes the three lines and shifts each byte in; spi_reg_bridge assembles the command and the four data bytes and, on the fifth, raises reg_write with reg_addr 3 and the byte in reg_wdata(7 downto 0).
2. uart_regs decodes that as tx_push, and its TX fifo stores the byte.
3. The **feeder** in uart_top sees the FIFO not empty and the transmitter not busy, and raises tx_load: uart_tx latches the byte into its frame, and the TX FIFO pops.
4. uart_tx drives the frame out on tx, one bit per sixteen baud_gen ticks.
5. The testbench's loopback, rx <= tx, returns it on the rx pin.
6. sync carries rx into the clock domain as rx_s2.
7. uart_rx, on the same ticks, finds the start edge, samples the eight bits and the stop bit, and pulses valid, which is rx_push.
8. uart_regs pushes the byte into its RX fifo, and STATUS bit 1 rises.
9. The testbench polls STATUS over SPI until it sees bit 1, then reads RX_DATA: spi_reg_bridge latches reg_rdata, which the bank's read mux fills from the RX FIFO's front, and spi_slave shifts it out on miso, most significant byte first.

The system testbench stops there, with the byte still in the RX FIFO: it never writes RX_POP.

B.6 Chapter 9: The Real Transport, and Both Toolchains

The code in this chapter is AvrSpi (Exercise 9.1 a), checked by the provided suite fw/test/transport/avr_spi_test.cpp over the mocked register file, and the bring-up main and the logging (Exercise 9.2 a and Exercise 9.3 a), which the bench checks. Everything else is answered here.

Exercise 9.1 AvrSpi

page 111

b) SPDR holds the incoming byte only once all eight bits have been shifted, eight SCK periods after the write, which at $f_{osc}/16$ is 128 CPU cycles, and SPIF is the only thing that says that moment has come: read earlier and you get a stale byte, write the next byte earlier and you collide with the transfer still in progress.

The spin is cheaper than it looks. Compiled, the whole of transfer() is six instructions, and the wait is a four-cycle loop that goes round about thirty times per byte:

```
out 0x2e, r22      ; SPDR = byte: the transfer starts
in  r0, 0x2d      ; read SPSR
sbrs r0, 7         ; SPIF set? skip the jump
rjmp .-6          ; not yet: read SPSR again
in  r24, 0x2e     ; return SPDR
ret
```

c) If SS (PB2) is an input and something drives it low while MSTR is set, the SPI peripheral concludes that another master has taken the bus and gets out of its way: **it clears MSTR in SPCR**, demoting itself to a slave, and sets SPIF, raising the SPI interrupt if one is enabled. The ATmega328P has no separate mode-fault flag; MSTR reading back as 0 is the only record of what happened.

For AvrSpi that is two failures in a row. The `transfer()` in progress sees SPIF and returns whatever is in SPDR. Every `transfer()` after it writes SPDR as a slave, where nothing is driving SCK, so SPIF never sets again and the poll spins for ever: the driver hangs in the first transaction after the glitch, and nothing sets MSTR back. Making PB2 an output removes the possibility, and costs nothing, because the transport drives that pin as the chip select anyway.

d) It returns the byte the *previous* transfer received, still sitting in SPDR's receive buffer, or whatever was there after reset if there was no previous transfer. The new byte does not exist yet: it arrives eight SCK periods later, 8 μ s or 128 CPU cycles after the write, and the read happens one cycle after it. Every reply is therefore shifted by one byte, so a `readReg` assembles the command byte's reply and three data bytes instead of four.

On the part it is worse than a shift. The next `transfer()` writes SPDR a few cycles later, while the byte before it is still shifting, which the datasheet calls a write collision: WCOL is set and the write is *ignored*, so most of the bytes of a transaction never leave the master at all, and the five-byte transaction is garbled in both directions.

The mock makes the same point more bluntly. It completes a transfer only when the driver reads SPSR, so with the poll gone no transfer ever completes and every read of SPDR returns the value left from reset. Three cases go red, and only these three:

```
Test case AvrSpi.TransferExchangesByte failed: EXPECT_EQ(rx, miso) failed: 0 != ↵
↵ 205
Test case AvrSpi.TransferIsFullDuplexInOrder failed: EXPECT_EQ(spi.transfer(mosi[i↵
↵ ]), miso[i])
failed: 0 != 170
Test case AvrSpi.DrivesTheDriverEndToEnd failed: EXPECT_EQ(uart.status(), data) ↵
↵ failed:
0 != 305419896
19 out of 22 test cases succeeded!
```

Every Uart case stays green, because those run over the scripted `transport::Stub`, not over AvrSpi: the bug is below the seam, and so is the only suite that can see it.

Exercise 9.2 The freestanding bring-up main

page 112

b) Not the provided demo itself: `source/main.cpp` was written in the AVR-portable subset, and it compiles and links unchanged with the `fw/avr` flags, into 1256 bytes of flash and 34 of RAM. It needs none of the idioms a freestanding build removes: its objects are automatic, with fixed arrays inside; the driver reports failure with a `bool`; and it prints nothing.

It is the idioms of ordinary host C++ that fail, and each has a replacement on the AVR:

Host idiom	In the AVR build	What replaces it
<code>#include <cstdint></code> , <code>std::uint8_t</code>	Does not compile: <code>avr-libc</code> has no C++ library, so fatal error: <code>cstdint: No such file or directory</code>	<code><stdint.h></code> and the bare <code>uint8_t</code>
<code>new</code> , <code>std::make_unique</code> , <code>std::vector</code>	No operator <code>new</code> , and no standard containers	Static and automatic storage, sized at compile time
<code>throw</code> , <code>try</code>	Rejected under <code>-fno-exceptions</code>	Return values, as <code>write()</code> returning false
<code>dynamic_cast</code> , <code>typeid</code>	Rejected under <code>-fno-rtti</code>	Virtual dispatch through an interface
<code>std::cout</code> , <code>printf</code> to a console	No iostreams, and no console	Bytes written to the ATmega's own USART, as in Exercise 9.3

c) With the reference firmware, the linker reports one symbol, three times:

```
In function `app::EchoNode::~EchoNode()':
  undefined reference to `operator delete(void*, unsigned int)'
In function `driver::transport::AvrSpi::~AvrSpi()':
  undefined reference to `operator delete(void*, unsigned int)'
In function `driver::uart::Uart::~Uart()':
  undefined reference to `operator delete(void*, unsigned int)'
```

The construct is the **virtual destructor**. Every class that has one gets a second, “deleting” destructor, which runs the destructor and then calls `operator delete`, so that `delete` through a base pointer frees the right object; the vtable points at it, so it is emitted, and names `operator delete`, even though nothing in the program ever deletes anything. From C++14 it calls the sized form, `operator delete(void*, size_t)`, and `size_t` is an unsigned int on the AVR. That is why the linker names the three concrete classes: each inherits a virtual destructor from its Interface.

The other symbols `env.cpp` defines turn up with other flags and other code, which is why it defines them all. At `-O0` the same link also misses `__cxa_pure_virtual`, eight times: the Interface classes’ own vtables name it for every pure virtual, and only the optimizer removes those vtables, because every Interface is constructed as the base of a concrete class that overwrites its vtable pointer at once. Without `-fno-threadsafe-statics`, a function-local static with a constructor, such as a static `driver::uart::Uart uart{spi};`

inside `main()`, adds `__cxa_guard_acquire` and `__cxa_guard_release`. And under `-fno-sized-deallocation` the missing symbol is the unsized operator `delete(void*)` instead.

The host build does not need the file because `g++` links `libstdc++`, which defines every one of these symbols, its static guards thread-safe. `env.cpp` lives in `avr/`, where the host Makefile never looks, precisely so that its single-threaded guards can never replace those.

Exercise 9.3 Logging and timing

page 112

b) Five bytes of eight bits is **40 SCK cycles**, which at 1 MHz is **40 μ s** of clocking. The analyzer shows more: five bursts of eight clocks, each 8 μ s long, with a gap between consecutive bursts, and SS falling a little before the first and rising a little after the last.

The gaps are software. SCK runs only while a byte is shifting, and between two bytes the CPU has to notice SPIF, return from `transfer()`, go round `readReg`'s loop and fold the byte into its 32-bit result, make the next virtual call through the transport interface, and write SPDR again. Counted from the reference build's disassembly, that path is about 30 CPU cycles, roughly 2 μ s at 16 MHz, so a register read takes about $5 \times 8 + 4 \times 2 = 48 \mu$ s between the first clock and the last, and about 50 μ s from SS falling to SS rising, once the `begin()` and `end()` calls on either side are counted. Your gaps depend on your code and your compiler, but not by more than a microsecond or two; what does not change is that the bus clocks for about four fifths of a transaction and waits for the CPU for the rest.

With the logging from part a) in the loop, the gaps *between* transactions are a different matter: a twenty-character line at 115200 baud takes 1.7 ms to leave the USART, more than thirty register reads.

c) The SPI is **synchronous**. The master's hardware makes SCK itself, by dividing the CPU clock by the prescaler chosen in `SPCR` and `SPSR`, and sends it on its own wire; the slave samples on the edges it receives. The code chooses only a ratio, $f_{\text{osc}}/16$, never a frequency, and the exact rate does not matter to either end so long as it stays within what the slave and the wiring can take. No number in the transport depends on the clock, so it needs no `F_CPU`.

The USART is **asynchronous**. There is no clock wire: the PC's end has a clock of its own and agrees with the ATmega only on a number, the baud rate, which the ATmega has to produce from its own clock by computing a divider, $\text{UBRR0} = \text{F_CPU} / (16 * \text{baud}) - 1$, rounded to an integer. The code can only compute that from the clock frequency, so `F_CPU` has to be the real one: with a wrong `F_CPU` every character leaves at the wrong rate and arrives as garbage.

The rounding matters at the log's end of the line too. At 16 MHz, 115200 baud needs a divider of 7.68, which rounds to 8 and gives 111111 baud, 3.5% slow, at the edge of what a frame tolerates. Setting `U2X0`, which halves the divisor's factor from 16 to 8, gives a divider of 16 and 117647 baud, 2.1% fast; 38400 baud (divider 25) and 9600 baud (divider 103) are both within 0.2%.

Exercise 9.4 Your own `uart_top` on the board

page 112

a) The **Fitter** report's resource usage summary gives the size: the logic utilization in ALMs, out of the 18 480 of the DE0-CV's Cyclone V (a 5CEBA4F23C7), the registers, and the pins. This peripheral is a small fraction of the device, well under a few per cent; the exact figure depends on your code and on the Quartus version, and is worth recording mainly so that you notice if a later change makes it jump.

The **Timing Analyzer** gives the other number. Read the worst-case **setup slack** and **hold slack** for the 50 MHz clock across all the timing corners Quartus analyzes, slow and fast silicon at the hot and the cold end of the temperature range; the multicorners summary gathers the worst of them in one table. **Positive slack on every path, in every corner, means the design met timing**; the restricted Fmax for the clock, which should come out comfortably above 50 MHz for a design this shallow, says the same thing another way. Check first that the clock is actually constrained, a 20 ns period on the 50 MHz pin in the project's timing constraints: an unconstrained clock is analyzed against nothing, and its slack means nothing. The asynchronous inputs, rx and the three SPI inputs, are expected to show up as unconstrained, because they enter through synchronizers and have no timing relationship to the clock to check.

The report is how you know, and the design working is not, because the analysis covers the slowest silicon, the lowest supply voltage and the highest temperature the part is specified for, and the board on your desk is one sample, at room temperature, today. A design with negative slack can work on the first try and fail warm, or on the next board; and nothing a working design does tells you how much margin it has left.

c) The pin loopback proves the **physical path**, which the simulation never had in it: that the FPGA is configured at all, that the wrapper's pin assignment puts rx and tx on the header pins the wires go to, and that the adapter really is at 3.3 V logic. It cannot tell you that the terminal's rate is right: the adapter sends and receives at the same setting, so a loopback agrees with itself at any rate, and the rate is first on trial when the peripheral talks to the terminal in Chapter 10. No simulation has a pin, a voltage, a USB adapter or a terminal in it.

`uart_top_tb` proves the **peripheral**, which the pin loopback never had in it: that a byte written to TX_DATA over SPI leaves `uart_tx` correctly framed at the rate BAUD_DIV sets, and comes back through sync, `uart_rx` and the RX FIFO to be read from RX_DATA. With rx wired straight to tx in the wrapper, none of your logic is in the path, so a character echoing in the terminal says nothing about it. Neither of the two says anything about timing, which is what part a) is for.

B.7 Chapter 10: Integration and Bring-Up

`app::EchoNode` and its tests (Exercise 10.1 a, b and d) are code, checked by `fw/test/app/echo_node_test.cpp` and the case you add to it. Assembling the bench, programming the two chips and climbing the ladder (Exercises 10.2 to 10.4) are procedures whose check is

the bench itself: each rung either behaves as its description says, or names the layer at fault. What is answered here is the reasoning in Exercise 10.1 and the closing trace.

Exercise 10.1 `app:EchoNode`

page 120

b) With nothing queued, the stub’s first `read()` finds its input already exhausted, sets `stop` and returns `false`; `run()` writes nothing, comes back to the top of its loop, sees `stop`, and returns. The recorded TX is empty.

The mistake only this case catches is **waiting for input before looking at stop**: a `run()` that blocks for a first byte, with `readBlocking()` or a loop of its own, and only then starts polling. With three bytes queued the first wait is satisfied at once and the flaw is invisible; with none, the byte it waits for never comes. Run against both cases, such a `run()` passes the provided one and hangs on the new one until the suite’s timeout kills it:

```
Test case EchoNode.EchoesQueuedBytesThenStops succeeded!
ERROR: uart_test did not finish within 5s and was killed.
```

On the bench that node is deaf to `stop` until somebody types. The other natural mistakes are caught by both cases: echoing without checking what `read()` returned sends a byte that never arrived (the three-byte case records 4 bytes instead of 3, the empty one 1 instead of 0), and calling `readBlocking()` inside the loop hangs both.

One mistake is caught by neither, and it is worth knowing about: a loop that ends when `read()` fails, while `(myUart.read(byte))`, instead of when `stop` is set. It passes both cases, because the stub sets `stop` at exactly the moment its input runs out, so no test built on it can tell “asked to stop” from “nothing waiting”. On the bench it returns the first time the terminal is idle, which is at once.

c) `readBlocking()` spins inside the helper until a byte arrives. While it spins, control never comes back to `run()`’s loop, so `stop` is never read again: the node cannot be shut down, and a test of it hangs, the three-byte case included, the moment its input runs out. The non-blocking `read()` returns on every pass, with a byte or without one, and that return is what brings the loop back round to look at `stop`.

That is also why `stop` is a reference read every pass. The flag belongs to someone outside `run()`, and changes while `run()` is running: in the test, the stub sets it from inside a `read()` that `run()` itself called. Passed by value, `run()` would hold a copy made at the call, `false` for ever. Returned, it could not stop a loop that has not returned yet. Checked once, at the top, it could only stop the loop before it started. A `const volatile bool&` lets `run()` see the owner’s current value on every pass while promising, through the `const`, never to change it.

And it is `volatile` because being read every pass is something the compiler must be told. `run()` never writes the flag, so where nothing the loop calls could write it either — an interrupt handler setting it, say — the compiler is free to load it once and test a register for ever. In the test the stub happens to set it from inside a `read()` the loop calls, which

already forces a fresh load; `volatile` makes that true however the flag is set. A byte read being indivisible on the ATmega is not the same as the compiler being obliged to repeat it.

Exercise 10.5 Trace a byte

page 124

One character, typed in the terminal and echoed back:

#	Where	What happens	Built in
<i>In, on the data plane</i>			
1	Terminal, adapter, rx pin	A UART frame at 115200 8N1 and 3.3 V: a start bit, eight data bits least significant first, a stop bit	external
2	sync	Two flip-flops take the pin into the 50 MHz domain as <code>rx_s2</code>	Chapter 3, placed in Chapter 4
3	<code>baud_gen</code>	Ticks at 16 times the baud rate, pacing the receiver	Chapter 2
4	<code>uart_rx</code>	Finds the start edge, samples each bit mid-bit, assembles the byte, pulses <code>valid</code> as <code>rx_push</code>	Chapter 3
5	RX FIFO, a <code>fifo</code> inside <code>uart_regs</code>	Stores the byte; <code>STATUS</code> bit 1 (RX valid) becomes 1	Chapter 4, Chapter 5
<i>Across the control plane and up the stack</i>			
6	<code>EchoNode::run()</code> , <code>Uart::read()</code>	The node polls; <code>readReg(STATUS)</code> finds RX valid set	Chapter 10, Chapter 7
7	<code>AvrSpi</code> , the level shifter, <code>spi_slave</code> , <code>spi_reg_bridge</code>	Each register access is one five-byte transaction; the bridge latches <code>reg_rdata</code> from the bank's read mux	Chapter 9; provided
8	<code>readReg(RX_DATA)</code>	The front byte comes back as <code>00 00 00 XX</code> on <code>MISO</code> and is assembled most significant first	Chapter 7
9	<code>writeReg(RX_POP, 1)</code>	<code>85 00 00 00 01</code> ; the bridge commits on the fifth byte and the FIFO advances	Chapter 7, Chapter 5
<i>Back down and out</i>			
10	<code>writeBlocking()</code> , <code>Uart::write()</code>	<code>readReg(STATUS)</code> finds TX ready; <code>writeReg(TX_DATA)</code> sends <code>83 00 00 00 XX</code>	Chapter 8, Chapter 7
11	<code>uart_regs</code> , TX FIFO	The committed write pulses <code>tx_push</code> ; the byte is queued	Chapter 5, Chapter 4
12	The TX feeder in <code>uart_top</code>	<code>tx_load</code> rises with the FIFO non-empty and the transmitter idle, starting <code>uart_tx</code> and popping the FIFO	Chapter 5
13	<code>uart_tx</code> , tx pin, adapter	The frame '1' & data & '0', one bit per 16 ticks, back to the terminal	Chapter 2

The byte changes representation at every boundary it crosses. It is a **serial frame**, bits in time, on each UART wire; a **parallel byte** from `uart_rx` onwards, in the RX FIFO and again in the TX FIFO and the transmitter's frame vector; the **low byte of a 32-bit register word** at the register bank, `RX_DATA` on the way in and `TX_DATA` on the way out; a **stream of SPI bytes**,

the fourth data byte of a five-byte transaction, on the wire between the chips; a `uint32_t` inside `readReg` and `writeReg`; and a `uint8_t` in the driver's interface and in `EchoNode`.

Worth noticing, now that every layer is in view: one echoed character costs at least five register transactions, `STATUS`, `RX_DATA` and `RX_POP` to read it, `STATUS` and `TX_DATA` to write it back, plus one `STATUS` read for every idle pass of `run()`. At about $50\text{ }\mu\text{s}$ a transaction (Exercise 9.3), that is around $250\text{ }\mu\text{s}$ per character, against $86.8\text{ }\mu\text{s}$ for one frame at 115200 8N1. Typing never notices. A pasted burst does: the RX FIFO absorbs the difference for a while, but in a continuous burst the backlog passes its eight entries somewhere around the thirteenth character, and from there bytes are dropped without a trace, because overrun has no producer in this build: the hardware never sets `ER_OVERRUN`. The bench works; that is its capacity.

The Self-Assessment Papers

This appendix and the four after it hold two complete four-hour papers for *UART: Hardware, Driver & Integration*, with worked solutions. Appendices [D](#) and [E](#) are the papers, questions only; Appendices [F](#) and [G](#) are their model answers, with marks.

What these are for

The book has **no exam and nothing is marked**. What you get instead is the exercises after every chapter, the provided self-checking testbench or host suite that grades each one, and the questions in every chapter's review. Roughly half the code the course produces lives in those exercises, and the testbenches in `hw/` and the suites in `fw/` are what say whether it works.

These papers do not replace that. **They are here for you to test your own skills and knowledge, nothing more.** They gate nothing, they are not a qualification, and no part of the book requires them. Nothing in the companion repository depends on them: no module is built from them, and neither `make build` nor `make format-check` knows they exist.

What they check is something the testbenches cannot. Every exercise is done at a keyboard, with the chapter open beside you, GHDL ready to say no and `make test` ready to say no twice. These papers ask what you can reconstruct with none of that in front of you — which of the rules you have absorbed, and which you have merely been looking up.

Take one after the last chapter. Every paper draws on all ten chapters and on both halves of the system, so sitting one partway through examines material you have not read yet, and the result says more about how far you have read than about what you have understood. The intended point is after Chapter [10](#), with the bench climbed and `app: :EchoNode` echoing.

The two papers

Both cover the whole book, Chapters 1 to 10, at the same weighting, in **eight questions: four of VHDL, three of C++, and one with both halves on the bench at once**. Two of the eight span a pair of chapters rather than one — Question 3 covers the receive side across Chapters 3 and 4, and Question 6 the driver and the suite that grades it across Chapters 7

and 8 — which is why ten chapters fit into eight questions without dropping anything. The papers share no question. Either can be used alone; use both as a main sitting and a resit, or in alternate years.

Question	Topic	Chapters	Side	Marks
1	The register map, the peripheral top, and binding	1	VHDL	12
2	Framing, the baud divider, and the transmitter	2	VHDL	13
3	The asynchronous line: synchronizer, oversampling, FIFO	3, 4	VHDL	14
4	The register bank: ports become registers	5	VHDL	13
5	The driver's contracts	6	C++	12
6	The driver and the five-byte transaction	7, 8	C++	13
7	The real transport and the freestanding target	9	C++	11
8	Integration and bring-up	10	Both	12
				100

Paper A leans towards reading and tracing. Most of its questions hand you something — a STATUS word, a listing, a recorded byte log, a bench result — and ask what it means: decode a status read and say what the driver does next, trace a byte through the transmitter's frame vector, work out how much baud mismatch the receiver's real sampling phase leaves, read five SPI transactions off a stub's record and name the driver call that produced each.

Five of its parts do ask for code, so that "leans towards" does not mean "never writes any", and they are chosen so that **no module is written on both papers**: fifo's clocked process and flags, the concurrent STATUS assembly, an AVR-portable header corrected, env.cpp's runtime symbols, and the main() that builds the stack on the target. Paper B writes none of those five.

Paper B leans towards writing and reviewing. Most of its questions ask you to produce a module — baud_gen in full, sync with its generic, the receiver's stop-bit branch, the two Interface headers, readReg and writeReg, AvrSpi, app::EchoNode — or to say what is wrong with one you are handed.

Both papers include at least one design that passes every check the course ships and is wrong anyway, because that is the failure mode this subject actually has. Two more questions, one per paper, are about the limits of the provided testbenches themselves, worked through a real gap that the course's testbenches once had and no longer do.

Why the papers ask for both languages

Because **the boundary is the subject**, and a boundary has two sides. A paper that examined the VHDL and stopped at the register map would be examining digital design with VHDL; one that started at readReg would be examining modern embedded C++. What this book adds is the thing in between: one contract, written down once in Appendix A, implemented independently on each side, and proven to agree only at the end.

So both papers cross the wire more than once. A question about the register bank's STATUS bits ends up in the driver's `write()`; a question about byte order in `writeReg` ends up in what `baud_gen` divides by. That is deliberate, and it is where the marks are.

What a written paper cannot do is run GHDL or make `test`, and it should not pretend to. The rubric says outright that syntax is not what is being marked: a missing semicolon costs nothing, and a process that describes a latch where the candidate meant a wire, or a `readReg` that assembles its bytes backwards, costs that part in full.

Conventions the papers assume

Both papers state these in their own rubric, so a candidate never has to have read this appendix.

- **The protocol is the single source of truth.** Where an implementation and the protocol disagree, the implementation is wrong, and an answer that cites the protocol beats one that cites code.
- **VHDL-93**, with `ieee.std_logic_1164`, and `ieee.numeric_std` where a conversion needs it.
- **The VHDL conventions are the book's:** ports declared all inputs first, then all outputs; every port `map` and generic `map` positional; an active-low signal named `_n`; a two-flop synchronized signal named `_s2`; every submodule taking `reset_s2_n` while `uart_top` alone takes `reset_n`; SPI traffic between the two transport blocks prefixed `spi_`.
- **C++ is written in the book's AVR-portable subset** wherever it would run on the target: `<stdint.h>` and the bare types, no `std::`, nested namespace blocks rather than namespace `driver::uart`, no `[[nodiscard]]`, no inline variables. Host-only code, meaning the test suites, may use full modern C++, and the papers say which is which.
- **The C++ house style is expected:** camelCase, a leading capital on type names, `my` on private members and `our` on private statics, brace initialization, `noexcept` on driver methods, and copy and move deleted on anything holding a reference member or a unique hardware resource.
- **The numbers are fixed:** the DE0-CV runs at 50 MHz (a 20 ns period), the Nano at 16 MHz, SCK at $f_{\text{osc}}/16$, which is 1 MHz, the receiver oversamples 16 times, and the default frame is 8N1.
- **Hardware answers, not code answers**, where a question asks what a design does. "It assigns `x`" is not an answer to "what reaches the wire".

Each paper supplies the clock rates, the `BAUD_DIV` formula and the SPI command-byte layout. It does **not** supply the register map: Question 1 of Paper A and Question 5 of Paper B ask for part of it, so it cannot be printed on the front page. Later questions consume it, which is what the follow-through rule below is for.

Marking

Every solution is written to be marked by somebody who has read the chapters and does not otherwise write VHDL or freestanding C++ daily, so each carries the reasoning rather than the answer alone. Marks are shown per part.

Four conventions are worth agreeing before a paper is marked:

- **Method carries the marks.** A correct derivation with a slip in it is worth more than a correct answer with no working, and both papers are built so that later parts consume earlier ones. Follow an error through rather than penalising it twice: a driver that faithfully polls the wrong bit of a STATUS word the candidate decoded wrongly in Question 1 loses nothing in Question 6.
- **Mark the design, not the punctuation.** Where a VHDL listing is asked for, the marks are in the sensitivity list, the reset branch, the direction of the shift, whether the pulse is one clock wide, and whether every path assigns its output. Where C++ is asked for, they are in the byte order, the `const` and `noexcept` qualifiers that carry meaning, the ordering of the three register accesses, and whether the seam is respected.
- **The named traps are worth full marks on their own.** Several parts exist entirely to see whether a candidate avoids one specific mistake: a bit *position* used as a mask, a `readReg` that assembles least significant byte first, an `RX_DATA` read that pops, a TX feeder that pops without watching busy, a `transfer()` with no SPIF poll, a receiver that leaves idle on a level rather than an edge, and a `run()` loop that blocks and can never be stopped. Where a solution flags one of these, a candidate who walks into it loses those marks and no others.
- **“State the consequence” means the consequence.** Naming a defect earns half the marks; saying what it does to the running system — which byte is lost, which poll never returns, what appears in the terminal — earns the other half.

There is a fifth, particular to this subject. Several questions ask what a design does in **simulation or on the host** and what it does **on the bench**, and the answer is that they differ. Two questions go further and ask what a provided testbench *cannot* catch. An answer that treats a green testbench as proof has missed the point of the question, however correct the rest of it is.

Self-Assessment Paper A

Time: 4 hours. **Closed book.** A basic calculator is permitted. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all ten chapters and on both halves of the system, so **it is meant to be taken once you have finished the book**, after Chapter 10 and the bring-up ladder. The model answers are in Appendix F.

Rubric

The protocol specification is the single source of truth. Where an implementation and the protocol disagree, the implementation is wrong. An answer that argues from the contract beats one that argues from code.

Both languages are the subject, so you write both here. VHDL is VHDL-93 using `ieee.std_logic_1164`, and `ieee.numeric_std` where a conversion needs it. C++ that would run on the ATmega328P is written in the book's AVR-portable subset: `<stdint.h>` and the bare types, no `std::`, nested namespace blocks rather than `namespace driver::uart`, no `[[nodiscard]]`, no inline variables. Host-only test code may use full modern C++, and a question says so when it means it.

Syntax is not what is being marked. The design is. A missing semicolon, a forgotten `is` or an absent `#include` costs nothing. A process that describes a latch where you meant a wire, or a `readReg` that assembles its four bytes backwards, costs that part in full, because that is a different system.

The book's conventions are the ones expected, and a design that breaks one should say why:

- VHDL ports are declared all inputs first, then all outputs, and every port map and generic map is **positional**.
- An active-low signal is named with an `_n` suffix; a signal that has been through a two-flop synchronizer is named with an `_s2` suffix. Every submodule takes `reset_s2_n`; only `uart_top` takes the raw `reset_n`. Signals carrying SPI traffic between the two transport blocks take a `spi_` prefix.
- Resets are asserted **asynchronously**, so `reset_s2_n` belongs in the sensitivity list.

- C++ uses camelCase, a leading capital on type names, my on private members and our on private statics, brace initialization, noexcept on driver methods, and copy and move deleted on any class holding a reference member or a unique hardware resource.

Where a question asks what reaches the wire, answer in hardware or in bytes, not in code. “It calls writeReg” is not an answer to “what does the peripheral do”.

Where a question asks for a stated number of defects, listing more is not penalised, but only that many are marked, so put your strongest answers first.

Supplied constants and formulas

Quantity	Value
DE0-CV system clock	50 MHz, a 20 ns period
Arduino Nano clock (f_{osc})	16 MHz
SPI SCK	$f_{\text{osc}}/16 = 1 \text{ MHz}$, mode 0, MSB first
Receiver oversampling	16 times
Default frame	8N1: 1 start bit, 8 data bits LSB first, 1 stop bit
Baud divider	$\text{BAUD_DIV} = \text{round}(50_000_000 / (16 * \text{baud}))$
SPI transaction	5 bytes: 1 command byte, then 4 data bytes MSB first
SPI command byte	bit 7 = W (1 write, 0 read), bits 6–4 = 0, bits 3–0 = register index

The register map itself is **not** supplied: Question 1 asks for part of it.

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	13	14	13	12	13	11	12

Question 1 The map, the top, and what a clean analysis does not prove

12 MARKS

(a) A driver reads STATUS over SPI and gets back 0x0000000D.

Name every STATUS bit, give its position, and say whether this value has it set or clear. Describe the state the peripheral is in, in one sentence. Then state what `Uart::write(0x41)` and `Uart::read(byte)` would each do if called immediately afterwards, including any register access each does **not** perform.

Finally, give the register index and the byte offset of ERROR_FLAGS, and the two command bytes that would read it and write it. (3 marks)

(b) `uart_top`’s entity declares eight ports in the order `clock`, `reset_n`, `sclk`, `mosi`, `ss`, `rx`, `miso`, `tx`. A colleague swaps `sclk` and `mosi` in the entity and changes nothing else.

State whether `ghdl -a` rejects the file, and why. State whether elaborating `uart_top_tb` rejects it. Name the first check anywhere in the course that fails, and the chapter in which that happens.

Then state the single change to the book's conventions that would have moved this failure forward to analysis time, and what that change costs. (3 marks)

(c) Two things about `reg_rdata` and one about `baud_div`.

(i) Chapter 1 leaves the placeholder `reg_rdata <= (others => '0')`; in `uart_top`, and Chapter 5 instantiates `uart_regs`, whose `reg_rdata` output drives the same signal. A candidate forgets to delete the placeholder. `uart_top_tb` writes `BAUD_DIV` with `0x00000004` and reads it back.

Name the VHDL mechanism that now governs `reg_rdata`, give the value each of the low four bits carries during the read-back, and state what the testbench reports. (2 marks)

(ii) Before Chapter 5, `baud_div` has no driver at all, and `uart_top` still has to hand `baud_gen` a natural range 1 to 65535. State the value `to_integer(unsigned(baud_div))` returns in that situation and the message it prints, and state why the value the library picks is worse than merely wrong here. Then give the guarded expression the book uses, and say which of its two cases covers this and which covers a legitimate value that arrives once the register bank exists. (2 marks)

(d) `reset_sync` and `sync` are both two flip-flops in series, and the book keeps them as separate modules.

Name the port on `sync` that makes it impossible for `sync` to be the reset synchronizer, and describe the circular dependency you would create by trying. Then state what “assert asynchronously, release synchronously” buys in a design of a few hundred flip-flops, and what could go wrong if the release were asynchronous too. (2 marks)

Question 2 The wire, the divider, and the transmitter

13 MARKS

(a) The byte `0x53` is sent 8N1 by `uart_tx`, which builds its frame as

```
frame <= STOP_BIT & data & START_BIT;
```

and then drives `tx <= frame(bit_idx)` with `bit_idx` walking from 0 to 9.

Give `frame` as a 10-bit vector, index 9 down to 0. Then give the ten levels `tx` carries in **time order**, naming the frame index each one comes from and what each bit is (start, data bit *n*, or stop).

State how many `baud_ticks` the whole frame takes, and how many 50 MHz clock cycles that is at 115200 baud. (4 marks)

(b) The peripheral is to run at 57600 baud from the 50 MHz clock.

Give the exact (unrounded) divider, the value written to BAUD_DIV, the baud rate actually achieved, and the error as a percentage with its sign. State why the receiver at the other end tolerates it, and name precisely what has to still be true, and at which bit of the frame, for the byte to be received correctly. (3 marks)

(c) Here is the state machine of uart_tx, unchanged:

```
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        tx <= '1'; done <= '0'; state <= STATE_IDLE;
        frame <= (others => '0'); bit_idx <= 0; ticks <= 0;
    elsif (rising_edge(clock)) then
        done <= '0';
        case (state) is
            when STATE_IDLE =>
                tx <= '1';
                if (start = '1') then
                    state <= STATE_SEND;
                    tx <= '0';
                    frame <= STOP_BIT & data & START_BIT;
                end if;
            when STATE_SEND =>
                tx <= frame(bit_idx);
                if (baud_tick = '1') then
                    if (ticks >= MAX_TICKS - 1) then
                        ticks <= 0;
                        if (bit_idx >= FRAME_WIDTH-1) then
                            done <= '1'; state <= STATE_IDLE; bit_idx <= 0;
                        else
                            bit_idx <= bit_idx + 1;
                        end if;
                    else
                        ticks <= ticks + 1;
                    end if;
                end if;
            when others =>
                state <= STATE_IDLE;
        end case;
    end if;
end process;
```

State at which clock edge tx first goes low relative to the edge on which start was seen, and say what that means for when the start bit begins relative to baud_gen's tick.

Then, for a divider `div`, give the **shortest** and the **longest** the start bit can be, in clock cycles, and explain where the variation comes from. Express the worst case as a fraction of a nominal bit period, and state why neither the data bits nor the stop bit inherit it. (3 marks)

(d) `uart_tx_tb` checks the eight data bits least significant first, and Chapter 2 describes this as the check that catches a transmitter sending most significant first. An earlier revision of the testbench sent `TXBYTE = x"A5"`; it sends `x"53"` today.

Write out the eight data-bit levels the line would carry under each of the two orderings for `0xA5`, and use them to show that the testbench could not have caught that bug at all. Name the property of `0xA5` responsible, and state how many of the 256 possible bytes share it.

Then repeat the exercise for `0x53`, state which data bit the testbench reports as wrong first, and say what the episode tells you about reading a passing test as proof. (3 marks)

Question 3 The asynchronous line

14 MARKS

(a) The `rx` pin arrives from another chip.

State what can happen to a flip-flop that samples it directly, and be precise about what the failure actually is: getting either level back from a mid-transition sample is *not* it. State why two flip-flops in series fix it and why one does not.

Then state which module in this design instantiates the synchronizer, which module deliberately does **not**, and what the receiver's port name for the line tells a reader about that decision. (3 marks)

(b) A correctly built `uart_rx` receives `0x53`, sent 8N1 least significant first, and stores each sampled bit with `frame(bit_idx) <= rx_s2`, walking `bit_idx` from 0 upward.

Give the eight data-bit levels in the order they arrive on the line, and the `bit_idx` each lands in. Show `frame` after the eighth, and confirm `data_out` carries `0x53`.

Then a colleague counts `bit_idx` **down** from 7 instead. Give the byte `data_out` would carry for this same frame, in hex, and state the general relationship between what was sent and what is delivered. (3 marks)

(c) The receiver aims to sample each bit at tick 8 of 16. It does not quite. Three things each cost one tick: the falling edge is only noticed at the next `baud_tick`; `ticks` is compared before it is incremented; and every `ticks <= 0` on a state change spends one more. The result, as built, is that the start-bit re-check arrives **nine** ticks after the detected edge rather than eight, the first data sample **seventeen** ticks after that re-check, the stop-bit sample **seventeen** after the last data bit, and data bit to data bit exactly **sixteen**.

(i) State how far past the centre of its own bit each of the start-bit re-check, a data bit and the stop bit is sampled, in ticks. State why the offset is fixed rather than accumulating across the byte. (1 mark)

(ii) Take an **ideal** receiver first, sampling every bit exactly at its centre. Working in ticks measured from the start-bit edge, give the tick at which the stop bit is sampled and the two

ticks at which the stop bit begins and ends. Hence derive the combined baud mismatch, as a percentage, at which the stop-bit sample first falls outside the stop bit. (2 marks)

(iii) Now do the same arithmetic for the receiver as built. Give the two limits, state which side of the ideal figure has been eaten into and by how much, and say in one sentence which physical situation that lost margin makes more dangerous. (2 marks)

(d) Both FIFOs in the finished peripheral are `WIDTH = 8`, `DEPTH = 8`.

Write `fifo's` clocked process, and the concurrent assignments that go with it for `rdata`, `empty` and `full`. Assume the declarations `entries`, `head`, `tail`, `count` and a helper that advances a pointer with wraparound; write everything else. Guard both directions — a write to a full FIFO and a read from an empty one must each do nothing — and be careful that a push and a pop landing on the same edge leave `count` where it was.

Then software stops reading while frames keep arriving. State how many back-to-back frames the RX path absorbs before anything is lost, what happens to the next one, and which line of what you just wrote decides it. Name the `ERROR_FLAGS` bit reserved for that loss, and state what it reads in this book's build and why that is a reservation rather than an omission. (3 marks)

Question 4 Ports become registers

13 MARKS

(a) `STATUS` is computed, never stored.

Write the concurrent assignments that build the whole 32-bit `status_reg` in `uart_regs`: all four defined bits, indexed by their `uart_def` names, in terms of the signals the bank actually has, and the reserved bits above them. Then say in a few words, for each defined bit, why it could not be a stored register that the datapath writes.

Then: `STATUS` is read at a moment when the TX FIFO holds two of its eight bytes, the transmitter is mid-frame, the RX FIFO is empty and no error has been latched. Give the 32-bit value the bank presents on `reg_rdata`. (3 marks)

(b) A candidate makes `RX_DATA` advance the FIFO as a side effect of being read, so that the driver needs no `RX_POP`.

Name the check in `uart_regs_tb` that fails and state what it asserts. Then, in terms of the SPI transport's abort rule, state why a read with a side effect is genuinely harder to get right than a pure read plus a separate write, and name the thing the bridge would have to tell the bank that it currently has no way to say.

Finally trace the driver side, with the popping read in place and the driver's own `RX_POP` write still present. The RX FIFO holds `0x41` then `0x42`, and nothing more arrives. Give what `uart::read()` hands back on the first call and on the second, and say what became of `0x42`. (4 marks)

(c) A candidate gates the TX FIFO push on `CTRL` bit 0, reasoning that a disabled peripheral should not transmit. The design analyzes and elaborates.

State what `uart_regs_tb` reports and why. State what `uart_top_tb` reports and why, and say why that second failure message is the less useful of the two.

Then name the fact about `uart_regs'` port list that settles, on its own, whether any CTRL bit may gate anything at all, and say why the bits exist in `uart_def` regardless. (3 marks)

(d) The TX feeder in `uart_top` is two concurrent lines:

```
tx_load <= (not tx_empty) and (not tx_busy);
tx_pop  <= tx_load;
```

State why this loads exactly one byte each time the transmitter goes idle with a non-empty FIFO, never zero and never two, naming what holds `tx_load` low for the rest of the frame.

Then a colleague simplifies it to `tx_load <= not tx_empty;`, keeping `tx_pop <= tx_load;`, arguing that `uart_tx` ignores start while it is sending anyway. They are right about start. Say what happens to a TX FIFO holding four bytes, how many of them reach the wire, and how long the rest survive. State whether `uart_top_tb` catches it, and why. (3 marks)

Question 5 The driver's contracts

12 MARKS

(a) Both sides of the wire store bit **positions**, not masks.

Give the C++ expression that tests RX-valid in a `uint32_t` `status`, and the VHDL expression that indexes the same bit of a `std_logic_vector`.

Then three one-line changes. For each, name the defect and state what the running system does — not merely that it is wrong.

- `if (status & status::RX_VALID) { ... }`
- `writeReg(reg::CTRL, ctrl::ENABLE);`
- `register_map.hpp` declares `constexpr uint8_t RX_VALID{1U << 1};` while `uart_def.vhd` keeps `ST_RX_VALID : natural := 1;`

(4 marks)

(b) `driver::transport::Interface` has three methods and knows nothing about registers.

Name the three, and name the two things the seam deliberately does not know. State what putting the seam at the **byte** level buys that a seam exposing `readReg` and `writeReg` would not, then name the two concrete implementations the book puts under it and state exactly what changes above the seam when they swap. (3 marks)

(c) This header compiles on the host and fails on the AVR toolchain the book targets:

```
#include <cstdint>

namespace driver::uart
```

```

{
    inline constexpr std::uint8_t TxReady{0U};

    class Interface
    {
    public:
        virtual ~Interface() noexcept = default;
        [[nodiscard]] virtual std::uint32_t status() const noexcept = 0;
    };
} // namespace driver::uart

```

Name **four** things in it that the AVR build rejects, and in each case say what the toolchain is actually missing rather than merely that it is old.

Then write the header out corrected, as the book would have it: the include, the namespaces, the constant and the class, with nothing left in it the AVR build cannot take.

(3 marks)

(d) write() and read() are non-blocking and each returns a bool.

State what each bool means. Name the STATUS bit that makes each answer possible, and the Chapter 5 mechanism behind that bit. Then state where the blocking versions live, why they are free functions taking Interface& rather than methods, and what would be lost if the interface itself blocked.

(2 marks)

Question 6 The five-byte transaction

13 MARKS

(a) A host test constructs a driver::transport::Stub, constructs a real Uart over it, and runs some driver calls. Afterwards the stub's recorded MOSI bytes are, in order:

```

82 00 00 00 1B
81 00 00 00 01
00 00 00 00 00
04 00 00 00 00
85 00 00 00 01

```

with beginCalls() and endCalls() both 5.

Decode every command byte: direction and register. Name the driver call that produced each transaction, and say which transactions belong together as one call.

State what the driver did with the bytes that came back during the third transaction and during the fourth, being specific about the order in which it assembled them.

Then state what beginCalls() and endCalls() prove that the byte log on its own does not, and describe the wrong behaviour that a byte log alone would fail to distinguish from this one.

(4 marks)

(b) A candidate sends the four data bytes least significant first in `writeReg`, and assembles the four returned bytes least significant first in `readReg`. The two are consistent with each other, so a stub round trip through this driver still agrees with itself.

(i) Give the five bytes that `configure(27)`'s first transaction now puts on the wire, and the 32-bit value the peripheral commits to `BAUD_DIV`. State the 16-bit value `baud_div` therefore carries, what `uart_top`'s guarded conversion does with it, and the baud rate the line actually runs at. (2 marks)

(ii) The peripheral presents a `STATUS` of `0x0000000B`. Give the `uint32_t` this driver ends up holding, and state what `write()` returns from then on and what `writeBlocking()` therefore does. (1 mark)

(c) The receive path is poll `STATUS`, read `RX_DATA`, write `RX_POP`.

State what happens if the `RX_POP` write is left out, and what `app::EchoNode` does on the bench as a result. State what happens if the pop is issued **before** the `RX_DATA` read.

Then name the host test that catches a driver which pops even when `RX-valid` is clear, state what that pop actually does to this peripheral, and say why the answer “nothing bad” is not a defence. (3 marks)

(d) `readReg` is a `const` member, and `status()` and `errorFlags()` are `const` because of it. The transport's `begin()`, `transfer()` and `end()` are not `const`.

State why those three are not `const`. State what `const_cast<transport::Interface&>(myTransport)` does, and why the compiler would in fact accept the calls without it as the class is written today. Name the change to `Uart` that would make the cast genuinely necessary.

Then state the one circumstance under which casting away `const` is undefined behaviour, and say why this is not it. (3 marks)

Question 7 The real transport, and the real toolchain

11 MARKS

(a) `AvrSpi`'s constructor is three statements:

```
DDRB |= (1U << SCK) | (1U << MOSI) | (1U << SS);
PORTB |= (1U << SS);
SPCR = (1U << SPE) | (1U << MSTR) | (1U << SPR0);
```

State what each statement achieves. The transport contract asks for mode 0, MSB first, and `SCK` at 1 MHz from a 16 MHz clock: state how this `SPCR` value delivers all three, naming the bits that are **not** set and what each one's absence means.

State why `MISO` appears in none of the three statements. Then state what would be on the `SS` line without the middle statement, and what the FPGA's bridge would make of the first transaction. (4 marks)

(b) A colleague writes:

```
uint8_t AvrSpi::transfer(const uint8_t byte) noexcept
{
    SPDR = byte;
    return SPDR;
}
```

State the byte this returns and where it came from. State how many SCK periods must pass before SPDR holds the byte that arrived on MISO, and how long that is at this bench's SCK rate.

State what the correct spin waits on, and name the side effect that reading SPSR and then SPDR has. Then say what the driver above would see, on a readReg (STATUS), with this transfer in place. (3 marks)

(c) SS/PB2 is configured as an output even though the transport drives the chip select itself.

State what the SPI peripheral does if SS is left an input and something pulls it low mid transaction, naming the SPCR bit the hardware changes. State what transfer()'s spin does in that moment, and give the symptom a bench would show from then on. (2 marks)

(d) Name two constructs used freely in the host build that do not survive the freestanding avr-gcc build, and what replaces each.

Then write out avr/env.cpp's definitions: every symbol the file has to supply for this book's code to link on the ATmega, with the right signatures and extern "C" where it belongs, and for each name the ordinary C++ construct that emits the reference. Finally state why the file lives in avr/ rather than source/. (2 marks)

Question 8 The bench

12 MARKS

(a) A user types A (0x41) into the PC terminal, and app::EchoNode echoes it.

Name, in order, every module the byte passes through from the USB-serial adapter until it reaches the terminal again. For each, name the chapter that built it (or mark it as provided), and say which of the two planes — control or data — it is on at that moment.

Then name the three representations the byte takes across that journey, and the two places where it changes from one to the next. (4 marks)

(b) A bench climbs the ladder. Rungs a, b and c pass. Rung d fails: bytes the driver writes appear in the terminal as wrong characters, and characters typed produce framing errors rather than bytes.

State what the ladder has already cleared, rung by rung, and name what is left that rung d is the first to exercise.

Name the most likely cause, and justify it by saying what rung c does that hides it. Then give the arithmetic that would confirm or exonerate BAUD_DIV = 27 as the culprit, and say what magnitude of error you would have to see before the divider itself is to blame. (3 marks)

(c) Write the `main()` that runs on the ATmega328P on bring-up day, in `fw/avr/`: the includes it needs from this book's headers, the objects it constructs and in what order, the `configure()` call for 115200 8N1, the flag it passes, and the call that starts the application. It allocates nothing and it never returns.

Then state why the flag this `main` passes is left `false` where the host test's is not, and why `EchoNode::run()` must poll the non-blocking `read()` rather than call `readBlocking()` — connecting that to how the host test manages to end the loop at all. (3 marks)

(d) One of the four SPI lines is wired 5 V straight to a 3.3 V FPGA pin, with no level shifter.

State the likely symptom and the risk. Name the first rung of the ladder that exposes it and state why the rung before it cannot.

Then name the one SPI line for which the omission is electrically safe, say which direction it travels and why, and state what could still go wrong with it. (2 marks)

Self-Assessment Paper B

Time: 4 hours. **Closed book.** A basic calculator is permitted. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all ten chapters and on both halves of the system, so **it is meant to be taken once you have finished the book**, after Chapter 10 and the bring-up ladder. The model answers are in Appendix G.

Rubric

The protocol is the single source of truth. Where an implementation and the protocol in Appendix A disagree, the implementation is wrong. An answer that argues from the contract beats one that argues from code.

Both languages are the subject, so you write both here. VHDL is VHDL-93 using `ieee.std_logic_1164`, and `ieee.numeric_std` where a conversion needs it; write the library and use clauses out at least once, after which they are assumed. C++ that would run on the ATmega328P is written in the book's AVR-portable subset: `<stdint.h>` and the bare types, no `std::`, nested namespace blocks rather than `namespace driver::uart`, no `[[nodiscard]]`, no inline variables. Host-only test code may use full modern C++, and a question says so when it means it.

Syntax is not what is being marked. The design is. A missing semicolon, a forgotten `is` or an absent `#include` costs nothing. A process that describes a latch where you meant a wire, or a `readReg` that assembles its four bytes backwards, costs that part in full, because that is a different system.

The book's conventions are the ones expected, and a design that breaks one should say why:

- VHDL ports are declared all inputs first, then all outputs, and every port map and generic map is **positional**.
- An active-low signal is named with an `_n` suffix; a signal that has been through a two-flop synchronizer is named with an `_s2` suffix. Every submodule takes `reset_s2_n`; only `uart_top` takes the raw `reset_n`. Signals carrying SPI traffic between the two transport blocks take a `spi_` prefix.

- Resets are asserted **asynchronously**, so `reset_s2_n` belongs in the sensitivity list.
- C++ uses camelCase, a leading capital on type names, `my` on private members and `our` on private statics, brace initialization, `noexcept` on driver methods, and `copy` and `move` deleted on any class holding a reference member or a unique hardware resource.

Where a question asks you to review a design, naming the defect earns half the marks and saying what it does to the running system earns the other half. “This is wrong” scores nothing.

Supplied constants and formulas

Quantity	Value
DE0-CV system clock	50 MHz, a 20 ns period
Arduino Nano clock (f_{osc})	16 MHz
SPI SCK	$f_{osc}/16$, which is 1 MHz; mode 0, MSB first
Receiver oversampling	16 times
Default frame	8N1: 1 start bit, 8 data bits LSB first, 1 stop bit
Baud divider	$BAUD_DIV = \text{round}(50_000_000 / (16 * \text{baud}))$
SPI transaction	5 bytes: 1 command byte, then 4 data bytes MSB first
SPI command byte	bit 7 = w (1 write, 0 read), bits 6–4 = 0, bits 3–0 = register index

The register map itself is **not** supplied: Question 5 asks you to write it out, and Questions 4 and 6 consume it, which is what the follow-through rule is for.

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	13	14	13	12	13	11	12

Question 1 Building the top before anything it holds

12 MARKS

(a) Write the complete `uart_top` entity, with its library and use clauses and the book’s banner comment naming the inputs and outputs. Eight ports, all inputs first, in the order the system testbench binds them.

Then name the use clause the file does **not** need in Chapter 1 and say why, name the chapter that adds it and the single expression that needs it, and state why `uart_top` never writes `use work._uart_def.all` at all. (3 marks)

(b) Here are the two provided transport blocks’ port lists, in declaration order.

<code>spi_slave</code>	1 clock	2 reset_s2_n	3 sclk	4 mosi	5 ss
	6 tx_data	7 miso	8 rx_data	9 rx_valid	10 ss_active

```
spi_reg_bridge 1 clock      2 reset_s2_n  3 ss_active  4 rx_data   5 rx_valid
               6 reg_rdata 7 tx_data    8 reg_addr   9 reg_wdata 10 reg_write
```

spi_slave's ports 1–5 are std_logic inputs, 6 is an 8-bit input vector, 7 is a std_logic output, 8 is an 8-bit output vector, and 9–10 are std_logic outputs. spi_reg_bridge's ports 1–3 are std_logic inputs, 4 is an 8-bit input vector, 5 is a std_logic input, 6 is a 32-bit input vector, 7 is an 8-bit output vector, 8 is a 4-bit output vector, 9 is a 32-bit output vector, and 10 is a std_logic output.

Declare **exactly** the internal signals these two instantiations and the provided reset_sync require, with their types, following the book's naming conventions, and write the three positional instantiations. Do not declare anything a later chapter will bring.

Then answer two things. spi_slave and spi_reg_bridge both have a port called rx_data: state why those are not one bus and which way each of your signals runs. And state why reg_addr is four bits wide when there are only seven registers. (5 marks)

(c) Chapter 1 ends with one placeholder assignment, because the register bank is four chapters away.

Write it. Then state what an SPI **read** transaction returns while it is in place and why that is harmless until Chapter 5, and state what a read would return instead if the placeholder were left out altogether — being precise about what “returns” means for a signal nobody drives. (2 marks)

(d) State what building uart_top before any block it instantiates fixes about every block you write later.

Then: baud_gen arrives in Chapter 2. State exactly what has to be true of its entity for its instantiation in uart_top to bind correctly without you touching the top again, and name the one thing about it that the book explicitly says is *not* part of the contract. (2 marks)

Question 2 The generator, the divider, and the frame

13 MARKS

(a) Write baud_gen.vhd complete: the library and use clauses, the entity with its four ports in order and their exact types, and the architecture. It produces a registered, one-clock-wide tick every div clocks, and its reset is the book's.

Then justify your comparison. State what counter >= div - 1 and counter = div - 1 each do when div is 1, and what each does if div could ever be 0, and say which keeps the counter inside its declared range. (4 marks)

(b) The peripheral is to run at 9600 baud.

Give the exact (unrounded) divider, the value written to BAUD_DIV, the baud rate achieved, and the error as a percentage with its sign.

Then state why the division is by $16 * \text{baud}$ rather than by baud , name the module that spends the sixteen ticks and say what it spends them on, and state what the transmitter does with the same tick stream. (3 marks)

(c) `uart_tx` produces 8N1 only. Extend it with `parity_en` and `parity_odd` inputs.

Give the new frame vector expression and its index range, say which index the parity bit occupies and which the stop bit now does, and give the expression that computes the parity bit from data for even parity and for odd.

Then a `two_stop` input is added as well. Name the only field of the frame whose length now varies, give the three possible frame lengths in bits, and state what the longest frame costs in transmitted bytes per second at 115200 baud compared with 8N1. (3 marks)

(d) A candidate's transmitter passes `uart_tx_tb`. Their busy is not the book's concurrent line but a registered flag, set inside the clocked process on the edge *after* the state becomes `STATE_SEND`, so it rises one clock late.

State what `uart_tx_tb` checks about busy and what it does not. Then state what `uart_top's` TX feeder does during that one clock, what `uart_tx` does with what the feeder sends, and what happens to the byte involved. Give the one-line fix and say which chapter's testbench is the first that could have caught it. (3 marks)

Question 3 The asynchronous line, written out

14 MARKS

(a) Write `sync.vhd` complete: the library and use clauses, the entity with its generic and its four ports in order, and the architecture. Two flip-flops per bit, the book's reset, and the output taken from the right one.

Then answer two things. State how many rising edges `sync_out` takes to follow `async_in`, and why neither one fewer nor one more is correct. And state what a `COUNT = 8` instance does **not** guarantee about eight bits that change on the same edge, why that is not a defect in `sync`, and one kind of signal you must therefore never put through a wide instance. (4 marks)

(b) Write the `STATE_IDLE` and `STATE_START` branches of `uart_rx's` state machine as the book specifies them, including the one flip-flop of history, where in the process it is updated, and why that position matters. Assume the surrounding `if baud_tick = '1'` and the tick counter are already written.

Then a candidate replaces the falling-edge test with a level test, `if rx_s2 = '0'` then. State what this does during a **break** — the line held low for longer than one frame — and then state precisely what happens to the frame that is still in flight at the moment the break *ends*. Name the testbench case that catches it and say what its message tells you. (4 marks)

(c) Write the `STATE_STOP` branch, and the output defaults at the top of the clocked process that go with it.

State what makes `valid` and `frame_err` exactly one clock wide, why the byte is dropped on a framing error rather than delivered alongside a flag, and what the register bank does with each of the two pulses. (3 marks)

(d) An earlier revision of `uart_rx_tb` sent `0xA5` in its clean-frame case and `0x3C` elsewhere, while `uart_top_tb` sent `0x5A` through a tx-to-rx loopback.

Show that a receiver which stores its data bits in the wrong order passed all three. Name the property `0xA5`, `0x3C` and `0x5A` share, state how many of the 256 bytes have it, and give the **second**, independent reason `uart_top_tb` could not have caught a bit-order convention shared by `uart_tx` and `uart_rx`, however good its constant was.

`uart_rx_tb`'s clean-frame case sends `0x53` today. State the value `data_out` carries under the faulty receiver, and hence what the testbench now reports. Then state which single kind of bit-order mistake remains beyond the reach of every testbench in hw/, and why. (3 marks)

Question 4 The bank, written out

13 MARKS

(a) Write the `ERROR_FLAGS` latch for the framing bit: its reset value, the condition that sets it, the condition that clears it, and what happens when both occur on the same clock. Use `uart_def`'s constants rather than numbers.

Then state why a single-cycle error pulse cannot be polled by software at all, and say what would actually be observed if `STATUS` bit 2 were wired straight to the receiver's `frame_err` output and a driver polled `STATUS` once per SPI transaction. (4 marks)

(b) Write the bank's read path: the combinational decode that drives `reg_rdata` from `reg_addr`, covering all seven registers and the reserved indices, using `uart_def`'s constants. Then write the two write **actions**, for `TX_DATA` and for `RX_POP`, as the book's bank implements them.

State why the read is combinational rather than registered, what the when others branch must produce and why getting it wrong would be a different kind of mistake here than in a purely combinational decoder, and what the bank relies on the provided bridge to guarantee about `reg_write`. (4 marks)

(c) Overrun is reserved in this course but not implemented. Design it.

Give the condition that detects an overrun, in terms of signals `uart_regs` already has. Give the `ERROR_FLAGS` bit it sets and its position, and the clear path it shares. State the one piece of information `uart_rx` would have to be given for it to detect the overrun itself instead, and why handing it that information is a worse design.

Then give a physical cause for an overrun and a different physical cause for a framing error. (3 marks)

(d) A candidate's bank stores `STATUS` in a 32-bit register, updated in the clocked process whenever `rx_push`, `tx_pop` or `frame_err` asserts. It passes a test they wrote themselves.

Give two distinct ways it disagrees with `uart_regs_tb`, and state the general rule the book draws from it in one sentence. (2 marks)

Question 5 Writing the contracts

12 MARKS

(a) Write `include/driver/transport/interface.hpp` and `include/driver/uart/interface.hpp` in full, in the book's AVR-portable style: the include guard or `#pragma`, the includes, the namespaces, the destructors, and every pure virtual method with its exact signature, parameters, return type and qualifiers.

`driver::transport::Interface` has three operations. `driver::uart::Interface` has six. (5 marks)

(b) Write the `reg` and `status` nested namespaces of `include/driver/uart/register_map.hpp`, with the correct names and values, in the book's style.

Then state why the constants are declared plain `constexpr` rather than `inline constexpr`, and what "internal linkage at namespace scope" costs here. And state why the `map` stores bit **positions** rather than masks, naming the file on the other side of the wire that this decision keeps identical and the one-line idiom that forms a mask at the use site. (3 marks)

(c) `driver::uart::Stub` holds `volatile bool& myStop` as a member.

State what that reference is for and who owns the flag it refers to. State what `read()` does with it when the scripted RX buffer runs out, and why that behaviour exists at all given that a stub has no reason to care how a caller's loop terminates.

Then state what holding a reference member forces about the class's copy constructor, move constructor and default constructor, and why. (2 marks)

(d) `driver::uart::Stub`'s `status()` returns 0, its `errorFlags()` returns 0 and its `clearErrors()` does nothing, even though a real UART's STATUS is the register the whole driver is built around.

State the general rule about what a test double must be faithful about and what it may ignore, and apply it here: say what would have to change about the stub before it could be used to test something new, and give an example of such a thing. (2 marks)

Question 6 Writing the protocol

13 MARKS

(a) Write `Uart::readReg(uint8_t addr) const` and `Uart::writeReg(uint8_t addr, uint32_t value)` in full, in the book's style, using only `myTransport`.

Both perform one five-byte transaction. Get the command byte, the framing and the byte order right, and say in one line each what `readReg` does with the reply to the command byte and what `writeReg` does with all four of its replies.

Then state what marking `readReg const` buys the class, and name the two public methods that depend on it. (5 marks)

(b) Write `Uart::read(uint8_t& byte)`.

Then, for one successful call on a peripheral whose RX FIFO front byte is `0x41`, write out the exact bytes a `driver::transport::Stub` would record, transaction by transaction, and give `beginCalls()` and `endCalls()`.

Finally, write the scripting the test must perform before the call. The stub's helpers are `injectRxByte(uint8_t)` and `injectRxWord(uint32_t)`, the latter queuing four bytes most significant first. State how many bytes in total have to be queued for this one call and explain the discrepancy between that number and the number of register values involved.

(4 marks)

(c) Write `writeBlocking()` and `readBlocking()` from `driver/uart/blocking.hpp`.

State why each is inline, why each takes `Interface&` rather than `Uart&`, and why they are free functions in a separate header rather than methods on the interface. (2 marks)

(d) A candidate implements `readReg` by calling the driver's own `write()` to send each byte, reasoning that `write()` is "the method that sends a byte".

State what happens when `status()` is called on this driver, in terms of the call sequence, and name the failure the ATmega328P actually exhibits. Then state the one-sentence rule that keeps the two apart. (2 marks)

Question 7 Writing the transport

11 MARKS

(a) Write `driver::transport::AvrSpi` in full: the constructor, the destructor and the three seam methods, in the book's style, reaching the registers through the platform header's `SCK`, `MOSI`, `MISO` and `SS` bit positions and the `DDRB`, `PORTB`, `SPCR`, `SPSR` and `SPDR` registers.

The class configures the ATmega328P as an SPI master at $f_{osc}/16$, mode 0, MSB first.

Then state exactly what the destructor must clear, what it must leave alone and why, and state why the constructor writes `SPCR = ...` rather than `SPCR |= ...`. (5 marks)

(b) A candidate ships `~AvrSpi() noexcept override = default;`, on the grounds that the class has no members to release and the program never destroys it anyway.

Name the principle broken and list what the constructor took that is now never given back. Then give one concrete way this harms code that runs afterwards, and one reason "the program never destroys it" is not a safe assumption in this codebase in particular. (3 marks)

(c) The SPI transport needs no `F_CPU`; the debug logging over the ATmega's own USART does.

State what sets the SPI `SCK` rate and what sets the USART's baud rate, and use that to explain the difference.

Then: the Nano's fuses actually run it at 8MHz, but the build still defines `F_CPU=16000000UL`. State what happens to `SCK`, in figures, and whether the SPI link still works. State what happens to the debug USART, in figures, and what appears

in the terminal. Say which of the two failures you would notice first and why that is unfortunate. (3 marks)

Question 8 The application, and the plan

12 MARKS

(a) Write `app::EchoNode` in full — `include/app/echo_node.hpp` and `source/app/echo_node.cpp` — in the book's AVR-portable style. It implements `app::Interface`, whose single operation is `void run(const volatile bool& stop) noexcept`.

Include the member, the constructor, the deleted operations and the loop body. State why the receive is the non-blocking `read()` while the echo is `writeBlocking()`, and why `stop` is a `volatile bool` passed by `const` reference rather than an `atomic` or a return value. (4 marks)

(b) Describe the host test for it precisely enough to write: which stub it runs over, what it queues, how the loop is made to terminate, and what it asserts and in what order. Host test code may use full modern C++.

Then state what an additional case that queues **nothing** proves, name the implementation mistake it isolates, and say whether the three-byte case can catch that mistake too. (3 marks)

(c) Give the five rungs of the bring-up ladder, in order, and for each name the single layer it adds that the previous rung did not exercise.

State why the order is forced rather than a matter of taste, naming the register that settles it and the reason it can only be reached one way.

Then a colleague proposes flashing `app::EchoNode` first: “if it echoes, everything works, and if it does not we start debugging.” Give two reasons this is a worse plan **even when it succeeds**. (3 marks)

(d) A candidate writes `EchoNode` to hold `driver::uart::Uart& myUart` rather than `driver::uart::Interface& myUart`. Everything compiles, the bench works, and the class is otherwise identical.

State what is now impossible, what has to be built before the class can be tested at all, and give the one-line fix. Then name the design decision of Chapter 6 that this makes concrete. (2 marks)

Model Answers to Paper A

Marks are shown per part. Method carries them: a correct derivation with a slip in it is worth more than a correct answer with no working, and later parts consume earlier ones, so an error should be followed through rather than penalised twice. A candidate who decodes the register map wrongly in Question 1 and then uses their own map consistently in Questions 4 and 6 loses the marks once.

Where a part asks for VHDL, mark the **hardware described**, not the syntax. Where it asks for C++, mark the **byte order, the ordering of register accesses, and the qualifiers that carry meaning**, not the punctuation.

Two parts, 2(d) and 4(d), ask what a provided testbench *cannot* catch. An answer that treats a passing testbench as proof has missed the point of the question however correct the rest of it is, and those parts are marked accordingly.

Question 1 The map, the top, and what a clean analysis does not prove

12 MARKS

(a)

3 MARKS

0x0000000D is 1101 in its low nibble.

Bit	Constant	Position	Meaning	In 0x0D
0	ST_TX_READY	0	the TX FIFO is not full	set
1	ST_RX_VALID	1	the RX FIFO is not empty	clear
2	ST_ERROR	2	one or more ERROR_FLAGS bits set	set
3	ST_TX_IDLE	3	the TX FIFO empty and the line idle	set

(1 mark for all four names and positions, correctly read against the value)

The state. The transmitter has nothing to send and nothing in flight — TX-idle implies TX-ready, which is why both are set — no received byte is waiting, and at least one error has been latched since the flags were last cleared. In this book's build that error can only be a framing error, since ER_FRAMING is the only flag with a producer.

What each call does.

- `write(0x41)` reads STATUS, finds TX_READY set, writes 0x41 to TX_DATA and returns true. Two transactions, ten bytes.
- `read(byte)` reads STATUS, finds RX_VALID clear, and returns false. It performs **no** RX_DATA read and **no** RX_POP write — that omission is the contract, not an optimization, and it is what one of the host tests exists to check.

(1 mark)

ERROR_FLAGS. Index 6, offset **0x18** (the index is the offset divided by four). The command byte to read it is **0x06**; to write it, **0x86**, bit 7 being the write bit.

(1 mark)

(b)

3 MARKS

ghdl -a accepts it. Both ports are in `std_logic`, so swapping them produces a perfectly legal entity: the analyzer checks that each formal has a type and a direction, and both still do. There is nothing here for it to object to.

(1 mark)

Elaboration accepts it too. `uart_top_tb` binds positionally, so it connects its own `sclk` signal to the entity's third port and its `mosi` signal to the fourth. Names never enter into it. Every type and direction matches, so elaboration succeeds — and the same is true inside `uart_top`, where the architecture's positional port `map` into `spi_slave` now hands the SPI clock line to `mosi` and the data line to `sclk`.

Where it surfaces. The first check anywhere in the course that fails is **`uart_top_tb`, at the end of Chapter 5** — the first time the system testbench runs at all. Before that it is *skipped*, not passed: its datapath blocks do not exist. So a wiring mistake made in Chapter 1 sits undetected for four chapters and then arrives as an SPI transaction that produces nothing.

(1 mark)

The change that would move it forward. Bind by **name** rather than by position, in the testbench and in every instantiation. With named association the entity's port *order* stops mattering: both the testbench and `uart_top`'s architecture refer to `sclk` by name, so moving it from position three to position four changes nothing and the transposition becomes a no-op instead of a bug.

The cost is the reason the book did not: named association is verbose, it lets the declared order drift from the documented one, and it changes what “the contract” means. This book made the **order and the types** the contract precisely so that a port table in a chapter is a complete specification of a module — rename every signal and everything still binds. That is a real benefit, and this failure is its real price.

(1 mark)

Accept “give the ports distinct subtypes so that a swap is a type error” as a second answer, with the observation that it is impractical for a bus of `std_logic` pins.

(c)

4 MARKS

(i) **The mechanism is driver resolution.** `std_logic` is a *resolved* type: a signal with more than one driver takes the value its resolution function computes from all of them, bit by bit. The placeholder is a concurrent assignment and therefore a driver; `uart_regs' reg_rdata` output is a second one. Resolving '1' against '0' gives 'X'; resolving '0' against '0' gives '0'. For a read-back of `0x00000004`, the low four bits carry:

Bit	Bank drives	Placeholder drives	Resolved
3	'0'	'0'	'0'
2	'1'	'0'	'X'
1	'0'	'0'	'0'
0	'0'	'0'	'0'

(1 mark)

What the testbench reports. The 'X' propagates out through the bridge and `spi_slave` onto `MISO` and into the testbench's captured word, so `rd(15 downto 0) = x"0004"` is false and the `BAUD_DIV` read-back assertion fires. The symptom generalizes: **every bit the bank drives '1' comes back unknown**, so the value looks like the one you wrote with all its ones missing. That is the signature to recognize, and the fix is to delete the Chapter 1 placeholder line, exactly as Exercise 5.2 instructs.

(1 mark)

An answer that says “two drivers, so it fails” without naming resolution or without the per-bit result gets one of the two marks.

(ii) **The value and the message.** `to_integer` on an all-'U' vector prints

```
NUMERIC_STD.TO_INTEGER: metavalue detected, returning 0
```

and returns 0.

(1 mark)

Why 0 is worse than wrong. `baud_gen`'s `div` port is declared natural range 1 to 65535, so 0 is not a bad baud rate — it is **outside the port's subtype**, and binding it is a run-time bound-check failure that aborts the simulation. A wrong divider you could at least see on a waveform; this one stops the run before there is a waveform. And the library chose the value, not the designer, which is the deeper objection: a warning that silently substitutes a value has made a design decision on your behalf.

The guard.

```
baud_div_int <= 1 when is_x(baud_div) or unsigned(baud_div) = 0
    else to_integer(unsigned(baud_div));
```

- `is_x(baud_div)` covers the **metavalue** case: before Chapter 5 nothing drives `baud_div`, so it sits at all-'U'.

- `unsigned(baud_div) = 0` covers the **legitimate zero** that arrives once `uart_regs` exists: `BAUD_DIV` holds whatever the bank reset it to until software writes a divider, and that reset value is a real, well-defined 0.

(1 mark)

1 is chosen because it is the smallest value the port accepts. Nothing useful is transmitted at either value, so the only thing that matters is that the design elaborates and runs. Worth a comment rather than a mark: or on `BOOLEAN` is short-circuit in VHDL, so the comparison is never evaluated on the all-`'U'` vector, and the metavalue case stays warning-free as well as legal.

(d)

2 MARKS

The port is `sync's reset_s2_n input`. `sync` consumes the synchronized reset; a module that requires a clean reset as an input cannot be the module that manufactures one. Try it and the dependency is circular: `sync's` own two flops would need `reset_s2_n` to start from a known state, and `reset_s2_n` is the thing they were supposed to produce. (1 mark)

What the pattern buys.

- **Assert asynchronously.** The instant `reset_n` falls, every flop clears with no clock edge needed. That matters because the clock may not be running, or may be about to start, and a reset that requires a clock cannot rescue a design whose clock has not arrived.
- **Release synchronously.** The release is lined up with `clock` by `reset_sync's` own two flops, so every flip-flop in the design leaves reset on the same edge.

Released asynchronously, the rising edge of `reset_n` could land inside some flops' setup-and-hold window and not others'. Those flops would leave reset one cycle later than the rest, so the design starts **half-reset**: a state machine already in `STATE_IDLE` beside a tick counter that never cleared, or a FIFO whose head reset while its tail did not. That is a state the design's own logic never anticipates, and it happens at a rate that depends on where the button bounced. (1 mark)

Question 2 The wire, the divider, and the transmitter 13 MARKS

(a) 4 MARKS

0x53 is 0101 0011, so data(7 downto 0) = "01010011".

The frame. Concatenation puts the leftmost item in the highest bits, so frame <= STOP_BIT & data & START_BIT gives

```
frame(9 downto 0) = '1' & "01010011" & '0' = "1010100110"
```

index	9	8	7	6	5	4	3	2	1	0
value	1	0	1	0	1	0	0	1	1	0
is	stop	d7	d6	d5	d4	d3	d2	d1	d0	start

(1 mark)

The wire order. bit_idx walks 0 upward, so the line carries frame(0) first:

time order	1	2	3	4	5	6	7	8	9	10
frame index	0	1	2	3	4	5	6	7	8	9
level	0	1	1	0	0	1	0	1	0	1
is	start	d0	d1	d2	d3	d4	d5	d6	d7	stop

(2 marks: 1 for the order, 1 for the levels)

That is the whole point of building the frame as a vector: read it from index 0 upward and it *is* the wire order, start bit first and data least significant first, so the transmitter never thinks about bit order again.

The timing. Ten bits at sixteen ticks each is **160 baud_ticks**. At 115200 baud BAUD_DIV = 27, so that is $160 \times 27 = 4320$ clock cycles, or $4320 \times 20 \text{ ns} = 86.4 \mu\text{s}$. (1 mark)

(b) 3 MARKS

```
exact divider = 50_000_000 / (16 x 57_600) = 50_000_000 / 921_600 = 54.2535
BAUD_DIV      = round(54.2535) = 54
```

(1 mark)

```
tick rate = 50_000_000 / 54 = 925_925.9 Hz
baud rate = 925_925.9 / 16 = 57_870.4 baud
error     = (57_870.4 - 57_600) / 57_600 = +0.47% (fast)
```

(1 mark)

Why it is tolerated, and what has to be true. The receiver does not run free: it hunts for the falling edge of every start bit and restarts its own tick counter there. So the error does not accumulate across a stream of bytes, only across the ten bit periods of a single frame.

The condition is therefore specific: the **stop-bit sample must still land inside the stop bit**, nine and a half bit periods after the edge that started the frame. That is the last and worst-placed sample in the frame, so it is the one that decides. Question 3 puts the budget at roughly plus or minus five percent combined for an ideally centred sample; 0.47 % at this end leaves the peer almost all of it. (1 mark)

An answer that says only “it resynchronizes each frame” earns half; the mark is for naming the stop bit as the condition.

(c)

3 MARKS

When tx goes low. start is looked at only in STATE_IDLE, and tx <= '0' is scheduled in the same clocked process at the same rising edge, so tx is low from just after **the very edge on which start was seen**. Nothing waits for a baud_tick. The start bit therefore begins on a clock edge, at whatever phase baud_gen's internal counter happens to be sitting. (1 mark)

How long the start bit lasts. The bit ends on the sixteenth baud_tick after entry to STATE_SEND: tick 1 finds ticks = 0 and increments, ..., tick 15 finds ticks = 14 and increments to 15, and tick 16 finds ticks = 15, satisfies ticks >= MAX_TICKS - 1, and advances bit_idx.

The **first** of those sixteen ticks arrives between 1 and div clocks after entry, because the divider's counter was already partway through a window. Every tick after it is exactly div clocks apart. One clock more follows: tx <= frame(bit_idx) reads bit_idx before the edge that advances it, so the line changes to data bit 0 one clock after the sixteenth tick. So, for k in 1...div:

```
start bit length = k + 15 x div + 1  clocks
                  = 15 x div + 2      at shortest
                  = 16 x div + 1      at longest
```

(1 mark)

A nominal bit is 16 x div clocks, so the start bit is at worst div - 2 clocks short, or one clock long: under one tick period either way, that is under **one sixteenth of a bit**. The spread of div - 1 clocks is the phase of baud_gen's counter when start arrives. Simulated at div = 4, the start bit measures 62 to 65 clocks and every data bit exactly 64. Accept 15 x div + 1 to 16 x div from a candidate who misses the one-clock lag but explains the tick phase; the variation is the point.

Why nothing else inherits it. ticks is cleared *on* the tick that ends the bit, and that tick is itself a tick boundary. So every bit after the first begins on a tick (one clock after it, the same lag every time) and lasts exactly sixteen tick periods. The error is confined to the start bit and does not accumulate down the frame.

And the receiver does not care: it triggers on the falling edge that opens the start bit and samples the start bit half a bit later. A start bit up to one sixteenth short is still comfortably low at its own midpoint, and every bit the receiver actually reads a value out of is full length. (1 mark)

(d)

3 MARKS

The two orderings. $0xA5 = 1010\ 0101$, so $d_7 = 1, d_6 = 0, d_5 = 1, d_4 = 0, d_3 = 0, d_2 = 1, d_1 = 0, d_0 = 1$.

Order	1st	2nd	3rd	4th	5th	6th	7th	8th
least significant first ($d_0 \dots d_7$)	1	0	1	0	0	1	0	1
most significant first ($d_7 \dots d_0$)	1	0	1	0	0	1	0	1

They are identical. A transmitter that packed its frame most significant first would put exactly the same eight levels on the line, in exactly the same order, and `uart_tx_tb` would have passed. (1 mark)

The property. $0xA5$ is a **bit-reversal palindrome**: reverse its eight bits and you get $0xA5$ again. The constant chosen was one of the few bytes that cannot possibly distinguish the two orderings.

How many bytes share it. A palindrome is determined entirely by its top four bits — b_7 fixes b_0 , b_6 fixes b_1 , b_5 fixes b_2 , b_4 fixes b_3 — so there are $2^4 = 16$ of them out of 256. Pick a byte at random and you have a 15-in-16 chance of catching the bug; the earlier testbench had picked one of the 16. (1 mark)

The constant the testbench uses today. $0x53 = 0101\ 0011$, and its reverse is $0xCA$:

Order	1st	2nd	3rd	4th	5th	6th	7th	8th
least significant first ($d_0 \dots d_7$)	1	1	0	0	1	0	1	0
most significant first ($d_7 \dots d_0$)	0	1	0	1	0	0	1	1

They differ at the **first** data bit, so the testbench fails immediately and reports **data bit 0** mismatched. Any non-palindromic byte would do — $0x01, 0x0F, 0x48, 0xB2$ — and the mark is for seeing that asymmetry is the property required, not for naming this particular byte.

The lesson. A green testbench proves the cases it contains, not the property you believed you were checking. The check itself was always real — the levels must match `TXBYTE` bit for bit in the order expected — but with a symmetric constant it had no power to distinguish an ordering at all, so a genuine check had been quietly turned into a decoration by its own test data. (1 mark)

Worth saying out loud, because the same reasoning applies to the other testbenches: `uart_rx_tb` used $0xA5$ and $0x3C$, and `uart_top_tb` used $0x5A$. All three were palindromes, so at that revision **no testbench in hw/ pinned down bit order at all**. All four constants are non-palindromic today. One case survives even so: `uart_top_tb` loops tx back to rx, so a transmitter and a receiver that are reversed **together** cancel exactly and pass whatever constant it sends. Only the two unit testbenches can catch that pair, which is exactly why a system-level loopback is not a substitute for them.

Full marks for reaching that conclusion by any route. A candidate who writes out the two orderings, sees that they match and says so has done the hard part.

Question 3 The asynchronous line

14 MARKS

(a)

3 MARKS

What can happen. rx is governed by another chip, so it can change at any instant relative to clock, including inside a flip-flop's **setup-and-hold window**. A flop clocked in that window can go **metastable**: its output sits between the two logic levels for an unbounded time before resolving, and resolves either way.

What the failure actually is. Sampling a signal mid-transition and getting either level back is **not** the failure. Either answer is legitimate for a signal that genuinely was changing, and one clock later you would have got the other one anyway. The failure is that for a while the output is **not a level at all**. Two gates reading the same flop can resolve it differently in the same cycle, so a state machine can take two branches at once and land in an encoding that does not exist, and a counter can advance by something other than one. It is a *late* answer, not a wrong one, and lateness is what breaks a synchronous design. (1 mark)

Why two flops. The first flop is the one exposed to the asynchronous edge and may go metastable. The second gives it a **full clock period** to settle before anything downstream sees it, and the probability of a metastable state surviving that long falls exponentially with the time allowed — which is what the mean-time-between-failures expression describes. One flop gives it no settling time whatever: whatever it is doing at the next edge is what the design consumes, so a single register is not a synchronizer at all, however much it looks like one in simulation. Nothing in a simulator is ever metastable, which is why sync_tb's "exactly two edges" check exists. (1 mark)

Where it lives. uart_top instantiates sync on the rx pin. uart_rx deliberately does **not** contain one: its input port is named **rx_s2**, and the **_s2** suffix is the announcement — this signal has already been through a two-flop synchronizer, so treat it as an ordinary synchronous input and sample it directly.

The principle is that the module which owns the pins owns the crossing. uart_top owns the pins, so it owns the crossing; uart_rx is then a purely synchronous design, which is easier to reason about and easier to test. (1 mark)

(b)

3 MARKS

0x53 = 0101 0011, so $d_0 = 1, d_1 = 1, d_2 = 0, d_3 = 0, d_4 = 1, d_5 = 0, d_6 = 1, d_7 = 0$. Sent least significant first, that is the arrival order, and `frame(bit_idx) <= rx_s2` with `bit_idx` counting up stores each where it belongs:

Arrival	1st	2nd	3rd	4th	5th	6th	7th	8th
level	1	1	0	0	1	0	1	0
bit_idx	0	1	2	3	4	5	6	7

(1 mark)

After the eighth, `frame(7 downto 0) = "01010011"`, and on a good stop bit that is what `data_out` carries: **0x53**, the byte that was sent. (1 mark)

Counting down. With `bit_idx` walking 7 down to 0, the first-arriving bit lands in index 7 and the last in index 0:

```
frame = d0 d1 d2 d3 d4 d5 d6 d7 = 1 1 0 0 1 0 1 0 = "11001010" = 0xCA
```

The delivered byte is the **bit reversal** of the byte sent. Note that it is not garbage and not noise: it is a well-formed byte, every frame arrives cleanly, valid pulses, no `frame_err` is raised, and everything downstream works perfectly on data that is silently wrong. (1 mark)

(c)

5 MARKS

(i) The centre of bit n is $8 + 16n$ ticks after the true falling edge. Against that:

Sample	Ticks after the true edge	Ideal	Past centre
start-bit re-check	1 (notice) + 9 = 10	8	2 ticks
first data bit	10 + 17 = 27	24	3 ticks
every later data bit	previous + 16	+16	3 ticks
stop bit	last data + 17 = 156	152	4 ticks

The offset is **fixed rather than accumulating** because data bit to data bit is exactly sixteen ticks. The extra ticks are spent once at each *state change* — idle to start, start to data, data to stop — not once per bit. So the phase error is paid twice in the whole frame and then held constant. (1 mark)

(ii) The stop bit is the tenth bit of an 8N1 frame, so $n = 9$.

```
ideal sample tick   = 8 + 16 x 9 = 152
stop bit begins at  = 16 x 9     = 144
stop bit ends at    = 16 x 10    = 160
```

(1 mark)

Write T_r for the receiver's tick period and T_t for the transmitter's. The receiver samples 152 of *its own* ticks after the edge; the transmitter's bit boundaries are 144 and 160 of *its own*. The sample stays inside the stop bit when

$$144 T_t < 152 T_r < 160 T_t \iff \frac{144}{152} < \frac{T_r}{T_t} < \frac{160}{152} \iff 0.9474 < \frac{T_r}{T_t} < 1.0526,$$

so the combined mismatch budget is **plus or minus 5.26 %**, symmetric, about five percent either way. (1 mark)

(iii) As built, the stop bit is sampled four ticks late, at 156:

$$144 T_t < 156 T_r < 160 T_t \iff \frac{144}{156} < \frac{T_r}{T_t} < \frac{160}{156} \iff 0.9231 < \frac{T_r}{T_t} < 1.0256,$$

so **−7.69 % / +2.56 %**.

(1 mark)

Which side was eaten into. $T_r/T_t > 1$ means the receiver's ticks are *longer* than the transmitter's — the receiver is slow, or equivalently **the transmitter is fast**. That limit falls from 5.26 % to 2.56 %, a little under half. The other side gains: a transmitter running slow now has 7.69 % instead of 5.26 %, which is no use to anybody, because a real link is limited by its worst direction.

When it bites. A peer transmitter that runs fast relative to this receiver. This is not hypothetical: each end rounds its own integer divider and runs from its own clock, so each is off by its own amount, and what the receiver sees is the difference. At 115200 this peripheral, with BAUD_DIV = 27, is itself **+0.47 % fast**, which happens to push that difference towards the roomy side; a divider that rounds up instead, making this end slow (9600, at −0.15 %, is one), or a peer whose own rate errs upward, spends the small one. The margin is still ample for a well-configured link — the point is that the design started with half the trailing margin the ideal tick-8 sample would have had, for three tick-sized implementation details nobody chose deliberately. (1 mark)

Accept a candidate who works in “percent baud error” rather than tick-period ratios, provided the asymmetry and the roughly halved fast-side figure come out.

(d)

3 MARKS

The FIFO.

```
rdata <= entries(tail);
empty <= empty_s;
full <= full_s;

full_s <= '1' when count = DEPTH else '0';
empty_s <= '1' when count = 0 else '0';

process(clock, reset_s2_n) is
variable push, pop: std_logic;
begin
    if (reset_s2_n = '0') then
        head <= 0;
        tail <= 0;
        count <= 0;
    elsif (rising_edge(clock)) then
        push := wr and (not full_s);
        pop := rd and (not empty_s);
        if (push = '1') then
            entries(head) <= wdata;
            advance_ptr(head);
        end if;
        if (pop = '1') then
            advance_ptr(tail);
```

```

    end if;
    if ((push = '1') and (pop = '0')) then
        count <= count + 1;
    elsif ((push = '0') and (pop = '1')) then
        count <= count - 1;
    end if;
end if;
end process;

```

(2 marks)

Four things carry the marks, and each is a place a plausible answer goes wrong:

- **The guards are folded into push and pop once**, as variables, rather than repeated at each use. Everything downstream then tests the *guarded* event, so a write to a full FIFO cannot move head and a read from an empty one cannot move tail.
- **count is adjusted only when the two disagree**. A simultaneous push and pop leaves it exactly where it was; incrementing in one branch and decrementing in another, unconditionally, is the classic bug here, and it desynchronizes the flags from the contents within one cycle.
- **rdata is a concurrent assignment from tail, not something the process writes**. That is the look-then-advance contract Chapter 5 depends on: the front byte is visible with no rd at all, and rd only advances.
- **Both flags come from count**, not from comparing head with tail — which cannot tell full from empty in a ring buffer without a spare slot or an extra bit.

Accept a registered rdata, or full and empty derived from pointers plus a wrap bit, only if the candidate says what they have paid for it. Accept advance_ptr written out inline.

How many frames. DEPTH = 8, so the RX path absorbs **eight** whole frames. The ninth valid pulse arrives with full_s set, so push is '0' and the byte is **simply dropped** — silently: no flag, no pulse, nothing on the wire. The line that decides it is push := wr and (not full_s). The FIFO's contract is to preserve order and keep its flags honest, not to tell anybody it threw something away, which is why the caller must watch the flags.

The reserved bit. ER_OVERRUN, **ERROR_FLAGS bit 2**. In this book's build nothing in the hardware ever sets it, because only frame_err has a producer, so it reads 0 unless software writes a 1 there; ER_PARITY (bit 1) is in the same situation.

That is a **reservation, not an omission**: the positions are fixed in the protocol, in uart_def.vhd and in register_map.hpp (error::OVERRUN = 2 on both sides), so an implementation that adds overrun detection later drops straight in without renumbering anything or breaking a driver that was already written against the map. A map that defined only the bits it currently implements would force exactly that renumbering, and every existing driver with it.

(1 mark)

Question 4 Ports become registers

13 MARKS

(a)

3 MARKS

The assignments.

```
status_reg(31 downto 4) <= (others => '0');
status_reg(ST_TX_READY) <= not tx_full_s;
status_reg(ST_RX_VALID) <= not rx_empty_s;
status_reg(ST_ERROR)    <= err_flags(0) or err_flags(1) or err_flags(2);
status_reg(ST_TX_IDLE)  <= tx_empty_s and (not tx_busy);
```

(1 mark)

Five concurrent assignments, not a process: there is no state here to clock. Accept a comparison with "000", or a helper function, in place of the OR for ST_ERROR, and accept any spelling of the FIFO flag signals, but **the bits must be indexed by their uart_def names**, not by literals — that is the whole point of the package, and a candidate who writes status_reg(1) has thrown away the protection the shared map exists to give. The reserved line is worth having: without it bits 31–4 are undriven, and a read returns 'U' in simulation rather than the zeros Part 2 of the protocol (§A.2) promises.

Bit	Constant	Expression	Why it cannot be stored
0	ST_TX_READY	not tx_full	It is a property of the FIFO's own count, changing on every push <i>and</i> every pop; a stored copy is wrong the moment one update is missed.
1	ST_RX_VALID	not rx_empty	The same — and note it cannot be the receiver's valid, which is one clock wide and long gone before software polls.
2	ST_ERROR	the OR of the ERROR_FLAGS latches	It is a <i>view</i> of other state, not a fact of its own; storing it means two places to keep in step.
3	ST_TX_IDLE	tx_empty and (not tx_busy)	It combines a FIFO flag with a datapath level, and no single writer knows both.

(1 mark for reasoning that generalizes)

The general statement is worth the second mark on its own: each bit reports a **state**, while the things that change that state are **pulses and edges**. A register written by events is a register that is wrong whenever an event is missed, whenever two arrive together, and immediately after reset unless its reset value is hand-maintained. Deriving the bit from the state it reports removes all three failure modes by construction.

The value. The TX FIFO holds 2 of 8, so tx_full = '0' and bit 0 is **1**. The RX FIFO is empty, so bit 1 is **0**. No error is latched, so bit 2 is **0**. tx_empty is false (two bytes waiting) *and* tx_busy is true, so bit 3 is **0**. Everything above bit 3 reads zero.

STATUS = 0x00000001

(1 mark)

(b)

4 MARKS

The failing check. A read has no strobe, so a popping read can only be built from the address: `rx_pop` also asserted whenever `reg_idx = REG_RX_DATA`. That pops on the edge after which `uart_regs_tb` samples, so the first check to fire is **Case 5’s RX_DATA read-back**, which asserts that `RX_DATA` returns the byte just pushed and reports `uart_regs_tb: RX_DATA mismatch, got 0x00!` — the byte is gone before the testbench looks. A variant that pops as the address *leaves* `RX_DATA` survives Case 5 and trips Case 6 instead, “`RX_DATA read must not pop!`”, which reads `STATUS` after a bare `RX_DATA` read and asserts that `RX-valid` is still set. Either is the split being pinned down; award the mark for either, with what it asserts. (1 mark)

Why a read with a side effect is harder. The transport’s abort rule says that if `SS` rises before the fifth byte the transaction is abandoned with **no side effects**. Writes get that for free: `spi_reg_bridge` asserts `reg_write` only when a write transaction completes, so a `reg_write` the bank sees is always a real commit — which is precisely what lets `TX_DATA` and `RX_POP` be one-cycle actions with no state machine of their own.

A read has no such strobe. `reg_rdata` is **combinational**: the bank presents a value for whatever `reg_addr` currently carries, and the bridge latches it once at the end of the command byte and then shifts it out. The bank therefore cannot distinguish a read that completed from one abandoned after the address byte — and it cannot even tell how many times the same address was presented.

So the thing the bridge would have to tell the bank, and currently has no way to say, is “**this read transaction completed**” — a read-side commit strobe, the mirror of `reg_write`. The bank would then have to hold the pop pending until that strobe arrived and drop it on an abort. That is the entire write-side machinery, duplicated, to make a read do something a separate write already does for free. Splitting `RX_DATA` from `RX_POP` sidesteps the question rather than solving it, which is why §5.1 calls it the single most important idea in the module. (2 marks)

The driver trace. The `RX FIFO` holds `0x41` then `0x42`.

Call	STATUS poll	RX_DATA read	RX_POP write	Returns	FIFO after
1	RX-valid set	returns <code>0x41</code> , and advances to 0x42	advances again, discarding <code>0x42</code>	<code>0x41</code> , true	empty
2	RX-valid clear	not issued	not issued	false	empty

`0x42` was **popped without ever being read**. The `FIFO` advanced twice for one delivered byte, so every second byte is lost — and lost invisibly, since nothing anywhere reports a discarded entry. A testbench that sends one byte at a time never sees it. (1 mark)

(c)

3 MARKS

uart_regs_tb. It does not write CTRL until its eighth case, so during the transmit case the enable bit holds the bank's reset value, 0. The TX_DATA write is therefore swallowed, the TX FIFO stays empty, and the case that checks the FIFO front reports uart_regs_tb: TX FIFO front should be 0x5A, got 0x00!. The message names the right register, so the cause is findable. (1 mark)

uart_top_tb. It never writes CTRL at all. The byte never enters the TX FIFO, so nothing is transmitted, nothing loops back, nothing arrives in the RX FIFO, and the testbench polls STATUS for RX-valid up to its poll limit and gives up.

That message is the **less useful** of the two because it is a timeout: it says the byte never came back and nothing about why. It is equally consistent with a broken transmitter, a broken receiver, a broken FIFO, a wrong BAUD_DIV, a missing feeder, or a bank waiting to be enabled — exactly the failure §5.1 warns about, “a timeout that says nothing about the cause”. (1 mark)

The fact that settles it. uart_regs has **no ctrl output**. Read the port list: there is no port by which CT_ENABLE, the parity select, the stop-bit select or either IRQ mask could reach uart_tx, uart_rx or baud_gen. If no wire leaves the module carrying CTRL, then nothing in this build may be gated on it, and the question is closed by the interface rather than by taste. (1 mark)

Why the bits exist anyway. The register map is the **shared contract** with register_map.hpp and with the protocol, and its positions are fixed so that an implementation which wires them later stays compatible with both halves as already written. The driver's configure() writes the enable bit for the same reason: the two sides agree on the wire, and the write simply has no effect on the hardware yet.

(d)

3 MARKS

Why exactly one byte. busy is a concurrent decode of the state:

```
busy <= '1' when state = STATE_SEND else '0';
```

so it is high in the same simulation cycle as the state change the load caused. The moment the transmitter accepts the byte it is in STATE_SEND, tx_busy is high, and tx_load has already gone low — one clock of tx_load, one start pulse, one tx_pop.

It stays low for the rest of the frame because tx_busy stays high for the rest of the frame; that is what holds it. And it cannot be **zero** loads, because the instant the transmitter returns to STATE_IDLE with a non-empty FIFO both terms are true again and the next byte goes.

(1 mark)

The simplification. They are right that uart_tx ignores start outside STATE_IDLE, so the transmitter is safe. The bug is in the *other* line: tx_pop is the same signal.

With tx_load <= not tx_empty, tx_load is high **continuously** for as long as the FIFO has anything in it. So the FIFO is popped **once per 50 MHz clock cycle**. A FIFO holding four

bytes empties in four consecutive clocks, about 80 ns, while the first byte's start bit is still on the wire and has roughly 8.6 μ s left to run.

One of the four bytes reaches the wire. The other three survive about 60 ns at most and are discarded unsent, with no flag and nothing to see. (1 mark)

Does `uart_top_tb` catch it? No. The testbench writes exactly one byte to `TX_DATA`, so the FIFO empties after a single pop either way and the two designs behave identically. It would take a testbench that queues two bytes to see the difference — which is the same lesson as Question 2(d) in different clothes: a passing testbench proves the cases it contains.

(1 mark)

Worth noting for a candidate who spots it: this is also why the feeder could not be written before Chapter 5. `tx_empty` comes from the register bank and `tx_busy` from the transmitter, so it needs a block from Chapter 2 and a block from Chapter 5 to exist at the same time.

Question 5 The driver's contracts

12 MARKS

(a)

4 MARKS

```
if (status & (1U << status::RX_VALID)) { /* a byte is waiting */ }
```

```
if (status(ST_RX_VALID) = '1') then -- the same bit, indexed by position
```

(1 mark for both)

if (status & status::RX_VALID). `status::RX_VALID` is 1, so the mask is 0x1 — which is `TX_READY`'s bit. The position has been used where a mask was meant, and it happens to be a legal mask for the wrong bit.

Consequence: `read()` returns true whenever the transmitter has room, which after reset it always does. It reads `RX_DATA` from an empty FIFO, so the bank hands back whatever the ring buffer holds at tail — reset data or a stale entry — and the driver returns that byte to the caller as if it had arrived on the wire. It then issues an `RX_POP` that the FIFO's not empty guard silently drops. `app::EchoNode` over this driver emits a stream of garbage bytes at the speed of its poll loop, having received nothing at all. (1 mark)

writeReg(reg::CTRL, ctrl::ENABLE); `ctrl::ENABLE` is 0, so this writes 0x00000000 to `CTRL` — clearing the register rather than setting bit 0. The correct form is `writeReg(reg::CTRL, 1U << ctrl::ENABLE)`.

Consequence, and this is the interesting part: **in this book's build, nothing happens.** `CTRL` gates nothing, so the peripheral transmits and receives exactly as before, and every testbench and every rung of the bench passes. Only the host suite notices — `Uart.ConfigureWritesBaudThenEnable` expects the `CTRL` write to end in 0x01 and gets 0x00 — and without that one assertion the bug is a time bomb: it becomes a peripheral that never enables on the first day somebody wires `CT_ENABLE` to something real, in a driver that has looked correct for months. (1 mark)

A mask stored where the map stores a position. RX_VALID is now 2, so `1U << status::RX_VALID` shifts by two and tests **bit 2**, the ERROR bit. `read()` returns `true` when an error is latched and `false` otherwise, so it never delivers a byte and does deliver on a framing error.

Worse than the behaviour is where the bug lives: the two sides of the wire now **disagree about the map**, since `uart_def.vhd` still has `ST_RX_VALID = 1`. Nothing on the host can find it, because the host tests script the stub with values produced from this same header — the driver and its tests are wrong together and agree perfectly. It is a bench-only bug, and “both sides store positions, transcribed from the protocol rather than from each other” is the rule that exists to prevent exactly it. (1 mark)

(b)

3 MARKS

The three methods. `begin()`, `transfer(uint8_t)` returning `uint8_t`, and `end()`. Select, exchange one byte in full duplex, deselect — the framing of Part 3 of the protocol and nothing more. (1 mark)

What it does not know. It knows nothing about **registers** — no indices, no command byte, no five-byte shape, no byte order — and nothing about the **UART** — no baud, no FIFOs, no status bits. It is a dumb byte pipe.

What the byte level buys. All the protocol knowledge stays in one place, the driver, so there is exactly one implementation of the transaction shape to get right and exactly one to test. And the layer below becomes something a stub can imitate **perfectly**, because “return a byte for a byte” has no behaviour to model — there is nothing for a fake to get subtly wrong.

A seam at `readReg` and `writeReg` would push the command byte and the byte order *below* the line. The stub would then have to reimplement them, and a test asserting that the driver produced the right transaction would really be asserting that the stub and the driver made the same mistake. The byte-level seam is what makes the test meaningful, not merely possible. (1 mark)

The two implementations, and what changes. `driver::transport::Stub` (Chapter 8, host, scripted) and `driver::transport::AvrSpi` (Chapter 9, target, real). What changes above the seam when they swap: **nothing at all**. The register map, `Uart`, the blocking helpers and `EchoNode` are recompiled, not rewritten — which is the entire return on the design work done in Chapter 6. (1 mark)

(c)

3 MARKS

1. **#include <stdint> and `std::uint8_t`, `std::uint32_t`.** The AVR target is *freestanding* and `avr-libc` ships **no C++ standard library at all**, so there is no `<stdint>` and no `std` namespace to qualify anything with. Replace them with `<stdint.h>` and the bare `uint8_t` and `uint32_t`.
2. **namespace `driver::uart` { ... }.** The compact nested-namespace definition is C++17, and the reference toolchain does not accept `-std=c++17` as an option at all. Replace it with nested blocks: `namespace driver { namespace uart { ... } }`.

3. **inline constexpr at namespace scope.** *inline variables* are also C++17. Plain `constexpr` is the replacement and costs nothing: at namespace scope it already has internal linkage, so each translation unit gets its own compile-time copy of a constant that never occupies storage.
4. **[[nodiscard]].** Rejected by that toolchain, so it is omitted from everything that has to cross-compile. The host-only test suites keep it, because they never build for the target.

The point in each case is that the toolchain is missing a **language level and a library**, not merely being conservative. The reference is the AVR/GNU C++ compiler bundled with Microchip Studio, several GCC releases behind a desktop g++, so “it compiled on my Linux avr-g++” is not evidence that it will build for the target the book names.

(2 marks: 1 for the four rejections, 1 for reasons that name the missing language level or library rather than “it is old”)

The corrected header.

```
#pragma once

#include <stdint.h>

namespace driver
{
    namespace uart
    {
        constexpr uint8_t TxReady{0U};

        class Interface
        {
        public:
            virtual ~Interface() noexcept = default;

            virtual uint32_t status() const noexcept = 0;
        };
    } // namespace uart
} // namespace driver
```

(1 mark)

`#pragma once` is not in the original listing and is not one of the four defects, but a header without an include guard is a defect of its own and the book’s headers all carry one; award the mark without it, and note it. `virtual` and `= 0` survive untouched, as do `noexcept`, `override`, `final` and `= default` — the subset is narrower than modern C++, not a different language.

(d)

2 MARKS

What each bool means. `write(byte)` returns `true` if the byte was accepted into the TX FIFO and `false` if the FIFO was full, in which case **nothing was written** — the driver does not queue, retry or buffer. `read(byte&)` returns `true` if a byte was placed in the out-parameter, and `false` if none was waiting, in which case no `RX_DATA` read and no `RX_POP` were issued.

(1 mark)

The bits behind them. `STATUS` bit 0, `TX_READY` (not `tx_full`), makes the write answer possible; bit 1, `RX_VALID` (not `rx_empty`), makes the read answer possible. The Chapter 5 mechanism behind both is that `STATUS` is **computed from the FIFO flags**, so each is a level that stays true until the condition changes — a pollable fact rather than an event that has to be caught.

The blocking versions. The free functions `writeBlocking` and `readBlocking` in `driver/uart/blocking.hpp`, taking `Interface&` so that they work over any implementation and dispatch to the concrete driver at run time. They are free functions rather than methods because they add no state and no new contract — putting them in the interface would oblige every implementation, including every stub, to reimplement a one-line spin loop.

If the interface itself blocked, two things would be lost. Every operation would become non-deterministic to test: a test would have to arrange for the stub to *eventually* succeed, and a mistake would hang rather than fail. And an application could no longer poll — `EchoNode` could not re-check its stop flag between reads, which is the exact property its test depends on.

(1 mark)

Question 6 The five-byte transaction

13 MARKS

(a)

4 MARKS

The command bytes.

Bytes	Bit 7	Index	Register	Direction
82 00 00 00 1B	1	2	BAUD_DIV	write 0x0000001B = 27
81 00 00 00 01	1	1	CTRL	write 0x00000001 = 1U << ctrl::ENABLE
00 00 00 00 00	0	0	STATUS	read, four dummy bytes
04 00 00 00 00	0	4	RX_DATA	read, four dummy bytes
85 00 00 00 01	1	5	RX_POP	write 1

(1 mark)

Which belong together.

- Transactions 1 and 2 are one **configure(27)**: the baud divider first, the enable afterwards, in that order, because the peripheral should know its rate before it is switched on.
- Transactions 3, 4 and 5 are one successful **read(byte)**: poll `STATUS`, pure read of `RX_DATA`, separate `RX_POP` write. Three register accesses, in that order, which is the Chapter 5 contract showing up on the other side of the wire.

(1 mark)

What the driver did with the returned bytes. In both reads, the reply that came back during the *command* byte is discarded — it is meaningless, since the slave does not yet know which register is being asked for. The four bytes returned during the four dummy exchanges are assembled **most significant byte first**: the first into bits 31–24, then 23–16, then 15–8, and the last into bits 7–0. In transaction 3 the driver then tested bit 1 (RX_VALID) of the assembled word; in transaction 4 it kept the low byte and copied it into the caller's out-parameter. (1 mark)

What the call counts prove. They prove each transaction was **framed exactly once**, and the two counts being equal means every SS that fell also rose — five complete, balanced transactions rather than one long one.

The byte log alone cannot distinguish this from a driver that called `begin()` once, pushed all twenty-five bytes, and called `end()` at the very end. On a stub that is invisible: the same bytes in the same order. On real hardware it is fatal — `spi_reg_bridge` would see `ss_active` assert once, decode the first five bytes as one BAUD_DIV write, and treat the remaining twenty as a continuation of a transaction that was already over, so nothing after the first would be decoded at all. Framing is not visible in a list of MOSI bytes, so it has to be counted separately, which is why the suite asserts on `beginCalls()` and `endCalls()` in eight different cases. (1 mark)

(b)

3 MARKS

(i) **The bytes.** Least significant first, `configure(27)`'s first transaction puts

```
82 1B 00 00 00
```

on the wire. (1 mark)

What the peripheral commits. The peripheral is not consulted about byte order — the protocol says the first data byte is bits 31–24. So it assembles `0x1B000000` and commits that to BAUD_DIV. BAUD_DIV is bits 15–0, so `baud_div` carries `x"0000"`.

`uart_top`'s guarded conversion then hits its **second** case, `unsigned(baud_div) = 0`, and substitutes 1. `baud_gen` divides the 50 MHz clock by one, so it ticks every clock:

```
tick rate = 50_000_000 Hz
baud rate = 50_000_000 / 16 = 3_125_000 baud
```

3.125 Mbaud, about 27 times the intended rate. The line runs, tx toggles, `uart_tx_tb` and `uart_regs_tb` are unaffected because neither involves the driver — and nothing at the far end can read a byte. (1 mark)

(ii) The peripheral shifts `0x0000000B` out most significant first, so the four bytes returned are `00 00 00 0B`. Assembled least significant first, the first goes to bits 7–0 and the last to bits 31–24:

```
driver holds 0x0B000000
```

status & (1U << status::TX_READY) is then 0. write() returns false on every call for ever, and writeBlocking() — a bare while (!uart.write(byte)) {} — spins for ever. The firmware hangs on the first byte it tries to send, with the peripheral perfectly healthy and reporting itself ready. (1 mark)

(c)

3 MARKS

No RX_POP. RX_DATA is a pure read, so the front byte is never discarded. RX-valid stays set, and every subsequent read() polls, finds it set, re-reads the same front byte and hands it back. app::EchoNode echoes the **first character it ever receives**, over and over, as fast as the poll loop runs — the terminal fills with a single repeated character and nothing typed afterwards has any effect. It is a very recognizable bench symptom, which is a small mercy. (1 mark)

Pop before read. The front byte is discarded before it is looked at, so the RX_DATA read returns whatever the FIFO shows *afterwards*: the next byte if one has already arrived, or stale empty-FIFO data if not. Every byte is either skipped outright or delivered one position late, and the stream is silently corrupted rather than stalled — which is harder to diagnose than the first mistake, not easier. (1 mark)

The pop on empty. The test is `Uart.ReadWhenEmptyReturnsFalseAndDoesNotPop`: read(), on no data, must report failure and issue no RX_POP, so the record holds one transaction, not three. What the stray pop actually does to *this* peripheral: fifo guards rd on not empty, so it is dropped and no queued byte is lost.

“Nothing bad happens” is not a defence, for three reasons:

- The guard is an implementation detail of fifo, **not a promise the register map makes**. The protocol says RX_POP discards the front byte; it does not say what happens when there is not one.
- It costs a five-byte SPI transaction on every idle poll, which on a poll loop with an empty FIFO is most of the bus traffic and most of the latency.
- A driver that depends on an unspecified guard breaks against the first compliant peripheral that does not have one, and it will break on the bench rather than on the host.

(1 mark)

(d)

3 MARKS

Why the three are not const. begin(), transfer() and end() drive SS and shift real bytes through real hardware. They are **actions with side effects**, not observations, and marking them const would be a lie about what calling them does — to the reader first and to the optimizer second. (1 mark)

What the cast does, and why it is not needed here. const_cast<transport::Interface&>(myTransport) yields a non-const reference the three methods can be called on — but it strips nothing, because the enclosing const never reached the transport.

The compiler would accept the calls **without** it as `Uart` is written today, because `myTransport` is a **reference member**: `const` does not propagate through a reference, so inside a `const` method `myTransport` still names a `transport::Interface&`. The cast is documentation of intent rather than a requirement.

It becomes genuinely required the moment the transport is held differently — by value, or behind a `const`-propagating handle — because then the object's `const` reaches the member and the calls stop compiling. Writing it now means that change is a one-word edit rather than a redesign. (1 mark)

The undefined case. Casting away `const` is undefined behaviour when the object being cast is **actually const** — declared `const`, or a temporary bound to a `const` reference — and is then modified through the resulting reference. That is not what happens here: `myTransport` refers to a real, non-`const` transport that the caller constructed and owns (a `Stub` on the host, an `AvrSpi` on the target), so driving it is defined. The one hard rule is to cast away `const` only on an object that is not genuinely `const`, and this obeys it. (1 mark)

Question 7 The real transport, and the real toolchain 11 MARKS

(a) 4 MARKS

What each statement achieves.

- `DDRB |= ...` makes `SCK` (PB5), `MOSI` (PB3) and `SS` (PB2) **outputs**.
- `PORTB |= (1U << SS)` idles the chip select **high**, that is deasserted, before any transaction.
- `SPCR = ...` enables the peripheral, selects master, and sets the prescaler.

(1 mark)

How `SPCR` delivers the contract, and the bits left clear.

Bit	State	What it gives
<code>SPE</code>	set	The SPI peripheral is enabled.
<code>MSTR</code>	set	Master: the ATmega generates <code>SCK</code> , and the FPGA is the slave.
<code>SPR0</code>	set	With <code>SPR1</code> clear and <code>SPI2X</code> clear, the prescaler is 16: 16MHz / 16 = 1 MHz .
<code>DORD</code>	clear	MSB first , in every byte, as the protocol requires.
<code>CPOL</code>	clear	<code>SCK</code> idles low .
<code>CPHA</code>	clear	Sample on the leading edge. With <code>CPOL</code> clear that is the rising edge.
<code>SPIE</code>	clear	No SPI interrupt. The transport polls, which is why it needs neither <code><avr/interrupt.h></code> nor any ISR.
<code>SPR1</code>	clear	Half of the prescaler selection above.

`CPOL = 0` together with `CPHA = 0` is SPI mode 0, which is what Part 3 of the protocol asks for. So all three requirements — mode, order and rate — are met by bits that are *not* set, which is exactly why an answer that lists only the three bits present is incomplete. (2 marks)

Why MISO appears nowhere. It is an input at reset and must stay one, so the constructor has nothing to do to it. The book's rule is that the constructor touches only what it must, so

that the destructor can put back exactly that and nothing else — MIS0 was never taken, so it is never given back. *(half of the last mark)*

Without the middle statement. PORTB resets to zero, so the instant DDRB makes PB2 an output the pin is actively driven **low**: the chip select is asserted from construction onward, with the bus idle. `spi_slave` sees `ss` fall and loads its first byte, and the bridge's byte counter waits at zero. The first transaction therefore still decodes, provided `SCK` has not moved: `begin()`'s drive low is a no-op, the five bytes arrive as a command and four data bytes, and `end()` raises `SS` for the first time, after which every transaction is framed normally. That is what makes the omission dangerous rather than loud: nothing on the bench and nothing in a MOSI byte log looks wrong, while the contract's idle-high chip select has been broken. Any `SCK` activity between construction and the first `begin()` would be counted as the start of a command byte and misalign that whole first transaction, because the only boundary the bridge knows is `SS` rising, which ties back to 6(a). *(half of the last mark)*

(b)

3 MARKS

What it returns. The byte left in `SPDR` from the **previous** exchange. The receive buffer still holds the last completed transfer's result, because the transfer just started has not shifted a single bit yet — the write to `SPDR` starts the shift, it does not perform it. *(1 mark)*

How long it takes. Eight `SCK` periods, one per bit. At 1 MHz that is **8 μ s**, which at 16 MHz is **128 ATmega clock cycles** — an eternity in instructions, and the reason the omission is both easy to make and impossible to get away with. *(1 mark)*

What the spin waits on, and the side effect. The spin waits on **SPIF** in `SPSR`:

```
SPDR = byte;
while (0U == (SPSR & (1U << SPIF))) {}
return SPDR;
```

Reading `SPSR` while `SPIF` is set and *then* reading `SPDR` **clears SPIF**, arming the next transfer. That is not incidental: without it the flag would still be set at the top of the next transfer() and the spin would fall straight through, so the correct code owes its correctness to the clearing sequence as much as to the wait.

What the driver sees. On the host mock, which models only the missing wait, every returned byte is one exchange stale: the reply to the command byte is discarded anyway, so `readReg` assembles the replies that belonged to the command exchange and the first three data exchanges — a `STATUS` word built from the tail of the previous transaction and the head of this one. On the ATmega it is worse, because the transmit side is single-buffered: a write to `SPDR` while a byte is still shifting sets `WCOL` and is discarded. The whole of `readReg` runs in about the time one byte takes at 1 MHz, so the dummy writes collide with the command byte and are lost, `end()` raises `SS` with the transaction nowhere near five bytes, and the bridge abandons it with no side effects — a `writeReg` built on this `transfer` commits nothing either. Every `SPDR` read returns whatever the receive buffer last completed, never the register's bytes, which is the worst kind of bug to meet first on a bench. *(1 mark)*

(c)

2 MARKS

What the hardware does. It clears **MSTR** in SPCR and sets SPIF. The ATmega demotes itself to a **slave** on the spot, stops driving SCK, and abandons the transfer in flight. This is why SS/PB2 must be an output in master mode even when something else is being used as the chip select — and here PB2 *is* the chip select, so driving it is the transport's job anyway. (1 mark)

What the spin does, and the symptom. The spin in flight ends **immediately**, because the demotion set SPIF, and `transfer()` returns a byte that was never clocked in: that call succeeds, wrongly. The next one is where it stops. The SPI is now a slave, so writing SPDR only loads the shift register and waits for a master's SCK that nobody drives; SPIF never sets, and the spin loops for ever.

On the bench: SCK stops moving on the analyzer, and the firmware freezes inside `transfer()` one byte after the demotion. Nothing recovers — the transport configured SPCR once in its constructor and never revisits it, so MSTR stays clear until the chip is reset.

(1 mark)

(d)

2 MARKS

Two host constructs that do not survive. Any two of:

- **Anything from the standard library, `std::cout` above all.** There are no iostreams, so debug output goes out the ATmega's **own** hardware USART: set UBRR0 from F_CPU and the target baud, enable the transmitter in UCSRB, then poll UCSRA & (1U << UDRE0) and write UDR0. (Not to be confused with the FPGA peripheral this book builds.)
- **Dynamic allocation**, `new` or `std::make_unique`. There is no operator `new`, so everything is static or automatic storage — which is why every seam in this book is a reference injected into an object living in `main`'s frame.
- **Exceptions and RTTI.** No `throw`, no `dynamic_cast`, no `typeid`; errors become return values, which is what `write()` returning `false` on a full FIFO already is.

(1 mark)

`env.cpp`.

```
void operator delete(void*) noexcept {}
void operator delete(void*, unsigned int) noexcept {}

extern "C" void __cxa_pure_virtual() {}

extern "C" int __cxa_guard_acquire(volatile void* g) { return !*(char*)g; }
extern "C" void __cxa_guard_release(volatile void* g) { *(char*)g = 1; }
extern "C" void __cxa_guard_abort(volatile void*) {}
```

What emits each reference:

- **operator delete** — a class with a **virtual destructor** names the deleting operator `delete` in its vtable, even though this code never deletes through a base pointer. Every Interface

in the book has one. **Both** forms are needed: from C++14 the compiler calls the sized form but requires the unsized one to exist beside it, and only the unsized one is emitted under `-fno-sized-deallocation`. On AVR `size_t` is unsigned `int`, so that is the second parameter's type; `std::size_t` is not available to name it.

- **__cxa_pure_virtual** — named by every **abstract class's** vtable, so `driver::uart::` Interface alone is enough to require it. It is called only if a pure virtual is somehow invoked, so an empty body is honest: there is nothing useful to do and nowhere to report it.
- **__cxa_guard_acquire, _release and _abort** — the guard around a **function-local static's** one-time initialization. Single-threaded definitions are all a bare-metal target needs.

The extern "C" on the three `__cxa_*` groups is not decoration: the compiler emits calls to them by their C names, and C++ mangling would leave the linker with unresolved symbols. `operator delete` is a C++ operator and must **not** be extern "C". A candidate who writes empty bodies but mangles the names has not linked anything.

Accept a subset with a note if the candidate says which construct in *their* design each is for; the mark is for knowing that these are compiler-emitted references with no library behind them, not for reciting all six.

Why it lives in avr/. So that the **host** link keeps `libstdc++`'s versions. Replacing the host's thread-safe static guards and its `operator delete` with these single-threaded, hand-written ones would change the behaviour of the build the tests actually run on, for no benefit whatever. The file exists to satisfy a linker that has no C++ runtime, and the host has one.

(1 mark)

Question 8 The bench

12 MARKS

(a)

4 MARKS

(2 marks for the order and completeness of the chain, 1 for the chapters and planes, 1 for the representations)

#	Module	Built in	Plane
1	the USB-serial adapter to the DE0-CV rx pin	provided hardware	data
2	sync on the rx pin	Chapter 3	data
3	uart_rx	Chapter 3	data
4	the RX fifo	Chapter 4, instantiated in Chapter 5	data
5	uart_regs	Chapter 5	the boundary between the two
6	spi_reg_bridge, then spi_slave	provided	control
7	AvrSpi	Chapter 9	control
8	Uart::readReg, then Uart::read	Chapter 7	control
9	app::EchoNode::run	Chapter 10	neither: it is the application
10	Uart::write, then Uart::writeReg	Chapter 7	control
11	AvrSpi, then spi_slave and spi_reg_bridge	Chapter 9, provided	control
12	uart_regs and the TX fifo	Chapters 5 and 4	the boundary again
13	the TX feeder in uart_top	Chapter 5	data
14	uart_tx	Chapter 2	data
15	the DE0-CV tx pin, the adapter, the terminal	provided hardware	data

baud_gen (Chapter 2) paces both 3 and 14 and is on the data plane throughout; reset_sync (provided) is on neither and is in every clock domain. uart_top (Chapter 1) is the file that holds all of it together and is not itself a step.

The three representations.

- 1. A **UART frame on the wire**: a low start bit, eight bits least significant first, a high stop bit, timed by the baud rate.
- 2. A **register field over SPI**: the low eight bits of a 32-bit word, carried most significant byte first inside a five-byte transaction.
- 3. A **uint8_t** in the driver and the application.

It changes at **uart_regs**, where a frame that has become a FIFO entry becomes a register field, and again at **Uart::readReg**, where four SPI bytes become a **uint32_t** and then a **uint8_t**. On the way out it changes back at the same two places, in the reverse order. Naming those two places is the point of the exercise: everything else on the list is transport.

(b)

3 MARKS

What the ladder has cleared.

- **Rung a**, the data-plane pin loopback: the USB-serial adapter, its 3.3 V logic levels, the two data-plane wires and their crossover, and the FPGA pin assignment. None of the candidate's logic is in that path, which is what makes it a clean result. What it cannot clear is the terminal's baud setting: the terminal talks to itself, and a loopback agrees with itself at any rate, for the same reason rung c does.
- **Rung b**, the control plane: the level shifter on all four SPI lines, AvrSpi's configuration, spi_slave and spi_reg_bridge, and the bank's write and read paths — proven by a BAUD_DIV write and read-back.
- **Rung c**, the peripheral loopback: the whole datapath on real silicon at a real 50 MHz — baud_gen, uart_tx, uart_rx, both FIFOs, the TX feeder and the register bank.

(1 mark)

What is left. The peripheral talking to a **different device**, with a **second, independently generated baud rate** that now has to agree with the peripheral's own. Rung d is the first rung at which any absolute timing requirement exists at all.

The likely cause, and why rung c hides it. A **baud mismatch**. In loopback the transmitter and receiver are driven by the *same* baud_gen, from the *same* 50 MHz clock, with the *same* BAUD_DIV. Any divider whatsoever passes rung c — including a wildly wrong one — because both ends are wrong by exactly the same amount and the frame is still sixteen ticks per bit at both ends. Rung d is the first time the number has to be right in absolute terms.

The two symptoms together point the same way: corruption in both directions is a *timing* disagreement. A broken, uncrossed or floating data-plane wire gives silence, not wrong characters.

(1 mark)

The arithmetic.

```

tick rate = 50_000_000 / 27      = 1_851_851.9 Hz
baud      = 1_851_851.9 / 16     = 115_740.7
error     = (115_740.7 - 115_200) / 115_200 = +0.47%
```

Question 3 puts this receiver's combined tolerance at roughly **+2.5 % / -7.7 %**, so 0.47 % is nowhere near the edge and BAUD_DIV = 27 is **exonerated**. To blame the divider you would need a percent-scale error, and the direction matters, because Question 3 found this receiver's budget lopsided. A BAUD_DIV of **28** gives 111 607 baud, -3.1 %: this receiver's ticks are then 3.2 % longer than the terminal's, beyond the +2.56 % fast-transmitter limit, so typed characters fail while the driver's own bytes, read by the terminal's receiver with its full five percent, still arrive. A BAUD_DIV of **26** gives 120 192 baud, +4.3 %: inside this receiver's 7.7 % on the other side, and only marginal at the terminal. Neither produces what this bench shows, corruption in *both* directions, which takes a bigger error: a terminal set to 57600 or 9600 rather than 115200 — the one setting rung a cannot check — or a board clock that is not

actually 50 MHz, which would move both halves of the peripheral together and so would also have been invisible at rung c. Those are the places to look, in that order. (1 mark)

(c)

3 MARKS

The main.

```
/**
 * @brief Application entry point for the ATmega328P.
 */
#include <stdint.h>

#include "app/echo_node.hpp"
#include "driver/transport/avr_spi.hpp"
#include "driver/uart/uart.hpp"

int main()
{
    constexpr uint16_t baudDiv{27U};

    driver::transport::AvrSpi transport{};
    driver::uart::Uart      uart{transport};
    uart.configure(baudDiv);

    app::EchoNode node{uart};

    volatile bool stop{false};
    node.run(stop);

    return 0;
}
```

(2 marks)

The marks are in the **order and the ownership**, not the punctuation:

- **Bottom-up construction.** Each layer is built before the one that references it, and each is handed up by reference: the transport into Uart, Uart into EchoNode. A candidate who constructs EchoNode first has nothing to give it.
- **configure() before run(),** and 27 for 115200 — the one number on this page that comes from Part 1's formula rather than from a header. Configure after starting the loop and the first bytes go out at whatever the reset divider produces.
- **Nothing is allocated.** Three plain locals in main's frame, living until the program ends, which on this target is never. This is the freestanding constraint paying off rather than biting: the design was reference-injected from Chapter 6, so the target needs no operator new and none is available. A candidate reaching for new here has not understood what Chapter 9 removed.

- **main is the only file that names concrete types.** EchoNode sees an interface, Uart sees an interface; swapping the transport for the Chapter 8 stub is an edit to these three lines and nothing else.

`return 0` is unreachable, and that is fine: `run()` never returns with `stop` held `false`. Accept an infinite loop after `run()` instead, or `for(;;)` in place of the `return`; do not accept a `main` that falls off the end into `avr-libc`'s exit path with the peripheral still live.

Why the flag is false here. On the bench there is nobody to stop it: the node should echo for as long as the board has power, so `stop` is a `volatile bool` that nothing ever writes and `run()` is an infinite loop by construction. In the host test the very same parameter is what *ends* the run — the UART stub of Chapter 6 sets the caller's flag `true` the moment its scripted RX buffer runs out. One flag, taken by `const volatile bool&` and read every pass, serves both: production runs forever because nothing sets it, and the test terminates because something does. That is why it is a reference rather than a return value or a one-time check.

Why not readBlocking(). It would spin inside `Interface::read()` until a byte arrived, and would therefore never return to the top of the loop — so `stop` would never be read again. The application could not be stopped by anybody, including its own test.

That connects directly to how the host test ends: the UART stub of Chapter 6 sets the caller's `stop` flag `true` the moment its scripted RX buffer runs out, and `run()` sees it on the **next pass**. With a blocking read there is no next pass; the test hangs instead of finishing, which is the test telling you the loop is not actually stoppable. (1 mark)

Worth crediting if a candidate volunteers it: the *write* is allowed to block, and the asymmetry is deliberate. The byte is already in hand, there is nothing else the loop could usefully do with it, and waiting cannot deadlock because the TX FIFO is drained by hardware whether software cooperates or not. Poll where you might have to give up; block where you have already committed.

(d)

2 MARKS

Symptom and risk. The FPGA input is being driven above its own 3.3 V supply. At best the pin misbehaves and the control plane is unreliable — a `BAUD_DIV` write that reads back wrong, or intermittently right, which is the worse of the two because it looks like a software problem. At worst the pin's protection diode conducts into the 3.3 V rail and the pin, or the I/O bank, is damaged. This is the one genuine electrical hazard on the bench, which is why the wiring is checked before power is applied.

The first rung that exposes it. Rung b, the control plane. Rung a cannot: the data-plane pin loopback has no SPI in it at all, running entirely between the adapter and the FPGA, both at 3.3 V, with the shifter not in the circuit. That is precisely the property that makes **rung a** a useful rung — it clears the things it clears and nothing else. (1 mark)

The safe line. MIS0. It is the only one of the four that travels from the FPGA to the Nano, 3.3 V up to 5 V, so nothing over-drives a 3.3 V input. It usually works unshifted because the ATmega's input-high threshold at $V_{cc} = 5\text{ V}$ is $0.6 \times V_{cc} = 3.0\text{ V}$, and 3.3 V clears it.

What can still go wrong: the margin is **0.3 V**. A loaded or slow edge, a long breadboard wire, a supply that has sagged under the FPGA's draw, or a warm room can drop a 1 below the threshold — producing an intermittent, wiring- and temperature-dependent read failure. That is considerably worse to debug than a line that never works at all, and it is why “it mostly works” is not an acceptable state for this line either. *(1 mark)*

Model Answers to Paper B

Marks are shown per part. Method carries them: a correct derivation with a slip in it is worth more than a correct answer with no working, and later parts consume earlier ones, so an error should be followed through rather than penalised twice. A candidate whose register map is wrong in Question 5 and who then uses their own map consistently in Question 6 loses the marks once.

Where a part asks for VHDL, mark the **hardware described**, not the syntax: the sensitivity list, the reset branch, whether a pulse is one clock wide, whether every path assigns its output, and which way a shift goes. Where it asks for C++, mark the **byte order, the ordering of register accesses, and the qualifiers that carry meaning** — `const`, `noexcept`, `override`, and what has been deleted — not the punctuation.

Where a part asks for a review, naming the defect earns half and saying what it does to the running system earns the other half. The questions are in Appendix E.

Question 1 Building the top before anything it holds 12 MARKS

(a) 3 MARKS

```
-----
-- UART peripheral top level.
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_n: Asynchronous active-low reset.
--   - sclk, mosi, ss: SPI from the processor.
--   - rx: UART serial input.
--
-- Outputs:
--   - miso: SPI to the processor.
--   - tx: UART serial output.
-----
```

```

library ieee;
use ieee.std_logic_1164.all;

entity uart_top is
  port(clock, reset_n: in std_logic;
        sclk, mosi, ss: in std_logic;
        rx          : in std_logic;
        miso, tx     : out std_logic);
end entity;

```

(2 marks: 1 for the eight ports in the right order with the right directions, 1 for the inputs-first grouping, the clauses and the banner.)

The clause it does not need. `ieee.numeric_std`. Nothing in the skeleton of Chapter 1 converts between a vector and an integer: the top wires bytes and vectors from block to block and never interprets one as a number.

It is added in **Chapter 2**, for the single guarded conversion that turns the register bank's `BAUD_DIV` vector into the natural `baud_gen` expects:

```

baud_div_int <= 1 when is_x(baud_div) or unsigned(baud_div) = 0
                else to_integer(unsigned(baud_div));

```

`unsigned` and `to_integer` come from `numeric_std`; `is_x` comes from `std_logic_1164`, so it needs no clause of its own. That expression is the only place in `uart_top` where a `std_logic_vector` becomes an integer.

Why never use `work.uart_def.all`. The top carries the register bus — `reg_addr`, `reg_wdata`, `reg_write`, `reg_rdata` — from the bridge to the bank without ever **naming** a register or a bit on it. It is wiring, not interpretation. `uart_regs` is the only synthesizable module that interprets what is on that bus, so it is the only one that needs the package (the testbenches use it too, mostly for `to_hex`). (1 mark)

(b)

5 MARKS

The signals. Read each entity and declare, for every port that does not connect straight to a `uart_top` port, a signal of the same type:

```

architecture behaviour of uart_top is
  signal reset_s2_n : std_logic;           -- from reset_sync, to ~
  -- everything
  signal spi_rx_data : std_logic_vector(7 downto 0); -- slave -> bridge
  signal spi_rx_valid : std_logic;           -- slave -> bridge
  signal spi_ss_active : std_logic;         -- slave -> bridge
  signal spi_tx_data : std_logic_vector(7 downto 0); -- bridge -> slave
  signal reg_addr : std_logic_vector(3 downto 0); -- bridge -> uart_regs (L05)

```

```

signal reg_wdata    : std_logic_vector(31 downto 0); -- bridge -> uart_regs (L05)
signal reg_write    : std_logic;                    -- bridge -> uart_regs (L05)
signal reg_rdata    : std_logic_vector(31 downto 0); -- uart_regs (L05) -> bridge
begin

```

(2 marks: 1 for the nine signals and their types, 1 for the `spi_` prefix and `_s2` suffix used correctly and for declaring nothing a later chapter brings.)

The instantiations. (2 marks)

```

reset_sync: entity work.reset_sync
    port map(clock, reset_n, reset_s2_n);

spi_slave: entity work.spi_slave
    port map(clock, reset_s2_n, sclk, mosi, ss, spi_tx_data,
            miso, spi_rx_data, spi_rx_valid, spi_ss_active);

spi_reg_bridge: entity work.spi_reg_bridge
    port map(clock, reset_s2_n, spi_ss_active, spi_rx_data, spi_rx_valid,
    reg_rdata,
            spi_tx_data, reg_addr, reg_wdata, reg_write);

```

Note that `miso` connects **straight to the entity port**, so it needs no signal of its own — that is the reason the list above has nine entries rather than ten, and it is the whole method of the exercise.

The two rx_data ports. (1 mark) They are not one bus, and they are not even the same direction. `spi_slave`'s `rx_data` is an **output**: bytes it has finished shifting in from `MOSI`. The bridge's `rx_data` is an **input**: the bytes it assembles into transactions. So `spi_rx_data` runs **slave to bridge**. `spi_tx_data` runs the other way, from the bridge's `tx_data output` into the slave's `tx_data input`, carrying the bytes to be shifted out on `MISO`. Read them as two one-directional lanes that happen to share a naming convention, and the wiring falls out.

Why reg_addr is four bits. The protocol's command byte reserves bits 3–0 for the register index, so the master can send any value from 0 to 15 and the bus has to be able to carry it. Indices 7–15 are *defined* as reserved — a read returns `0x00000000` and a write is ignored — which is a rule the bank can only implement if it can see the out-of-range index in the first place. Three bits would make an illegal address indistinguishable from a legal one.

(c)

2 MARKS

```
reg_rdata <= (others => '0');
```

(Half a mark — the line is trivial; the marks are in what follows.)

With it in place, an SPI read transaction returns `0x00000000` for every register index. That is harmless until Chapter 5 because there is no register bank yet: nothing in the design has a value to return, and no testbench asks for one — `uart_top_tb` is *skipped* until its datapath exists, and the two transport testbenches drive the provided blocks directly. (1 mark)

Without it, `reg_rdata` has **no driver at all**. That is legal VHDL and the file still analyzes. But an undriven signal of a resolved type sits at the **left-most value of its type**, which for `std_logic` is 'U', *uninitialized*. So “returns” is the wrong word: the bridge latches all-'U', shifts 'U' out on MISO, and the metavalue propagates into anything that looks at it. It is not zero and it is not garbage-but-defined; it is the simulator’s marker for “nothing has ever driven this”, which is exactly the information the placeholder exists to suppress while the bank is missing. (Half a mark.)

The other half of the mark is for saying **when it must go**: in Chapter 5, in the same edit that instantiates `uart_regs`. Leave it and the vector has two drivers.

(d)

2 MARKS

What the top fixes. Every block’s **port contract**: the number of ports, their order, their directions and their types. The moment `uart_top` writes port `map(clock, reset_s2_n, baud_div_int, baud_tick)`, `baud_gen` has no interface decisions left to make — and neither does its testbench, which was written against the same contract before either existed. Building top-down means a block arrives into a shape that is already waiting for it, rather than the top being rewritten around each block as it lands. (1 mark)

What has to be true of `baud_gen`. Its entity must declare exactly four ports, in this order and with these types:

#	Dir	Type
1	in	<code>std_logic</code>
2	in	<code>std_logic</code>
3	in	natural range 1 to 65535
4	out	<code>std_logic</code>

What is not part of the contract: the names. `hw/README.md` says so outright — rename every port and everything still binds, because nothing in this course associates a port by name. The chapters keep their names anyway so that the prose and the code talk about the same signals, but that is a convenience for readers, not a rule the tools enforce. It is also, as Question 1 of Paper A explores, the reason a transposed pair of same-type ports is invisible until the system testbench runs. (1 mark)

Question 2 The generator, the divider, and the frame 13 MARKS

(a)

4 MARKS

```
-----  
-- Baud rate generator.  
--  
-- Inputs:  
--   - clock: 50 MHz system clock.  
--   - reset_s2_n: Active-low synchronized reset.  
--   - div: Baud rate divider. Must be greater than 0.  
--  
-- Outputs:  
--   - tick: Baud rate tick.  
-----  
  
library ieee;  
use ieee.std_logic_1164.all;  
  
entity baud_gen is  
    port(clock, reset_s2_n: in std_logic;  
          div           : in natural range 1 to 65535;  
          tick          : out std_logic);  
end entity;  
  
architecture behaviour of baud_gen is  
    signal counter: natural;  
begin  
    process(clock, reset_s2_n) is  
    begin  
        if (reset_s2_n = '0') then  
            tick    <= '0';  
            counter <= 0;  
        elsif (rising_edge(clock)) then  
            if (counter >= div - 1) then  
                tick    <= '1';  
                counter <= 0;  
            else  
                tick    <= '0';  
                counter <= counter + 1;  
            end if;  
        end if;  
    end process;  
end architecture;
```

(3 marks: 1 for the entity with `div` as a `natural` and the four ports in order; 1 for the asynchronous reset with `reset_s2_n` in the sensitivity list and both outputs cleared; 1 for a **registered**, one-clock-wide tick with the counter reloading to zero.)

Deduct for a tick driven combinationaly, for a synchronous reset, or for a counter that reloads to 1.

The comparison. (1 mark)

At `div = 1`, `div - 1` is 0 and the counter is always 0 when the test runs, so both forms fire on **every** clock. They are indistinguishable, and the same is true for any stable `div`: the counter reloads the instant it reaches the threshold and never passes it.

At `div = 0` they diverge completely. `div - 1` evaluates to -1:

- `counter >= -1` is **always true**, so the block ticks every cycle and the counter never leaves zero. Useless output, but the design keeps running and stays inside its declared range.
- `counter = -1` is **never true**, so the counter is incremented on every edge for ever. `counter` is a `natural`, so this ends in a **run-time bound check failure** that aborts the simulation, and in synthesis it is a counter that free-runs and wraps.

So `>=` is the one that keeps the counter inside its declared range. It also corrects a `div` changed mid-count on the next edge rather than after a wrap. One character of typing for both.

Worth noting for a candidate who raises it: `div` is declared `natural range 1 to 65535`, so 0 cannot legally be *bound* to it. The `>=` is defensive against the case the subtype is supposed to rule out, which is exactly when defensiveness is cheap and worth having.

(b)

3 MARKS

```
exact divider = 50_000_000 / (16 x 9600) = 50_000_000 / 153_600 = 325.52
BAUD_DIV      = round(325.52) = 326

tick rate = 50_000_000 / 326 = 153_374.2 Hz
baud rate = 153_374.2 / 16   = 9585.9 baud
error      = (9585.9 - 9600) / 9600 = -0.15%    (slow)
```

(2 marks: 1 for the divider, 1 for the achieved rate and a correctly signed error.)

Why 16 * baud. (1 mark) The receiver cannot see where a bit begins — there is no clock on the wire, only the line itself. So it **oversamples**: it watches `rx` sixteen times per bit, finds the falling edge that starts a frame, and then aims for the middle of each bit rather than its edge. For the transmitter and the receiver to share **one** time base rather than each keeping its own, that base has to tick at the finer rate.

uart_rx spends the sixteen. It uses them for resolution: one to notice the start edge, eight to reach the middle of the start bit for the re-check that rejects a glitch, and then one sample every sixteen thereafter, at each bit's centre.

uart_tx does not need them and simply counts: sixteen ticks to a bit, changing the line only on bit boundaries. It shares the generator not because it needs the resolution but so

that both halves are paced by the same divider from the same clock, with no handshake between them.

(c)

3 MARKS

The frame. (1 mark)

```
frame <= STOP_BIT & parity_bit & data & START_BIT;  -- frame(10 downto 0)
```

Index	0	1 .. 8	9	10
Carries	start	data(0) .. data(7)	parity	stop

The vector is now 11 bits and `bit_idx` is declared natural range 0 to 10; the terminal test compares `bit_idx` with the last index of the frame actually being sent, 10 with parity and 9 without. The parity bit goes **between the last data bit and the stop bit**, which is where the line protocol puts it, and the stop bit moves from index 9 to index 10.

The parity bit. (1 mark)

```
even_parity <= data(0) xor data(1) xor data(2) xor data(3)
               xor data(4) xor data(5) xor data(6) xor data(7);

parity_bit <= even_parity when parity_odd = '0' else not even_parity;
```

Even parity makes the total number of ones — data plus parity — **even**, and the XOR of the data bits is exactly the bit that achieves it. Odd parity is its complement. (With `parity_en` clear the parity field is not sent at all: the stop bit stays at index 9 and the frame ends there, exactly as in 8N1. A design that always sends eleven bits and drives the parity position '0' when parity is off is *wrong*: an 8N1 receiver samples that '0' where it expects the stop bit, and reports a framing error on every byte.)

The variable field, and the cost. (1 mark) With a `two_stop` input, the **stop field** is the only field whose length varies — 1 or 2 bits. Parity is a different matter: it adds a field rather than lengthening one.

Frame lengths: **10 bits** (8N1), **11** (either parity or a second stop bit), **12** (both).

At 115200 baud, throughput is `baud / bits-per-frame`:

```
10 bits: 115_200 / 10 = 11_520 bytes/s
12 bits: 115_200 / 12 = 9_600 bytes/s
```

so the longest frame costs **1920 bytes/s**, a drop of **16.7 %**, for the same line rate. The wire runs at exactly the same speed; you are simply spending a third of it on framing (4 bits in 12) rather than data, where 8N1 spent a fifth (2 bits in 10).

(d)

3 MARKS

What the testbench checks about busy. (1 mark) Nothing. `uart_tx_tb` binds `busy` and `done` but never asserts on either — §2.2 says so in as many words. What it does check is that `tx` idles high, goes low within two bit periods of start, that the start bit is low at its centre, that the eight data bits match least significant first, and that the stop bit is high. A busy that rises one clock late is entirely invisible to it.

What the feeder does. (1 mark) The feeder is

```
tx_load <= (not tx_empty) and (not tx_busy);
tx_pop  <= tx_load;
```

On the edge that loads a byte, the FIFO is popped and the transmitter enters `STATE_SEND`. On the **next** clock, `tx_busy` is still low — it rises a clock late — and `tx_empty` is still low if any byte remains queued. So `tx_load` is high for a **second** clock, producing a second start pulse and a second `tx_pop`.

What happens to the byte. `uart_tx` ignores start outside `STATE_IDLE`, so the second start does nothing — the candidate is right about that, and it is what makes the bug survive review. But `tx_pop` is not ignored: the FIFO honours it and **discards its front byte**. So the byte behind the one in flight is popped and never transmitted, once per frame.

Note the edge case that hides it: with exactly one byte queued, the FIFO is already empty after the first pop, so `tx_empty` is high on the second clock and `tx_load` is low. The bug only appears from the second queued byte onward.

The fix and the first testbench that could catch it. (1 mark)

```
busy <= '1' when state = STATE_SEND else '0';
```

A concurrent decode of the state, so `busy` is high in the same simulation cycle as the state change rather than a clock later.

The first testbench that **could** have caught it is `uart_top_tb`, in **Chapter 5** — the first testbench in which the feeder, the FIFO and the transmitter exist at the same time. Be honest about the rest: as written it writes exactly **one** byte to `TX_DATA`, so it lands squarely in the edge case above and does **not** catch it. Catching it needs a testbench that queues two bytes behind one in flight and checks that all of them leave. Full marks for the honest answer; half for “the `uart_top_tb` of Chapter 5” without the caveat.

Question 3 The asynchronous line, written out

14 MARKS

(a)

4 MARKS

```

-----
-- Generic double-flop synchronizer.
--
-- Generics:
--   - COUNT: Number of inputs to synchronize. Must be greater than 0.
--
-- Inputs:
--   - clock: 50 MHz system clock.
--   - reset_s2_n: Active-low synchronized reset.
--   - async_in: Inputs to synchronize.
--
-- Outputs:
--   - sync_out: Synchronized signals.
-----

library ieee;
use ieee.std_logic_1164.all;

entity sync is
    generic(COUNT: natural range 1 to 15 := 1);
    port(clock, reset_s2_n: in std_logic;
         async_in      : in std_logic_vector(COUNT-1 downto 0);
         sync_out       : out std_logic_vector(COUNT-1 downto 0));
end entity;

architecture behaviour of sync is
    signal sync_s1, sync_s2: std_logic_vector(COUNT-1 downto 0);
begin
    sync_out <= sync_s2;

    process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            sync_s1 <= (others => '0');
            sync_s2 <= (others => '0');
        elsif (rising_edge(clock)) then
            sync_s1 <= async_in;
            sync_s2 <= sync_s1;
        end if;
    end process;
end architecture;

```

(2 marks: 1 for the generic with its default and the vector ports sized from it; 1 for two flops in series with the **asynchronous** reset and sync_out taken from the **second**.)

The commonest error worth deducting for is `sync_out <= sync_s1`, which throws away the entire point: the output must come from the flop that has had a full clock period to settle.

Two edges. (1 mark) `sync_out` follows `async_in` after **exactly two** rising edges.

- **One is not a synchronizer.** A single register gives a metastable output no settling time at all: whatever it is doing at the next edge is what the design consumes. It looks identical in simulation — nothing in a simulator is ever metastable — which is precisely why `sync_tb`'s “exactly two edges” check exists and why deleting a flop is the bug that testbench is *for*.
- **Three is not wrong, it is unspecified.** An extra stage buys more settling time and would be right at a much higher clock or for a far more hostile input, but here it adds a clock of latency, fails `sync_tb`, and changes the receiver's edge-detection timing. The contract is two, so the module delivers two.

A COUNT = 8 instance. (1 mark) Each bit crosses **independently**, through its own pair of flops. Two bits that change on the same source edge can therefore resolve on different clocks, so `sync_out` may show one bit's new value beside another's old one for a cycle.

That is **not a defect in sync**: each bit individually is correctly synchronized, which is everything a synchronizer promises. The promise was never coherence across bits.

So the kind of signal you must never put through a wide instance is a **multi-bit value whose bits must stay coherent** — a counter, an address, any encoded number crossing clock domains. The receiving side can read a value the source never held: a counter going from 0111 to 1000 can be sampled as anything from 0000 to 1111. Use a Gray code (only one bit changes per step, so the worst case is the old value or the new one), a request/acknowledge handshake, or an asynchronous FIFO. `rx` is a single bit, so the question does not arise here — but it arises the first time somebody reaches for a wide instance.

(b)

4 MARKS

```
when STATE_IDLE =>
    -- Leave idle only on a high-to-low transition, never on a level.
    if ((rx_s2 = START_BIT) and (rx_prev = STOP_BIT)) then
        state <= STATE_START;
        ticks <= 0;                -- restart the count from this edge
    end if;

when STATE_START =>
    -- Half a bit on from the edge, check the line is still low.
    if (sample_start_bit = '1') then
        if (rx_s2 = '0') then
            state <= STATE_DATA;
```

```

        ticks <= 0;
    else
        state <= STATE_IDLE; -- a glitch, not a start bit
    end if;
end if;

```

and, **last on the tick**, after the case and inside the if `baud_tick = '1'`:

```

rx_prev <= rx_s2;

```

(2 marks: 1 for the edge test and the glitch re-check with both outcomes; 1 for the history flop and its placement.)

Why the placement matters. `rx_prev` is updated **on the tick**, inside if `baud_tick = '1'`, which is what makes the comparison “this tick against the previous tick”. Update it on every clock instead and it holds the line as it was one clock ago, not one tick ago: a falling edge is then seen only if it arrives in the last clock before a tick, so the receiver misses nearly every start bit, and `uart_rx_tb` fails its first case, “expected exactly one valid byte”. Where it sits *within* the tick does not change the hardware: `rx_prev` is a signal, so the case reads the value it had before this clock edge whatever the textual order, and a receiver with the assignment moved above the case passes `uart_rx_tb` unchanged. Writing it last is for the reader, since it puts the update where it takes effect. (Were it a *variable*, writing it first really would compare `rx_s2` with itself, and the receiver would never leave idle.) Worth noting alongside: the `ticks <= 0` in the idle branch overrides the tick-counter block above it, because that block comes first in the process and VHDL gives the **last** assignment to a signal in a process the final say.

The level test and the break. (2 marks)

With if `rx_s2 = '0'` then, the receiver re-arms the moment the previous frame’s stop bit has been judged. During a break — the line held low for longer than a frame, which is what a transmitter losing power or a cable coming loose looks like — it marches through back-to-back **all-zero frames**. Each of those ends on a low stop bit and is correctly rejected as a framing error, and that part is fine.

The failure is the frame still **in flight when the break ends**. Its stop bit is sampled after the line has returned high, so as far as the receiver can tell it is a well-formed frame: valid pulses and a byte assembled out of nothing — the tail of the break plus the returning idle — is pushed into the RX FIFO as real data. Software receives a byte nobody sent.

With an edge test there is no second falling edge until the line has gone high again, so no frame is ever in flight across that boundary. One flip-flop, and it is the difference between a receiver that works on clean data and one that survives a cable being unplugged.

The testbench case is `uart_rx_tb`’s break case: it holds the line low for longer than one frame and then idles long enough for a frame in flight to complete. Its message names the fix outright — “a break produced a byte - leave `STATE_IDLE` on a falling edge” — and a second

check requires the break to be reported as at least one framing error, so a receiver that goes silent instead of wrong does not pass either.

(c)

3 MARKS

```

elsif (rising_edge(clock)) then
    -- Defaults first: any pulse from the previous cycle lasts exactly one
    -- clock.
    data_out <= (others => '0');
    valid    <= '0';
    frame_err <= '0';
    ...
    when STATE_STOP =>
        if (sample_data = '1') then
            if (rx_s2 = '1') then
                data_out <= frame;      -- the stop bit is where it should-
            be
                valid    <= '1';
            else
                frame_err <= '1';      -- framing is broken
                frame     <= (others => '0');
            end if;
            state <= STATE_IDLE;      -- either way
        end if;

```

(2 marks: 1 for both outcomes and the unconditional return to idle, 1 for the defaults.)

What makes the pulses one clock wide. The defaults are written **first on every clock edge**, and only the specific case overrides them. So a `valid` set at one edge is cleared at the next, with no extra counter, no extra state and nothing to get out of step. Writing the default first and letting the specific case override it is the book's idiom for a single-cycle strobe, and it is the same one `uart_tx` uses for done.

Why the byte is dropped. (1 mark) A framing error means the receiver's idea of *where the bits were* is wrong: the stop bit was not high where it should have been, so the eight bits it sampled were sampled at times it cannot vouch for. Delivering them alongside a flag would put a byte into the RX FIFO that software cannot distinguish from good data once it has popped past it — `ERROR_FLAGS` is **global and sticky**, not attached to a FIFO entry, so there is no way to say "this byte in particular is suspect". Dropping it preserves the invariant that every byte in the RX FIFO is a byte the receiver believes in, and the sticky flag still tells software that something went wrong.

What the bank does with each pulse. `valid` becomes `uart_regs' rx_push`, so the byte enters the RX FIFO and `STATUS` bit 1 (RX-valid) goes high, where software can poll it. `frame_err` is latched into `ERROR_FLAGS` bit 0, which stays set until software writes zero to

clear it, and STATUS bit 2 is the OR of the error flags. Both pulses are one clock wide and neither could be polled directly — turning them into levels is the register bank’s entire job.

(d)

3 MARKS

All three passed. (1 mark)

Byte	Binary	Reversed	Same?
0xA5	1010 0101	1010 0101	yes
0x3C	0011 1100	0011 1100	yes
0x5A	0101 1010	0101 1010	yes

A receiver that stores its data bits in the reverse order delivers, for each of these, **exactly the byte that was sent**. Every check comparing the received byte with the expected one passed, and no other check in either testbench looks at bit order at all — so at that revision the whole of hw/ was blind to it.

The property and the count. They are **bit-reversal palindromes**. A palindrome is determined by its top four bits — b7 fixes b0, b6 fixes b1, and so on — so there are $2^4 = 16$ of them out of 256. All three constants happen to be among the sixteen.

The second, independent reason for uart_top_tb. (1 mark) It **loops tx back to rx**. A design built by one person from one set of chapters will get the order the same way in both halves, and a matched pair of reversals **cancels exactly**: the transmitter puts the byte on the wire backwards and the receiver reads it backwards, so the byte read back over SPI is the byte written, whatever the constant. Changing 0x5A to something asymmetric would let the testbench catch a *one-sided* flip — a receiver reversed while the transmitter is not — but it structurally cannot catch the matched pair, which is the likelier bug.

What the testbench reports today. (1 mark) 0x53 is 0101 0011 and its reverse is 1100 1010, so with the reversed-order receiver in place data_out carries **0xCA**. uart_rx_tb’s clean-frame case then fails and names both values, since its message already prints the expected and the received byte through to_hex:

```
uart_rx_tb: received byte mismatch, expected 0x53, got 0xCA!
```

The fix that got it there was a constant or two per testbench — TXBYTE in uart_tx_tb, BYTE_A and BYTE_B in uart_rx_tb, and the transmitted byte in uart_top_tb — which is worth noting in its own right: the gap was a handful of constants wide and had nothing to do with the checks themselves.

What is still out of reach. A transmitter and a receiver that are reversed **together**. uart_tx_tb and uart_rx_tb each catch their own half, so the pair cannot survive both unit testbenches — but uart_top_tb alone never could, for the structural reason above, and a candidate who says “the loopback is fixed now” has missed it. No constant makes a loopback able to test a convention that both ends of it share.

Full marks for reaching the conclusion by any route. The key insight, worth the bulk of the marks, is that a loopback testbench cannot test a property that both halves get wrong together, and that a symmetric constant cannot test an ordering.

Question 4 The bank, written out

13 MARKS

(a)

4 MARKS

```

process(clock, reset_s2_n) is
    variable addr: natural range 0 to 15;
begin
    if (reset_s2_n = '0') then
        err_flags <= (others => '0');
    elsif (rising_edge(clock)) then
        addr := to_integer(unsigned(reg_addr));

        -- Software clear: a write of zero to ERROR_FLAGS clears every flag.
        if ((reg_write = '1') and (addr = REG_ERR_FLAGS) and (unsigned(reg_wdata) = 0)) then
            err_flags <= (others => '0');
        end if;

        -- The receiver's pulse sets the framing bit. Written last, so it wins a tie.
        if (frame_err = '1') then
            err_flags(ER_FRAMING) <= '1';
        end if;
    end if;
end process;

```

(2 marks: 1 for a reset value of zero with a defined set and a defined clear, 1 for using uart_def's constants and for a clear that is a write of zero to the right index.)

Both on the same clock. The set is written **after** the clear in the same process, and VHDL gives the last assignment the final say, so **the set wins**: an error arriving in the same cycle as a clear is not lost. That is the right way round. Losing an error because software happened to clear in the same 20 ns window is a silent data-integrity hole that would be essentially impossible to reproduce; a flag that survives one extra clear merely makes software clear twice, which it was polling anyway. A candidate who orders it the other way should say why — the honest reason would be “so a clear always leaves the register readable as zero”, and it is not a good enough one.

Why a pulse cannot be polled. (1 mark) frame_err is high for **one clock**: 20 ns. Software polls over SPI, and one five-byte transaction at 1 MHz SCK takes at least **40 µs** — two thousand

times longer than the pulse, before any inter-byte overhead. The probability that a STATUS read lands on precisely the clock the pulse is high is effectively zero, and even then the pulse would have to coincide with the exact cycle in which the bridge latches the read value. A pollable bit has to be a **level that persists until acknowledged**, and manufacturing that level from a pulse is what the latch is for.

Wired straight through. (1 mark) STATUS bit 2 would read 0 on **practically every poll, for ever**, no matter how many framing errors occurred: a poll sees it only if the bridge's one-clock latch happens to fall on the one clock the pulse is high. The system would appear to have no errors at all.

Worse, it would appear so *consistently*: a driver would never see the bit flicker, never see it disagree with ERROR_FLAGS, and have nothing at all to suggest the mechanism was broken. A flag that is always wrong in the same direction is far harder to notice than one that is sometimes wrong, and the first evidence would be a bench full of corrupted bytes and a peripheral cheerfully reporting itself healthy.

(b)

4 MARKS

The read decode. (1 mark)

```
with to_integer(unsigned(reg_addr)) select reg_rdata <=
    status_reg          when REG_STATUS,
    ctrl_reg            when REG_CTRL,
    x"0000" & baud_div_reg when REG_BAUD_DIV,
    (others => '0')      when REG_TX_DATA,    -- write-only
    x"000000" & rx_front  when REG_RX_DATA,
    (others => '0')      when REG_RX_POP,      -- write-only
    x"0000000" & '0' & err_flags when REG_ERR_FLAGS,
    (others => '0')      when others;         -- reserved: 7-15
```

An equivalent case inside a combinational process with reg_addr and every source register in the sensitivity list is just as good; mark the decode, not the form. What must be right is that the two write-only registers and every reserved index read **zero**, and that every path assigns.

The write actions. (1 mark)

```
tx_push <= '1' when reg_write = '1' and reg_idx = REG_TX_DATA else '0';
rx_pop  <= '1' when reg_write = '1' and reg_idx = REG_RX_POP  else '0';
```

Two concurrent strobes, outside any process, where reg_idx is reg_addr converted to an integer: tx_push is the TX FIFO's wr, with reg_wdata(7 downto 0) bound to its wdata, and rx_pop is the RX FIFO's rd. In the clocked process the two indices do **nothing**: each FIFO registers its own strobe on the same edge, and writing FIFO logic into the process as well is the most common way to get two bytes out of one write. A candidate who registers the strobes inside the process instead, with defaults of '0' at the top and '1' in the two branches,

builds a working bank one clock slower; give the mark if the strobes are provably one clock wide.

Both are **edge events**: one write, one action, never a held level. That is what makes them safe to be one-cycle strobes into the FIFOs.

Why the read is combinational. (1 mark) The transport contract says the bridge **latches the addressed register's value once**, at the end of the command byte, and then shifts those four bytes out while the master sends dummies. For that latch to capture the right value, `reg_rdata` must already reflect `reg_addr` in the same cycle the address becomes known. A registered read would present its value one clock late, so the bridge would latch the answer to the *previous* question — every read would return the previous read's register, which is a bug that looks like a random off-by-one until somebody notices the pattern.

when others, and why it is a different mistake here. It must produce (`others => '0'`) on the read side and `null` on the write side, because the protocol **defines** indices 7–15 as reserved: a read returns `0x00000000` and a write is ignored.

VHDL does not let you leave the branch out altogether: the choices of a case or a selected assignment must cover every value of the selector, and an integer selector has far more than seven, so a missing `others` does not even analyze. What a decoder *can* get wrong is an `others` that assigns nothing — `when others => null;` in a combinational process with no default assignment above the case — and in an ordinary decoder that is a *synthesis* mistake: it infers a latch, because the output has no value for the uncovered case and must therefore hold. That is true here too. But here it is also a **protocol violation**: a driver is entitled to probe an unimplemented register and get zero, and a bank that returned something else — or held the previous value — would be non-compliant even if it inferred no latch at all. Two independent reasons for the same line.

What the bank relies on. (1 mark) That `spi_reg_bridge` asserts `reg_write` **only on a completed, non-aborted write transaction** (Part 3 of the protocol, §A.3). A strobe the bank sees is therefore always a real commit. That single guarantee is what lets `TX_DATA` and `RX_POP` be simple one-cycle actions with no commit-and-abort machinery of their own — the bank never has to think about `SS` at all. If a half-finished transaction could reach it as a `reg_write`, every write action in the bank would need to be held pending and rolled back, and the read/pop split would stop being an elegance and start being a necessity.

(c)

3 MARKS

The condition. (1 mark)

```
overrun <= rx_push and rx_full;
```

The receiver delivered a byte while the RX FIFO had no room, so `fifo's not full` guard dropped it. Both signals are already inside `uart_regs` — `rx_push` is the receiver's valid and `rx_full` is the FIFO's own flag — so the detection is one AND gate. It sets the flag through the same latch as `frame_err`.

The bit and the clear path. `ER_OVERRUN`, **ERROR_FLAGS bit 2**, cleared by the same write of `0x00000000` to `ERROR_FLAGS` that clears the other two, and visible through `STATUS bit 2` (the OR of the flags) so that a polling driver sees it without an extra register read.

What `uart_rx` would need, and why it is worse. (1 mark) It would have to be told **whether the previous byte has been consumed** — in practice the RX FIFO's full flag, or a “byte still unread” level, as a new input port.

That is a worse design because it inverts the dependency. `uart_rx`'s job is to recover bytes from a wire; it is a purely serial, purely synchronous module with no knowledge of buffering and no reason to acquire any. Giving it a port that reports the state of a buffer it does not own couples it to the register bank's internal storage, and means `uart_rx_tb` would have to model a FIFO in order to test a receiver. The bank already holds both facts, one clock apart, in one module.

The two physical causes. (1 mark)

- **Overrun:** the wire is fine and the *software* is late. A host busy elsewhere, a poll loop that stalled, or simply a line faster than the loop can drain: at 115200 a byte arrives every 86.8 μ s, a single five-byte SPI transaction costs at least 40 μ s, and receiving one byte takes three of them — the `STATUS` poll, the `RX_DATA` read and the `RX_POP` write — so at least 120 μ s. A continuous stream at 115200 outruns even a driver that does nothing else, and the eight-entry FIFO only postpones the loss. It is a timing failure above the peripheral.
- **Framing error:** the *wire* is not fine. A baud mismatch large enough to move the stop-bit sample out of its bit, a break from a transmitter that lost power or a cable that came loose, noise or a reflection on a long unterminated run, or a level-shifting problem. It is an electrical or configuration failure below the peripheral.

One is caused by the reader, the other by the sender or the medium, which is exactly why they are separate bits rather than one “something went wrong” flag.

(d)

2 MARKS

Two disagreements with `uart_regs_tb`. (1 mark each, any two of the following)

1. **After reset.** The testbench checks that `STATUS` immediately after reset shows **TX-ready and TX-idle set** with `RX-valid` and Error clear. A register updated only when `rx_push`, `tx_pop` or `frame_err` asserts has seen none of those, so unless its reset value is hand-written to `0b1001` it reads zero and the check fails — and a hand-written reset value is a second copy of the truth to keep in step with the FIFOs for ever.
2. **`RX_POP`.** Popping the RX FIFO empties it and must clear `RX-valid`, and the testbench checks exactly that. But `RX_POP` is a **register write**, not one of the three events listed, so `RX-valid` stays set after the FIFO is empty. A driver would then poll, see `RX-valid`, read `RX_DATA` from an empty FIFO and pop it, for ever.
3. **`TX_DATA`.** A write pushes a byte and must clear `TX-idle`; a push is also not in the list, so the testbench's “a `TX_DATA` write must clear `TX-idle`” check fails for the same reason.

4. **tx_busy.** It changes with no event at all — the transmitter simply enters and leaves a state — so TX-idle goes stale whenever the datapath moves without the bank being told.

The rule. Derive a status bit from **the state it reports**, never from the events that changed that state. A register written by events is wrong whenever an event is missed, whenever two coincide, whenever something changes without an event, and immediately after reset.

Question 5 Writing the contracts

12 MARKS

(a)

5 MARKS

```

/**
 * @file Byte transport interface.
 */
#pragma once

#include <stdint.h>

namespace driver
{
    namespace transport
    {
        /**
         * @brief Byte-level SPI transport interface.
         */
        class Interface
        {
        public:
            /** @brief Destructor. */
            virtual ~Interface() noexcept = default;

            /** @brief Start a transaction by pulling SS low. */
            virtual void begin() noexcept = 0;

            /**
             * @brief Exchange a single byte in full duplex.
             *
             * @param[in] byte Byte to send.
             * @return The byte received at the same time.
             */
            virtual uint8_t transfer(uint8_t byte) noexcept = 0;

            /** @brief End the transaction by releasing SS. */

```

```

    virtual void end() noexcept = 0;
};
} // namespace transport
} // namespace driver

```

(2 marks: 1 for the three pure virtuals with correct signatures, 1 for the *virtual*, *noexcept*, = default destructor and the nested-block namespaces.)

```

/**
 * @file UART driver interface.
 */
#pragma once

#include <stdint.h>

namespace driver
{
    namespace uart
    {
        /**
         * @brief UART driver interface.
         */
        class Interface
        {
        public:
            /** @brief Destructor. */
            virtual ~Interface() noexcept = default;

            /** @brief Configure the peripheral: set the baud divider and enable it. */
            virtual void configure(uint16_t baudDiv) noexcept = 0;

            /** @brief Send one byte. @return True if accepted, false if the TX FIFO was ↵
             ↵ full. */
            virtual bool write(uint8_t byte) noexcept = 0;

            /** @brief Receive one byte. @return True if a byte was retrieved, false ↵
             ↵ otherwise. */
            virtual bool read(uint8_t& byte) noexcept = 0;

            /** @brief Read the raw STATUS register. */
            virtual uint32_t status() const noexcept = 0;

            /** @brief Read the raw ERROR_FLAGS register. */

```

```

virtual uint32_t errorFlags() const noexcept = 0;

/** @brief Clear the error flags. */
virtual void clearErrors() noexcept = 0;
};
} // namespace uart
} // namespace driver

```

(3 marks: 1 for configure, write and read with the right parameter and return types — note that read takes a **reference** out-parameter and returns bool; 1 for the three status methods with status() and errorFlags() **const** and clearErrors() not; 1 for noexcept throughout and for the AVR-portable style: <stdint.h>, bare types, nested blocks, no [[nodiscard]].)

Deduct for std::uint8_t, for namespace driver::uart, for [[nodiscard]], for a non-virtual destructor, and for read returning the byte instead of a bool.

(b)

3 MARKS

```

namespace reg
{
/** UART status register. */
constexpr uint8_t STATUS{0U};

/** UART control register. */
constexpr uint8_t CTRL{1U};

/** UART baud rate divider register. */
constexpr uint8_t BAUD_DIV{2U};

/** UART TX data register. */
constexpr uint8_t TX_DATA{3U};

/** UART RX data register. */
constexpr uint8_t RX_DATA{4U};

/** UART RX pop register. */
constexpr uint8_t RX_POP{5U};

/** UART error flag register. */
constexpr uint8_t ERROR_FLAGS{6U};
} // namespace reg

namespace status
{

```

```

/** Transmitter is ready for another byte. */
constexpr uint8_t TX_READY{0U};

/** An RX byte is available. */
constexpr uint8_t RX_VALID{1U};

/** UART is in an error state. */
constexpr uint8_t ERROR{2U};

/** Transmitter is idle. */
constexpr uint8_t TX_IDLE{3U};
} // namespace status

```

(1 mark for the seven indices, 1 for the four status positions — both sets in the right order with the right values.)

Why plain constexpr. (1 mark, shared with the next point) inline **variables** are a C++17 feature, and the oldest AVR toolchain the course targets, the avr-gcc Microchip Studio ships, does not have them — it does not accept `-std=c++17` at all. Plain `constexpr` compiles everywhere the course needs it to.

What internal linkage costs here: nothing. At namespace scope `constexpr` already implies internal linkage, so each translation unit gets its own copy of a **compile-time** constant. That copy is folded into the instruction stream at every use site and occupies no storage, so there is no duplication to pay for and no one-definition-rule problem, because there is no object to define twice.

Positions rather than masks. It keeps the C++ **identical to `uart_def.vhd`**, which stores natural bit positions because that is what indexes a `std_logic_vector`, and identical to the protocol’s own “bit N” wording. The two sides are then each checked against the specification rather than against each other, which is the only arrangement in which a disagreement is findable. The mask is formed at the use site:

```

if (status & (1U << status::RX_VALID)) { /* a byte is waiting */ }

```

which is explicit about which bit is meant and cannot be mistaken for a value.

(c)

2 MARKS

What it is for, and who owns it. (1 mark) `myStop` refers to a flag the **caller** owns — the very flag the application’s `run(const volatile bool& stop)` is watching. The stub sets it true the moment its scripted RX buffer is exhausted, so a single-threaded test can end an application loop that would otherwise never return.

`read()` therefore does two things when the buffer runs out: it sets `myStop` to true and returns false. Both matter — the false says “no byte this pass”, the flag says “and there will not be another”.

Why a stub should care at all. Because the thing it doubles for is a **UART**, and a real UART never runs out of input; it merely has none *yet*. There is no in-band way to say “that was all the input there will ever be”, so the double supplies an out-of-band one. It is the double being faithful about the test’s needs rather than about the hardware, which is exactly what a double is for.

What the reference member forces. (1 mark) The language forces two things. The **default constructor** is implicitly deleted, because a reference member must be bound at construction — there is no way to build a Stub without a flag to point at. And the copy and move **assignment** operators are implicitly deleted, because a reference cannot be rebound after construction. The copy and move **constructors** are *not* forced: the implicit ones would compile, and bind the new stub to the same flag. The book deletes them anyway, explicitly, because a copy that shares its original’s stop flag but not its buffers is a trap, and a class that cannot be assigned but can be copied invites exactly that mistake. Uart and EchoNode delete the same five for the same reason. Award the mark for saying which one is forced, and why the other two are deleted by choice.

(d)

2 MARKS

The rule. (1 mark) A test double must be faithful about **everything the code under test depends on**, and may ignore everything else. It is a fixture, not a model of the hardware: fidelity it is not asked for is fidelity nobody checks, and unchecked behaviour in a double is a liability the day it disagrees with the real thing.

Applied here. The only application tested against `driver::uart::Stub` is `app::EchoNode`, which drives the UART entirely through `read()` and `write()`. It never calls `status()`, `errorFlags()` or `clearErrors()`. So returning 0 and doing nothing is not laziness — it is the correct amount of fidelity, and inventing a status word would mean shipping behaviour with nothing to check it against.

What would have to change. (1 mark) The moment an application makes a **decision** from the status word, the stub needs a scriptable one: a settable `uint32_t`, a helper such as `setStatus(uint32_t)` and `setErrorFlags(uint32_t)`, and a `clearErrors()` that actually zeroes what `errorFlags()` returns so that the clear-then-recheck path can be exercised.

Examples of such a thing: a node that stops or reports when STATUS bit 2 goes high; one that waits for TX-idle before letting the MCU sleep; one that counts framing errors and resets the link after a threshold. Each of those is a decision the current stub cannot script, and each is a good reason to extend it — but only when it exists.

Question 6 Writing the protocol

13 MARKS

(a)

5 MARKS

```
// -----
uint32_t Uart::readReg(const uint8_t addr) const noexcept
{
    constexpr uint8_t dummy{0x00U};
    constexpr uint32_t shift{8U};
    constexpr uint32_t size{static_cast<uint32_t>(sizeof(uint32_t))};
    constexpr uint32_t last{size - 1U};

    uint32_t value{};
    auto& transport = const_cast<transport::Interface&>(myTransport);

    transport.begin();
    transport.transfer(addr);           // bit 7 clear => read. Reply ignored.

    for (uint8_t i{}; i < size; ++i)
    {
        const auto idx = last - i;      // 3, 2, 1, 0 => most significant byte ←
        ← first
        const uint8_t response{transport.transfer(dummy)};
        value |= static_cast<uint32_t>(response) << (shift * idx);
    }
    transport.end();
    return value;
}

// -----
void Uart::writeReg(const uint8_t addr, const uint32_t value) noexcept
{
    constexpr uint8_t writeMask{0x80U};
    constexpr uint32_t shift{8U};
    constexpr uint32_t size{static_cast<uint32_t>(sizeof(uint32_t))};
    constexpr uint32_t last{size - 1U};

    myTransport.begin();
    myTransport.transfer(static_cast<uint8_t>(addr | writeMask)); // bit 7 set ←
    ← => write

    for (uint32_t i{}; i < size; ++i)
    {
        const auto idx = last - i;      // most significant byte first

```

```

        myTransport.transfer(static_cast<uint8_t>(value >> (shift * idx)));
    }
    myTransport.end();
}

```

(4 marks: 1 for the command bytes — the raw index for a read, index | 0x80 for a write; 1 for the begin(), five transfers, end() framing in both; 1 for **most significant byte first** in the read assembly; 1 for **most significant byte first** in the write split.)

Byte order is the part to be strict about. Reverse either loop and the same four bytes cross the wire carrying a different 32-bit value, and neither the compiler nor a round trip through a symmetrically-wrong stub will say a word.

The replies. readReg **discards** the reply to the command byte: the slave was shifting that out before it knew which register was being asked for, so it is meaningless by construction. writeReg discards **all four** of its replies as well — on a write the master has nothing to learn from MIS0, and the transfers happen only because SPI is duplex and a byte out costs a byte in. (Half a mark.)

What const buys. (Half a mark.) It lets status() and errorFlags() — the two public methods that are pure observations — be const too. Those are the two that depend on it. Without it, a caller holding a const Uart& could not read the peripheral's status at all, which would be an odd thing for a const handle to forbid.

(b)

4 MARKS

```

// -----
bool Uart::read(uint8_t& byte) noexcept
{
    const auto status = readReg(reg::STATUS);
    const auto rxValid = static_cast<bool>(status & (1U << status::RX_VALID));

    if (rxValid)
    {
        constexpr uint32_t popCount{1U};
        byte = readReg(reg::RX_DATA);           // pure read: does not pop
        writeReg(reg::RX_POP, popCount);        // separate write: advances the FIFO
    }
    return rxValid;
}

```

(1 mark: poll, then read, then pop, in that order, with nothing issued when RX_VALID is clear.)

The recorded bytes. (1 mark)

```
transaction 1 STATUS read      : 00 00 00 00 00
transaction 2 RX_DATA read    : 04 00 00 00 00
transaction 3 RX_POP write    : 85 00 00 00 01
```

beginCalls() == 3, endCalls() == 3, and txLen() == 15.

The scripting. (1 mark)

```
stub.injectRxByte(0x00U);           // command-phase placeholder
stub.injectRxWord(1U << status::RX_VALID); // STATUS = 0x00000002
stub.injectRxByte(0x00U);           // command-phase placeholder
stub.injectRxWord(0x00000041U);      // RX_DATA = 0x41
```

Ten bytes in total, for **two** register values. (1 mark)

The discrepancy. The stub is a **byte pipe**: it returns the next queued byte on *every* transfer() call, and a five-byte read transaction makes five of them. The reply the driver actually uses is only four bytes long, because the reply to the command byte is discarded — but it is still *consumed*. So each read costs one placeholder plus the four data bytes.

The third transaction needs nothing queued: it is a write, the driver ignores all five replies, and the stub returns 0x00 once its buffer is exhausted anyway.

Getting this wrong is spectacular and confusing: queue only the four data bytes and every read is shifted by one byte. The command phase swallows the most significant byte, so readReg assembles the value's low three bytes, shifted up by eight bits, together with the first byte queued for the *next* read: STATUS reads back as 0x00000200, RX-valid is clear, and read() returns false with a byte waiting. Every later read in the test is misaligned the same way. This is precisely why the provided suite wraps the scripting in a scriptRead(stub, value) helper rather than leaving it at each call site.

(c)

2 MARKS

```
inline void writeBlocking(Interface& uart, const uint8_t byte) noexcept
{
    while (!uart.write(byte)) {}
}

inline void readBlocking(Interface& uart, uint8_t& data) noexcept
{
    while (!uart.read(data)) {}
}
```

(1 mark for both, inline and noexcept, in the driver::uart namespace.)

Why inline. (1 mark, shared) They are **defined in a header**, so every translation unit that includes it emits a definition. Without `inline` that is a one-definition-rule violation and a duplicate-symbol error at link time the moment two files include the header.

Why Interface& and not Uart&. So they work over **any** implementation — the concrete `Uart`, the driver `::uart::Stub` of Chapter 6, any future driver — dispatching virtually at run time. Taking the concrete type would tie a convenience function to one implementation for no reason.

Why free functions in a separate header. They add **no state and no new contract**: they are a spin loop over an operation the interface already provides. Putting them in the interface would oblige every implementation, including every stub, to reimplement that loop, and would push the decision to block *below* the seam, so a test could no longer opt out of it. The core stays deterministic and testable, and the spinning lives where a caller chooses it.

(d)

2 MARKS

The call sequence. (1 mark)

```
status() -> readReg(STATUS) -> write(cmd) -> readReg(STATUS) -> write(cmd) ->
    ...
```

`Uart::write` polls `STATUS` before it sends anything, and `readReg` is what polls. So a `readReg` implemented in terms of `write()` calls the very method that calls it, with **no base case**: infinite mutual recursion, entered the first time anything reads a register: the first `read()`, `write()` or `status()` after `configure()`, which only writes, and every one thereafter.

What the ATmega328P exhibits. (1 mark) Each level pushes a stack frame. The ATmega328P has **2 KB of SRAM**, the stack grows down from the top of it toward the statically allocated data, and there is no MMU, no stack guard, no exception and no fault handler. So the stack simply runs into `.data` and `.bss` and starts overwriting live variables, then keeps going.

The symptom is not a clean crash: it is **memory corruption followed by a jump to a garbage address**, which typically presents as a device that appears to reset in a loop the moment the driver is first used, or that behaves erratically before it does. On the host the same bug is a segmentation fault, which is the kinder outcome and the reason the host suite finds it first — one more argument for building the driver on the host before it ever reaches a chip.

The rule. `transfer()` moves a **byte on the wire**; `write()` sends a **byte over the UART**. They are two different layers that happen to have the same shape, and the private register core may only ever touch `myTransport`.

Question 7 Writing the transport

11 MARKS

(a)

5 MARKS

```
// -----
AvrSpi::AvrSpi() noexcept
{
    DDRB |= (1U << SCK) | (1U << MOSI) | (1U << SS);    // SCK, MOSI, SS out; MISO ↪
    ↪ stays in.
    PORTB |= (1U << SS);                                // Idle the chip select ↪
    ↪ high.
    SPCR = (1U << SPE) | (1U << MSTR) | (1U << SPR0); // Enable, master, mode 0, ↪
    ↪ MSB, f_osc/16.
}

// -----
AvrSpi::~AvrSpi() noexcept
{
    DDRB &= ~(1U << SCK) | (1U << MOSI) | (1U << SS));
    PORTB &= ~(1U << SS);
    SPCR = 0U;
}

// -----
void AvrSpi::begin() noexcept { PORTB &= ~(1U << SS); }

// -----
uint8_t AvrSpi::transfer(const uint8_t byte) noexcept
{
    SPDR = byte;
    while (0U == (SPSR & (1U << SPIF))) {} // Spin until the transfer completes.
    return SPDR;                          // Now SPDR holds the received byte.
}

// -----
void AvrSpi::end() noexcept { PORTB |= (1U << SS); }
```

(3 marks: 1 for the constructor's three statements with SS idled high and MISO untouched; 1 for begin() low and end() high; 1 for transfer() with the SPIF spin between the write and the read.)

SPR0 set with SPR1 clear and SPI2X clear gives $f_{\text{osc}}/16$, which is 1 MHz. D0RD, CP0L and CPHA are all clear, which is MSB first, mode 0 — so the contract is met by bits that are *absent* as much as by bits that are present.

The destructor. (1 mark) It must clear **exactly what the constructor set and nothing else**:

- the three DDRB direction bits for SCK, MOSI and SS, putting those pins back to inputs;
- the SS bit in PORTB, which was the chip-select drive;
- all of SPCR, written to 0, which disables the peripheral.

It must **leave alone**: MISO, because the constructor never touched it — it is an input at reset and stays one, and returning something you did not take is not RAIL, it is trespass. Also SPSR, which the constructor never wrote: its one configuration bit, SPI2X, is left at its reset value of 0, which $f_{osc}/16$ relies on. And every other bit of DDRB and PORTB, which belong to whatever else is using port B. The `&=` and `|=` forms are what confine the edits to those bits.

Why `SPCR = ...` and not `SPCR |= ...` (1 mark) The plain assignment **guarantees** that every bit the design relies on being clear is clear: D0RD, CP0L, CPHA, SPIE and SPR1. With `|=`, whatever was in SPCR beforehand survives — anything that ran first, an Arduino core's `SPI.begin()` or a library of your own, could have left D0RD set (LSB first) or SPR1 set (a different prescaler), and the transport would silently run at the wrong bit order or the wrong rate with no line in the source to blame it on.

The general rule, worth stating: a register you **own** is *set*; a register you **share** is *merged*. SPCR is entirely this class's, so it is assigned. DDRB and PORTB are shared with every other pin on port B, so they are read-modify-written.

(b)

3 MARKS

The principle. (1 mark) **RAII**: what the constructor acquires, the destructor releases. This class acquires the SPI hardware and three port pins, and a defaulted destructor gives none of them back. The Interface declares a **virtual** destructor precisely so that destroying an `AvrSpi` through an `Interface&` runs the derived one; providing that mechanism and then making it do nothing throws away the thing the base class went to the trouble of arranging.

What is never given back. PB5, PB3 and PB2 are left as **outputs** in DDRB; PB2 is left **driven high** in PORTB; and SPCR still has SPE and MSTR set, so the peripheral is still enabled and still owns those three pins.

Concrete harm. (1 mark) Any code that runs afterwards and wants PB2, PB3 or PB5 as a general-purpose **input** finds them driven as outputs and reads its own drive rather than the world. Any code that wants the SPI peripheral in a different configuration — as a slave, at a different prescaler, LSB first — inherits a half-configured master rather than the reset state, and a transport that uses `|=` on SPCR (see (a)) would inherit MSTR and never notice. And with SPE still set the peripheral keeps driving SCK and MOSI, so anything else trying to use those pins fights the hardware for them.

Why “never destroyed” is unsafe here in particular. (1 mark) Because this codebase builds its objects as **automatic variables and injects them by reference**, and the **host test suite** does exactly that: it constructs an `AvrSpi` inside a scope, exercises it over the mocked register file, lets it go out of scope, and constructs another for the next case. With a do-nothing destructor the peripheral it configured outlives it, so a defaulted destructor is observably wrong even on the host. The suite defends itself today by calling `resetHardware()` at the top of every case; without that defence the cases would become **order-dependent** and the

configuration assertions could pass for the wrong reason — the bits right because the last test set them, not because this constructor did. The point is that a class should not need its tests to clean up after it.

(c)

3 MARKS

What sets each. (1 mark)

- **SCK** is set by the SPI **prescaler**, a hardware divider off the CPU clock selected by `SPR1:SPR0` and `SPI2X`: $f_{\text{osc}}/4, /16, /64, /128$ and the doubled variants. It divides whatever the chip is *actually* running at, and no software constant takes part — which is exactly why `AvrSpi` needs no `F_CPU`.
- The **USART's baud** is set by **UBRR0**, a number the software computes: $\text{UBRR0} = F_CPU / (16 * \text{baud}) - 1$. The compiler substitutes `F_CPU` at compile time, so the number is only right if the macro matches reality. The logging path needs `F_CPU` because it does arithmetic with it; the transport does not because it does none.

At 8 MHz with `F_CPU` still 16000000. (2 marks)

- **SCK** becomes $8 \text{ MHz} / 16 = 500 \text{ kHz}$ instead of 1 MHz. The link **still works**: the FPGA synchronizes and edge-detects `sck` into its own 50 MHz domain, so it keeps up with far more than 1 MHz and there is no minimum rate. Everything is simply twice as slow — a five-byte transaction goes from 40 μs to 80 μs .
- The **USART** computes `UBRR0` for a 16 MHz clock while the hardware divides a real 8 MHz one, so the actual baud is **half** of what the divider was computed for: at a 115200 log rate, about 57600. A terminal set to 115200 sees framing errors and garbage, with the occasional plausible character by luck.

Which you notice first, and why that is unfortunate. The **terminal**, immediately and unmistakably — garbage on a console is impossible to miss, while a `readReg` that takes 80 μs instead of 40 μs looks exactly like one that takes 40 μs unless you are measuring.

That is unfortunate because the loud symptom points at the **logging**, the one part of the system that does not matter, and hides the fact that the SPI link is also running at half speed — which is precisely what Exercise 9.3 asks you to measure against the 1 MHz budget. A candidate who “fixes” it by halving the log baud has made the symptom go away and left the wrong clock in place, and every timing figure they take afterwards is out by a factor of two. The fix is to `F_CPU`, so that it states the clock the part really runs at (or to the fuses, if 16 MHz was what was wanted).

Question 8 The application, and the plan

12 MARKS

(a)

4 MARKS

```

/**
 * @file Echo node implementation.
 */
#pragma once

#include "app/interface.hpp"

// clang-format off
namespace driver
{
    /** UART driver interface. */
    namespace uart { class Interface; }
} // namespace driver
// clang-format on

namespace app
{
    /**
     * @brief Echo node implementation.
     *
     * This class is non-copyable and non-movable.
     */
    class EchoNode final : public Interface
    {
    public:
        /** @brief Constructor. @param[in] uart UART driver instance. */
        explicit EchoNode(driver::uart::Interface& uart) noexcept;

        /** @brief Destructor. */
        ~EchoNode() noexcept override = default;

        /** @brief Run application. @param[in] stop Set true to stop the application. ↵
         ↵ */
        void run(const volatile bool& stop) noexcept override;

        EchoNode() = delete;
        EchoNode(const EchoNode&) = delete;
        EchoNode(EchoNode&&) = delete;
        EchoNode& operator=(const EchoNode&) = delete;
        EchoNode& operator=(EchoNode&&) = delete;
    };
}

```

```
private:
    /** UART driver instance. */
    driver::uart::Interface& myUart;
};
} // namespace app
```

```
/**
 * @file Echo node implementation details.
 */
#include "app/echo_node.hpp"
#include "driver/uart/blocking.hpp"
#include "driver/uart/interface.hpp"

namespace app
{
    // -----
    EchoNode::EchoNode(driver::uart::Interface& uart) noexcept
        : myUart{uart}
    {}

    // -----
    void EchoNode::run(const volatile bool& stop) noexcept
    {
        while (!stop)
        {
            uint8_t rxByte{};

            if (myUart.read(rxByte)) { driver::uart::writeBlocking(myUart, rxByte); }
        }
    }
} // namespace app
```

(3 marks: 1 for the reference member to the **interface** with the five deleted operations; 1 for the explicit, `noexcept` constructor that does no I/O and the override on `run`; 1 for a loop that re-checks `stop` every pass and echoes only on a successful read.)

Why non-blocking read, blocking write. (1 mark) A blocking read would spin inside `Interface::read()` until a byte arrived and would never return to the top of the loop, so `stop` would never be read again — the application could not be stopped by anybody, including its own test, which would hang. Polling `read()` returns every pass, which is the only thing that gives the flag a chance to be seen.

The echo write may block because the byte is **already in hand**: there is no decision left to make, nothing else the loop could usefully do, and no deadlock available, since the TX FIFO is drained by hardware whether software cooperates or not. Poll where you might have to give up; block where you have already committed.

Why a `volatile bool` by `const` reference. Freestanding `avr-libc` ships no `<atomic>`, so `std::atomic<bool>` is not available — and on the single-core ATmega328P a byte read is already indivisible, so it would not buy anything. It is `volatile` so that every pass reads it from memory: `run()` never writes it, and when it is set from outside anything the loop calls, by an interrupt handler for instance, a plain `bool` could be read once and kept in a register. It is passed **by reference** and re-read every pass so that the owner can set it at any moment; a return value would only be examined when `run()` returned, which is exactly what it will not do until the flag is set. And it is `const` because the application only ever **reads** it: setting it is the caller's business.

(b)

3 MARKS

The test. (2 marks)

It runs over the `driver::uart::Stub` of Chapter 6 — the UART-level double, not the transport stub, since `EchoNode` sits above the driver and knows nothing of SPI. Host test code may use full modern C++.

```
volatile bool stop{false};
driver::uart::Stub stub{stop};
app::EchoNode node{stub};

stub.injectRxByte(0x00U);
stub.injectRxByte(0x41U);
stub.injectRxByte(0xFFU);

node.run(stop);

EXPECT_EQ(stub.txLen(), 3U);
EXPECT_EQ(stub.txBuf()[0], 0x00U);
EXPECT_EQ(stub.txBuf()[1], 0x41U);
EXPECT_EQ(stub.txBuf()[2], 0xFFU);
```

- **What it queues:** the scripted RX bytes, `0x00`, `0x41` and `0xFF`. The two extremes are chosen deliberately — they are the values most likely to expose a truncation, a sign-extension or a “treat zero as no data” mistake.
- **How the loop terminates:** the stub sets `stop` to `true` when `read()` finds its RX buffer exhausted, and `run()` sees it on the next pass. That is also a **check on the implementation**: because termination depends on `run()` polling and re-checking, a `run()` that blocked in

`readBlocking()` would hang the test rather than fail it — the test telling you the loop is not actually stoppable.

- **What it asserts:** the recorded TX length equals the number queued, and each recorded byte equals the queued byte at the same position — equality **in order**, because a UART is a byte stream and order is half the contract.

The empty case. (1 mark) Queue nothing, call `run(stop)`, and assert that it returns and that `txLen()` is 0.

It proves two things: that the loop's exit does **not** depend on having echoed anything, and that the first thing the loop does before any I/O is test stop — so a caller who sets the flag before `run()` is even entered gets an immediate return.

The implementation mistake it isolates is a `run()` that **ignores `read()`'s return value** and echoes `rxByte` regardless. With nothing queued, that is the only thing the loop can do wrong, and it shows as one spurious `0x00` where there should be none. The three-byte case catches it too, but only through its length assertion: every byte it compares is correct, and the bug adds a fourth, spurious `0x00` after them, so `txLen()` reads 4 instead of 3. A test that checked only the first n bytes would pass it, which is why the length is asserted as well as the contents.

(c)

3 MARKS

The five rungs, and what each adds. (1 mark)

Rung	What it is	The one layer it adds
a	Data-plane pin loop-back : the wrapper wires <code>rx</code> straight back to <code>tx</code> , bypassing <code>uart_top</code>	The USB-serial adapter, its 3.3 V levels, the terminal's baud and frame settings, the two data-plane wires and their crossover, and the FPGA pin assignment. None of your logic.
b	The control plane : write <code>BAUD_DIV</code> over SPI and read it back	The four SPI lines and the level shifter, <code>AvrSpi</code> , the provided <code>spi_slave</code> and <code>spi_reg_bridge</code> , and the register bank's write and read paths.
c	Peripheral loopback : <code>tx</code> tied to <code>rx</code> on the board	The whole datapath on real silicon at a real 50 MHz — <code>baud_gen</code> , <code>uart_tx</code> , <code>uart_rx</code> , both FIFOs and the TX feeder.
d	The real data plane : against the terminal	A second device with an independently generated baud rate that must now agree with the peripheral's own.
e	app::EchoNode	The application.

Why the order is forced. (1 mark) **BAUD_DIV.** The peripheral cannot transmit or receive anything usable until a divider has been written to it — `baud_gen` has no default and the guarded conversion substitutes 1, which is 3.125 Mbaud and useless. And the **only** route to `BAUD_DIV` is an SPI transaction: it is not reachable from the data plane, not from a pin, not from a strap, not from its reset value, which is zero.

So the control plane must be proven before **any** rung that uses the peripheral. Any ladder that puts a peripheral rung first is exercising two layers at once and calling it one, and its first failure names both.

Two reasons EchoNode-first is worse even when it succeeds. (1 mark)

1. **A pass produces no record.** It tells you the whole stack works right now; it does not tell you which layers you may trust tomorrow. The next change — a re-synthesis, a different adapter, a wire moved, a new bench — puts you back to a single binary result with no cleared layers to fall back on. The ladder's product is not the final pass, it is the sequence of things each rung ruled out, and one end-to-end test produces none of them.
2. **It hides marginal failures.** The rungs are designed to stress *different* things: rung a exercises the levels and the terminal settings with none of your logic in the path, rung c removes the second baud source entirely, rung d puts it back. A stack that echoes correctly today may still be running with a 0.3 V MIS0 margin, an unshifted line that happens to work at this temperature, or a baud error that is inside tolerance now and outside it later. Each rung would have failed on exactly one of those; a passing end-to-end test reports none.

(The usual argument — that it is also worse when it *fails*, because nothing is localized — is correct but was not what the question asked.)

(d)

2 MARKS

What is now impossible. (1 mark) **Host-testing EchoNode without a transport.** A Uart can only be constructed over a `driver::transport::Interface&`, so the test must build a `driver::transport::Stub`, then a Uart over it, then script **every SPI transaction the echo produces**: for each byte received, a STATUS read, an RX_DATA read and an RX_POP write; for each byte echoed, a STATUS read and a TX_DATA write — five bytes each, and each read needing its command-phase placeholder plus four data bytes queued. Every transfer consumes a queued byte, the writes' included, so each echoed byte costs twenty-five bytes of script and three bytes of echo about eighty, and the test is then re-testing the register protocol rather than the echo logic. Worse, it cannot end: the flag that stops `run()` was set by the UART stub, and a transport stub knows nothing of it, so once the script runs out every STATUS poll reads zero and `run()` spins for ever.

It is also impossible to run EchoNode over anything **else** — the `driver::uart::Stub` of Chapter 6, a loopback double, a future driver variant, a driver over a different transport — because the type is nailed down at compile time.

What has to be built first: the whole test harness of Chapter 8, one layer too low, and a transport stub taught to set the stop flag, a double doing an application's job, before a single line of application logic can be checked.

The fix. One word:

```
driver::uart::Interface& myUart;
```

The decision of Chapter 6 it makes concrete. (1 mark) That `driver::uart::Interface` is **abstract so that the application codes against a promise rather than against a driver**. The interface exists for exactly this: the application and its tests are written against the contract, so the same object runs unchanged over the stub in a host test and over the real Uart on the

bench, and arrives at the bench already proven. Depending on the concrete class throws the abstraction away while still paying for it — the vtable, the indirection and the extra header are all still there, buying nothing.